User Manual

AWTK 开发实践

用户手册

V1.1.4 Date:2025/12/15

类别	内容
关键词	AWTK、开发基础、经典案例
摘要	本文主要介绍了 AWTK 的基础知识以及示例项目，帮助读者快速入门

ZLG 致远电子

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

修订历史

版本	日期	原因
0.0.0	2025/12/15	• [功能] NewFeature: Added a new feature to improve user experience • [功能] ModuleName: Introduced a new module for better functionality • [improve features] ExistingFeature: Enhanced performance of an existing feature • [improve features] Component: Optimized the interface of a specific component • [不兼容] DeprecatedModule: Removed a deprecated module from the system • [不兼容] FunctionName: Changed the signature of a function, might break existing calls

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

目 录

1. 初识 AWTK 2
1.1 简介 2
1.2 跨平台 2
1.3 特色 2
1.4 环境搭建 3
1.4.1 Windows（x64） 3
1.4.2 Linux（x64） 5
1.4.3 macOS 5
1.5 编译构建 5
1.5.1 下载 AWTK 5
1.5.2 编译 AWTK 6
1.6 运行示例 6
1.7 AWTK 目录 7
1.8 配置文件 8
1.8.1 渲染模式选择 8
1.8.2 是否支持 png/jpeg 图片 10
1.8.3 是否使用点阵字体 10
1.8.4 是否启用软键盘 10
1.8.5 是否启用中文输入法 10
1.8.6 是否启用联想功能 10
1.8.7 是否有标准的内存分配函数 10
1.8.8 是否支持文件系统 11
1.8.9 是否在嵌入式系统运行 11
2. AWTK 应用构建 12
2.1 简介 12
2.2 应用目录 13
2.2.1 应用的目录结构 13
2.2.2 应用的资源目录 13
2.2.3 images 目录说明 14
2.3 界面设计 14
2.3.1 手动编写 UI 设计界面 14
2.3.2 使用 AWTK Designer 设计 UI 界面 14
2.4 打包资源 15
2.5 代码编辑 15
2.5.1 代码结构 15
2.5.2 主函数入口 16
2.5.3 应用程序初始化 16
2.5.4 窗口初始化 17
2.5.5 打开并初始化窗口 17

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

2.5.6 响应按钮事件 18
2.6 编译构建 19
2.7 应用调试 20
2.7.1 Visual Studio 20
2.7.2 Visual Studio Code 20
2.8 应用发布 22
3. 开发基础 23
3.1 简介 23
3.2 控件样式 23
3.2.1 控件样式结构 23
3.2.2 控件样式属性 24
3.2.3 控件状态 25
3.2.4 使用控件样式 26
3.2.5 每个窗口支持独立的控件样式 26
3.2.6 inline style 动态修改控件样式 28
3.2.7 注意事项 29
3.3 布局管理器 29
3.3.1 为什么需要布局参数 29
3.3.2 概述 29
3.3.3 控件自身的布局 30
3.3.4 子控件的布局 32
3.4 资源管理器 39
3.4.1 资源的生成 40
3.4.2 相关工具 40
3.4.3 初始化 40
3.4.4 使用方法 41
3.4.5 资源的名称 42
3.5 事件处理 43
3.5.1 事件处理机制 43
3.5.2 事件分发 43
3.5.3 事件处理 47
3.5.4 事件类型 49
3.6 定时器 54
3.6.1 函数 54
3.6.2 定时器回调函数返回值 54
3.6.3 示例 54
3.7 idle 定时器 55
3.7.1 函数 55
3.7.2 idle 定时器回调函数返回值 56
3.7.3 示例 56
3.8 在多线程环境中发送事件 56
3.8.1 相关函数 56

ZLG ©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

3.8.2 示例 ····· 57
3.9 基本函数库 ····· 58
3.9.1 容器与基本数据结构 ····· 58
3.9.2 流 ····· 59
3.9.3 并发 ····· 60
3.9.4 压缩和解压 ····· 60
3.9.5 平台相关 ····· 60
3.9.6 数据格式 ····· 60
3.9.7 工具类 ····· 61
4. 控件 ····· 62
4.1 简介 ····· 62
4.2 widget ····· 62
4.2.1 函数 ····· 62
4.2.2 属性 ····· 68
4.2.3 事件 ····· 70
4.2.4 示例 ····· 71
4.2.5 控件的可见性 ····· 74
4.3 window_manager ····· 74
4.3.1 窗口管理器 ····· 74
4.3.2 函数 ····· 74
4.3.3 属性 ····· 75
4.3.4 事件 ····· 75
4.3.5 示例 ····· 76
4.4 窗口 ····· 76
4.4.1 window_base ····· 76
4.4.2 window ····· 79
4.4.3 dialog ····· 82
4.4.4 system_bar ····· 85
4.4.5 calibration_win ····· 85
4.4.6 overlay ····· 86
4.4.7 popup ····· 87
4.4.8 component ····· 88
4.5 控件的属性 ····· 89
4.6 基本控件 ····· 90
4.6.1 button ····· 90
4.6.2 label ····· 94
4.6.3 edit ····· 97
4.6.4 image ····· 103
4.6.5 spin_box ····· 106
4.6.6 combo_box ····· 109
4.6.7 combo_box_item ····· 112
4.6.8 color_tile ····· 113

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.6.9 dialog_title ····· 115
4.6.10 dialog_client ····· 117
4.6.11 slider ····· 118
4.6.12 progress_bar ····· 122
4.6.13 tab_control ····· 125
4.6.14 tab_button ····· 127
4.6.15 check_button ····· 129
4.6.16 radio_button ····· 132
4.6.17 dragger ····· 134
4.6.18 digit_clock ····· 137
4.7 通用容器控件 ····· 141
4.7.1 row ····· 142
4.7.2 column ····· 143
4.7.3 grid ····· 143
4.7.4 view ····· 144
4.7.5 clip_view ····· 145
4.7.6 grid_item ····· 145
4.7.7 group_box ····· 146
4.7.8 app_bar ····· 147
4.7.9 pages ····· 147
4.7.10 button_group ····· 148
4.7.11 tab_button_group ····· 149
4.8 扩展控件 ····· 151
4.8.1 canvas_widget ····· 152
4.8.2 color_picker ····· 154
4.8.3 color_component ····· 158
4.8.4 combo_box_ex ····· 158
4.8.5 gif_image ····· 159
4.8.6 gauge ····· 160
4.8.7 gauge_pointer ····· 162
4.8.8 image_animation ····· 164
4.8.9 image_value ····· 167
4.8.10 keyboard ····· 170
4.8.11 candidates ····· 171
4.8.12 lang_indicator ····· 172
4.8.13 progress_circle ····· 173
4.8.14 rich_text ····· 177
4.8.15 slide_menu ····· 180
4.8.16 slide_view ····· 184
4.8.17 svg_image ····· 186
4.8.18 switch ····· 189
4.8.19 text_selector ····· 191

ZLG ©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.8.20 time_clock ····· 195
4.8.21 list_view ····· 197
4.8.22 list_view_h ····· 199
4.8.23 list_item ····· 201
4.8.24 list_item_seperator ····· 203
4.8.25 scroll_bar ····· 204
4.8.26 scroll_view ····· 206
4.8.27 mutable_image ····· 208
4.8.28 mledit ····· 209
4.8.29 line_number ····· 211
4.8.30 slide_indicator ····· 213
4.8.31 hscroll_label ····· 216
4.8.32 serial_widget ····· 218
4.8.33 timer_widget ····· 219
4.8.34 vpage ····· 220
4.9 自定义控件 ····· 221
4.9.1 注册控件 ····· 222
4.9.2 创建控件 ····· 223
4.9.3 使用控件 ····· 229
5. 动画 ····· 230
5.1 简介 ····· 230
5.2 窗口动画 ····· 230
5.2.1 动画类型 ····· 230
5.2.2 使用方法 ····· 231
5.2.3 动画参数 ····· 231
5.2.4 示例 ····· 231
5.2.5 自定义动画 ····· 232
5.3 控件动画 ····· 232
5.3.1 动画类型 ····· 232
5.3.2 特色 ····· 233
5.3.3 函数 ····· 233
5.3.4 XML 参数 ····· 233
5.3.5 插值算法名称（easing） ····· 235
5.3.6 示例 ····· 236
6. 画布 ····· 238
6.1 普通画布 ····· 238
6.1.1 简介 ····· 238
6.1.2 函数 ····· 238
6.1.3 属性 ····· 239
6.1.4 示例 ····· 240

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

6.2 矢量图画布 ····· 240
6.2.1 简介 ····· 240
6.2.2 函数 ····· 241
6.2.3 属性 ····· 242
6.2.4 示例 ····· 243
6.3 离线画布 ····· 244
6.3.1 简介 ····· 244
6.3.2 函数 ····· 245
6.3.3 属性 ····· 245
6.3.4 示例 ····· 245
7. 输入法 ····· 247
7.1 简介 ····· 247
7.2 软键盘 ····· 247
7.3 键盘类型 ····· 248
7.4 键盘映射 ····· 248
7.5 示例 ····· 249
7.6 自定义键盘 ····· 250
7.7 软键盘的 action 按钮 ····· 252
7.7.1 添加 action 按钮 ····· 253
7.7.2 修改 action 按钮显示文本 ····· 253
7.7.3 实现 action 按钮功能 ····· 254
7.8 加入中文输入法 ····· 254
7.9 更新拼音输入法字典和联想字库 ····· 256
7.9.1 更新拼音输入法字典 ····· 257
7.9.2 更新联想字库 ····· 258
8. 多国语言互译 ····· 260
8.1 简介 ····· 260
8.2 语言文件介绍 ····· 260
8.3 文本翻译 ····· 260
8.3.1 设置需要翻译的文本 ····· 261
8.3.2 编写 strings.xml 文件 ····· 262
8.3.3 实时切换语言 ····· 262
8.3.4 重新生成资源和编译应用程序 ····· 263
8.3.5 重新设置缺省字体 ····· 264
8.4 图片翻译 ····· 264
9. 经典案例 ····· 266
9.1 简介 ····· 266
9.2 洁净新风系统 ····· 266
9.2.1 功能详解 ····· 266
9.2.2 应用实现 ····· 266

ZLG ©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

9.3 炫酷图表……276
9.3.1 功能详解……276
9.3.2 应用实现……276
9.4 音乐播放器……288
9.4.1 功能详解……288
9.4.2 应用实现……289
9.5 智能手表……297
9.5.1 功能详解……297
9.5.2 应用实现……298
10. 把应用部署到嵌入式开发环境……313
10.1 把应用部署到嵌入式 Linux……313
10.2 把应用部署到 AWorksOS……315
10.2.1 使用 Eclipse 构建应用……315
10.2.2 使用 Keil 构建应用……320
10.2.3 如何升级 AWTK……322
10.3 把应用部署到其他平台……322

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

文档导读

本文主要介绍了 AWTK 的基础知识以及示例项目，其中包括搭建开发环境、构建 AWTK 应用以及 AWTK 中的控件、动画、输入法和多国语言互译等各种功能，帮助读者快速入门。

需要注意的是：**由于 AWTK 1.7 升级了内置的示例程序 demoui，因此本文中提到的示例程序 demoui 其实是老版本使用的 demouiold，其源码文件由原来的 demo_ui_app.c 改为了 demo_ui_old_app.c**。

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

1. 初识 AWTK

本章导读：

随着手机、智能手表等便携式设备的普及，用户对 *GUI* 的要求越来越高，嵌入式系统对 *GUI* 的需求也越来越迫切，本章将为大家介绍一个轻型、占用资源少、高性能、高可靠、便于移植、可配置及美观的 *GUI* 编程框架。

1.1 简介

AWTK 全称为 Toolkit AnyWhere，是 ZLG 倾心打造的一套基于 C 语言开发的 GUI 框架。旨在为用户提供一个功能强大、高效可靠、简单易用、可轻松做出炫酷效果的 GUI 引擎，并支持跨平台同步开发，一次编程，终生使用。

1.2 跨平台

AWTK 目前支持的平台如下：

- **桌面操作系统：**

  Windows、Linux、macOS

- **嵌入式操作系统：**

  AWorksOS、嵌入式 Linux、RT-Thread、TencentOS、FreeRTOS、SylixOS、MS-RTOS。

  想进一步了解，AWTK 支持的平台与芯片请看：awtk_ecology.md[1]

1.3 特色

1. 高效。采用脏矩形裁剪算法，每次只绘制和更新变化的部分，极大提高运行效率。
2. 稳定。通过良好的架构设计、编程风格、单元测试、动态（valgrind）检查和 Code Review 保证其运行的稳定性。
3. 丰富的 GUI 组件。提供窗口、对话框和各种常用的组件（用户可以配置自己需要的组件，降低对运行环境的要求）。
4. 支持多种字体格式。内置位图字体（并提供转换工具），也可以使用 stb_truetype 或 freetype 加载 ttf 字体。
5. 支持多种图片格式。内置位图图片（并提供转换工具），也可以使用 stb_image 加载 png/jpg 等格式的图片。
6. 支持裸系统，无需 OS 和文件系统。字体、图片、窗体样式和界面描述数据都编译到代码中，以常量数据的形式存放，运行时无需加载到内存。
7. 内置 nanovg 实现高质量的矢量动画。
8. 支持窗口动画、控件动画、滑动动画和高清 LCD 等现代 GUI 常见特性。
9. 支持国际化（Unicode、字符串翻译和输入法等）。
10. 可移植。支持移植到各种 RTOS 和嵌入式 Linux 系统，并通过 SDL 在各种流行的 PC/手

[1] https://github.com/zlgopen/awtk/blob/master/docs/awtk_ecology.md

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

机系统上运行。

11. 支持硬件 2D 加速（目前支持 STM32 的 DMA2D 和 NXP 的 PXP）和 GPU 加速（OpenGL/OpenGLES/DirectX/Metal），充分挖掘硬件潜能。
12. 采用 LGPL 协议开源发布，在商业软件中使用时无需付费。

1.4 环境搭建

本章节将向大家介绍如何在本地搭建 AWTK 开发环境。

1.4.1 Windows（x64）

在 Windows 平台需要安装的软件有：Python（x64 版本 >=2.7）、Scons（版本 >=3.0.0）、Node.js（版本 >=10.0.0）和 Visual Studio C++（版本 >=2015）。需要安装好 Python 后才能安装 SCons。

（1）安装 Python 时，需要将 Python 的安装路径添加到系统环境变量，如下图所示，其他按缺省步骤操作。下载 python-3.8.2-amd64.exe[2]。

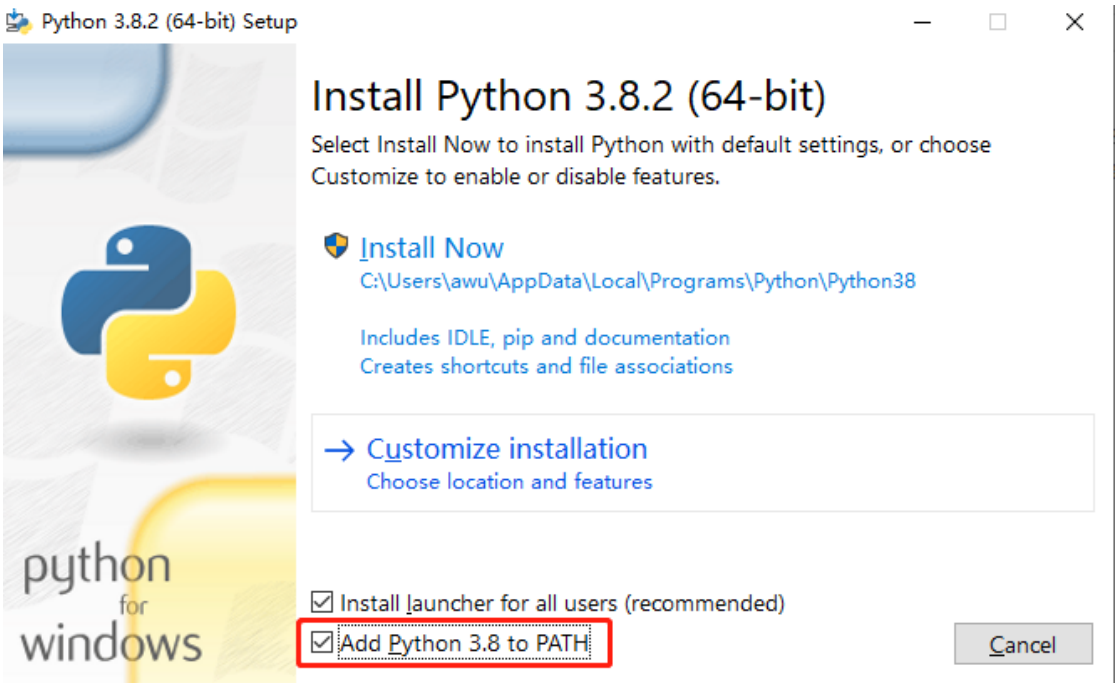


图 1.1 安装 Python

（2）下载 SCons 源码安装包，下载完成后解压并在解压目录打开终端，并执行以下命令，完成安装。下载 SCons-4.3.0.zip[3]。

```
python setup.py install
```

注：如果是安装 Visual Studio 2019，需要安装 SCons（版本 >=3.1.0）。

（3）安装 Node.js 时，安装程序会自动将 Node.js 的安装路径添加到系统环境变量，如下图所示，用户只需按缺省步骤操作即可。下载 node-v12.18.1-x64.msi[4]。

[2] https://www.python.org/ftp/python/3.8.2/python-3.8.2-amd64.exe
[3] http://sourceforge.net/projects/scons/files/scons/4.3.0/SCons-4.3.0.zip/download
[4] https://nodejs.org/dist/v12.18.1/node-v12.18.1-x64.msi

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

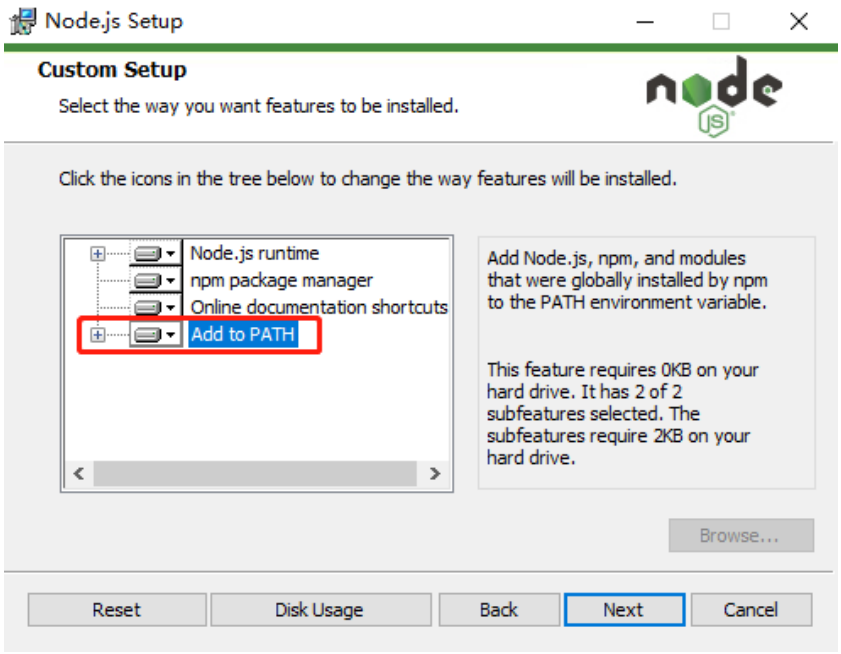


图 1.2　安装 Node.js

（4）安装 Visual Studio

- 如果安装的是 Visual Studio 2015，请选择自定义安装，然后选择 Visual C++，详见下图，其他按缺省步骤操作。

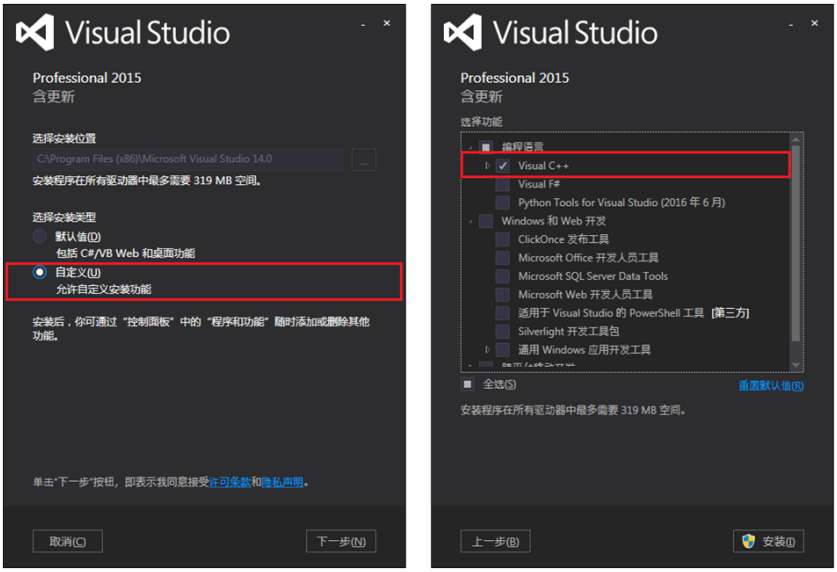


图 1.3　安装 Visual Studio 2015

- 如果安装的是 Visual Studio 2019，安装时请勾选”使用 C++ 的桌面开发”，如下图所示，其他按缺省步骤操作。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.



图 1.4　安装 Visual Studio 2019

1.4.2　Linux（x64）

下面以 Ubuntu（版本 >=16）为例，如果没有安装 SCons 和其他依赖的软件包，请在终端运行下面的命令：

```
sudo apt-get install gcc g++ scons
sudo apt-get install libsndio-dev libgtk-3-dev libglu1-mesa
sudo apt-get install libglu1-mesa-dev libgl1-mesa-glx
sudo apt-get install libgl1-mesa-dev libasound2-dev
sudo apt-get install libibus-1.0-dev fcitx-libs-dev git vim
sudo apt-get install clang-format libharfbuzz-dev nodejs libreadline-dev
```

1.4.3　macOS

如果没有安装 SCons 和 SDL2，请在终端运行下面的命令（假定已安装 Brew）：

```
brew install scons sdl2
```

1.5　编译构建

1.5.1　下载 AWTK

在 AWTK 官网可下载：最新源码、AWTK Designer 界面设计器、入门指南与手册、演示例子以及整合发布包等，下载地址如下：

- AWTK 官网[5]：找到资源下载专栏。
- AWTK 的 VS 工程（基于 CMake）[6]

注：下载 AWTK 源码的时候，不要保存含有中文的路径。另外，如果从 github 使用”Download ZIP”的方式下载上面的源码，下载的 zip 文件夹会包含”-master”；如果是这种情况，在解压后请将”-master”去

[5] https://www.zlg.cn/index/pub/awtk.html
[6] https://github.com/zlgopen/awtk-msvc

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

掉，如：将”awtk-master”重命名”awtk”。

1.5.2 编译 AWTK

在编译 AWTK 之前请参考本章 1.4 的内容，确认已在对应平台上搭建好开发环境，然后下载好 AWTK 源码后，就可以编译 AWTK 了，在 awtk 根目录下打开终端，执行以下命令：

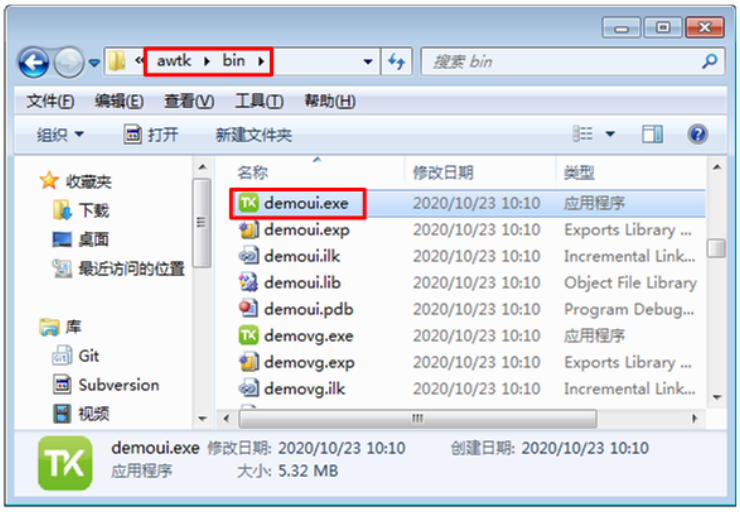
```
scons
```

注：检查 Windows 下是否已经正确安装 Python、SCons 和 Visual Studio C++ 相应的版本，请运行 AWTK Designer（下载地址请看 1.5.1）安装目录下的：Tools/awtk-tools-checker.exe。

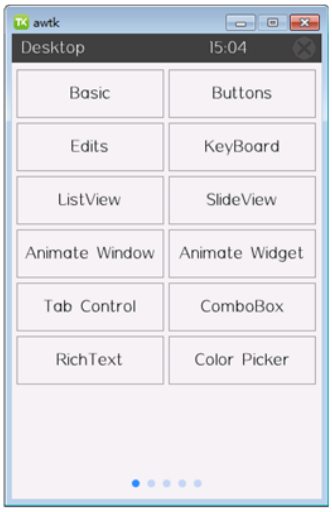
1.6 运行示例

AWTK 源码中内置了示例程序（demoui），该示例程序演示了 AWTK 中各个控件及其基本功能的效果，编译源码成功后，可运行 awtk/bin/demoui 示例，此处以 Windows 平台为例，在 awtk 根目录下打开终端，执行以下命令，运行效果下图右侧所示。

```
bin\demoui.exe
```



awtk/bin/demoui.exe



demoui运行效果图

图 1.5 demoui 所在目录及其运行效果

注：demoui 示例仅做功能展示，其源文件位于 awtk/demos/demo_ui_app.c。各个控件的具体用法请参考示例程序 awtk-c-demos[7]。

[7] https://github.com/zlgopen/awtk-c-demos

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

1.7 AWTK 目录

编译 AWTK 源码后目录如下图所示，其中 bin 目录和 lib 目录是编译后新增的，具体说明详见下表。

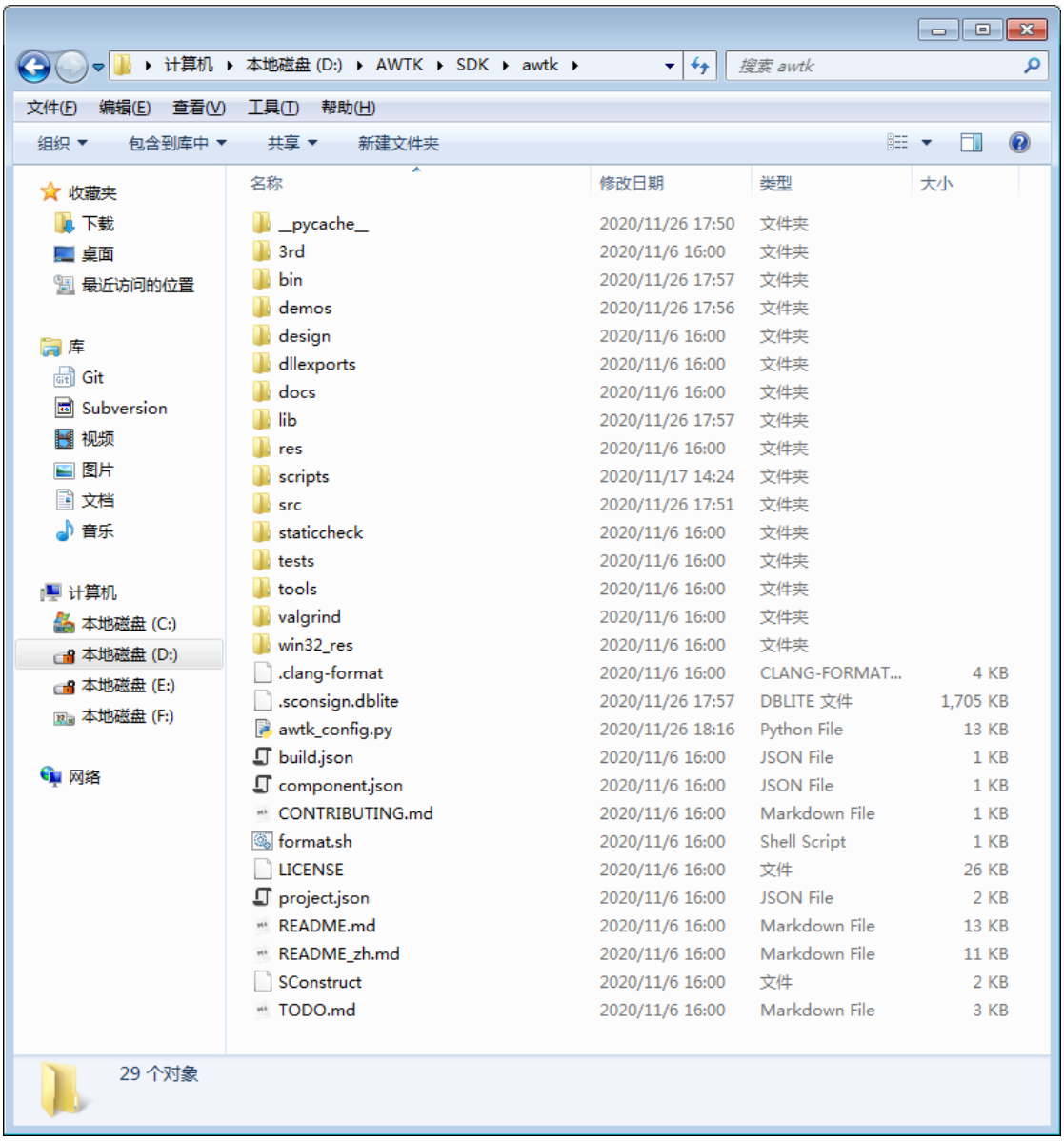


图 1.6 AWTK 目录结构

目录/文件	说明
3rd	AWTK 使用的第三方库目录
bin	编译 AWTK 后生成的可执行文件，包括动态链接库（awtk.dll）、示例程序和内置工具
demos	AWTK 内置示例程序的代码目录，代码文件对应的示例程序详见目录下的 README.md
design	使用 AWTK Designer 设计 demoui 所用的资源目录（AWTK Designer 详见 2.3.2）
dllexports	AWTK 动态链接库（awtk.dll）导出表
docs	AWTK 帮助文档目录
lib	编译 AWTK 后生成的静态链接库
res	demoui 资源文件目录

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

目录/文件	说明
scripts	AWTK 工具脚本目录，具体使用说明请查阅该目录下的 README.md
src	AWTK 源代码
staticcheck	静态代码检查，具体说明请查阅该目录下的 README.md
tests	AWTK 的单元测试代码目录
tools	AWTK 内置工具的源代码目录
valgrind	AWTK 用于检查内存泄漏的 valgrind 工具
win32_res	AWTK 应用程序在 Windows 下的图标资源
awtk_config.py	AWTK 配置文件，配置方法详见 1.8
build.json	特殊平台（Web、Android、IOS）编译配置文件，具体用法请查阅 docs/build_config.md
project.json	demoui 的项目配置文件，可使用 AWTK Designer 打开该项目文件做界面设计
README.md	AWTK 介绍文档（英文版）
README_zh.md	AWTK 介绍文档（中文版）
SConstruct	SCons 脚本

AWTK Designer 的具体用法请参阅其安装目录下 Docs 文件夹中的《AWTK Designer 用户手册》，下载地址本文 1.5.1。

AWTK 的示例程序主要是 awtk/bin 目录中以 demo 开头的可执行程序，比如 demoui、demotr 等。AWTK 的内置工具源代码位于 awtk/tools，编译后的可执行程序位于 awtk/bin，这些工具主要用于生成资源文件，比如 xml_to_ui，该工具可将程序界面的 xml 文件打包生成二进制的 bin 文件，相关工具详见 3.4.2。

注：

1. 上表中的目录下通常都有一个 README.md 文档，该文档通常包含目录的说明和具体的使用方法。
2. awtk/docs/README.md 文档包含了 docs 目录下 AWTK 帮助文档的具体说明。
3. awtk/docs/changes.md 文档包含了 AWTK 的最新动态。

1.8 配置文件

在 AWTK 目录下有个 awtk_config.py 文件（被 SConstruct 引用），在这个文件中可以通过定义不同含义的宏实现不同的效果，接下来介绍一些常用的宏及其说明。

注：

1. 修改 awtk_config.py 文件后，需要重新编译 awtk，编译方法请看 1.5.2。
2. 修改 awtk_config.py 文件只在 Windows、Linux、macOS 平台有效果，在嵌入式平台无效。

1.8.1 渲染模式选择

AWTK 支持多种渲染模式，不同的渲染模式适用于不同的硬件平台，显示效果也略有不同。比如：PC 操作系统平台 Windows、Linux、macOS 可以使用 OpenGL 模式，而普通嵌入式平台则可以使用 AGGE 模式。通过修改 awtk_config.py 文件中的渲染模式变量，可以在 PC 上模拟在不同的渲染模式时应用的运行效果。

常见的渲染模式如下：

- **GL3/ GLES2/ GLES3**：基于 OpenGL/ GLES 实现（nanovg 内置），由宏 WITH_NANOVG_GL 决定。适合支持 OpenGL 硬件加速的平台（比如：Windows、Linux、macOS）使用

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

- **AGGE**：基于 agge 实现。纯软件实现，渲染效果一般，由宏 WITH_NANOVG_AGGE 决定。适合没有 GPU 的嵌入式平台使用

awtk_config.py 文件中渲染模式变量代码如下：

```
# awtkawtk_config.py
...
VGCANVAS='NANOVG'
if OS_NAME == 'Windows':
  TK_ROOT=TK_ROOT.replace('\\', '\\\\');
  NANOVG_BACKEND='GLES2'
else:
  NANOVG_BACKEND='GL3'

#VGCANVAS='CAIRO'
#NANOVG_BACKEND='GLES2'
#NANOVG_BACKEND='GLES3'
#NANOVG_BACKEND='AGG'
#NANOVG_BACKEND='BGFX'
#NANOVG_BACKEND='AGGE'
...
```

当 VGCANVAS=’NANOVG’时，对于常用的 AGGE 和 OpenGL 渲染模式又可以细分为下面几种：

(1) AGGE

- AGGE-BGR565：表示运行时绘图使用 AGGE 引擎，LCD Frame Buffer 的格式为 BGR565。
- AGGE-BGRA8888：表示运行时绘图使用 AGGE 引擎，LCD Frame Buffer 的格式为 BGRA8888。
- AGGE-MONO：表示运行时绘图使用 AGGE 引擎，LCD Frame Buffer 的格式为 MONO（黑白）。

它们在 awtk_config.py 中的参数设置如下：

```
# 使用 AGGE 渲染模式
NANOVG_BACKEND='AGGE'
# 基于 Frame Buffer 实现彩色 LCD（BGR565或BGRA8888格式使用该参数）
LCD='SDL_FB'

# 基于 Frame Buffer 实现黑白 LCD（MOMO格式使用该参数）
# LCD='SDL_FB_MONO'


# FRAME_BUFFER_FORMAT='bgr565'   # 帧缓冲颜色格式为 BGR565
# FRAME_BUFFER_FORMAT='bgra8888' # 帧缓冲颜色格式为 BGRA8888
```

(2) OpenGL

- OpenGL-GLES2：表示运行时绘图使用 GLES2 引擎，建议在 Windows 平台使用。
- OpenGL-GL3：表示运行时绘图使用 GL3 引擎，建议非在 Windows 平台使用。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

它们在 awtk_config.py 中的参数设置如下：

```
NANOVG_BACKEND='GLES2'  # 使用 OpenGL-GLES2 渲染模式
# NANOVG_BACKEND='GL3'  # 使用 OpenGL-GL3 渲染模式

LCD='SDL_GPU'  # 基于 GPU 实现 LCD
```

1.8.2 是否支持 png/jpeg 图片

如果要支持 png/jpeg 图片，请定义宏 **WITH_STB_IMAGE**，可以使用下面两种方式加载图片：

（1）有文件系统：即同时定义了宏 **WITH_FS_RES**，AWTK 会通过读取文件方式直接读取 assets/default/raw/images 目录下的原始 png/jpeg 图片。

（2）无文件系统：即没有定义宏 **WITH_FS_RES**，AWTK 会使用 assets/default/inc/images 目录下”*.res”文件（res 文件是未经解码的原始 png/jpeg 文件数据）

注：假如宏 **WITH_FS_RES** 和 **WITH_STB_IMAGE** 都没有定义，则使用 assets/inc/images 目录下”*.data”文件（data 文件是 png/jpeg 解码后的位图数据）。

1.8.3 是否使用点阵字体

程序默认启用宏 **WITH_STB_FONT**，即使用 TrueType 字体。如果需要使用点阵字体请定义宏 **WITH_BITMAP_FONT**，一般只在 RAM 极小时，且需要支持预先解码的图片，才启用本宏。

1.8.4 是否启用软键盘

如果应用程序不需要弹出软键盘，请定义宏 **WITH_NULL_IM**。如果需要使用输入法，请看第 7 章。

1.8.5 是否启用中文输入法

如果应用程序不需要中文输入法，但又需要软键盘，请定义宏 **WITH_IME_NULL**。

1.8.6 是否启用联想功能

如果启用了软键盘，但不想启用联想功能，请定义宏 **WITHOUT_SUGGEST_WORDS**。

1.8.7 是否有标准的内存分配函数

如果有标准的 malloc/free/calloc 等函数，请定义本宏 **HAS_STD_MALLOC**。否则将由 AWTK 自己管理动态内存，如果系统提供了标准的 malloc 函数建议打开此项。

ZLG ©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

1.8.8 是否支持文件系统

有文件系统：即在 awtk_config.py 文件中定义了宏 WITH_FS_RES，AWTK 将加载应用程序 default/raw 目录下的资源文件（字体、图片、窗体样式和界面描述数据等），运行应用程序时，依赖 default/raw 目录下的资源文件。代码如下：

```
# awtk/awtk_config.py
...
COMMON_CCFLAGS=COMMON_CCFLAGS+' -DWITH_ASSET_LOADER -DWITH_FS_RES -DWITH_ASSET_
↪LOADER_ZIP '
...
```

** 无文件系统：**即在 awtk_config.py 文件中没有定义宏** WITH_FS_RES，** 就无需 OS 和文件系统，AWTK 将加载应用程序 default/inc 目录下的资源文件，AWTK 将这些资源编译到代码中，以常量数据的形式存放，嵌入到固件中，运行应用程序时，无需加载这些资源文件。

1.8.9 是否在嵌入式系统运行

默认程序入口是 WinMain() 或 main()，如果应用程序是在嵌入式系统运行的话，请定义宏 **USE_GUI_MAIN**，这种情况下程序入口点变为 gui_app_start()，具体可以看示例 HelloWorld.Xml-Demo/src/app_main.c 文件。

注: 更多的宏说明请看：awtk/docs/porting_common.md。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

2. AWTK 应用构建

本章导读：

在学习计算机语言的时候，通常接触的第一个程序也就是最经典的 Hello World 程序。与之类似，要掌握 AWTK 的开发流程，也先创建一个 Hello World 应用程序，通过该程序了解 AWTK 的相关知识。

2.1 简介

本章以 HelloWorld.Xml-Demo 为例，介绍 AWTK 应用的构建流程。示例程序的主要功能如下：

- 一个文本框用于显示”Hello World”；
- 一个按钮用于改变自身文本。

注: 本章节所使用到的代码可以在 HelloWorld.Xml-Demo 目录下找到。

一般 AWTK 应用的实现流程详见下图，图中背景比较深的部分（如设计 UI 界面文件，实现业务逻辑）是编写应用程序的主要工作，其他部分可以复用，应用实现后就可以打包资源并编译程序代码了。

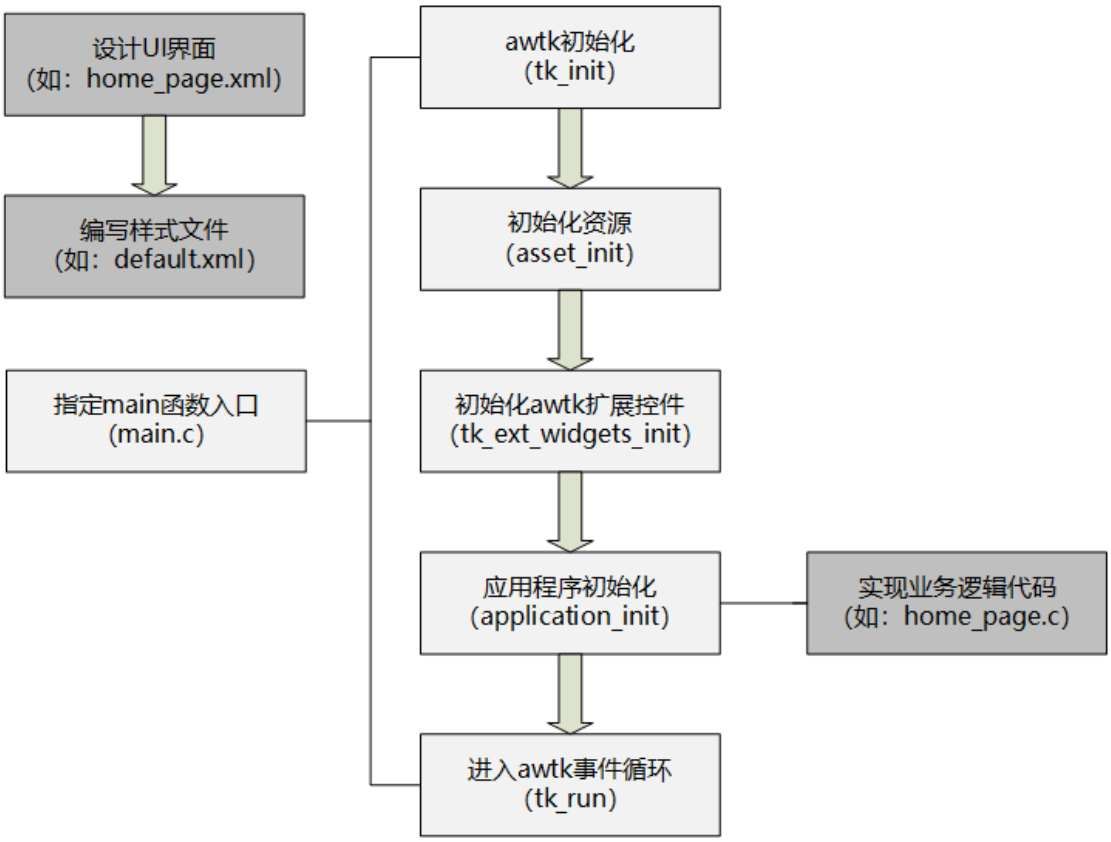


图 2.1 AWTK 应用的实现流程

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

2.2 应用目录

2.2.1 应用的目录结构

此处以 HelloWorld.Xml-Demo 为例，简单介绍 AWTK 应用的目录结构：

目录/文件	说明
bin	可执行文件目录
design	原始资源目录（存放 AWTK Designer 设计时使用的资源）
res	经打包后的资源目录（打包时自动生成，应用程序运行时会使用该目录下的资源）
scripts	工具脚本目录（具体使用说明请查阅该目录下的 README.md）
src	源代码目录
tests	单元测试代码目录
project.json	项目描述文件
SConstruct	SCons 编译脚本

建议使用 AWStudio[8] 中内置的 AWTK Designer 工具新建项目，以上文件以及目录都会自动生成。

2.2.2 应用的资源目录

在 AWTK 中，规定应用设计时的资源目录为 design，并且约定 design/default 的目录结构（此处的 default 为主题名称），各目录说明如下表所示（注意目录位置和名称不可随意更改）：

目录	说明
fonts	字体文件目录，AWTK 目前支持矢量字库和点阵字
images	图片文件目录（包含 x1、x2、x3、xx 目录），AWTK 目前支持的格式有 png、jpg、bmp、gif、svg
strings	多国语言文件目录，包含使用 xml 描述的翻译文本信息，所有语言环境下的文本信息都保存在 strings.xml 中，类似字典文件
styles	样式文件目录，包含使用 xml 描述的界面外观信息，即样式文件，比如下文所介绍的 default.xml
ui	UI 文件目录，包含使用 xml 描述的界面结构信息，即 UI 文件，比如下文所介绍的 home_page.xml

注：应用运行时将使用 res 目录下的资源，该目录通过打包资源自动生成，如果修改了 design 目录下的资源，则需要重新打包资源让其生效。

[8] https://awtk.zlg.cn

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

2.2.3 images 目录说明

design/default/images 目录下可以建立的文件夹详见下表。

目录	说明
x1	存放普通密度屏幕上使用的图片
x2	存放 2 倍密度屏幕上使用的图片
x3	存放 3 倍密度屏幕上使用的图片
xx	存放与屏幕密度无关的图片
svg	存放 svg 图片

AWTK 运行时会优先在 x1、x2、x3 目录中的其中一个查找图片（比如 LCD 的 DPR=1，则在 x1 目录），如果找不到，则在 xx 目录中查找。对于嵌入式系统，一般只需要 x1 或 xx 的图片。如果开发环境使用高清的 PC 显示器，为了方便 PC 上看效果，建议也准备一套 x2 或 x3 的图片。

注: AWTK 中使用资源时，资源的名称一般为文件名，且不包含文件后缀。

2.3 界面设计

2.3.1 手动编写 UI 设计界面

可以用文本编辑器（如：VSCode、Notepad++ 等）编写 UI 界面的 XML 文件，代码如下，

```
<!-- HelloWorld.Xml-Demo/design/default/ui/home_page.xml -->
<window name="home_page">
  <label name="label" x="c" y="m" w="30%" h="20%" style:normal:font_size="30" text=
↪"Hello World"/>
  <button name="button" x="c" y="m:100" w="100" h="36" text="True"/>
</window>
```

以上代码中使用的控件所代表的具体含义请看第 4 章。

2.3.2 使用 AWTK Designer 设计 UI 界面

AWTK Designer 是 AWStudio 中内置的工具，专门用来制作 AWTK 应用程序的 UI 界面的实用工具，只要通过拖曳和点击就可以完成复杂的界面设计，而且可以随时预览效果图。

前往 AWStudio 云平台[9] 下载并安装 AWTK Designer 后，可以阅读 AWTK 在线帮助文档[10] 中的《AWTK Designer 用户手册》，了解如何使用 AWTK Designer 完成界面设计。

[9] https://awtk.zlg.cn/
[10] https://awtk.zlg.cn/docs

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

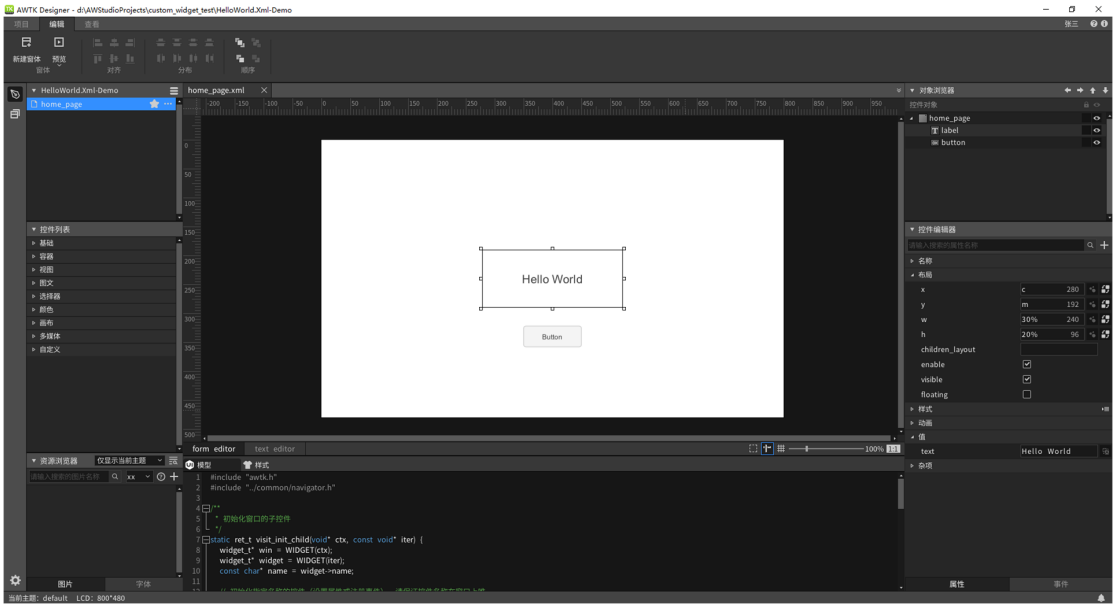


图 2.2　AWTK Designer 主界面

2.4　打包资源

在设计完 UI 界面后，需要将文件打包为 AWTK 可以使用的资源，在 HelloWorld.Xml-Demo 目录下打开终端，执行以下命令，生成样式、翻译文件、字体、图片、UI 等资源文件。

```
python ./scripts/update_res.py all
```

执行完上面的命令后会在 res 文件夹中生成对应的资源文件，并实现资源读取的函数文件。

注: 打包资源的注意事项如下:

1. 在生成资源之前，请先参考上一章节编译 AWTK。
2. 如果 design/default 目录下的资源文件发生变化（增加、删除，修改等），都需要重新执行该命令对资源文件进行打包。
3. 该命令的参数可以查看 scripts 目录下的 README.md。

另外，也可以在 AWTK Designer 中打开项目，点击”项目”页面下的”打包”进行资源的打包。

2.5　代码编辑

2.5.1　代码结构

HelloWorld.Xml-Demo 的源代码文件在项目的 src 目录下，具体说明见下表：

源文件	说明
common	存放自动生成的导航器（navigator）源代码，用于页面跳转，可直接使用
pages	存放页面源代码，文件名与页面名相同
application.c	存放当前项目的初始化与反初始化源代码，可直接使用
main.c	存放当前项目的应用程序入口，可直接使用

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

源文件	说明
SConscript	Scons 编译脚本

2.5.2 主函数入口

main.c 中包含着 main 函数的入口（在 awtk_main.inc 中），这个文件用户可以直接套用到自己项目中，通常不需要修改，主要完成设置屏幕大小、资源的初始化以及 AWTK 框架的加载等，代码如下：

```
/* HelloWorld.Xml-Demo/src/main.c */
#include "awtk.h"

BEGIN_C_DECLS
#ifdef AWTK_WEB
#include "assets.inc"
#else /*AWTK_WEB*/
#include "../res/assets.inc"
#endif /*AWTK_WEB*/
END_C_DECLS

extern ret_t application_init(void);

extern ret_t application_exit(void);

#include "awtk_main.inc"
```

2.5.3 应用程序初始化

在 main.c 文件中额外声明了两个函数，分别是 application_init 和 application_exit，这两个函数分别在进入 awtk 事件循环的前后执行，它们在 application.c 中实现，用户可以在这两个函数中添加项目的初始化和反初始化代码：

```
/* HelloWorld.Xml-Demo/src/application.c */
......
/* 当程序初始化完成时调用，全局只触发一次。有初始化需求可以自己在此处添加 */
static ret_t application_on_launch(void) {
  return RET_OK;
}

/* 当程序退出时调用，全局只触发一次。如有初始化的数据需要释放，在此函数内释放 */
static ret_t application_on_exit(void) {
  return RET_OK;
}

/* 初始化程序 */
ret_t application_init(void) {
  custom_widgets_register();
  application_on_launch();
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  if (strlen(APP_SYSTEM_BAR) > 0) {
    /* 如果存在指定的系统栏，则打开指定的顶部系统栏 */
    navigator_to(APP_SYSTEM_BAR);
  }

  if (strlen(APP_BOTTOM_SYSTEM_BAR) > 0) {
    /* 如果存在指定的系统栏，则打开指定的底部系统栏 */
    navigator_to(APP_BOTTOM_SYSTEM_BAR);
  }
  /* 打开设定的主界面窗口 */
  return navigator_to(APP_START_PAGE);
}

/* 退出程序 */
ret_t application_exit(void) {
  application_on_exit();
  log_debug("application_exit\n");

  return RET_OK;
}
```

2.5.4 窗口初始化

在 AWTK 中，通常会为每个窗口配置一个 c 文件，例如 home_page.c，并在该文件中进行窗口初始化，用户可以按需在初始化函数中添加内容，对该窗口的操作也在这个文件中实现，代码如下所示：

```
/* HelloWorld.Xml-Demo/src/pages/home_page.c */
ret_t home_page_init(widget_t* win, void* ctx) {
  (void)ctx;
  return_value_if_fail(win != NULL, RET_BAD_PARAMS);
  /* 遍历 win 窗口的子控件，visit_init_child 是遍历时的回调函数 */
  widget_foreach(win, visit_init_child, win);

  return RET_OK;
}
```

2.5.5 打开并初始化窗口

在项目代码中可以使用 AWTK 原生的 API 接口打开并初始化窗口，此处以 home_page 为例，代码如下：

```
ret_t application_init(void) {
  /* 调用 AWTK 原生 API 打开 home_page 窗口 */
  widget_t* win = window_open("home_page");
  /* 初始化 home_page 窗口 */
  home_page_init(win, NULL);
  return RET_OK;
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
}
```

另外，也可以使用项目 src/common/navigator.h 中的导航器接口打开窗口，代码如下：

```
/* HelloWorld.Xml-Demo/src/application.c */
/* 初始化程序 */
ret_t application_init(void) {
  ......
  /* 打开并初始化 界面窗口，APP_START_PAGE 为窗口名称"home_page" */
  return navigator_to(APP_START_PAGE);
}
```

navigator 的相关代码是 AWTK Designer 新建项目时自动生成的，navigator_to 函数本质上内部也是调用 AWTK 原生的 API 来打开窗口，并在 navigator_window_init 函数中根据窗口名称来调用对应的窗口初始化函数，代码如下，更具体的代码详见 src/common/navigator.c。

```
/* HelloWorld.Xml-Demo/src/common/navigator.c */
extern ret_t home_page_init(widget_t* win, void* ctx);

static ret_t navigator_window_init(const char* name, widget_t* win, void* ctx) {
  if (tk_str_eq(name, "home_page")) {
    return home_page_init(win, ctx);
  }

  return RET_OK;
}
```

2.5.6 响应按钮事件

在本 demo 中，会简单实现一个按下按钮改变按钮文字的功能来展示如何为一个控件注册事件并实现响应函数，实现步骤如下：

- 根据按钮名称调用 widget_on 函数注册事件，一般在窗口初始化代码中提到的遍历子控件的函数进行注册；
- 实现注册的回调函数，代码如下所示。

```
/* 按钮注册的回调函数 */
static ret_t on_button_click(void* ctx, event_t* e) {
  /* e->target为当前触发事件的控件*/
  widget_t* btn = e->target;
  char btn_text[10] = "";
  /* 获取按钮的文本 */
  widget_get_text_utf8(btn, btn_text,sizeof(btn_text));
  /* 根据文本内容改变文本 */
  if(strcmp("True", btn_text) == 0) {
    widget_set_text_utf8(btn, "False");
  } else {
    widget_set_text_utf8(btn, "True");
  }
  return RET_OK;
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
}

/* 初始化窗口的子控件 */
static ret_t visit_init_child(void* ctx, const void* iter) {
  widget_t* win = WIDGET(ctx);
  widget_t* widget = WIDGET(iter);
  const char* name = widget->name;

  /* 初始化指定名称的控件（设置属性或注册事件），请保证控件名称在窗口上唯一 */
  if (name != NULL && *name != '\0') {
    /* 在此处对指定名称的控件进行注册事件，此处的事件为指针按下 */
    if (tk_str_eq(name, "button")) {
      widget_on(widget, EVT_CLICK, on_button_click, win);
    }
  }
  return RET_OK;
}
```

注：这里的 visit_init_child 函数是在 widget_foreach 遍历接口中注册的回调函数，每遍历一个控件就会调用一次这个回调函数，因此需要对控件名称进行判断。

2.6 编译构建

Windows 和 Linux 下编译源代码的命令都是一样的，下面以 Windows 平台为例：

在 HelloWorld.Xml-Demo 目录下打开终端，执行以下命令：

```
scons
```

编译后生成的可执行文件在 HelloWorld.Xml-Demo/bin 目录下，名称为 demo，运行效果如下：

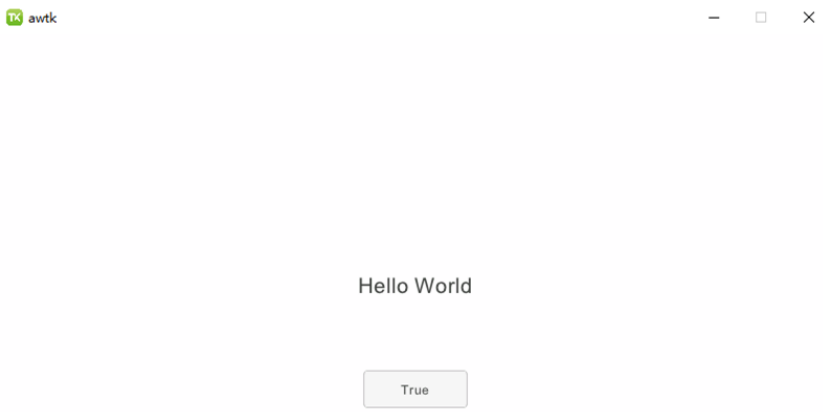


图 2.3 hello world 运行效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

2.7 应用调试

有两种方式调试 AWTK 应用程序：使用 Visual Studio 或者 Visual Studio Code，下面分别介绍如何使用这两种方式。

2.7.1 Visual Studio

使用 scons 命令编译时并没有生成 Visual Studio 的工程，如果需要在 Visual Studio 中调试 AWTK 应用程序，可按下列步骤进行：

1. 打开 Visual Studio；
2. 在”文件”菜单中点击”打开”并选中”项目/解决方案”；
3. 选择 bin/demo.exe（或者其它要调试的可执行文件）；
4. 在项目设置中设置调试参数（可选）；
5. 将要调试的文件拖到 Visual Studio 中，在要调试的地方打断点进行调试。

2.7.2 Visual Studio Code

使用 Visual Studio Code 调试时，可按下列步骤进行：

1. 如果还没有安装 C/C++ 插件的话，请安装该插件，如下图所示：

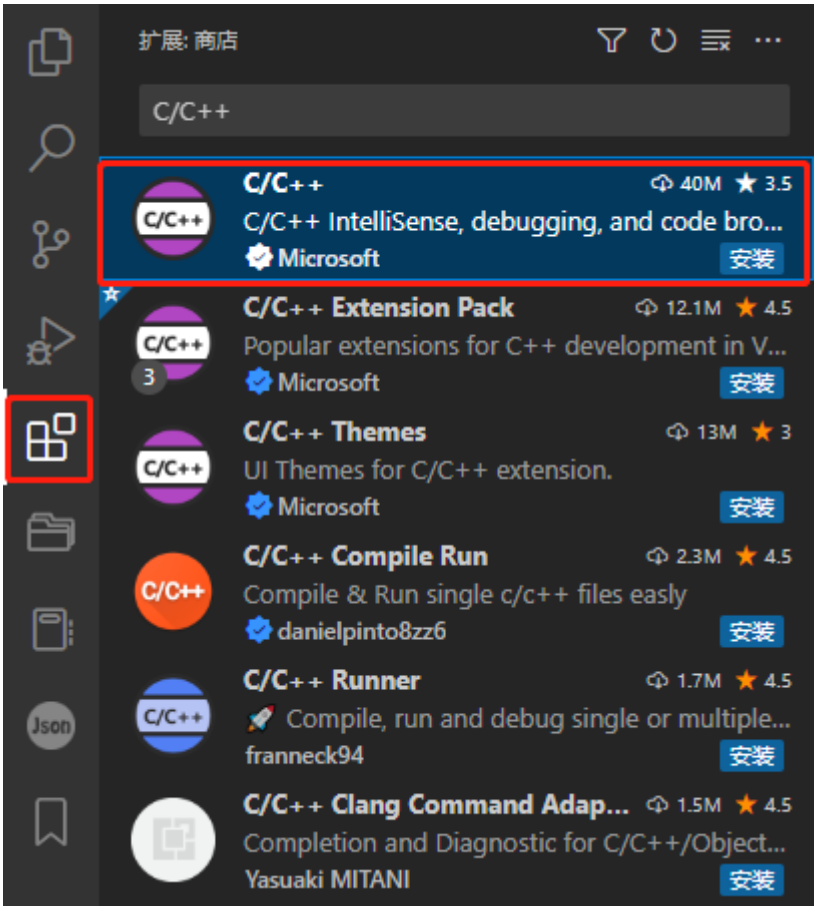


图 2.4　安装 C/C++ 插件

2. 选择文件 → 文件夹，选择要调试的应用程序（如:HelloWorld.Xml-Demo）。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

3. 选择运行与调试，如下图所示：

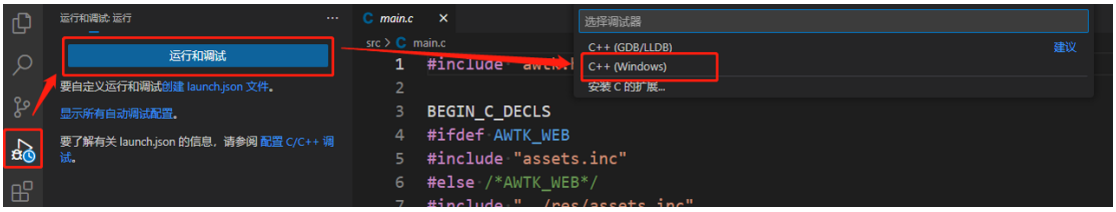


图 2.5　添加配置

4. 选择 C++(Windows）后，弹出界面，如下图所示：

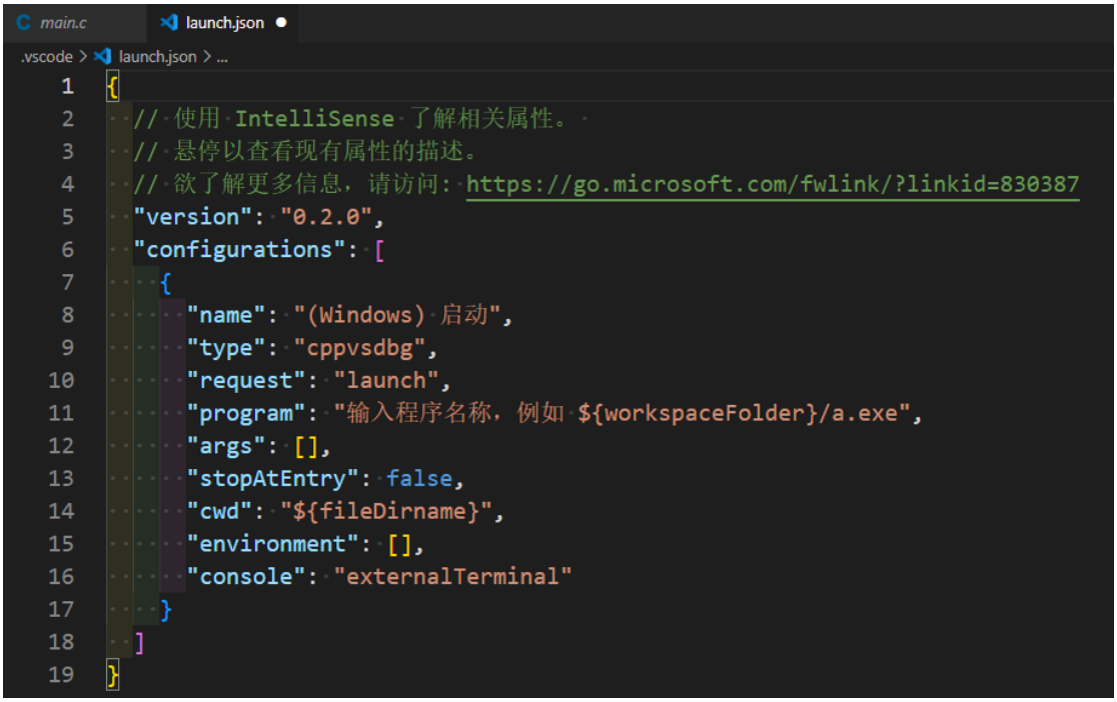


图 2.6　弹出界面

5. 如果没有弹出界面，可以手动生成该文件，如下图所示：



图 2.7　创建 launch.json

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

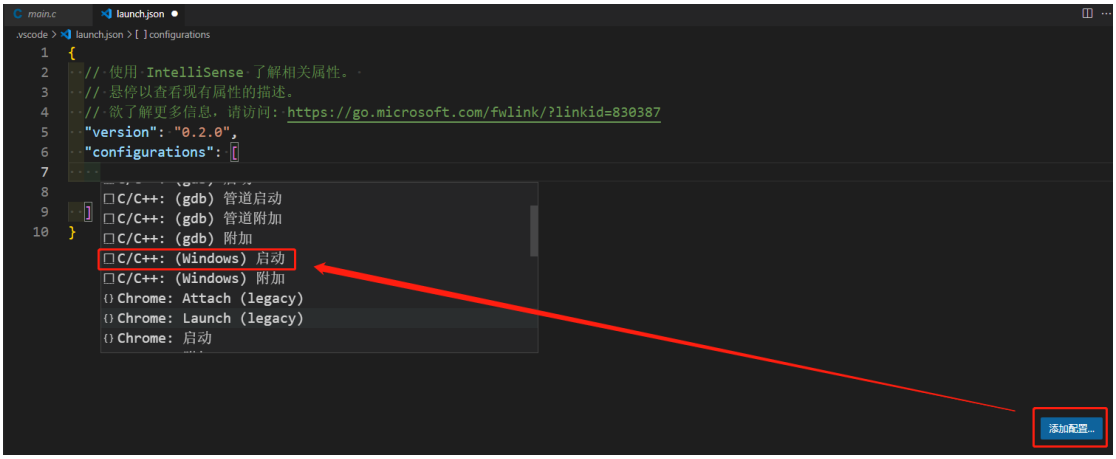


图 2.8　添加配置

6. 将 launch.json 文件中的 program 修改为要调试的应用程序的路径，即将”program”: “输入程序名称, 例如 example ${workspaceFolder}/a.exe”，修改成如下：

```
"program": "${workspaceFolder}/bin/demo.exe",
```

7. 按 F5 启动调试，然后就可以在源文件里面下断点了。

注: 建议安装 C/C++ Intellisense 插件，该插件主要用于跳转到定义、自动提示等。

2.8　应用发布

在开发完成后，通常需要将可执行文件和运行时需要的资源拷贝出来，发给客户或者放到嵌入式开发板上运行。AWTK 提供了一个脚本：scripts/release.py，将可执行文件和运行时需要的资源拷贝出来，放到当前目录的 release 子目录下。

例如，当前的项目是 HelloWorld.Xml-Demo，此处以 Windows 平台为例，在 HelloWorld.Xml-Demo 目录下打开终端，执行 release.py 脚本，命令如下：

```
python scripts/release.py
```

运行成功后，可以看见项目目录下生成了 release 文件夹，其中包含 assets 目录和 bin 目录，存放的内容如下：

- assets 目录：存放了项目运行时需要的资源；
- bin 目录：存放了项目的可执行文件。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

3. 开发基础

本章导读：

学习一门新的开发语言，首先得学习一些关于该门语言的基础知识，如：语法。同样的在学习 *AWTK* 的过程中，也需要学习一些基本的基础知识，在了解了这些知识之后，才能更加好的学习后面的章节。

3.1 简介

本章将按照 AWTK 框架的构成，阐述 AWTK 开发过程中所需的基础知识，包括控件样式、布局管理器、定时器、事件处理等。本章以 AWTK 源码目录下自带的例子（awtk/demos）来介绍关于 AWTK 的基础知识，所以本章涉及的代码均可以在 AWTK 源码目录下找到。

3.2 控件样式

设计漂亮的界面并非程序员的强项，AWTK 通过窗体提供这样一种机制，让设计漂亮的界面变得非常容易。通过控件样式，可以改变控件的背景颜色、边框颜色、字体颜色、字体、字体大小、背景图片、背景图片的显示方式和图标等属性。同时 AWTK 也提供了一些控件样式重用的机制，让控件样式文件的开发和维护变得容易。

3.2.1 控件样式结构

AWTK 的控件样式按控件进行分类，每种控件可以有多种不同的样式，每种样式下又有不同状态下的配置，代码如下：

```
<!--awtk/demos/assets/default/raw/styles/default.xml -->
...
<label>
  <style name="default">
    <normal text_color="black" />
  </style>
  <style name="left">
    <normal text_color="red" text_align_h="left" border_color="#a0a0a0" margin="4"
↪/>
  </style>
...
</label>
```

上面是文本框样式配置（可以按需增加 style 的数量），其中定义两种不同的文本样式：

- default 为缺省的文本样式（如果控件中不指定样式将使用 default 样式）
- left 为文本水平居左样式

控件样式的各个属性，具体请看 3.2.2 章节，可以分别在控件、style 或者状态中定义，优先级从底到高，重复定义时，后者会覆盖前者。具体说明如下：

（1）在控件中指定样式的属性，代码如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
<label text_color="red">
  <style name="default">
    <normal border_color="black" />
  </style>
</label>
```

（2）在 style 中指定样式的属性，代码如下：

```
<label >
  <style name="default" text_color="red">
    <normal border_color="black" />
  </style>
</label>
```

（3）在 style 中指定样式的属性，代码如下：

```
<label >
  <style name="default" >
    <normal text_color="red" border_color="black" />
  </style>
</label>
```

注：在自己的应用程序 assets/default/raw/styles 目录下必须有 default.xml 文件（缺省控件样式），否则控件将不能正常显示，并且最好在 default.xml 文件中给各个控件指定一个 default 缺省样式。

3.2.2 控件样式属性

AWTK 为控件提供了丰富的属性，详见下表，其中颜色可使用标准名称、# 开头的 16 进制值和 rgba 合成的值，例如，在 3.2.1 章节中给 label 文本控件设置文本颜色为黑色。

在下表中的属性一般每个控件都可以使用，例如 button 按钮、label 控件都可以设置 bg_color 属性。除非特别说明，例如 selected_bg_color 属性只能在 edit 编辑器中使用，具体每个控件可以设置哪些属性可以参考第 4 章下各个子章节的”样式”章节。

属性	说明
bg_color	背景颜色
fg_color	前景颜色
mask_color	蒙版颜色
font_name	字体名称
font_size	字体大小
text_color	文本颜色
tips_text_color	提示文本颜色
text_align_h	文本水平对齐的方式（取值：center|left|right）
text_align_v	文本垂直对齐的方式（取值：middle|top|bottom）
border_color	边框颜色
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称
bg_image_draw_type	背景图片的显示方式
icon	图标的名称

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

属性	说明
fg_image	前景图片的名称
fg_image_draw_type	前景图片的显示方式
spacer	间距
margin	边距
margin_left	左边距
margin_right	右边距
margin_top	顶边距
margin_bottom	底边距
icon_at	图标的位置（取值：top|bottom|left|right）
active_icon	active 图标的名称
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
selected_bg_color	编辑器（包括 edit 和 mledit 控件）中选中区域的背景颜色
selected_fg_color	编辑器（包括 edit 和 mledit 控件）中选中区域的前景颜色
selected_text_color	编辑器（包括 edit 和 mledit 控件）中选中区域的文本颜色
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

3.2.3 控件状态

在 3.2.1 章节中，为 label 指定了 normal（正常状态）。AWTK 为各种控件提供了不同状态的属性，详见下表，可以根据不同的状态设置不一样的样式。具体每个控件可以设置哪些状态可以参考第 4 章下各个子章节的”状态”章节，示例可以参考：awtk/demos/assets/default/raw/styles/default.xml。

状态	说明
normal	正常状态
pressed	指针按下状态
over	指针悬浮状态
disable	禁用状态
focused	聚焦状态
checked	勾选状态
unchecked	没勾选状态
empty	编辑器无内容状态
empty_focus	编辑器无内容同时聚焦的状态
error	输入错误状态
selected	选中状态
normal_of_checked	正常状态 (选中项)
pressed_of_checked	指针按下状态 (选中项)
over_of_checked	指针悬浮状态 (选中项)
focused_of_checked	焦点状态 (选中项)

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

状态	说明
normal_of_active	正常状态 (当前项)
pressed_of_active	指针按下状态 (当前项)
over_of_active	指针悬浮状态 (当前项)
focused_of_active	焦点状态 (当前项)

3.2.4 使用控件样式

在控件样式文件（通常是：default.xml）中定义好了控件的样式后，就可以在代码中使用了。例如，在 3.2.1 章节中定义了 label 控件的 left 样式，可以使用下面两种方式设置 label 控件的样式为 left：

（1）UI 文件方式，在 XML 文件中通过 style 属性设置 label 控件的样式为 left，代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/basic.xml -->
<window anim_hint="htranslate">
  ...
  <row x="0" y="80" w="100%" h="30" children_layout="default(r=1,c=3,xm=2,s=10)">
    <label style="left" name="left" text="Left"/>
    <label style="center" name="center" text="Center"/>
    <label style="right" name="right" text="Right"/>
  </row>
  ...
</window>
```

（2）C 代码方式，在 C 源文件中可以调用 widget_use_style() 函数设置 label 控件的样式为 left，代码如下：

```
/* awtk/demos/demo1_app.c */
ret_t application_init() {
  ...
  label = label_create(win, 10, 40, 80, 30);
  widget_set_text(label, L"Left");
  widget_use_style(label, "left");
  ...
}
```

3.2.5 每个窗口支持独立的控件样式

在 AWTK 中为每个窗口 UI 文件可以有自己的控件样式文件，可以通过下面方式指定窗口自己的控件样式文件：

- 通过窗口的 theme 属性来指定控件样式文件名（方便多个窗口共用一个控件样式文件），代码如下：

```
<!-- default/ui/home_page.xml -->
<window theme="home_page">
  <label name="label" x="187" y="151" w="160" h="28" text="Label"/>
</window>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

- 如果没有指定 theme 属性，以窗口的 name 属性作为窗口的控件样式文件名，代码如下：

```
<!-- default/ui/home_page.xml -->
<window name="home_page">
  <label name="label" x="187" y="151" w="160" h="28" text="Label"/>
</window>
```

注: 建议窗口的控件样式文件名与 UI 的文件名相同，例如：上面代码中窗口的控件样式文件名和 UI 的文件名都是 home_page。窗口的控件样式文件与缺省控件样式文件（default.xml）放在同一目录下。

AWTK 在查找控件设置的样式时，优先在窗口自己的控件样式文件中查找；如果没有找到，再去为缺省的控件样式文件（default.xml）中查找；如果该样式在缺省的控件样式文件也没有找到（可能会因为样式字符串的书写错误，而找不到），则用缺省控件样式文件中该控件的 default 样式。

例如，在 awtk/demos/assets/default/raw/ui/目录下”kb_*.xml”键盘文件中，通过设置”theme”属性，多个键盘文件共用同一个控件样式文件”keyboard.xml”，代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/kb_default.xml -->
<keyboard theme="keyboard" x="0" y="bottom" w="100%" h="40%">
  <pages x="0" y="bottom" w="100%" h="-28" active="4">
    <view name="upper" x="0" y="0" w="100%" h="100%" children_layout="default(r=4,
↪c=1,s=2,m=2)">
      <group_box children_layout="default(r=1,c=10,s=2,m=2)">
        <button repeat="300" name="Q" text="Q"/>
        <button repeat="300" name="W" text="W"/>
        ...
      </group_box>
        ...
</keyboard>
```

```
<!-- awtk/demos/assets/default/raw/styles/keyboard.xml -->
...
<button>
  <style name="highlight" border_color="#a0a0a0"  text_color="black">
    <normal     bg_color="#a8b0b0" />
    <pressed    bg_color="#c0c0c0" />
    <over       bg_color="#e0e0e0" />
    <disable    bg_color="gray" text_color="#d0d0d0" />
  </style>
</button>
...
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

3.2.6 inline style 动态修改控件样式

控件样式数据是只读的，它的好处是速度快，占用内存少。但在一些特殊情况下，希望通过函数直接修改控件的 style，或者在 UI 描述的 XML 文件中直接设置控件的 style，把这类 style 称为 inline style，具体用法如下：

- 在 XML UI 描述文件中使用 inline style。

控件的属性名以”style:”开头表示这是一个 inline 属性：

```
style:状态:名称
```

下面表示设置正常状态的字体大小为 16：

```
style:normal:font_size="16"
```

状态可以省略，如果省略，表示正常状态（normal），下面这个和上面的功能一样：

```
style:font_size="16"
```

完整示例：

```
<label x="0" y="0" w="100%" h="100%" text="hello" style:font_size="24" style:text_
↪color="green"/>
<button name="dec_value" text="Dec" focusable="true" style:focused:text_color="red
↪"/>
```

- 在 C 代码中使用 inline style。

在 AWTK 中，便于用户使用 inline style，提供了下表中的函数，具体请查看 AWTK_API 手册，这里不累赘了。

函数名称	说明
widget_set_style_int	设置整数类型的 style
widget_set_style_str	设置字符串类型的 style
widget_set_style_color	设置颜色类型的 style

示例：

```
/* 设置button控件的字体大小为24 */
widget_set_style_int(button, "font_size", 24);

/* 设置button控件正常状态下的背景色 */
widget_set_style_color(button, "normal:bg_color", 0xff0000ff);

/* 设置button控件被按下时的背景色 */
widget_set_style_str(button, "pressed:bg_color", "red");
```

注: inline style 会消耗更多内存，而且不方便切换控件样式，一般应该尽量避免使用。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

3.2.7 注意事项

建议直接将 awtk/demos/assets/default/raw/styles/default.xml 文件复制到自己的应用程序目录中，如：res/assets/default/raw/styles 目录下。

如果需要添加新的样式直接在该文件中添加，需要注意不要删除原有的样式，如果删除可能会造成某些控件无法正常显示。例如，不能删除 combobox_down 样式，否则 combo_box 控件不能显示下拉按钮，代码如下：

```
<!-- awtk/demos/assets/default/raw/styles/default.xml -->
...
<button>
  <style name="default" border_color="#a0a0a0"  text_color="black">
    <normal     bg_color="#f0f0f0" />
    <pressed    bg_color="#c0c0c0" x_offset="1" y_offset="1"/>
    <over       bg_color="#e0e0e0" />
    <focused    bg_color="#e0e0e0" />
    <disable    bg_color="gray" text_color="#d0d0d0" />
  </style>

  <style name="combobox_down" border_color="#a0a0a0">
    <normal     bg_color="#f0f0f0" icon="arrow_down_n"/>
    <pressed    bg_color="#c0c0c0" icon="arrow_down_p"/>
    <over       bg_color="#e0e0e0" icon="arrow_down_o"/>
  </style>
...
```

3.3 布局管理器

3.3.1 为什么需要布局参数

如果界面上元素是预先知道的，而且窗口的大小也是固定的，通过可视化的工具，以所见即所得的方式，去创建界面是最轻松的方式。但是在下列情况下，使用布局参数却是更好的选择：

（1）窗口的大小是可以动态调整的。

（2）需要适应不同大小的屏幕。

（3）界面上的元素是动态的，需要用程序创建界面。

3.3.2 概述

AWTK 的布局器（layouter）分为两类，一类用于对控件自身进行布局，另一类用于对子控件进行布局详见下图。

- self_layout 对控件自身进行布局
- children_layout 用于对子控件进行布局

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

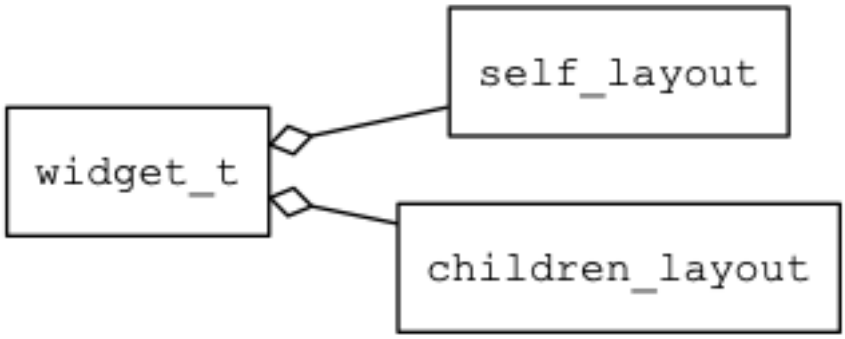


图 3.1　layouter 布局器

AWTK 提供了灵活的扩展机制，可以方便的扩展新的布局方式，所以 self_layouter 和 children_layouter 都是接口。

3.3.3　控件自身的布局

AWTK 目前仅仅实现了缺省布局方式，以后陆续实现 css、flex 等布局方式，详见下图。

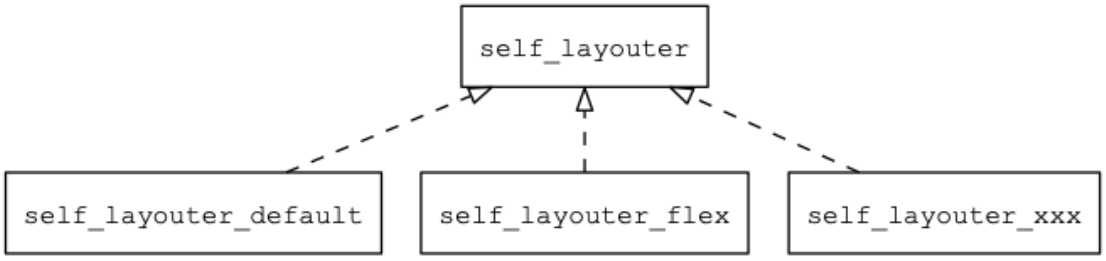


图 3.2　self_layouter 布局

控件自身布局有下面两种写法：

方式一：使用 self_layout 属性指定控件的 x、y、w、h，示例如下：

```
<button text="ok" self_layout="default(x=10,y=10,w=100,h=100)"/>
```

方式二：使用缺省布局参数指定控件的 x、y、w、h，示例如下：

```
<button text="ok" x="10" y="10" w="100" h="100" />
```

由于这两种写法的效果都是一样的，在下面的示例中以第二种写法为例。

1. 参数

缺省布局中有 5 个参数，详见下表：

参数	说明
x	x 坐标
y	y 坐标
w	控件宽度
h	控件高度
floating	是否为浮动布局。如果设置为 true，该控件不受父控件的 children_layouter 的影响

2. 使用方法

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

AWTK 提供了多种控件自身布局方式，开发者可以根据自己的需求选择其中一种方式，具体的方式请看下文。

(1) 像素方式

直接指定控件的 x/y/w/h 的像素值，这是缺省的方式，也是最不灵活的方式。示例如下：

- 在 XML 界面描述文件中：

```
<button x="10" y="5" w="80" h="30" text="ok"/>
```

- 在 C 代码中：

```
widget_move_resize(btn, 10, 5, 80, 30);
```

(2) 百分比

x/w 的值如果包含”%“，则自动换算成相对其父控件宽度的百分比。y/h 的值如果包含”%”，则自动换算成相对其父控件高度的百分比。示例如下：

- 在 XML 界面描述文件中：

```
<button x="10%" y="10" w="50%" h="30" text="ok"/>
```

- 在 C 代码中：

```
widget_set_self_layout(btn, "default(x=10%,y=10,w=50%,h=30)");
widget_layout(btn);
```

(3) 水平居中

让控件在水平方向上居中，只需要将 x 的值设置成”c”或者”center”即可。示例如下：

```
<button x="center" y="10" w="50%" h="30" text="ok"/>
```

(4) 垂直居中

让控件在垂直方向上居中，只需要将 y 的值设置成”m”或者”middle”即可。示例如下：

```
<button x="center" y="middle" w="50%" h="30" text="ok"/>
```

(5) 位于右边

让控件位于父控件的右侧，只需要将 x 的值设置成”r”或者”right”即可。示例如下：

```
<button x="right" y="10" w="50%" h="30" text="ok"/>
```

如果还想离右侧有一定距离，可以在 right 后指定距离的像素。示例如下：

```
<button x="right:20" y="10" w="50%" h="30" text="ok"/>
```

(6) 位于底部

让控件位于父控件的底部，只需要将 y 的值设置成”b”或者”bottom”即可。示例如下：

```
<button x="10" y="bottom" w="50%" h="30" text="ok"/>
```

如果还想离底部有一定距离，可以在 bottom 后指定距离的像素。示例如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
<button x="10" y="bottom:20" w="50%" h="30" text="ok"/>
```

(7) 宽度和高度为负数

无论是像素模式还是百分比模式，宽度和高度均可为负数。

- 宽度为负数。其值为父控件的宽度 + 该负值
- 高度为负数。其值为父控件的高度 + 该负值

(8) 浮动布局

如果 floating 设置为 true，该控件不受父控件的 children_layouter 的影响。示例如下：

```
<button x="10" y="20" w="50" h="30" floating="true" "text="ok"/>
```

3.3.4 子控件的布局

AWTK 目前仅仅实现了缺省布局方式，详见下图，以后将会陆续实现 css flex 等布局方式。

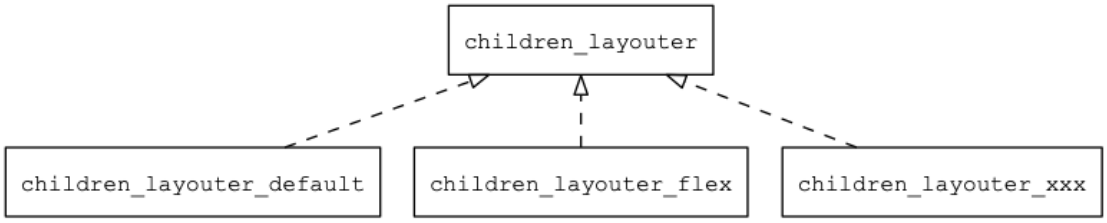


图 3.3 子控件的布局

1. 语法

子控件布局器统一使用 children_layout 属性指定，其语法为：

```
缺省子控件布局器 => default '(' PARAM_LIST ')'
PARAM_LIST => PARAM | PARAM ',' PARAM_LIST
```

示例：

```
<view x="0" y="0" w="100%" h="100%" children_layout="default(c=2,r=8,m=5,s=5)">
```

2. 参数

为了方便父控件布局子控件，AWTK 提供了下面几个参数：

参数	简写	说明
rows	r	行数
cols	c	列数
width	w	子控件的宽度，可以用来计算列数，与 cols 互斥
height	h	子控件的高度，可以用来计算行数，与 rows 互斥
x_margin	xm	水平方向的边距
y_margin	ym	垂直方向的边距
spacing	s	子控件之间的间距
keep_invisible	ki	是否给不可见的控件留位置（缺省否）
keep_disable	kd	是否给不用的控件留位置（缺省是）

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

在 C 代码中，可以通过下面的函数设置这几个参数：

```
/**
 * @method widget_set_children_layout
 * 设置子控件的布局参数。
 * @annotation ["scriptable"]
 * @param {widget_t*} widget 控件对象。
 * @param {const char*} params 布局参数。
 * @return {ret_t} 返回RET_OK表示成功，否则表示失败。
 */
ret_t widget_set_children_layout(widget_t* widget, const char* params);
```

示例：

```
widget_set_children_layout(w, "default(r=2,c=2)");
```

XML 中，可以通过 children_layout 属性设置:

```
 <column x="20" y="160" w="50%" h="60" children_layout="default(r=2,c=1,ym=2,s=10)"
↪>
    <check_button name="c1" text="Book"/>
    <check_button name="c2" text="Food"/>
  </column>
```

下面介绍通过调整 rows/cols 两个参数，来实现不同的布局方式。

3. 使用方法

(1) 缺省

在没有设置子控件布局参数时，采用缺省的布局方式，父控件啥事也不做，完全由子控件自己的布局参数决定。

(2)hbox 水平布局

当 rows=1,cols=0 时，所有子控件在水平方向排成一行，可以实现水平布局功能，子控件的参数详见下表：

属性	说明
x	从左到右排列，由布局参数计算而出
y	为 y_margin
w	由子控件自己决定
h	为父控件的高度-2*y_margin

示例：

```
<window>
  <view x="c" y="m" w="300" h="30" children_layout="default(r=1,c=0,s=5)">
      <button text="1" w="20%"/>
      <button text="2" w="30%"/>
      <button text="3" w="30%"/>
      <button text="4" w="20%"/>
  </view>
</window>
```

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

例如，将文件保存为 t.xml，可用 preview_ui（在 awtk/bin 目录下）预览效果详见下图，命令如下：

```
bin/preview_ui.exe t.xml 480 320
```



图 3.4　水平布局

(3)vbox 垂直布局

当 cols=1,rows=0 时，所有子控件在垂直方向排成一列，可以实现垂直布局功能，子控件的参数详见下表：

属性	说明
x	x_margin
y	从上到下排列，由布局参数计算而出
w	为父控件的宽度-2*x_margin
h	由子控件自己决定

示例：

```
<window>
  <view x="c" y="m" w="80" h="200" children_layout="default(r=0,c=1,s=5)">
      <button text="1" h="20%"/>
      <button text="2" h="30%"/>
      <button text="3" h="30%"/>
      <button text="4" h="20%"/>
  </view>
</window>
```

例如，将文件保存为 t.xml，可用 preview_ui（在 awtk/bin 目录下）预览效果详见下图，命令如下：

```
bin/preview_ui.exe t.xml 480 320
```

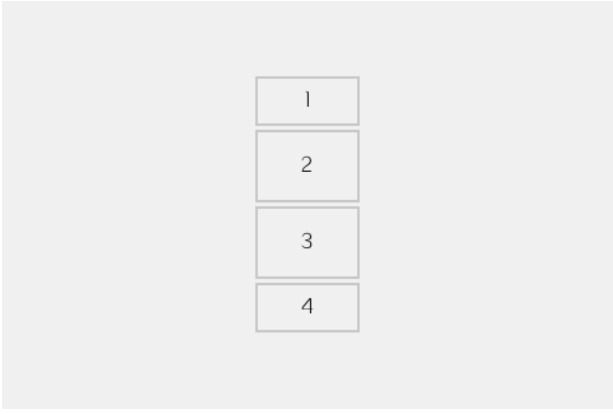


图 3.5　垂直布局

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

(4)listbox 列表布局

当 cols=1,rows=N 时，所有子控件在垂直方向排成一列，可以实现列表布局功能，子控件的参数详见下表：

属性	说明
x	x_margin
y	从上到下排列，由布局参数计算而出
w	为父控件的宽度-2*x_margin
h	为父控件的高度（减去边距和间距）分成成 N 等分

示例：

```
<window>
  <view x="c" y="m" w="200" h="200" children_layout="default(r=4,c=1,s=5)">
      <button text="1" />
      <button text="2" />
      <button text="3" />
      <button text="4" />
  </view>
</window>
```

例如，将文件保存为 t.xml，可用 preview_ui（在 awtk/bin 目录下）预览效果详见下图，命令如下：

```
bin/preview_ui.exe t.xml 480 320
```



图 3.6　列表布局

(5)grid 网格布局

当 cols=N,rows=N 时，所有子控件放在 M x N 的网格中，可以实现网格布局功能，子控件无需指定 x/y/w/h 等参数。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

示例：

```
<window>
  <view x="c" y="m" w="200" h="200" children_layout="default(r=2,c=2,s=5)">
      <button text="1" />
      <button text="2" />
      <button text="3" />
      <button text="4" />
  </view>
</window>
```

例如，将文件保存为 t.xml，可用 preview_ui（在 awtk/bin 目录下）预览效果详见下图，命令如下：

```
bin/preview_ui.exe t.xml 480 320
```

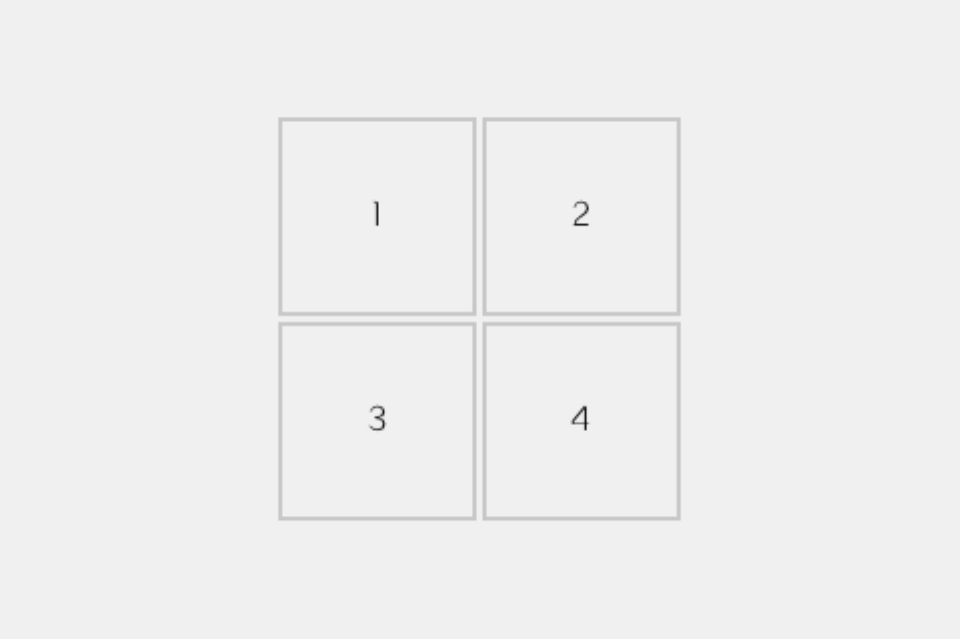


图 3.7　网格布局

(6) floating 浮动布局

如果子控件的 floating 属性设置为 true，其不受 children_layout 的限制。

示例：

```
<window>
  <view x="c" y="m" w="200" h="200" children_layout="default(r=2,c=2,s=5)">
      <label text="1" />
      <label text="2" />
      <label text="3" />
      <label text="4" />
      <button text="floating" floating="true" x="c" y="m" w="80" h="30"/>
  </view>
</window>
```

例如，将文件保存为 t.xml，可用 preview_ui（在 awtk/bin 目录下）预览效果如详见下图，

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

命令如下：

```
bin/preview_ui.exe t.xml 480 320
```



图 3.8　浮动布局

4. 高级用法

(1) 子控件布局器和子控件自身的布局参数结合。

为了更大的灵活性，缺省子控件布局器可以和子控件自身的参数结合起来。

示例：

```
<window>
  <view x="c" y="m" w="200" h="200" children_layout="default(r=2,c=2,s=5)">
      <button text="1" x="0" y="0" w="50%" h="50%"/>
      <button text="2" x="r" y="m" w="60%" h="60%"/>
      <button text="3" x="c" y="m" w="70%" h="70%"/>
      <button text="4" x="c" y="m" w="80%" h="80%"/>
  </view>
</window>
```

例如，将文件保存为 t.xml，可用 preview_ui（在 awtk/bin 目录下）预览效果如详见下图，命令如下：

```
bin/preview_ui.exe t.xml 480 320
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

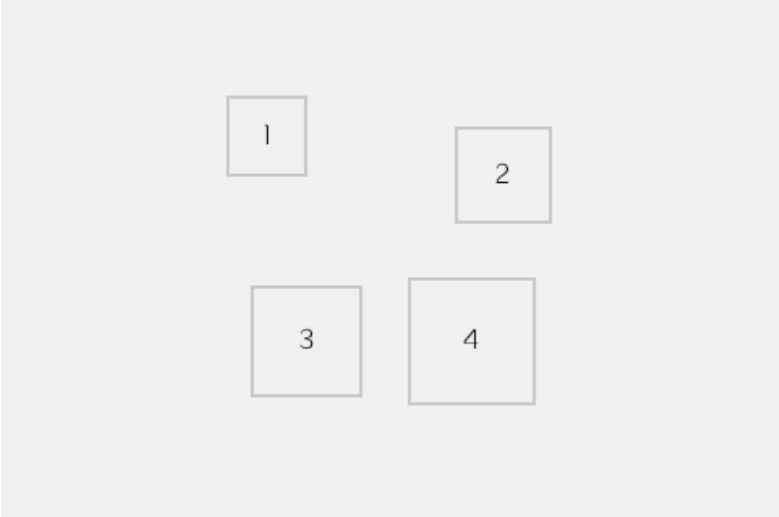


图 3.9　高级用法一

(2) 子控件自身的布局参数 x/y/w/h 均为像素方式时，需要用 self_layout 参数指定。

示例：

```
<window>
  <view x="c" y="m" w="200" h="200" children_layout="default(r=2,c=2,s=5)">
      <button text="1" self_layout="default(x=0,y=0,w=50,h=50)" />
      <button text="2" x="r" y="m" w="60%" h="60%"/>
      <button text="3" x="c" y="m" w="70%" h="70%"/>
      <button text="4" x="c" y="m" w="80%" h="80%"/>
  </view>
</window>
```

例如，将文件保存为 t.xml，可用 preview_ui（在 awtk/bin 目录下）预览效果如详见下图，命令如下：

```
bin/preview_ui.exe t.xml 480 320
```

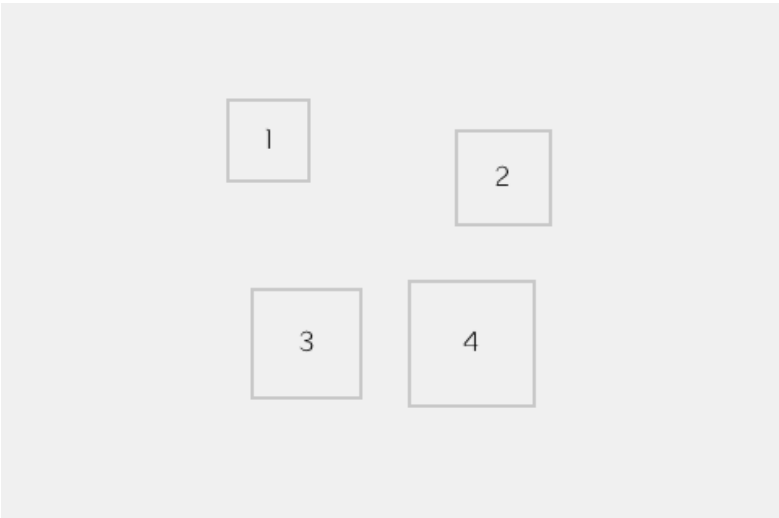


图 3.10　高级用法二

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

3.4 资源管理器

这里的资源管理器并非 Windows 下的文件浏览器，而是负责对各种资源，比如字体、控件样式、图片、界面数据、字符串和其它数据的进行集中管理的组件。引入资源管理器的目的有以下几个：

- 让上层不需要了解存储的方式。在没有文件系统时或者内存紧缺时，把资源转成常量数组直接编译到代码中。在有文件系统而且内存充足时，资源放在文件系统中。在有网络时，资源也可以存放在服务器上（暂未实现）。资源管理器为上层提供统一的接口，让上层而不用关心底层的存储方式。
- 让上层不需要了解资源的具体格式。比如一个名为 earth.png 的图片，没有文件系统或内存紧缺，图片直接用位图数据格式存在 ROM 中；而有文件系统时，则用 PNG 格式存放在文件系统中。资源管理器让上层不需要关心图片的格式，访问时指定图片的名称即可（不用指定扩展名）。
- 让上层不需要了解屏幕的密度。不同的屏幕密度下需要加载不同的图片，比如 MacPro 的 Retina 屏就需要用双倍解析度的图片，否则就出现界面模糊。AWTK 以后会支持 PC 软件和手机软件的开发，所以资源管理器需要为此提供支持，让上层不需关心屏幕的密度。
- 对资源进行内存缓存。不同类型的资源使用方式是不一样的，比如字体和控件样式加载之后会一直使用，UI 文件在生成界面之后就暂时不需要了，PNG 文件解码之后就只需要保留解码的位图数据即可。资源管理器配合图片管理器等其它组件实现资源的自动缓存。

资源管理器和资源管理相关的组件详见下图，网络加载暂未实现。

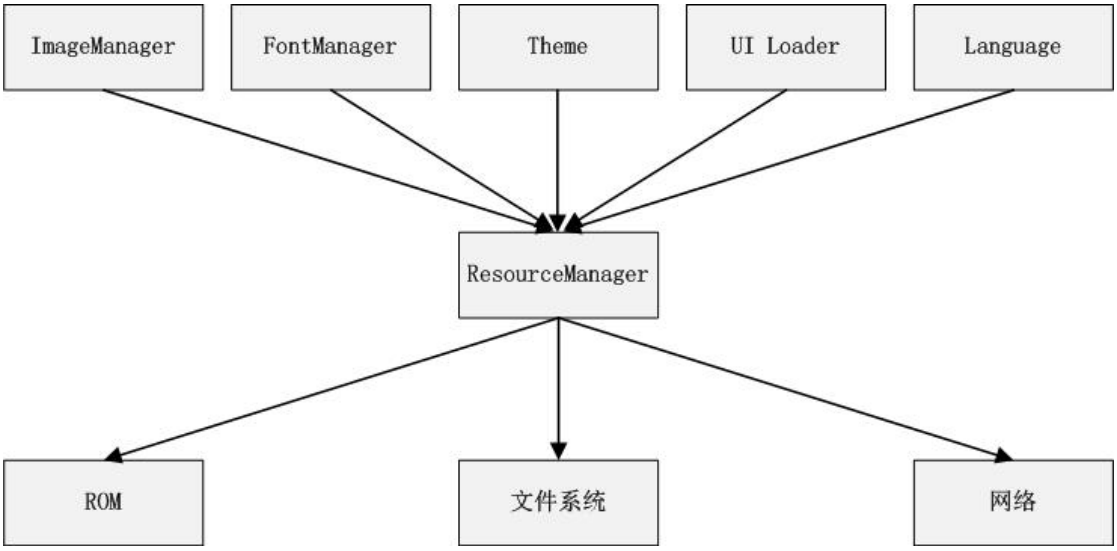


图 3.11 资源管理器

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

3.4.1 资源的生成

AWTK 中的资源需要进行格式转换才能使用：

- 在没有文件系统时或者内存紧缺时，需要把资源转成常量数组直接编译到代码中。
- XML 格式的 UI 文件需要转换成二进制的格式。
- XML 格式的控件样式文件需要转换成二进制的格式。
- TTF 字体可以根据需要转换成位图字体。
- PNG 图片可以根据需要转换成位图图片。

3.4.2 相关工具

在 awtk/bin 目录下提供了多种工具用于生成资源，详见下表：

工具	说明
fontgen	位图字体生成工具
imagegen	位图图片生成工具
resgen	二进制文件生成资源常量数组
themegen	XML 控件样式转换成二进制的控件样式
xml_to_ui	XML 的界面描述格式转换二进制的界面描述格式

上面工具的使用方法请看：awtk/tools 目录对应文件夹下的 README.md 文档。

awtk/scripts/update_res.py 脚本通过调用以上工具批量转换整合项目的资源。

3.4.3 初始化

将资源生成常量数组直接编译到代码中时，其初始化过程为：

（1）包含相应的资源数据文件，代码如下：

```
/* awtk/demos/assets_default.inc */
...
#include "assets/default/inc/strings/en_US.data"
#include "assets/default/inc/strings/zh_CN.data"
#include "assets/default/inc/styles/default.data"
#include "assets/default/inc/images/check.res"
#include "assets/default/inc/images/checked.res"
#include "assets/default/inc/fonts/default.res"
#include "assets/default/inc/ui/main.data"
...
```

（2）将资源添加到资源管理器分两种情况：

- 没有文件系统时（即没有定义宏 WITH_FS_RES），调用 assets_manager_add() 函数将资源添加到资源管理器中，代码如下：

```
/* awtk/demos/assets_default.inc */
...
assets_manager_add(rm, ui_kb_ascii);
assets_manager_add(rm, ui_vgcanvas);
assets_manager_add(rm, ui_rich_text1);
```

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
assets_manager_add(rm, ui_slide_menu);
assets_manager_add(rm, ui_radial_gradient);
assets_manager_add(rm, ui_color_picker_simple);
...
```

- 有文件系统时（即有定义宏 WITH_FS_RES 时，不需要（1）包含相应的数据文件这一步），一般不需要特殊处理，不过可以用 assets_manager_preload() 函数预加载资源，代码如下：

```
/* awtk/demos/assets_default.inc */
...
assets_manager_preload(am, ASSET_TYPE_FONT, "default");
assets_manager_preload(am, ASSET_TYPE_STYLE, "default");
...
```

3.4.4 使用方法

- 加载图片

调用 image_manager_get_bitmap() 函数获取位图数据，并指定图片的名称即可。例如，有一张 earth.png 的图片，并且需要加载这张图片，代码如下：

```
bitmap_t img;
image_manager_get_bitmap(image_manager(), "earth", &img);
```

- 使用 UI 数据

调用 window_open() 函数打开 UI 文件，并指定 UI 文件的名称即可。例如，有个 name.xml 文件，并且需要打开这个 UI 文件，代码如下：

```
widget_t* win = window_open(name);
```

- 使用字体

一般在控件样式文件中指定 font_name 属性即可。如果字体文件不是 default.ttf，才需要指定 font_name 属性。例如，有个 ap.ttf 的字体文件，并且 label 文件控件中的 center_ap 样式需要使用 ap.ttf 字体，代码如下：

```
<!-- awtk/demos/assets/default/raw/styles/default.xml -->
<label>
    <style name="center_ap">
        <normal text_color="green"  font_name="ap" font_size="12"/>
    </style>
</label>
```

- 使用控件样式

一般在 UI 界面描述文件中指定 style 属性即可。例如，为 label 文本框控件指定 left 样式，代码如下：

```
<!--awtk/demos/assets/default/raw/ui/basic.xml -->
<row x="0" y="80" w="100%" h="30" children_layout="default(r=1,c=3,xm=2,s=10)">
        <label style="left" name="left" text="Left"/>
```

ZLG ©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
</row>
```

3.4.5 资源的名称

1. 基本用法

资源名称一般就是资源的文件名，不带文件扩展名。比如：图片名为 test.png，那资源名称就是 test。如果因为某种原因，把 test.png 换成了 test.jpg，对代码并无影响。

注: 对于 assets/default/raw/data 目录下的资源，由于其扩展名不固定，所以需要带扩展名才能访问。比如：资源文件名为”app.json”的文件，不能用”app”访问，而是用”app.json”才能访问。

2. 高级用法

（1）简介

对于图片和 UI 资源名称，AWTK 还支持一种高级用法。例如，有下面几种场景：

- 需要支持不同的分辨率。而在不同分辨率里，要使用不同的背景图片。
- 需要支持竖屏和横屏。而有的界面自动排版不能满足需求，需要为竖屏和横屏各写一个 XML 文件。
- 需要支持不同的语言。而在不同的语言里，有的图片是语言相关的。

为了解决上面这些问题，AWTK 提供了名称表达式：

- 名称中可以带变量和表达式。变量用 ${xxx} 表示，xxx 将被替换成实际的值。
- 可以指定多个名称，依次匹配，直到找到的为止。多个名称之间用逗号分隔。

（2）示例

示例 1：支持竖屏和横屏下使用不同图片。

例如：在 images/xx 目录下有 bg_portrait_1.jpg 和 bg_landscape_1.jpg 两张图片，在竖屏时，加载 bg_portrait_1.jpg；在横屏时，加载 bg_landscape_1.jgp，代码如下：

```
<!-- awtk/demos/assets/default/raw/styles/default.xml -->
<style name="sky">
    <normal bg_image="bg_${device_orientation}_1"/>
</style>
```

在竖屏下，相当于：

```
<style name="sky">
    <normal bg_image="bg_portrait_1"/>
</style>
```

在横屏下，相当于：

```
<style name="sky">
    <normal bg_image="bg_landscape_1"/>
</style>
```

示例 2: 支持不同的语言时使用不同图片。

例如：在 images/xx 目录下有 flag_CN.png 和 flag_US.png 两张图片，在当前语言为中文时，加载 flag_CN.png；为英文时，加载 flag_US.png，代码如下：

ZLG ©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
<image image="flag_${country},flag_none" x="c" y="m:-80" w="80" h="80"/>
```

在 locale 为 zh_CN 时，依次查找 flag_CN 和 flag_none 两张图片，相当于：

```
<image image="flag_CN,flag_none" x="c" y="m:-80" w="80" h="80"/>
```

在 locale 为 en_US 时，依次查找 flag_US 和 flag_none 两张图片，相当于：

```
<image image="flag_US,flag_none" x="c" y="m:-80" w="80" h="80"/>
```

3.5 事件处理

事件一般用来响应各种行为。例如当键盘按下或者抬起时，会产生键盘按下或抬起事件。当某个窗口打开或者关闭的时候，产生一个打开或者关闭事件，用户可以在打开或者关闭窗口的时候，注册一个回调函数完成某些功能。大多数事件是作为用户行为的响应而产生的，但也有例外，如 timer 定时器事件、idle 定时器事件。

3.5.1 事件处理机制

在 AWTK 内部，AWTK 通过函数 tk_run 启动的主事件循环不停的捕捉用户触发的事件和事件队列中的事件，然后将它们转换成对应的事件，如：EVT_KEY_DOWN 键盘按下事件、EVT_POINTER_DOWN 指针按下事件等，并最终将这些事件发送到目的控件对象来完成上述一系列事件的处理，大致流程请看下图。

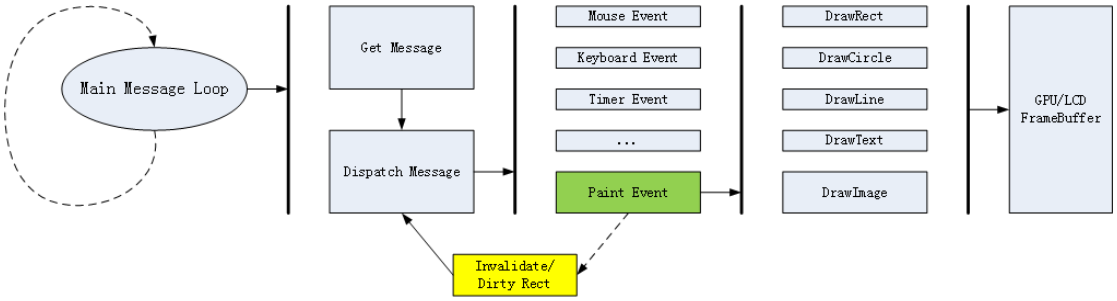


图 3.12 事件处理机制

例如，当点击一个 button 按钮会触发 EVT_POINTER_DOWN 指针按下事件，主事件循环捕获到该消息，最终会将该消息发送给 button 控件的 button_on_event 函数处理。

3.5.2 事件分发

事件分发的形式有多种，最常见的是通过 widget_dispatch() 函数分发一个事件，例如，AWTK 中比较常见的按钮点击事件 EVT_CLICK（详见 3.5.4 章节）就是通过 widget_dispatch 分发的。还有一些其他事件，则可以通过 main_loop_queue_event() 函数分发。

1. widget_dispatch

widget_dispatch() 函数的参数列表如下：

参数	类型	说明
返回值	ret_t	返回 RET_OK 表示成功，否则表示失败
widget	widget_t*	控件对象
e	event_t*	事件

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

widget_dispatch 用于分发一个事件，只能用于 GUI 线程，如果需要用到多线程的话，请用 idle_queue 或 timer_queue 函数（可参考 3.8 章节）。

例如，在 HelloWorld.Xml-Demo 的基础上修改代码，当用户点击”inc”按钮的时候，调用 widget_dispatch 分发一个自定义事件 EVT_USER_EVT，然后调用 widget_on（可参考 3.5.3 章节）捕获该事件，实现下图红色方框中的两个 label 文本控件的内容同步更新，代码如下：

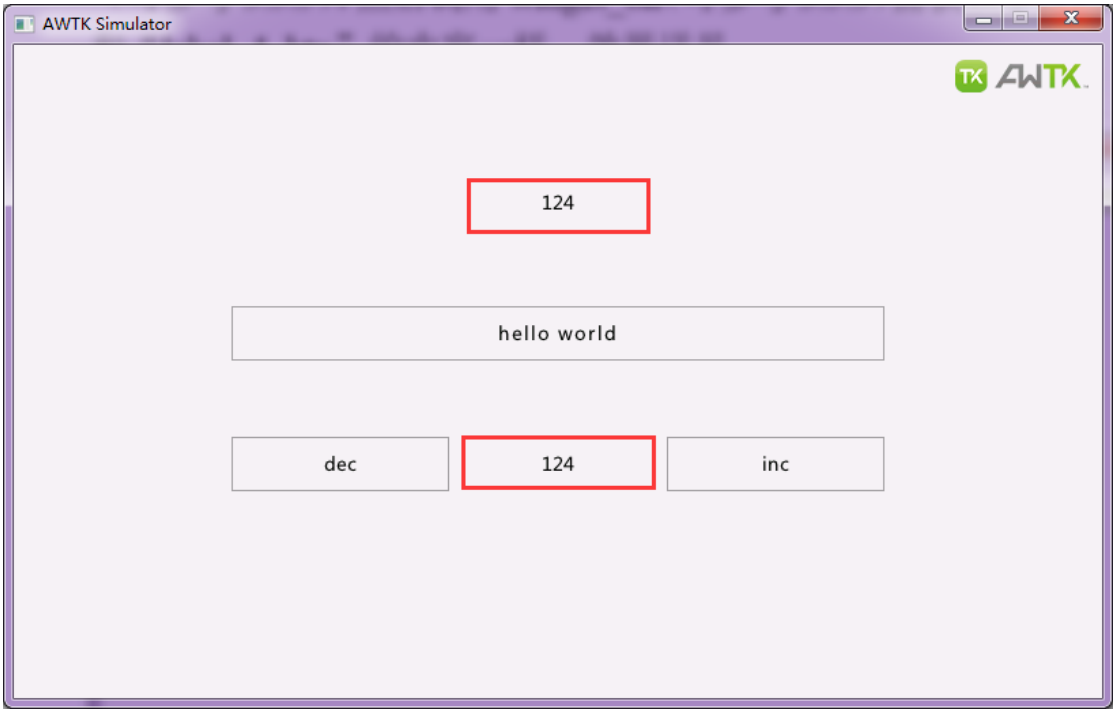


图 3.13　HelloWorld.Xml-Demo

```
/* HelloWorld.Xml-Demo/src/window_main.c */
#define EVT_USER_EVT (EVT_USER_START + 1)

static ret_t on_inc_click(void* ctx, event_t* e) {
  widget_t* win = WIDGET(ctx);
  widget_t* label = widget_lookup(win, "label_4_btn", TRUE);
  label_add(label, 1);

  event_t user = event_init(EVT_USER_EVT, NULL);
  widget_dispatch(window_manager(), &user);

  return RET_OK;
}

static ret_t on_user_evt(void* ctx, event_t* e) {
  (void)e;

  widget_t* win = WIDGET(ctx);
  widget_t* label_4_edit = widget_lookup(win, "label_4_edit", TRUE);
  widget_t* label_4_btn = widget_lookup(win, "label_4_btn", TRUE);
  widget_set_text(label_4_edit, widget_get_text(label_4_btn));
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  return RET_OK;
}

static ret_t on_changing(void* ctx, event_t* evt) {
  widget_t* target = WIDGET(evt->target);
  widget_t* win = WIDGET(ctx);
  widget_t* label = widget_lookup(win, "label_4_edit", TRUE);
  widget_set_text(label, target->text.str);

  return RET_OK;
}

static ret_t init_widget(void* ctx, const void* iter) {
  (void)ctx;
  widget_t* widget = WIDGET(iter);
  widget_t* win = widget_get_window(widget);

  if (widget->name != NULL) {
    const char* name = widget->name;
    if (tk_str_eq(name, "edit")) {
      widget_on(widget, EVT_VALUE_CHANGING, on_changing, win);
    } else if (tk_str_eq(name, "dec_btn")) {
      widget_on(widget, EVT_CLICK, on_dec_click, win);
    } else if (tk_str_eq(name, "inc_btn")) {
      widget_on(widget, EVT_CLICK, on_inc_click, win);
    }
  }

  return RET_OK;
}

ret_t application_init() {
  widget_t* win = window_open("main");
  if (win) {
    init_children_widget(win);
    widget_on(window_manager(), EVT_USER_EVT, on_user_evt, win);
  }
  return RET_OK;
}
```

2. main_loop_queue_event

main_loop_queue_event() 函数的参数列表如下：

参数	类型	说明
返回值	ret_t	返回 RET_OK 表示成功，否则表示失败
l	main_loop_t*	主循环事件对象，改值一般填写 main_loop()
e	const event_queue_req_t*	事件请求对象

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

事件请求 event_queue_req_t 成员详见以下代码：

```
typedef union _event_queue_req_t {
  event_t event;
  key_event_t key_event;
  pointer_event_t pointer_event;
  add_idle_t add_idle;
  add_timer_t add_timer;
} event_queue_req_t;
```

main_loop_queue_event 用于非 GUI 线程（可用于多线程）增加一个事件，本函数向主循环事件队列中发送一个增加事件请求。event_queue_req_t 可以请求多种事件，如 key_event_t 键盘事件、pointer_event_t 指针事件、idle 定时器事件以及 timer 定时器事件，对于这几种事件 AWTK 已经封装好了对应的 API，用户可以直接调用，具体请看下表，相关示例可以参考 awtk/demos/demo_thread_app.c，涉及传参部分的内容请查看 AWTK_API 手册。

事件请求类型	对应的 API
key_event_t	main_loop_post_key_event
pointer_event_t	main_loop_post_pointer_event
add_idle_t	idle_queue
add_timer_t	timer_queue

虽然 main_loop_queue_event 可用于多线程分发自定义事件，但在使用的时候，如果在非 GUI 线程中访问 GUI 控件对象，可能存在安全隐患，请用 idle_queue 或 timer_queue 函数代替（可参考 3.8）。

注: main_loop_post_key_event() 接口只能在关闭宏 WITH_SDL 的情况下使用，即该接口仅支持嵌入式平台。

3. 手动触发键盘或者鼠标事件

AWTK 已经封装好了键盘或者鼠标事件的函数，有关函数的定义具体请看：awtk/src/main_loop/main_loop_simple.h

- 触发键盘事件

```
main_loop_post_key_event(main_loop(), TRUE, TK_KEY_F1);
```

其中，在 main_loop_post_key_event() 函数中第一个参数 main_loop() 是固定的，第二个参数表示键盘是否按下，第三个参数表示按下的键盘值（具体可参考本文 7.4 章节）。

- 触发鼠标事件

```
main_loop_post_pointer_event(main_loop(), TRUE, 10, 20);
```

其中，在 main_loop_post_pointer_event() 函数中第一个参数 main_loop() 是固定的，第二个参数表示鼠标是否按下，第三、四个参数分别表示 x、y 坐标。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

3.5.3 事件处理

通常事件分发后将会调用相应的回调函数，例如，在本文 3.5.2 章节的第一小节中，点击”inc”按钮后，AWTK 会分发 EVT_CLICK（可参考本文 3.5.4 章节）事件，然后通过 widget_on 注册控件事件，最后调用注册的 on_inc_click 回调函数，实现每按下该按钮加 1 的功能。

1. 注册控件事件

(1) 函数原型

```
uint32_t widget_on(widget_t* widget, event_type_t type, event_func_t on_event,␣
↪void* ctx);
```

(2) 参数说明

widget_on() 函数的参数说明详见下表：

参数	类型	说明
返回值	uint32_t	用于 widget_off
widget	widget_t*	控件对象
type	event_type_t	事件类型
on_event	event_func_t	事件处理函数
ctx	void*	事件处理函数上下文

2. 注销控件事件

由于窗口关闭时会销毁其中所有控件，所以一般不需要手工去注销。如果确实需要，可以使用 widget_off。例如，使用 widget_on 给某个控件注册了事件后，在符合某些场景下不需要该控件继续响应该事件，就可以使用 widget_off 注销事件。

(1) 函数原型

```
ret_t widget_off(widget_t* widget, uint32_t id);
```

(2) 参数说明

widget_off() 函数的参数说明详见下表：

参数	类型	说明
返回值	ret_t	返回 RET_OK 表示成功，否则表示失败
widget	widget_t*	控件对象
id	uint32_t	widget_on 返回的 IDs

3. 注册事件回调函数返回值

通过 widget_on() 注册控件事件，在其回调函数中需要设置返回值，可以设置的返回值通常有：RET_OK、RET_REMOVE 和 RET_STOP（也可以是 RET_FAIL 等其他值，其效果和 RET_OK 一样，但不建议这么做），其具体说明如下：

（1）RET_OK

返回 RET_OK，表示可重复执行。

例如，在下图的程序界面中，XML 代码如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.



图 3.14　事件返回值

```
<window name="record" anim_hint="htranslate(easing=cubic_out)" style:normal:bg_
↪color="#000000">
  ...
  <list_view name="list_view" x="4" y="76" w="-8" h="400" default_item_height="60"␣
↪>
    <scroll_view name="scroll_view" x="0" y="0" w="100%" h="100%" >
      <list_item name="list_item_test" h="40" style="odd_clickable" >
        <button  w="100%" h="100%" name="button_test" tr_text="button_test"/>
      </list_item>
          ...
  </list_view>
</window>
```

上图对应的 C 逻辑代码如下，给按钮”button_test”注册 EVT_CLICK 事件的回调函数 on_button_test，其返回值为 RET_OK，表示每次点击”button_test”按钮的时候，都会执行该函数。

```
static ret_t on_list_item_test(void* ctx, event_t* e) {
  (void)e;
  (void)ctx;
  return RET_OK;
}

static ret_t on_button_test(void* ctx, event_t* e) {
  (void)e;
  (void)ctx;
  return RET_OK;
}
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
static ret_t on_button_test_pointer_up(void* ctx, event_t* e) {
  (void)e;
  (void)ctx;
  return RET_STOP;
}

static ret_t init_widget(void* ctx, const void* iter) {
  ...
  if (widget->name != NULL) {
    const char* name = widget->name;
    if (tk_str_eq(name, "list_item_test")) {
      widget_on(widget, EVT_CLICK, on_list_item_test, win);
    } else if (tk_str_eq(name, "button_test")) {
      widget_on(widget, EVT_CLICK, on_button_test, win);
      widget_on(widget, EVT_POINTER_UP, on_button_test_pointer_up, win);
    }
  }

  return RET_OK;
}
```

（2）RET_REMOVE

返回 RET_REMOVE，表示只执行一次。

例如，上述 C 代码中修改回调函数 on_button_test 的返回值为 RET_REMOVE，on_button_test 函数只会执行一次，以后就不会再执行了。

（3）RET_STOP

返回 RET_STOP，表示停止后续操作。

例如，在上图中 list_view 中 list_item_test 和 button_test 控件都注册了 EVT_CLICK 事件。如果只想执行 button_test 控件注册的 EVT_CLICK 事件回调函数，而不执行 list_item_test 控件的。这个时候可以给 button_test 控件注册 EVT_POINTER_UP 事件，并设置其回调函数的返回值为 RET_STOP，那么将不会执行 list_item_test 注册的 EVT_CLICK 事件的回调函数 on_list_item_test。

3.5.4 事件类型

AWTK 提供了丰富的事件类型，如鼠标按下、移动、抬起事件；键盘按下、抬起事件等，详见下表，一般在 widget_on 函数中使用。

事件名称	说明
EVT_POINTER_DOWN	指针按下事件名 (pointer_event_t)
EVT_POINTER_DOWN_BEFORE_CHILDREN	指针按下事件名，在子控件处理之前触发 (pointer_event_t)
EVT_POINTER_MOVE	指针移动事件名 (pointer_event_t)
EVT_POINTER_MOVE_BEFORE_CHILDREN	指针移动事件名，在子控件处理之前触发 (pointer_event_t)

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

事件名称	说明
EVT_POINTER_UP	指针抬起事件名 (pointer_event_t)
EVT_POINTER_UP_BEFORE_CHILDREN	指针抬起事件名，在子控件处理之前触发 (pointer_event_t)
EVT_WHEEL	滚轮事件名 (pointer_event_t)
EVT_POINTER_DOWN_ABORT	取消前一个指针按下事件名 (pointer_event_t)
EVT_CONTEXT_MENU	右键/长按弹出上下文菜单的事件名 (pointer_event_t)
EVT_POINTER_ENTER	指针进入事件名 (pointer_event_t)
EVT_POINTER_LEAVE	指针离开事件名 (pointer_event_t)
EVT_LONG_PRESS	长按事件名 (pointer_event_t)
EVT_CLICK	点击事件名 (pointer_event_t)
EVT_FOCUS	得到焦点事件名 (event_t)
EVT_BLUR	失去焦点事件名 (event_t)
EVT_KEY_DOWN	键按下事件名 (key_event_t)
EVT_KEY_DOWN_BEFORE_CHILDREN	键按下事件名，在子控件处理之前触发 (key_event_t)
EVT_KEY_REPEAT	按键 repeat 事件名 (key_event_t)
EVT_KEY_UP	键抬起事件名 (key_event_t)
EVT_KEY_UP_BEFORE_CHILDREN	键抬起事件名，在子控件处理之前触发 (key_event_t)
EVT_WILL_MOVE	即将移动 Widget 的事件名 (event_t)
EVT_MOVE	移动 Widget 的事件名 (event_t)
EVT_WILL_RESIZE	即将调整 Widget 大小的事件名 (event_t)
EVT_RESIZE	调整 Widget 大小的事件名 (event_t)
EVT_WILL_MOVE_RESIZE	即将调整 Widget 大小/位置的事件名 (event_t)
EVT_MOVE_RESIZE	调整 Widget 大小/位置的事件名 (event_t)
EVT_VALUE_WILL_CHANGE	控件的值即将改变的事件名 (event_t)
EVT_VALUE_CHANGED	控件的值改变的事件名 (event_t)
EVT_VALUE_CHANGING	控件的值持续改变（如编辑器正在编辑）的事件名 (event_t)
EVT_PAINT	绘制的事件名 (paint_event_t)
EVT_BEFORE_PAINT	即将绘制的事件名 (paint_event_t)
EVT_AFTER_PAINT	绘制完成的事件名 (paint_event_t)
EVT_PAINT_DONE	绘制完成（canvas 状态已经恢复）的事件名 (paint_event_t)
EVT_LOCALE_CHANGED	locale 改变的事件 (event_t)
EVT_ANIM_START	控件动画开始事件 (event_t)
EVT_ANIM_STOP	控件动画被主动停止的事件 (event_t)
EVT_ANIM_PAUSE	控件动画被暂停的事件 (event_t)
EVT_ANIM_ONCE	控件动画 yoyo/repeat 时，完成一次的事件 (event_t)
EVT_ANIM_END	控件动画完成事件 (event_t)
EVT_WINDOW_LOAD	窗口加载完成事件 (event_t)
EVT_WINDOW_WILL_OPEN	窗口即将打开事件 (event_t) 如果有窗口动画，在窗口动画开始前触发如果没有窗口动画，在窗口被加载后的下一次循环中触发

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

事件名称	说明
EVT_WINDOW_OPEN	窗口打开事件 (event_t) 如果有窗口动画，在窗口动画完成时触发如果没有窗口动画，在窗口被加载后的下一次循环中触发
EVT_WINDOW_TO_BACKGROUND	窗口被切换到后台事件 (event_t) 打开新窗口时，当前窗口被切换到后台时，对当前窗口触发本事件
EVT_WINDOW_TO_FOREGROUND	窗口被切换到前台事件 (event_t) 关闭当前窗口时，前一个窗口被切换到前台时，对前一个窗口触发本事件
EVT_WINDOW_CLOSE	窗口关闭事件
EVT_REQUEST_CLOSE_WINDOW	请求关闭窗口的事件 (event_t)
EVT_TOP_WINDOW_CHANGED	顶层窗口改变的事件 (window_event_t)
EVT_IM_COMMIT	输入法提交输入的文本事件 (im_commit_event_t)
EVT_IM_SHOW_CANDIDATES	输入法请求显示候选字事件 (im_candidates_event_t)
EVT_IM_ACTION	软键盘 Action 点击事件 (event_t)
EVT_IM_ACTION_INFO	请求更新软键盘上的 Action 按钮的信息 (im_action_button_info_event_t)
EVT_DRAG_START	开始拖动 (event_t)
EVT_DRAG	拖动 (event_t)
EVT_DRAG_END	结束拖动 (event_t)
EVT_SCREEN_SAVER	在指定的时间内 (WITH_SCREEN_SAVER_TIME)，没有用户输入事件，由窗口管理器触发
EVT_LOW_MEMORY	内存不足
EVT_OUT_OF_MEMORY	内存耗尽
EVT_ORIENTATION_WILL_CHANGED	屏幕即将旋转
EVT_ORIENTATION_CHANGED	屏幕旋转
EVT_WIDGET_CREATED	控件创建事件
EVT_REQ_START	event queue 其它请求编号起始值
EVT_USER_START	用户定义事件起始值

在 AWTK 中 event_t 是事件基类，常用的 pointer_event_t 和 key_event_t 等都是它的子类。例如，只有将 event_t 类型的对象转换为具体子类 key_event_t 类型才能获取键盘具体按下的是哪个键。AWTK 中具体有哪些事件类型请看：awtk/src/base/events.h

1. 键盘事件

例如，进入下文第二小节示例图中右侧的界面时，按下”F2”键，界面将回到图中左侧的界面。要完成这样的功能，可以使用 widget_on 注册 EVT_KEY_DOWN 键盘按下事件，然后根据键盘按下的是哪个键，在做相应的处理，代码如下。其中，F2 对应 AWTK 中的键盘映射关系请看本文 7.4 章节。

```
/* awtk/demos/demo_ui_app.c */
static ret_t on_key_back_or_back_to_home(void* ctx, event_t* e) {
  key_event_t* evt = (key_event_t*)e;
  if (evt->key == TK_KEY_F2) {
    window_manager_back(WIDGET(ctx));
  } else if (evt->key == TK_KEY_F3) {
    window_manager_back_to_home(WIDGET(ctx));
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  }

  return RET_OK;
}

ret_t application_init() {
  widget_t* wm = window_manager();

  tk_ext_widgets_init();

  /* enable screen saver */
  window_manager_set_screen_saver_time(wm, 180 * 1000);
  widget_on(wm, EVT_SCREEN_SAVER, on_screen_saver, NULL);

  widget_on(wm, EVT_KEY_DOWN, on_key_back_or_back_to_home, wm);
  widget_on(wm, EVT_BEFORE_PAINT, wm_on_before_paint, wm);
  widget_on(wm, EVT_AFTER_PAINT, wm_on_after_paint, wm);
  widget_on(wm, EVT_LOW_MEMORY, wm_on_low_memory, wm);
  widget_on(wm, EVT_OUT_OF_MEMORY, wm_on_out_of_memory, wm);
  widget_on(wm, EVT_REQUEST_QUIT_APP, wm_on_request_quit, wm);

  return show_preload_res_window();
}
```

2. 鼠标事件

例如，点击 awtk/bin/demoui.exe 中的”Inc”按钮将增加进度条的值，详见下图：

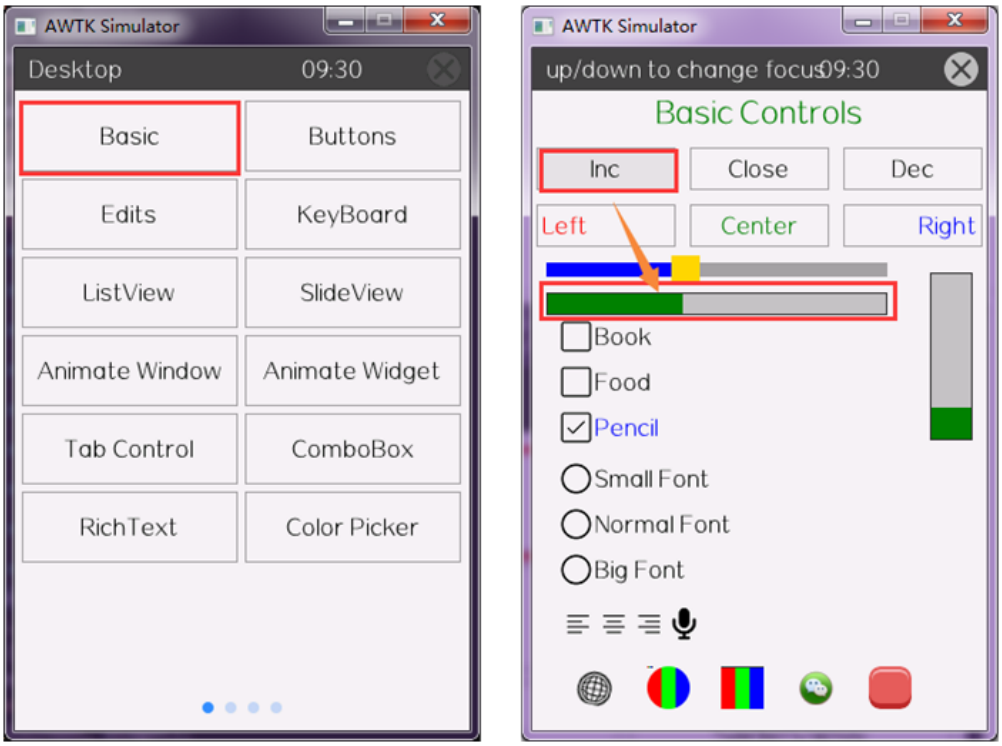


图 3.15　点击事件

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

使用 widget_on 函数注册 EVT_CLICK 点击事件对应的回调函数，代码如下：

```
/* awtk/demos/demo_ui_app.c */
static ret_t on_inc(void* ctx, event_t* e) {
  widget_t* win = WIDGET(ctx);
  (void)e;

  progress_bar_animate_delta(win, "bar1", 10);
  progress_bar_animate_delta(win, "bar2", 10);

  return RET_OK;
}

static ret_t install_one(void* ctx, const void* iter) {
  widget_t* widget = WIDGET(iter);
  widget_t* win = widget_get_window(widget);

  if (widget->name != NULL) {
    const char* name = widget->name;
    if (strstr(name, "open:") != NULL) {
      widget_on(widget, EVT_CLICK, on_open_window, (void*)(name + 5));
      widget_on(widget, EVT_LONG_PRESS, on_open_window, (void*)(name + 5));
    } else if (tk_str_eq(name, "inc_value")) {
      widget_t* win = widget_get_window(widget);
      widget_on(widget, EVT_CLICK, on_inc, win);
    }
  }

  return RET_OK;
}
```

3. 控件事件

控件事件一般是指控件响应某种行为而触发的事件，例如，edit 单行编辑器控件，在用户正在输入文本的时候会触发 EVT_VALUE_CHANGING（文本正在改变事件），在输入完成后则会触发 EVT_VALUE_CHANGED（文本已改变事件）。

例如，在 3.5.2 章节的第一小节中，当用户在 edit 控件输入文本的时候会触发 EVT_VALUE_CHANGING 事件，通过 widget_on 给该事件注册回调函数 on_changing，实现 label 文本控件和 edit 控件内容同步更新的功能。

控件可以触发哪些事件，可以参考第 4 章中各章节下的”事件”。例如，edit 可以触发的事件可以参考 4.6.3 章节。

4. 控件动画事件

控件动画事件是指控件动画在不同的状态时触发的事件，例如，开始、暂停、停止、结束等，具体的可以参考 3.5.4 章节表格中以”EVT_ANIM_”开头的事件，另外有关控件动画方面的介绍请看 5.3 章节。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

3.6 定时器

AWTK 提供了很便捷的定时器，包括添加、删除定时器等。

3.6.1 函数

timer 提供的函数详见下表，具体请查看 AWTK_API 手册，这里不累赘了。

函数名称	说明
timer_add	增加一个 timer
timer_count	返回 timer 的个数
timer_modify	修改指定的 timer 的 duration，修改之后定时器重新开始计时
timer_next_time	返回最近的 timer 到期时间
timer_queue	用于非 GUI 线程增加一个 timer，本函数向主循环的事件队列中发送一个增加 timer 的请求
timer_remove	删除指定的 timer
timer_reset	重置指定的 timer，重置之后定时器重新开始计时
timer_set_on_destroy	设置一个回调函数，在 timer 被销毁时调用 (方便脚本语言去释放回调函数)

3.6.2 定时器回调函数返回值

调用 timer_add 函数添加定时器的时候，在其回调函数中需要设置返回值，可以设置的返回值通常有：RET_REPEAT 和 RET_REMOVE，其具体说明如下。

（1）返回 RET_REPEAT，表示可重复执行。例如，在 3.6.3 章节的代码中定时器的回调函数 on_timer 的返回值为 RET_REPEAT，表示每隔 500ms 都会执行该函数。

（2）返回 RET_REMOVE，表示只执行一次。如果修改 3.6.3 章节的代码，定时器的回调函数 on_timer 的返回值为 RET_REMOVE，表示该定时器只执行一次，以后就不再执行了。

3.6.3 示例

下面代码是 awtk/bin/demo1.exe 下定时更新进度条的一个示例，效果如下图所示。调用 timer_add 函数每隔 500ms 更新进度条的数值，代码如下：

```
/* awtk/demos/common.inc */
static ret_t on_timer(const timer_info_t* timer) {
  widget_t* progress_bar = (widget_t*)timer->ctx;
  uint8_t value = (PROGRESS_BAR(progress_bar)->value + 5) % 100;
  progress_bar_set_value(progress_bar, value);
  return RET_REPEAT;
}

/* awtk/demos/demo1_app.c */
ret_t application_init() {
  ...
  progress_bar = progress_bar_create(win, 10, 80, 168, 20);
  widget_set_value(progress_bar, 40);
  timer_add(on_timer, progress_bar, 500);
  ...
}
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

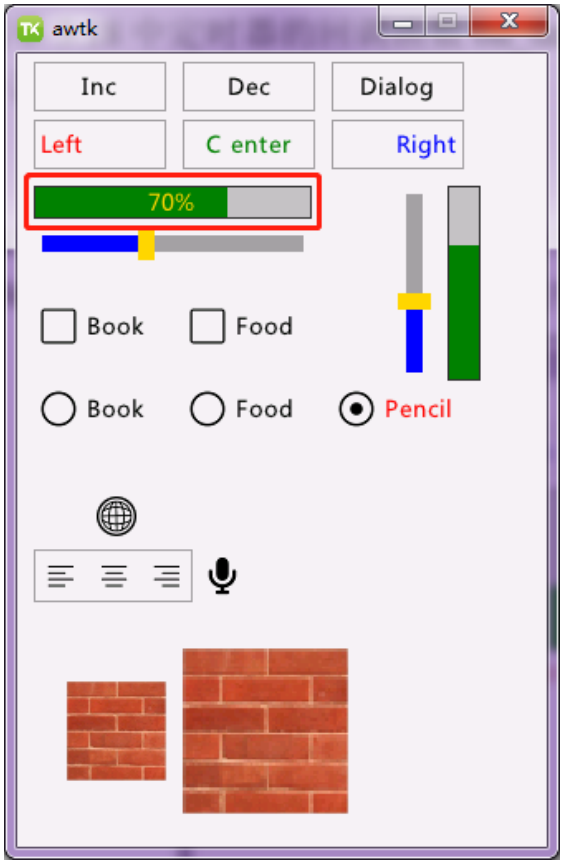


图 3.16　定时器示例

3.7　idle 定时器

idle 可以看作是 duration 为 0 的定时器，idle 可以用来实现一些异步处理。

3.7.1　函数

idle 提供的函数详见下表：

函数名称	说明
idle_add	增加一个 idle
idle_count	返回 idle 的个数
idle_queue	用于非 GUI 线程增加一个 idle，本函数向主循环事件队列中发送一个增加 idle 的请求
idle_remove	删除指定的 idle
idle_set_on_destroy	设置一个回调函数，在 idle 被销毁时调用（方便脚本语言去释放回调函数）

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

3.7.2 idle 定时器回调函数返回值

idle 定时器中回调函数的返回值与 timer 定时器的一样，请看 3.6.2 章节。

3.7.3 示例

在以下代码中调用 idle_add 添加一个 idle 定时器，因为 duration 为 0，不用等到某个时间点到了才执行。在 AWTK 每次消息循环的时候都会调用 idle_add 中的回调函数 something_on_idle。

```
static ret_t something_on_idle(const idle_info_t* info) {
widget_t* widget = WIDGET(info->ctx);
  edit_t* edit = EDIT(widget);
  ...
  return RET_REPEAT;
}

ret_t application_init() {
  ...
  idle_add(something_on_idle, edit);
  ...
}
```

3.8 在多线程环境中发送事件

GUI 控件只能在 GUI 线程进行操作，非 GUI 线程想操作 GUI 控件，必须用 idle_queue 或 timer_queue 进行串行化。

3.8.1 相关函数

（1）idle_queue 函数

- 函数功能：

向主循环的事件队列提交一个增加 idle 的请求，GUI 线程的主循环在处理事件队列时，会把该 idle 函数放到 idle 管理器中，在分发 idle 时，该 idle 函数在 GUI 线程执行。

- 函数原型：

```
ret_t idle_queue (idle_func_t on_idle, void* ctx);
```

- 参数说明：

参数	类型	说明
返回值	ret_t	返回 RET_OK 表示成功，否则表示失败
on_idle	idle_func_t	idle 回调函数
ctx	void*	idle 回调函数的上下文

（2）timer_queue 函数

- 函数功能：

向主循环的事件队列提交一个增加 timer 的请求，GUI 线程的主循环在处理事件队列时，

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

会把该 timer 函数放到 timer 管理器中，在分发 timer 时，该 timer 函数在 GUI 线程执行。

- 函数原型：

```
ret_t timer_queue(timer_func_t on_timer, void* ctx, uint32_t duration);
```

- 参数说明：

参数	类型	说明
返回值	ret_t	返回 RET_OK 表示成功，否则表示失败
on_timer	timer_func_t	timer 回调函数
ctx	void*	timer 回调函数的上下文
duration	uint32_t	时间

需要注意的是，之所以 idle_queue 和 timer_queue 是少数几个可以在非 GUI 线程安全调用的函数，是因为这两个函数向主循环的事件队列添加 idle 或 timer 事件请求时是有加锁的，GUI 线程的主循环在处理事件队列时，根据 idle 或 timer 请求类型（REQ_ADD_IDLE 或 REQ_ADD_TIMER），最终调用 idle_add 和 timer_add 函数，串行化执行。

3.8.2 示例

在以下代码中，调用 tk_thread_create 创建一个线程，在线程的 test_timer_queue 回调函数中调用 timer_queue 添加 timer 事件请求，在 timer 定时器中更新 label 文本框的内容。

```
static widget_t* s_label = NULL;

static ret_t update_label(int nr) {
  if (s_label) {
    char str[16] = {0};
    tk_snprintf(str, sizeof(str), "%d", nr);
    widget_set_text_utf8(s_label, str);
  }

   return nr > 0 ? RET_OK : RET_REMOVE;
}

/* GUI线程中可调用与GUI相关的函数，如：widget_lookup */
static ret_t on_timer(const timer_info_t* timer) {
   return update_label(*(int*)(timer->ctx));
}

/* 非GUI线程中不可调用与GUI相关的函数，如：widget_lookup */
void* test_timer_queue(void* args) {
  static int nr = 500000;
  while (nr-- > 0) {
    timer_queue(on_timer, &nr, 30);
    sleep_ms(30);
  }

  return NULL;
}
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
ret_t application_init() {
  tk_thread_t* thread = NULL;

  widget_t* win = window_create(NULL, 0, 0, 0, 0);
  widget_t* label = label_create(win, 10, 10, 300, 20);
  s_label = label;

  thread = tk_thread_create(test_timer_queue, NULL);
  tk_thread_start(thread);

  return RET_OK;
}

ret_t application_exit() {
  log_debug("application_exit\n");
  return RET_OK;
}

#include "awtk_main.inc"
```

在上面的代码中，需要注意的是：如果在调用 tk_thread_create 函数创建线程的时候，使用 label 对象作为线程的参数，并在 update_label 函数中使用该对象更新 label 文本框的内容，会存在安全隐患。因为 label 是 GUI 界面中的一个文本框控件对象，在线程操作的过程中，如果此时 label 对象在其他地方被销毁，再更新其文本内容就会出现问题。

3.9 基本函数库

AWTK 提供容器与基本数据结构、流、并发、事件与事件源、压缩与解压等 API 接口，相关 API 的用法请查看 AWTK_API.chm 手册（下载地址请看：1.5.1），这里不累赘了。另外，对相关 API 的用法也可参考 AWTK 源码下的 tests 目录。

3.9.1 容器与基本数据结构

AWTK 提供了一些常用的操作数组、链表、字符串等 API 接口，详见下表：

类型	说明
darray	动态数组
slist	单向链表
wbuffer	Write Buffer
rbuffer	Read Buffer
ring_buffer	循环缓存区
str	可变长度的 UTF8 字符串
wstr	可变长度的宽字符字符串
point	点
rect	矩形
color	颜色对象
pointf	点 (浮点数格式)

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

类型	说明
int_str	数字-字符串类型
str_str	字符串-字符串类型
named_value	命名的值
value	一个通用数据类型，用来存放整数、浮点数、字符串和其它对象
rgba	颜色值

3.9.2 流

AWTK 提供了一些常用的文件、套接字、串口等输入输出流 API 接口, 详见下表：

类型	说明
istream	输入流的接口
ostream	输出流的接口
iostream	输入/出流的接口
iostream_mem	内存输入输出流
istream_mem	内存输入流
ostream_mem	内存输出流
istream_file	文件输入流
ostream_file	文件输出流
iostream_tcp	TCP 输入输出流
istream_tcp	TCP 输入流
ostream_tcp	TCP 输出流
iostream_udp	UDP 输入输出流
istream_udp	UDP 输入流
ostream_udp	UDP 输出流
iostream_serial	串口输入输出流
istream_serial	串口输入流
ostream_serial	串口输出流
istream_buffered	缓冲输入流
ostream_buffered	缓冲输出流
iostream_noisy	故障注入流
ostream_noisy	故障注入输出流
iostream_shdlc	SHDLC 输入输出流
istream_shdlc	SHDLC 输入流
ostream_shdlc	SHDLC 输出流
ostream_retry	重传输出流

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

3.9.3 并发

AWTK 提供了互斥锁、线程、信号量和条件变量的 API 接口，详见下表：

类型	说明
mutex	互斥锁
thread	线程
semaphor	信号量
cond_var	简化版条件变量

3.9.4 压缩和解压

AWTK 提供了压缩和解压的 API 接口，详见下表：

类型	说明
compressor	压缩解压接口
compressor_miniz	基于 miniz 实现的压缩解压接口

3.9.5 平台相关

AWTK 提供了操作文件、目录以及获取时间等 API 接口，详见下表：

类型	说明
fs	文件系统
fs_dir	目录
fs_file	文件
fs_item	目录或目录
path	路径相关的工具函数
memory	内存管理相关函数和宏
date_time	时间日期函数
time_now	获取当前时间的函数
platform	平台接口，包括：获取时间、休眠等函数
socket_pair	生成两个可以互相通信的 socket 句柄

3.9.6 数据格式

AWTK 提供了 json 的相关 API 接口，详见下表：

类型	说明
ubjson_parser	json 解析
ubjson_writer	json 写
ubjson_reader	json 读

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

3.9.7 工具类

AWTK 提供了一些工具类、chat 和 wchar 互转等 API 接口，详见下表：

类型	说明
utils	工具类
utf8	wchar_t 和 char 类型转换接口
tokenizer	从字符串中解析出一个一个的 token

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4. 控件

本章导读：

AWTK 框架为开发者提供了丰富的基础控件，开发者通过组合这些基础控件可以快速开发、创建复杂的 *GUI* 界面。控件是视图层的基本组成元素，是构建 *GUI* 界面的重要元素。熟练的使用控件是高效开发 *AWTK* 应用程序的必备技能。

4.1 简介

本章将阐述 AWTK 四大基本控件：窗口、基本控件、通用容器和扩展控件，内容包括各个控件提供的函数、属性、事件以及相关用法的示例。AWTK 包括的控件及其继承关系详见 awtk/docs/images/widget_overview.png。

4.2 widget

widget_t 是所有控件、窗口和窗口管理器的基类。widget_t 也是一个容器，可放其它 widget_t 到它的内部，形成一个树形结构。通常 widget_t 通过一个矩形区域向用户呈现一些信息，接受用户的输入，并据此做出适当反应。它负责控件的生命周期、通用状态、事件分发和 Style 的管理。

本类提供的接口（函数和属性）除非特别说明，一般都适用于子类控件。例如，list_view_t 是 widget_t 的子类控件，在 list_view 控件中没有提供专门的 API 用于添加、删除 list_item 控件，但可以调用 widget 控件提供的 widget_add_child 和 widget_remove_child 函数完成。

4.2.1 函数

widget 提供的函数详见下表，具体请查看 AWTK_API 手册，这里不累赘了。

函数名称	说明
widget_count_children	获取子控件的个数。
widget_get_child	获取指定索引的子控件。
widget_find_parent_by_name	通过名称查找父控件。
widget_find_parent_by_type	通过类型查找父控件。
widget_get_focused_widget	获取当前窗口中的焦点控件。
widget_get_native_window	获取原生窗口对象。
widget_index_of	获取控件在父控件中的索引编号。
widget_close_window	关闭控件所在的窗口。
widget_close_window_force	关闭控件所在的窗口。
widget_back	请求返回到前一个窗口。
widget_back_to_home	请求返回到 home 窗口。
widget_move	移动控件。
widget_move_to_center	移动控件到父控件中间。
widget_resize	调整控件的大小。
widget_move_resize	移动控件并调整控件的大小。
widget_move_resize_ex	移动控件并调整控件的大小。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
widget_get_value	获取控件的值。只是对 widget_get_prop 的包装，值的意义由子类控件决定。
widget_set_value	设置控件的值。
widget_add_value	增加控件的值。
widget_get_value_int	获取控件的值。只是对 widget_get_prop 的包装，值的意义由子类控件决定。
widget_set_value_int	设置控件的值。
widget_add_value_int	增加控件的值。
widget_animate_value_to	设置控件的值 (以动画形式变化到指定的值)。
widget_set_text	设置控件的文本。（如果字符串相同，则不会重复设置以及触发事件）
widget_set_text_ex	设置控件的文本。
widget_get_window_theme	获取控件的窗体样式。
widget_is_style_exist	查询指定的 style 是否存在。
widget_is_support_highlighter	判断 widget 是否支持高亮。
widget_use_style	启用指定的 style。
widget_set_text_utf8	设置控件的文本。（如果字符串相同，则不会重复设置以及触发事件）
widget_set_text_utf8_ex	设置控件的文本。
widget_get_text_utf8	获取控件的文本。
widget_set_child_text_utf8	设置子控件的文本。
widget_get_child_text_utf8	获取子控件的文本。
widget_set_child_text_with_double	用一个浮点数去设置子控件的文本。
widget_set_child_text_with_int	用一个整数去设置子控件的文本。
widget_set_tr_text	获取翻译之后的文本，然后调用 widget_set_text。
widget_get_enable	获取控件 enable 属性值。
widget_get_floating	获取控件 floating 属性值。
widget_get_auto_adjust_size	获取控件 auto_adjust_size 属性值。
widget_get_with_focus_state	获取控件 with_focus_state 属性值。
widget_get_focusable	获取控件 focusable 属性值。
widget_get_sensitive	获取控件 sensitive 属性值。
widget_get_visible	获取控件 visible 属性值。
widget_get_feedback	获取控件 feedback 属性值。
widget_get_text	获取控件的文本。
widget_to_local	将屏幕坐标转换成控件内的本地坐标，即相对于控件左上角的坐标。
widget_to_global	将控件内的本地坐标转换成全局坐标。
widget_to_screen_ex	将控件内的本地坐标根据祖父级控件转换成相对于祖父级控件的屏幕相对坐标。
widget_to_screen	将控件内的本地坐标转换成屏幕上的坐标。
widget_set_name	设置控件的名称。
widget_set_theme	设置 theme 的名称，用于动态切换主题。名称与当前主题名称相同，则重新加载全部资源。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
widget_get_theme_name	获取 theme 的名称
widget_set_pointer_cursor	设置鼠标指针的图片名。
widget_set_animation	设置控件的动画参数 (仅用于在 UI 文件使用)。
widget_create_animator	创建动画。
widget_start_animator	播放动画。
widget_set_animator_time_scale	设置动画的时间倍率，<0: 时间倒退，<1: 时间变慢，>1 时间变快。
widget_pause_animator	暂停动画。
widget_find_animator	查找指定名称的动画。
widget_stop_animator	停止动画 (控件的相应属性回归原位)。
widget_destroy_animator	销毁动画。
widget_set_enable	设置控件的可用性。
widget_set_feedback	设置控件是否启用反馈。
widget_set_auto_adjust_size	设置控件是否根据子控件和文本自动调整控件自身大小。
widget_set_floating	设置控件的 floating 标志。
widget_set_focused	设置控件是否获得焦点。
widget_set_focusable	设置控件是否可获得焦点。
widget_set_state	设置控件的状态。
widget_set_opacity	设置控件的不透明度。
widget_set_dirty_rect_tolerance	设置控件脏矩形超出控件本身大小的最大范围 (一般不用指定)。
widget_destroy_children	销毁全部子控件。
widget_add_child	加入一个子控件。
widget_remove_child	移出指定的子控件 (并不销毁)。
widget_insert_child	插入子控件到指定的位置。
widget_restack	调整控件在父控件中的位置序数。
widget_child	查找指定名称的子控件 (同 widget_lookup(widget, name, FALSE))。
widget_lookup	查找指定名称的子控件 (返回第一个)。
widget_lookup_by_type	查找指定类型的子控件 (返回第一个)。
widget_set_visible	设置控件的可见性。
widget_set_visible_only	设置控件的可见性 (不触发 repaint 和 relayout)。
widget_set_sensitive	设置控件是否接受用户事件。
widget_on	注册指定事件的处理函数。
widget_on_with_tag	注册指定 tag 的事件处理函数。
widget_off	注销指定事件的处理函数。
widget_child_on	为指定名称的子控件注册指定事件的处理函数。
widget_off_by_func	注销指定函数的事件处理函数。
widget_off_by_ctx	注销指定 ctx 的事件处理函数。
widget_off_by_tag	注销指定 tag 的事件处理函数。
widget_invalidate	请求重绘指定的区域
widget_invalidate_force	请求强制重绘控件。

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
widget_paint	绘制控件到一个 canvas 上。
widget_draw_text_in_rect	在 canvas 绘制一行文本。
widget_dispatch	分发一个事件。
widget_dispatch_async	异步分发一个事件。
widget_dispatch_simple_event	分发一个简单事件。
widget_dispatch_model_event	分发 model 事件 (分发到各个窗口)。
widget_dispatch_recursive	分发一个事件控件本身及所有子控件。
widget_get_prop	获取控件指定属性的值。
widget_get_prop_default_value	获取控件指定属性的缺省值 (在持久化控件时，无需保存缺省值)。
widget_set_prop	设置控件指定属性的值。
widget_set_props	设置多个参数。
widget_set_prop_str	设置字符串格式的属性。
widget_get_prop_str	获取字符串格式的属性。
widget_set_prop_pointer	设置指针格式的属性。
widget_set_prop_pointer_ex	设置指针格式的属性。
widget_get_prop_pointer	获取指针格式的属性。
widget_set_prop_float	设置浮点数格式的属性。
widget_get_prop_float	获取浮点数格式的属性。
widget_set_prop_int	设置整数格式的属性。
widget_get_prop_int	获取整数格式的属性。
widget_set_prop_bool	设置布尔格式的属性。
widget_get_prop_bool	获取布尔格式的属性。
widget_is_window_opened	判断当前控件所在的窗口是否已经打开。
widget_is_window_created	判断窗口及子控件创建或加载是否完成。
widget_is_parent_of	判断当前控件是否是指定控件的父控件 (包括非直系)。
widget_is_direct_parent_of	判断当前控件是否是指定控件的直系父控件。
widget_is_window	判断当前控件是否是窗口。
widget_is_system_bar	检查控件是否是 system bar 类型。
widget_is_normal_window	检查控件是否是普通窗口类型。
widget_is_fullscreen_window	检查控件是否是全屏窗口。
widget_is_dialog	检查控件是否是对话框类型。
widget_is_popup	检查控件是否是弹出窗口类型。
widget_is_overlay	检查控件是否是 overlay 窗口类型。
widget_is_always_on_top	检查控件是否总在最上层。
widget_is_opened_dialog	检查控件弹出对话框控件是否已经打开了（而非挂起状态）。
widget_is_opened_popup	检查控件弹出窗口控件是否已经打开了（而非挂起状态）。
widget_is_keyboard	判断当前控件是否是 keyboard。
widget_is_designing_window	判断当前控件是否是设计窗口。
widget_is_window_manager	判断当前控件是否是窗口管理器。
widget_grab	让指定子控件抓住事件。

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
widget_ungrab	让指定子控件放弃抓住事件。
widget_foreach	遍历当前控件及子控件。
widget_get_window	获取当前控件所在的窗口。
widget_get_window_manager	获取当前的窗口管理器。
widget_get_type	获取当前控件的类型名称。
widget_clone	clone。
widget_equal	判断两个 widget 是否相同。
widget_add_timer	创建定时器。
widget_remove_timer	删除指定的 timer。
widget_add_idle	创建 idle。
widget_remove_idle	删除指定的 idle。
widget_load_image	加载图片。
widget_unload_image	卸载图片。
widget_load_asset	加载资源。
widget_load_asset_ex	加载资源。
widget_unload_asset	卸载资源。
widget_cast	转换为 widget 对象 (供脚本语言使用)。
widget_destroy	从父控件中移除控件，并调用 unref 函数销毁控件。
widget_destroy_async	从父控件中移除控件，并调用 unref 函数销毁控件。
widget_ref	增加控件的引用计数。
widget_unref	减少控件的引用计数。引用计数为 0 时销毁控件。
widget_paint_helper	帮助子控件实现自己的绘制函数。
widget_draw_background	根据控件的 style 绘制背景矩形。
widget_stroke_border_rect	根据控件的 style 绘制边框矩形。
widget_fill_bg_rect	根据控件的 style 绘制背景矩形。
widget_fill_fg_rect	根据控件的 style 绘制前景矩形。
widget_prepare_text_style	从 widget 的 style 中取出字体名称、大小和颜色数据，设置到 canvas 中。
widget_prepare_text_style_ex	从 widget 的 style 中取出字体名称、大小和颜色数据，设置到 canvas 中。
widget_measure_text	计算文本的宽度。
widget_dispatch_event_to_target_recursive	递归分发事件到 targe 控件。
widget_is_point_in	判断一个点是否在控件内。
widget_dispatch_to_target	递归的分发一个事件到所有 target 子控件。
widget_dispatch_to_key_target	递归的分发一个事件到所有 key_target 子控件。
widget_find_target	查找 x/y 坐标对应的子控件。
widget_re_translate_text	语言改变后，重新翻译控件上的文本 (包括子控件)。
widget_init	初始化控件。仅在子类控件构造函数中使用。
widget_create	创建控件。仅在子类控件构造函数中使用。
widget_get_style_type	获取 widget 对应风格类型
widget_update_style	让控件根据自己当前状态更新 style。
widget_update_style_recursive	让控件及子控件根据自己当前状态更新 style。
widget_set_as_key_target	递归的把父控件的 key_target 设置为自己。

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
widget_focus_next	把焦点移动下一个控件。
widget_focus_prev	把焦点移动前一个控件。
widget_get_state_for_style	把控件的状态转成获取 style 选要的状态，一般只在子类中使用。
widget_layout	布局当前控件及子控件。
widget_layout_children	layout 子控件。
widget_set_self_layout	设置控件自己的布局参数。
widget_set_children_layout	设置子控件的布局参数。
widget_set_self_layout_params	设置控件自己的布局 (缺省布局器) 参数 (过时，请用 widget_set_self_layout)。
widget_set_style_int	设置整数类型的 style。
widget_set_style_str	设置字符串类型的 style。
widget_set_style_color	设置颜色类型的 style。
widget_take_snapshot	创建一个 bitmap 对象，将控件绘制到 bitmap 上，并返回 bitmap 对象。
widget_take_snapshot_rect	创建一个 bitmap 对象，将控件绘制到 bitmap 上并且可以设置该控件的截屏区域，返回 bitmap 对象。
widget_get_canvas	获取 canvas 对象。
widget_reset_canvas	重置 canvas 对象。for designer only, 调用者需要 unload 全部图片
widget_on_paint_background	绘制背景。
widget_on_paint_self	绘制自身。
widget_on_paint_children	绘制子控件。
widget_on_paint_border	绘制边框。
widget_is_instance_of	检查控件是否是指定的类型。
widget_set_need_relayout_children	设置控件需要 relayout 标识。
widget_set_need_relayout	设置控件需要 relayout 标识。
widget_ensure_visible_in_viewport	使控件滚动到可见区域。
widget_focus_first	置焦点于第一个控件。
widget_get_locale_info	获取 locale_info 对象。
widget_get_image_manager	获取 image_manager 对象。
widget_get_assets_manager	获取 assets_manager 对象。
widget_get_font_manager	获取 font_manager 对象。
widget_update_pointer_cursor	更新鼠标指针。
widget_begin_wait_pointer_cursor	开始等待鼠标指针。
widget_end_wait_pointer_cursor	结束等待鼠标指针。
widget_set_style	设置 widget 私有样式。
widget_get_style	获取 widget 样式。
widget_get_content_area	获取 widget 去掉 margin 之外的区域。
widget_calc_icon_text_rect	计算 icon text 的位置。
widget_auto_scale_children	根据缩放子控件的位置和大小。
widget_set_need_update_style	设置需要更新 Style。
widget_set_need_update_style_recursive	让控件及子控件设置需要更新 Style。

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
widget_on_pointer_down_children	处理子控件的 pointer down 事件。
widget_on_pointer_move_children	处理子控件的 pointer move 事件。
widget_on_pointer_up_children	处理子控件的 pointer up 事件。
widget_is_change_focus_key	是否是切换焦点的键。
widget_add_child_default	加入一个子控件默认实现 (供子类调用)。
widget_on_keydown	处理 key down 事件。
widget_on_keyup	处理 key up 事件。
widget_get_widget_vtable	获取 widget 虚表。
widget_to_xml	将 widget 转换成 xml。

4.2.2 属性

1. 属性列表

widget 提供了下面属性，widget 的属性均适用于 widget 的子类控件，详见下表：

属性名称	类型	说明
x	xy_t	x 坐标 (相对于父控件的 x 坐标)。
y	xy_t	y 坐标 (相对于父控件的 y 坐标)。
w	wh_t	宽度。
h	wh_t	高度。
name	char*	控件名字。
pointer_cursor	char*	鼠标光标图片名称。
tr_text	char*	保存用于翻译的字符串。
style	char*	style 的名称。
animation	char*	动画参数。请参考控件动画[11]
enable	bool_t	启用/禁用状态。
feedback	bool_t	是否启用按键音、触屏音和震动等反馈。
visible	bool_t	是否可见。
sensitive	bool_t	是否接受用户事件。
focusable	bool_t	是否支持焦点停留。
with_focus_state	bool_t	是否支持焦点状态。
auto_adjust_size	bool_t	是否根据子控件和文本自动调整控件自身大小。
focused	bool_t	是否得到焦点。
auto_created	bool_t	是否由父控件自动创建。
dirty	bool_t	标识控件是否需要重绘。
floating	bool_t	标识控件是否启用浮动布局，不受父控件的 children_layout 的控制。
need_update_style	bool_t	标识控件是否需要 update style。
ref_count	int32_t	引用计数，计数为 0 时销毁。
initializing	bool_t	标识控件正在初始化。
loading	bool_t	标识控件正在加载。
destroying	bool_t	标识控件正在被销毁。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

属性名称	类型	说明
state	char*	控件的状态 (取值参考 widget_state_t)。
opacity	uint8_t	不透明度 (0-255)，0 完全透明，255 完全不透明。
dirty_rect_tolerance	uint16_t	脏矩形超出控件本身大小的最大范围 (一般不用指定)。
text	wstr_t	文本。用途视具体情况而定。
parent	widget_t*	父控件
target	widget_t*	接收事件的子控件。
grab_widget	widget_t*	grab 事件的子控件。
grab_widget_count	int32_t	有时子控件和控件自己都会 grab widget，所需要 grab 的计数。
key_target	widget_t*	接收按键事件的子控件。
children	darray_t*	全部子控件。
emitter	emitter_t*	事件发射器。
astyle	style_t*	Style 对象。
children_layout	children_layouter_t*	子控件布局器。请参考控件布局参数[12]
self_layout	self_layouter_t*	控件布局器。请参考控件布局参数[13]
custom_props	tk_object_t*	自定义属性。
vt	const widget_vtable_t*	虚函数表。

2. 设置属性

上个小节表格中的属性，可以在编写 UI 界面的 XML 文件通过控件的属性指定，比如，设置 label 控件的 name、x、y 等属性，代码如下：

```
<window name="home_page">
    <label name="label" x="79" y="192" w="160" h="28" text="Label"/>
</window>
```

注: 因为 label 控件是 widget 的子类控件，所以 widget 的属性均适用于 label 控件。

3. 获取属性

在以上代码中，设置了 label 控件的 x 属性，如果想要获取 x 属性对应的属性值”79”。可以通过 widget_lookup() 函数获取 label 控件的指针对象，然后通过箭头运算符（->）获取该属性值，代码如下：

```
widget_t* label = widget_lookup(win, "label", TRUE);
int32_t x = label->x;
```

4. 使用 widget_get_prop 和 widget_set_prop 函数获取或设置控件属性

除了上面提到的可以通过箭头运算符获取控件的属性外，AWTK 还提供 widget_get_prop 和 widget_set_prop 函数用于获取和设置控件的属性，这两个函数的函数原型如下：

[11] https://github.com/zlgopen/awtk/blob/master/docs/widget_animator.md
[12] https://github.com/zlgopen/awtk/blob/master/docs/layout.md
[13] https://github.com/zlgopen/awtk/blob/master/docs/layout.md

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
ret_t widget_get_prop(widget_t* widget, const char* name, value_t* v);
ret_t widget_set_prop(widget_t* widget, const char* name, const value_t* v);
```

其中，上面两个函数中 name 的取值详见下表，由于 prop 属性很多该表没有列出全部，具体请看 AWTK_API 手册根据关键字”widget_prop_t”查找，示例请看下文章节。

名称	说明
WIDGET_PROP_X	X 坐标
WIDGET_PROP_Y	Y 坐标
WIDGET_PROP_W	宽度
WIDGET_PROP_H	高度
WIDGET_PROP_CANVAS	Canvas
WIDGET_PROP_NATIVE_WINDOW	Native Window
WIDGET_PROP_HIGHLIGHT	dialog highlight
WIDGET_PROP_BAR_SIZE	slider 中的 bar 的的宽度或高度
WIDGET_PROP_OPACITY	不透明度
WIDGET_PROP_MIN_W	最小宽度
WIDGET_PROP_MAX_W	最大宽度
WIDGET_PROP_CHILDREN_LAYOUT	子控件布局参数
WIDGET_PROP_LAYOUT	子控件布局参数 (过时)
WIDGET_PROP_SELF_LAYOUT	控件布局参数
WIDGET_PROP_LAYOUT_W	layout 宽度
WIDGET_PROP_LAYOUT_H	layout 高度
…	…

4.2.3 事件

widget 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_WILL_MOVE	event_t	控件移动前触发。
EVT_MOVE	event_t	控件移动后触发。
EVT_WILL_RESIZE	event_t	控件调整大小前触发。
EVT_RESIZE	event_t	控件调整大小后触发。
EVT_WILL_MOVE_RESIZE	event_t	控件移动并调整大小前触发。
EVT_MOVE_RESIZE	event_t	控件移动并调整大小后触发。
EVT_THEME_CHANGED	event_t	主题变化事件。
EVT_FOCUS	event_t	控件得到焦点时触发。
EVT_BLUR	event_t	控件失去焦点时触发。
EVT_WIDGET_ADD_CHILD	event_t	控件加载新的子控件。
EVT_WIDGET_REMOVE_CHILD	event_t	控件移除子控件。
EVT_WIDGET_WILL_RESTACK_CHILD	event_t	控件即将调整子控件位置序数。
EVT_WIDGET_RESTACK_CHILD	event_t	控件调整子控件位置序数。
EVT_PROP_WILL_CHANGE	prop_change_event_t	控件属性改变前触发 (通过 set_prop 设置属性，才会触发)。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

事件名称	类型	说明
EVT_PROP_CHANGED	prop_change_event_t	控件属性改变后触发 (通过 set_prop 设置属性，才会触发)。
EVT_PAINT	paint_event_t	控件绘制时触发。
EVT_BEFORE_PAINT	paint_event_t	控件绘制前触发。
EVT_AFTER_PAINT	paint_event_t	控件绘制完成时触发。
EVT_PAINT_DONE	paint_event_t	控件绘制完成时 (canvas 状态已经恢复) 触发。
EVT_WHEEL_BEFORE_CHILDREN	wheel_event_t	鼠标滚轮事件，在子控件处理之前触发。
EVT_WHEEL	wheel_event_t	鼠标滚轮事件。
EVT_POINTER_LEAVE	pointer_event_t	鼠标指针离开控件时触发。
EVT_POINTER_ENTER	pointer_event_t	鼠标指针进入控件时触发。
EVT_KEY_DOWN_BEFORE_CHILDREN	key_event_t	键按下事件，在子控件处理之前触发。
EVT_KEY_DOWN	key_event_t	键按下事件。
EVT_KEY_LONG_PRESS	key_event_t	键长按事件。
EVT_KEY_UP_BEFORE_CHILDREN	key_event_t	键释放事件，在子控件处理之前触发。
EVT_KEY_UP	key_event_t	键释放事件。
EVT_POINTER_DOWN_BEFORE_CHILDREN	pointer_event_t	指针设备按下事件，在子控件处理之前触发。
EVT_POINTER_DOWN	pointer_event_t	指针设备按下事件。
EVT_POINTER_DOWN_ABORT	pointer_event_t	取消指针设备按下事件。
EVT_POINTER_MOVE	pointer_event_t	指针设备移动事件。
EVT_POINTER_MOVE_BEFORE_CHILDREN	pointer_event_t	指针设备移动事件，在子控件处理之前触发。
EVT_POINTER_UP	pointer_event_t	指针设备释放事件。
EVT_POINTER_UP_BEFORE_CHILDREN	pointer_event_t	指针设备释放事件，在子控件处理之前触发。
EVT_DESTROY	event_t	控件销毁时触发。

4.2.4 示例

上面介绍了有关 widget 的函数、属性以及事件等，下面以一些简单的示例来说明一下，如何调用这些函数。

1. 查找控件 widget_lookup

例如，要查找 awtk/bin/demoui.exe 中下图所示的”n/a”文本框，步骤如下：

首先，需要在 XML 中设置文本框的 name 属性，详见以下代码：

```
<!-- awtk\demos\assets\default\raw\ui\memtest.xml -->
<window anim_hint="htranslate" x="0" y="0" w="320" h="480">
    <button name="memtest" x="center" y="bottom:100" w="50%" h="30" text="Start"/>
    <button name="close" x="center" y="bottom:50" w="50%" h="30" text="Close"/>
    <label name="cost" x="center" y="10" w="100%" h="30" text="n/a"/>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    <label name="memset" x="center" y="50" w="100%" h="30" text="n/a"/>
    <label name="memcpy" x="center" y="90" w="100%" h="30" text="n/a"/>
</window>
```

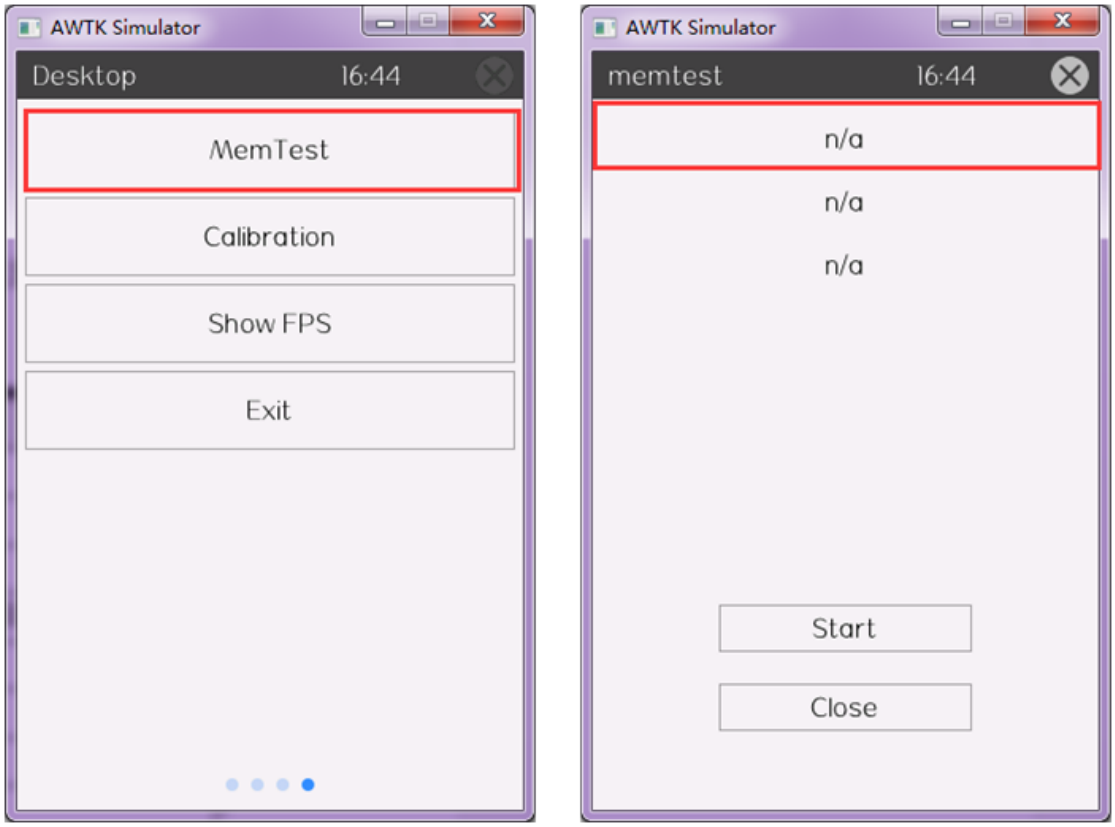


图 4.1　widget_lookup

然后，在 C 代码中调用 widget_lookup 函数查找该控件，代码如下：

```
/* awtk/demos/demo_ui_app.c */
static ret_t on_mem_test(void* ctx, event_t* e) {
  ...
  widget_t* win = WIDGET(ctx);
  widget_t* label_cost = widget_lookup(win, "cost", TRUE);
  ...
  return RET_OK;
}
```

2. 设置文本内容 widget_set_text_utf8

例如，要设置上一小节示例图中”n/a”文本框的内容，通过 widget_lookup 查找到控件后，再调用 widget_set_text_utf8 设置文本框的内容，代码如下：

```
/* awtk/demos/demo_ui_app.c */
static ret_t on_mem_test(void* ctx, event_t* e) {
  char text[32];
  uint32_t size = 100 * 1024;
  uint32_t memset_speed = 0;
  uint32_t memcpy_speed = 0;
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  widget_t* win = WIDGET(ctx);
  widget_t* label_cost = widget_lookup(win, "cost", TRUE);

  void* buff = TKMEM_ALLOC(size);
  uint32_t cost = tk_mem_speed_test(buff, size, &memcpy_speed, &memset_speed);
  TKMEM_FREE(buff);

  tk_snprintf(text, sizeof(text), "%ums", cost);
  widget_set_text_utf8(label_cost, text);
  ...
  return RET_OK;
}
```

3. 获取控件指定属性的值 widget_get_prop

在 4.2.1 章节的表格中有多个以 widget_get_prop 开头的函数，其中 widget_get_prop_xx 是对 widget_get_prop 进一步封装，如：widget_get_prop_int 用于获取整数格式的属性；而 widget_get_prop_bool 则用于获取布尔格式的属性。至于使用 widget_get_prop_int 还是 widget_get_prop_bool 要看获取属性值类型。

例如，要获取 label 控件的宽和高，因为宽和高是 int 类型，所以使用 widget_get_prop_int，代码如下：

```
widget_t* label = widget_lookup(win, "label", TRUE);
int32_t w = widget_get_prop_int(label, WIDGET_PROP_W, 0);
int32_t h = widget_get_prop_int(label, WIDGET_PROP_H, 0);
```

4. 更新样式 widget_use_style

例如，要更新 label 控件的样式，可以使用 widget_use_style 函数，代码如下：

```
widget_t* label = widget_lookup(win, "label", TRUE);
widget_use_style(label, "green");
```

上述代码中的”green”为样式名称，该样式的代码如下，当然也可以设置为其他样式：

```
<!-- awtk/demos/assets/default/raw/styles/default.xml -->
<label>
  <style name="default">
    <normal text_color="black" />
  </style>
  <style name="green">
    <normal text_color="green" />
  </style>
  ...
</label>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.2.5 控件的可见性

在 AWTK 中，控件的 visible 属性决定了控件是否可见，默认为可见的。例如，设置 button 按钮控件不可见，代码如下：

```
<window name="home_page">
  <button name="button" x="33" y="26" w="100" h="36" visible="false" text="Button"/
↪>
</window>
```

注: 在 C 代码中，可以通过 widget_set_visible 函数设置控件是否可见。

需要注意的是，控件是否可见并不影响其是否接收用户事件（比如：鼠标事件）。假设不可见的 button 控件注册了 EVT_CLICK 点击事件，则该事件的回调函数还是会执行。

在 AWTK 中，窗口内的控件是否接收用户事件由控件的 enable（是否使能）和 sensitive（是否接受用户事件）属性决定。只要 enable 和 sensitive 属性有一个值为 false，就不会接收用户事件。

因此，如果不想 button 控件的事件回调函数被执行，可以设置 button 控件的 enable 或者 sensitive 属性为 false，代码如下：

```
<window name="home_page">
  <button name="button" x="33" y="26" w="100" h="36" visible="false" text="Button"␣
↪enable="false"/>
</window>
```

注: 也可以在 C 代码的程序中调用 widget_set_enable 或 widget_set_sensitive 函数完成。

设置 enable 和 sensitive 属性的差异主要表现在外观上：设置 enable 属性为 false，则使用 disable 状态的样式；而设置 sensitive 属性，不管其值是 true 还是 false，都使用 normal 状态的样式。

4.3 window_manager

4.3.1 窗口管理器

window_manager_t 继承 widget_t，负责对 AWTK 应用程序的各个窗口进行集中管理。例如，通过 window_manager_t 类提供的函数可以回到指定的窗口、获取最上面的窗口、获取鼠标指针的坐标、设置鼠标指针等，以上功能请参考下文示例。

4.3.2 函数

window_manager 提供的函数详见下表，具体请查看 AWTK_API 手册，这里不累赘了。在下表中，函数的第一个形参传递的值通常是 window_manager()。

函数名称	说明
window_manager	获取全局 window_manager 对象
window_manager_cast	转换为 window_manager 对象 (供脚本语言使用)。
window_manager_set	设置缺省的窗口管理器。
window_manager_get_top_main_window	获取最上面的主窗口。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
window_manager_get_top_window	获取最上面的窗口。
window_manager_get_prev_window	获取前一个的窗口。
window_manager_get_pointer_x	获取指针当前的 X 坐标。
window_manager_get_pointer_y	获取指针当前的 Y 坐标。
window_manager_get_pointer_pressed	获取指针当前是否按下。
window_manager_is_animating	获取当前窗口动画是否正在播放。
window_manager_open_window	打开窗口。
window_manager_close_window_force	强制立即关闭窗口。
window_manager_paint	绘制。
window_manager_dispatch_input_event	分发输入事件。
window_manager_set_show_fps	设置是否显示 FPS。
window_manager_set_max_fps	限制最大帧率。
window_manager_set_ignore_input_events	设置是否忽略用户输入事件。
window_manager_set_screen_saver_time	设置屏保时间。
window_manager_set_cursor	设置鼠标指针。
window_manager_back	请求关闭顶层窗口。
window_manager_back_to_home	回到主窗口，关闭之上的全部窗口。
window_manager_back_to	回到指定的窗口，关闭之上的全部窗口。
window_manager_switch_to	切换到指定窗口。
window_manager_dispatch_native_window_event	处理 native window 事件。
window_manager_begin_wait_pointer_cursor	开始等待鼠标指针。
window_manager_end_wait_pointer_cursor	结束等待鼠标指针。
window_manager_resize	调整原生窗口的大小。
window_manager_close_all	关闭全部窗口。
window_manager_destroy	销毁 window_manager。
window_manager_init	初始化 window_manager。

4.3.3 属性

window_manager 还提供了下面属性，详见下表：

属性名称	类型	说明
global_emitter	emitter_t*	全局事情分发器。

4.3.4 事件

window_manager 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_TOP_WINDOW_CHANGED	window_event_t	顶层窗口改变的事件。
EVT_SCREEN_SAVER	window_event_t	在指定的时间内，没有用户输入事件，由窗口管理器触发。
EVT_ORIENTATION_CHANGED	event_t	屏幕旋转事件。
EVT_SYSTEM	system_event_t	SDL 系统事件。

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.3.5 示例

1. 回到指定的窗口 window_manager_back_to

如果要回到指定的窗口，并关闭之上的全部窗口，比如 main 窗口，代码如下：

```
window_manager_back_to(window_manager(), "main");
```

2. 获取最上面的窗口 window_manager_get_top_window

如果要获取最上面的窗口，代码如下：

```
widget_t* top_win = window_manager_get_top_window(window_manager());
```

3. 获取指针当前的 X 坐标 window_manager_get_pointer_x

如果要获取指针当前的 X 坐标，代码如下：

```
int32_t x = window_manager_get_pointer_x(window_manager());
```

4. 设置鼠标指针 window_manager_set_cursor

请参考 HowTo 中的如何设置鼠标指针[14]。

4.4 窗口

常用的窗口详见下表：

窗口	说明
window_base	窗口基类（一般不直接使用）
dialog	对话框
popup	弹出窗口（一般不直接使用）
window	普通窗口
system_bar	系统状态窗口
calibration_win	电阻屏校准窗口
overlay	悬浮窗口

4.4.1 window_base

window_base_t 继承 widget_t，是所有窗口的基类。window_base_t 是一个抽象类，不能进行实例化。请在应用程序中使用中具体的窗口类，例如 window_t、dialog_t、system_bar_t、calibration_win_t、overlay_t、popup_t。

1. 函数

window_base 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
window_base_on_destroy	窗口 on_destroy 函数的缺省实现。
window_base_on_event	窗口 on_event 函数的缺省实现。
window_base_invalidate	窗口 on_invalidate 函数的缺省实现。
window_base_on_paint_begin	窗口 on_paint_begin 函数的缺省实现。

[14] https://awtk.zlg.cn/docs/awtk_docs/HowTo/how_to_enable_mouse_cursor.html

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
window_base_on_paint_end	窗口 on_paint_end 函数的缺省实现。
window_base_on_paint_self	窗口 on_paint_self 函数的缺省实现。
window_base_get_prop	窗口 get_prop 函数的缺省实现。
window_base_set_prop	窗口 set_prop 函数的缺省实现。
window_base_create	创建 window base 对象。
window_base_cast	转换为 window_base 对象 (供脚本语言使用)。
window_base_set_need_relayout	设置是否需要 relayout
window_base_on_copy	默认拷贝函数。
window_base_auto_scale_children	根据参数自动缩放子控件。
window_base_get_widget_vtable	获取 window_base 虚表。

2. 属性

window_base 还提供了下面属性，详见下表：

属性名称	类型	说明
theme	char*	窗体样式资源的名称。
design_w	uint16_t	设计时宽度。
design_h	uint16_t	设计时高度。
auto_scale_children_x	bool_t	窗口大小与设计时大小不同时，是否自动调整子控件的 x 坐标。
auto_scale_children_y	bool_t	窗口大小与设计时大小不同时，是否自动调整子控件的 y 坐标。
auto_scale_children_w	bool_t	窗口大小与设计时大小不同时，是否自动调整子控件的宽度。
auto_scale_children_h	bool_t	窗口大小与设计时大小不同时，是否自动调整子控件的高度。
disable_anim	bool_t	禁用窗口动画。
closable	window_closable_t	收到 EVT_REQUEST_CLOSE_WINDOW 是否自动关闭窗口。
open_anim_hint	char*	打开窗口动画的名称。
close_anim_hint	char*	关闭窗口动画的名称。
stage	window_stage_t	窗口当前处于的状态。
theme_obj	theme_t*	窗口的常量窗体样式数据。
image_manager	image_manager_t*	获取图片管理器对象。
font_manager	font_manager_t*	获取字体管理器对象。
assets_manager	assets_manager_t*	获取资源管理器对象。
locale_info	locale_info_t*	获取多国语言字符串管理器对象。
move_focus_prev_key	char*	向前移动焦点的键值。
move_focus_next_key	char*	向后移动焦点的键值。
move_focus_up_key	char*	向上移动焦点的键值。
move_focus_down_key	char*	向下移动焦点的键值。
move_focus_left_key	char*	向左移动焦点的键值。
move_focus_right_key	char*	向右移动焦点的键值。
applet_name	char*	小应用程序 (applet) 的名称。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

属性名称	类型	说明
single_instance	bool_t	单例。如果窗口存在，先关闭再打开。
strongly_focus	bool_t	点击非 focusable 控件时，是否让当前焦点控件失去焦点。比如点击窗口空白区域，是否让编辑器失去焦点。

3. 样式

window_base 可设置的样式在所有窗口中均可设置，样式列表详见下表：

样式属性	说明
bg_color	背景颜色
border_color	边框颜色
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称
bg_image_draw_type	背景图片的显示方式（取值：详见 image 控件一节）
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

4. 状态

window_base 在样式文件中可以设置的状态同样适用于所有窗口，状态列表详见下表：

状态	说明
normal	正常状态
disable	禁用状态
focused	聚焦状态

5. 事件

window_base 提供的事件在所有窗口中均可使用，事件列表详见下表，窗口事件分发的时间点在窗口生命周期中的位置如下图所示。

事件名称	类型	说明
EVT_WINDOW_WILL_OPEN	event_t	窗口即将打开事件。如果有窗口动画，在窗口动画开始前触发。如果没有窗口动画，在窗口被加载后的下一次循环中触发。
EVT_WINDOW_OPEN	event_t	窗口打开事件。如果有窗口动画，在窗口动画完成时触发。如果没有窗口动画，在窗口被加载后的下一次循环中触发。
EVT_WINDOW_TO_BACKGROUND	event_t	窗口被切换到后台事件。打开新窗口时，当前窗口被切换到后台时，对当前窗口触发本事件。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

事件名称	类型	说明
EVT_WINDOW_TO_FOREGROUND	event_t	窗口被切换到前台事件。关闭当前窗口时，前一个窗口被切换到前台时，对前一个窗口触发本事件。
EVT_WINDOW_CLOSE	event_t	窗口关闭事件。
EVT_WINDOW_LOAD	event_t	窗口加载完成事件。
EVT_REQUEST_CLOSE_WINDOW	event_t	请求关闭窗口的事件。
EVT_LOCALE_CHANGED	event_t	locale 改变的事件。

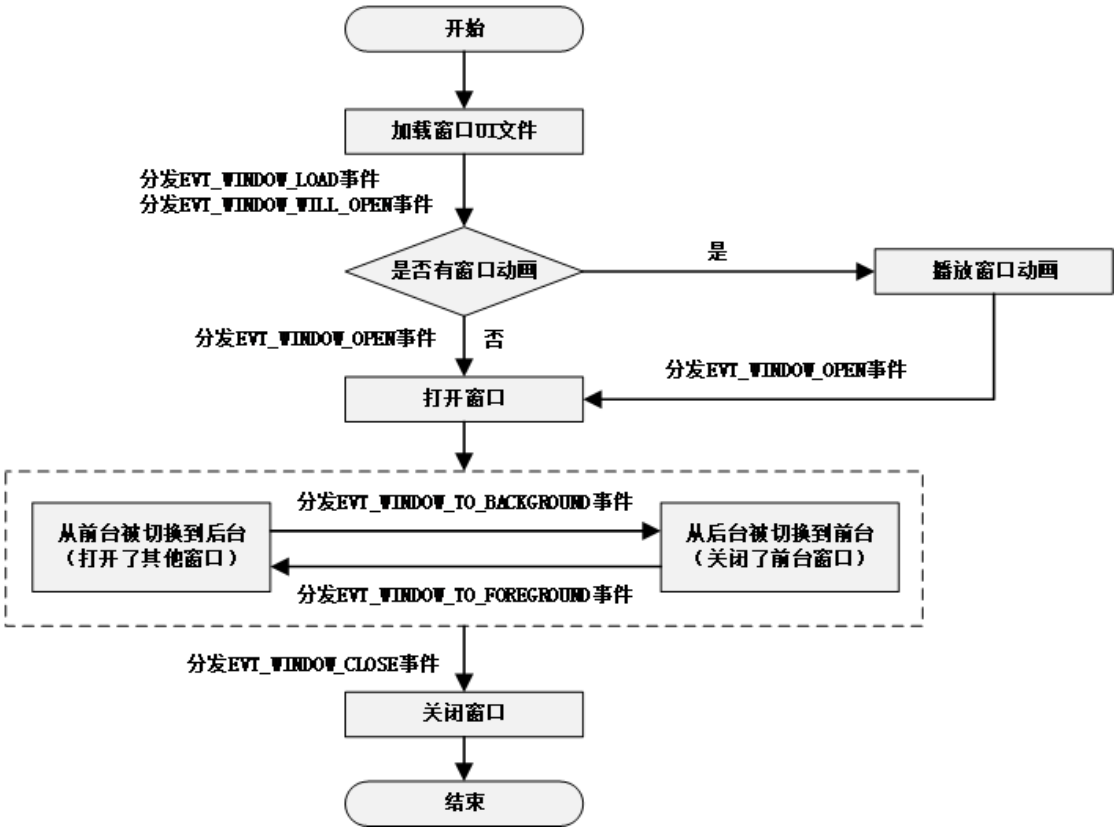


图 4.2　常用窗口生命生命周期

4.4.2　window

缺省的应用程序窗口，占用除 system_bar_t 之外的整个区域。window_t 是 window_base_t 的子类控件，window_base_t 的函数均适用于 window_t 控件。

1. 函数

window 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
window_create	创建 window 对象
window_create_default	以缺省的方式创建 window 对象。
window_set_fullscreen	设置为全屏窗口。
window_set_auto_scale_children	当设计分辨率和实际分辨率不一致时，自动调整子控件的位置和大小。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
window_open	从资源文件中加载并创建 window_base 对象。本函数在 ui_loader/ui_builder_default 里实现。
window_open_and_close	从资源文件中加载并创建 window 对象。本函数在 ui_loader/ui_builder_default 里实现。
window_close	关闭窗口。
window_close_force	立即无条件关闭窗口 (无动画)。
window_cast	转换为 window 对象 (供脚本语言使用)。
window_get_widget_vtable	获取 window 虚表。

2. 属性

window 还提供了下面属性，详见下表：

属性名称	类型	说明
fullscreen	bool_t	是否全屏。

3. 示例

(1) 创建窗口

- 在 xml 中，使用”window”标签创建窗口。无需指定坐标和大小，可以指定窗体样式和动画名称，代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/main.xml -->
<window closable="no" text="Desktop" anim_hint="htranslate">
        ...
</window>
```

- 在 C 代码中，使用函数 window_create 创建窗口，无需指定父控件、坐标和大小，使用 0 即可，代码如下：

```
/* awtk/demos/demo1_app.c */
ret_t application_init() {
  ...
  widget_t* win = window_create(NULL, 0, 0, 0, 0);
  ...
}
```

(2) 打开窗口 window_open

通常，在 assets/default/raw/ui 目录下的 XML 文件中使用”window”标签创建好窗口，然后调用 window_open 函数打开窗口，需要传递给该函数的字符串为 XML 的文件名，代码如下：

```
/* awtk-examples/HelloWorld.Xml-Demo/src/window_main.c */
ret_t application_init() {
  widget_t* win = window_open("main");
  if (win) {
    init_children_widget(win);
  }
  return RET_OK;
```

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
}
```

(3) 关闭窗口 window_close

例如，点击下图中右侧的”close”按钮关闭当前窗口，需要调用 window_close，代码如下：

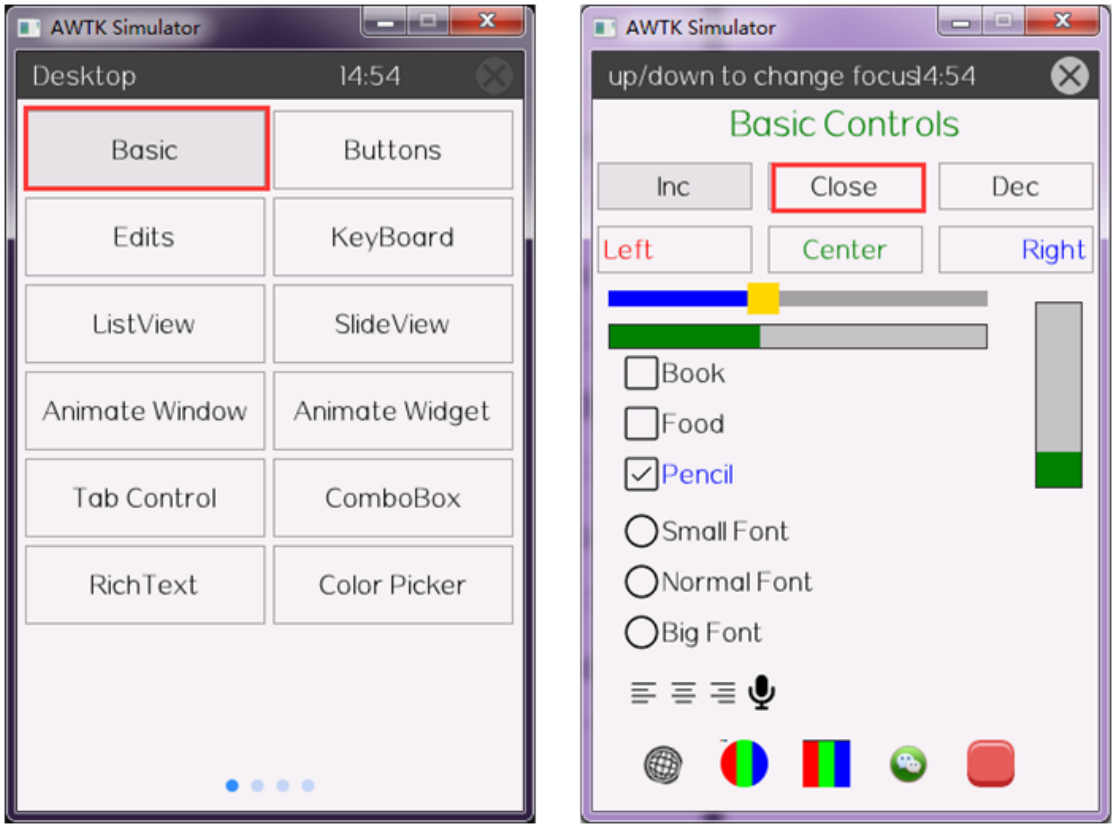


图 4.3　关闭窗口

```c
/* awtk/demos/demo_ui_app.c */
static ret_t on_close(void* ctx, event_t* e) {
  widget_t* win = WIDGET(ctx);
  (void)e;
  return window_close(win);
}

static ret_t install_one(void* ctx, const void* iter) {
  widget_t* widget = WIDGET(iter);
  widget_t* win = widget_get_window(widget);

  if (widget->name != NULL) {
    const char* name = widget->name;
    if (strstr(name, "open:") != NULL) {
      widget_on(widget, EVT_CLICK, on_open_window, (void*)(name + 5));
      widget_on(widget, EVT_LONG_PRESS, on_open_window, (void*)(name + 5));
    } else if (tk_str_eq(name, "close")) {
      widget_on(widget, EVT_CLICK, on_close, win);
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    }
    ...
  return RET_OK;
}
```

4.4.3 dialog

对话框是一种特殊的窗口，大小和位置可以自由设置。AWTK 中的对话框可以是模态的，也可以是非模态的。

对话框通常由对话框标题和对话框客户区两部分组成详见下图。dialog 是 window_base_t 的子类控件，window_base_t 的函数均适用于 dialog_t 控件。

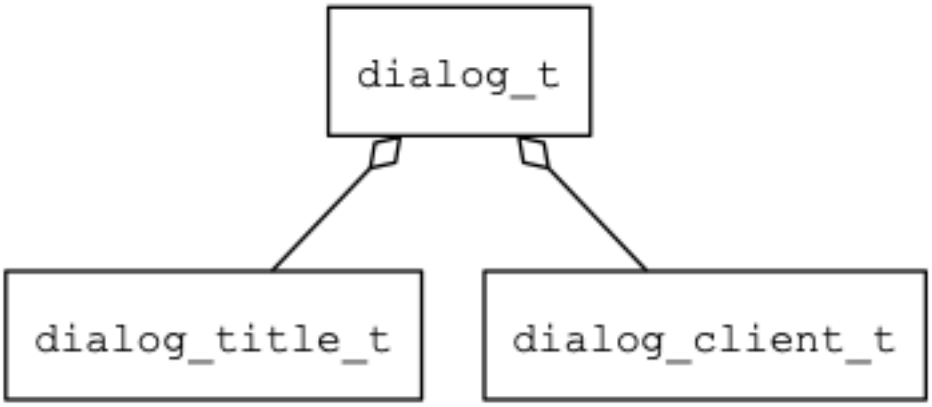


图 4.4 对话框的组成

注: 由于浏览器中无法实现主循环嵌套，因此无法实现模态对话框。如果希望自己写的 AWTK 应用程序可以在浏览器（包括各种小程序）中运行或演示，请避免使用模态对话框。

1. 函数

dialog 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
dialog_create	创建 dialog 对象。
dialog_create_simple	创建 dialog 对象，同时创建 title/client。
dialog_cast	转换 dialog 对象 (供脚本语言使用)。
dialog_get_title	获取 title 控件。
dialog_get_client	获取 client 控件。
dialog_open	从资源文件中加载并创建 Dialog 对象。
dialog_set_title	设置对话框的标题文本。
dialog_modal	模态显示对话框。
dialog_quit	退出模态显示，关闭对话框。
dialog_is_quited	检查对话框是否已经退出模态。
dialog_is_modal	检查对话框是否为模态对话框。
dialog_toast	显示『短暂提示信息』对话框。
dialog_info	显示『提示信息』对话框。
dialog_warn	显示『警告』对话框。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
dialog_confirm	显示『确认』对话框。
dialog_create_with_ok_cancel	创建一个带有 OK/Cancel 按钮的对话框。
dialog_simple_show	根据所选标题，内容与主题等创建一个对话框。
dialog_get_widget_vtable	获取 dialog 虚表。

2. 属性

dialog 还提供了下面属性，详见下表：

属性名称	类型	说明
highlight	char*	对话框高亮策略。

3. 示例

(1) 创建对话框

- 在 xml 中，使用”dialog”标签创建对话框，代码如下：

```
<dialog theme="dialog" style="custom" highlight="default(alpha=40)"
  x="c" y="m" w="80%" h="160">
  <dialog_title style="custom" x="0" y="0" w="100%" h="30" text="Hello AWTK" />
  <dialog_client style="custom" x="0" y="bottom" w="100%" h="-30">
    <label x="center" y="middle:-20" w="200" h="30" text="AWTK is cool!"/>
    <button style="green_btn" name="quit" x="10" y="bottom:10" w="40%" h="30" text=
↪"确定"/>
    <button style="red_btn" name="quit" x="right:10" y="bottom:10" w="40%" h="30"␣
↪text="取消"/>
  </dialog_client>
</dialog>
```

- 在 C 代码中，使用函数 dialog_create_simple 创建对话框，代码如下：

```
/* awtk/demos/demo1_app.c */
static ret_t on_show_dialog(void* ctx, event_t* e) {
  ...
  widget_t* win = dialog_create_simple(NULL, 0, 0, 240, 160);
  dialog_t* dialog = DIALOG(win);

  code = dialog_modal(win);
  ...
}
```

创建之后，再创建子控件，最后调用 dialog_modal 显示对话框。对话框关闭之后 dialog_modal 才会返回。

另外，关于非模态和模态对话框在使用上的区别请看：

- 非模态对话框[15]
- 模态对话框[16]

[15] https://github.com/zlgopen/awtk-c-demos/blob/master/demos/dialog.c
[16] https://github.com/zlgopen/awtk-c-demos/blob/master/demos/dialog_modal.c

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

(2) 打开对话框 dialog_open

通常，在 assets/default/raw/ui 目录下的 XML 文件中使用”dialog”标签创建好窗口，然后调用 dialog_open 函数打开窗口，需要传递给该函数的字符串为 XML 的文件名，打开之后还要调用 dialog_modal 模态显示对话框，代码如下：

```
/* awtk-examples/MusicPlayer-Demo/src/dialog_advanced.c */
ret_t open_play_list_window(widget_t* s_parent) {
  widget_t* dialog = dialog_open("play_list");

  if (dialog) {
    widget_on(dialog, EVT_POINTER_DOWN, on_dialog_state, dialog);
    m_win = s_parent;
    init_children_widget(dialog);
    return dialog_modal(dialog);
  }

  return RET_FAIL;

}
```

(3) 关闭对话框 dialog_quit

要关闭对话框只需要调用 dialog_quit 就可以了，代码如下：

```
/* awtk-examples/MusicPlayer-Demo/src/dialog_advanced.c */
static ret_t on_close(void* ctx, event_t* e) {
  widget_t* dialog = (widget_t*)ctx;
  (void)e;

  return dialog_quit(dialog, RET_QUIT);
}

static ret_t init_widget(void* ctx, const void* iter) {
  widget_t* widget = WIDGET(iter);
  (void)ctx;
  if (widget->name != NULL) {
    const char* name = widget->name;
    if (tk_str_eq(name, "close")) {
      widget_t* dialog = widget_get_window(widget);
      widget_on(widget, EVT_CLICK, on_close, dialog);
    }
    ...
  }
  return RET_OK;
}
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.4.4 system_bar

system_bar 窗口是一种特殊的窗口，独占 LCD 顶部区域，用来显示当前窗口的标题和关闭按钮等内容。system_bar 窗口需要在打开第一个应用程序窗口之前打开。system_bar_t 是 window_base_t 的子类控件，window_base_t 的函数均适用于 system_bar_t 控件。system_bar 对两个子控件会做特殊处理：

- 名为”title”的 label 控件，自动显示当前主窗口的 name 或 text
- 名为”close”的 button 控件，向当前主窗口发送 EVT_REQUEST_CLOSE_WINDOW 消息，如果点击该控件的话

1. 函数

system_bar 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
system_bar_create	创建 system_bar 对象。
system_bar_cast	转换为 system_bar 对象 (供脚本语言使用)。
system_bar_get_widget_vtable	获取 system_bar 虚表。

2. 示例

- 在 xml 中，使用”system_bar”标签创建 system_bar 窗口，代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/system_bar.xml -->
<system_bar h="30">
  <column x="0" y="0" w="-40" h="100%">
    <label style="title" x="10" y="m" w="55%" h="100%" name="title"/>
    <digit_clock style="time" x="r" y="m" w="40%" h="100%" format="hh:mm"/>
  </column>
  <button style="close" x="r:5" y="m" w="26" h="26" name="close" text="x"/>
</system_bar>
```

- 在 C 代码中，使用函数 system_bar_create 创建 system_bar 窗口，代码如下：

```
widget_t* win = system_bar_create(NULL, 0, 0, 320, 30);
```

创建之后，就和使用普通窗口一样了。

4.4.5 calibration_win

电阻屏校准窗口。calibration_win_t 是 window_base_t 的子类控件，window_base_t 的函数均适用于 calibration_win_t 控件。

1. 函数

calibration_win 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
calibration_win_create	创建 calibration_win 对象
calibration_win_set_on_done	设置校准完成的处理函数。
calibration_win_set_on_click	设置校准点击事件的处理函数。
calibration_win_cast	转换为 calibration_win 对象 (供脚本语言使用)。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
calibration_win_get_widget_vtable	获取 calibration_win 虚表。

2. 示例

- 在 xml 中，使用”calibration_win”标签创建电阻屏校准窗口，代码如下：

```
<!--awtk/demos/assets/default/raw/ui/calibration_win.xml -->
<calibration_win name="cali" w="100%" h="100%" text="Please click the center of␣
↪cross">
</calibration_win>
```

- 在 C 代码中，使用 calibration_win_create 创建电阻屏校准窗口，代码如下：

```
widget_t* win = calibration_win_create(NULL, 0, 0, 320, 480);
```

4.4.6 overlay

overlay 窗口。overlay 窗口有点类似于非模态的 dialog，但是它位置和大小是完全自由的，窗口管理器不会对它做任何限制。如果 overlay 窗口有透明或半透效果，则不支持窗口动画，但可以通过移动窗口位置来实现类似动画的效果。overlay_t 是 window_base_t 的子类控件，window_base_t 的函数均适用于 overlay_t 控件。

1. 函数

overlay 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
overlay_create	创建 overlay 对象
overlay_set_click_through	设置是否启用点击穿透。
overlay_set_always_on_top	设置是否总是在最上面。
overlay_set_modeless	设置是否非模态窗口模式。
overlay_cast	转换为 overlay 对象 (供脚本语言使用)。
overlay_get_widget_vtable	获取 overlay 虚表。

2. 属性

overlay 还提供了下面属性，详见下表：

属性名称	类型	说明
click_through	bool_t	点击穿透。点击没有子控件的位置，是否穿透到底层窗口。
always_on_top	bool_t	是否总在最上面。
modeless	bool_t	非模态窗口。

3. 示例

- 在 xml 中，使用”overlay”标签创建 overlay 控件，代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/overlay.xml -->
<overlay text="overlay" x="0" y="0" w="100%" h="100%">
  <svg image="ball" x="c" y="m" w="200" h="200"/>
  <button name="close" image="ball" x="c" y="m:150" w="80" h="30" text="Close"/>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
</overlay>
```

4.4.7 popup

popup 弹出窗口。弹出窗口是一种特殊的窗口，大小和位置可以自由设置，主要用来实现右键菜单和 combo_box 的下列列表等功能。popup_t 是 window_base_t 的子类控件，window_base_t 的函数均适用于 popup_t 控件。

1. 函数

popup 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
popup_create	创建 popup 对象。
popup_cast	转换为 popup 对象 (供脚本语言使用)。
popup_set_close_when_click	设置点击时是否关闭窗口。
popup_set_close_when_click_outside	设置点击窗口外部时是否关闭窗口。
popup_set_close_when_timeout	设置超时关闭时间 (ms)。
popup_get_widget_vtable	获取 popup 虚表。

2. 属性

popup 还提供了下面属性，详见下表：

属性名称	类型	说明
close_when_click	bool_t	点击时是否关闭窗口。
close_when_click_outside	bool_t	点击到窗口外时是否关闭窗口。
close_when_timeout	uint32_t	超时后自动关闭窗口 (ms)。

3. 示例

- 在 xml 中，使用”popup”标签创建 popup 控件，代码如下：

```
<popup close_when_click_outside="true" x="c" y="m" w="80%" h="80" >
  <list_view x="0"  y="0" w="100%" h="100%" item_height="30">
    <scroll_view name="view" x="0"  y="0" w="-12" h="100%">
      <combo_box_item tr_text="english"/>
      <combo_box_item tr_text="chinese" />
    </scroll_view>
    <scroll_bar_d name="bar" x="right" y="0" w="12" h="100%" value="0"/>
  </list_view>
</popup>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.4.8 component

可复用的 UI 组件。Component 严格意义上来说并不是一个窗口，而是一段通用的 UI 片段，可以在不同的窗口上使用，或者在一个窗口上复用多次，主要用来解决 UI 复用的问题。

注: Component 是一个独立的 xml 文件，通常用 view 控件作为根节点容器控件，可以往这个 view 控件中添加各种类型子控件组合成组件。

1. 示例

- 在 xml 中，使用”view”标签创建 Component，代码如下：

```
<!-- component_name.xml -->
<view name="view" x="0" y="0" w="100" h="100">
  <button name="button" x="0" y="36" w="100" h="36" text="Button"/>
</view>
```

注: 使用 AWTK Designer 可以在”新建窗口”页面中直接创建 Component 类型的界面。

- 在窗口 UI 中，可以通过 Component 界面的文件名直接加载，代码如下：

```
<!-- home_page.xml -->
<window name="home_page">
  <?include filename="component_name.xml" ?>
</window>
```

- 在 C 代码中动态加载 Component

在 C 代码中调用 `ui_loader_load_widget_with_parent()` 函数即可动态加载 Component，接口声明如下：

```
/**
 * @method ui_loader_load_widget_with_parent
 *
 * 从指定的资源加载widget对象，并为wid-
get对象指定父控件对象，通常用于加载非窗口的widget。
 *
 * @param {const char*} name 资源名。
 * @param {widget_t*} parent 父控件对象。
 *
 * @return {widget_t*} 返回widget对象。
 *
 */
widget_t* ui_loader_load_widget_with_parent(const char* name, widget_t* parent);
```

动态加载 Component 时，需要传入资源文件名和父控件，比如此处将上文中创建的 Component 加载到某个窗口对象中，代码如下：

```
ret_t home_page_init(widget_t* win, void* ctx) {
  /* 将 win 窗口作为父控件加载组件，并获取组件对象 component */
  widget_t* component = ui_loader_load_widget_with_parent("component_name", win);

  /* 初始化 component 对象 */
  ...
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  return RET_OK;
}
```

注: 无论是在窗口 UI 中加载 Component，还是在代码中加载 Component，加载完毕后都需要对其进行初始化操作。

4.5 控件的属性

在使用 XML 编写 UI 界面的时候，控件包含的属性一般包括两方面：父控件的属性和本身的属性。例如，button_t 是 widget_t 的子类控件，widget_t 的属性列表详见 widget 一节的属性小节，button_t 的属性列表详见 button 一节的属性小节。

在以下代码中 button 控件的 name、style、x、y 等属性来自于其父控件 widget_t，而 enable_long_press 属性则来自于自身。

```
<!-- awtk/demos/assets/default/raw/ui/button.xml -->
<window anim_hint="htranslate">
  <button name="open:dialog1" style="round" enable_long_press="true"
    x="c" y="10" w="128" h="30" text="Long Press"/>
  <button style="red_btn" x="c" y="50" w="128" h="30" text="Text"/>
  <button style="icon" x="c" y="90" w="128" h="30" text="Fullscreen" name=
↪"fullscreen"/>
  <button x="c" y="130" w="128" h="30">
    <label y="0" x="0" w="100%" h="100%" text="Text"/>
    <image image="earth" draw_type="icon" y="0" x="right" w="30" h="100%"/>
  </button>

  <button style="icon" x="c" y="170" w="60" h="60" text="Text"/>
  <button style="icon" x="c" y="240" w="60" h="60"/>
  <button style="image" x="c" y="310" w="30" h="30"/>

  <button name="close" x="center" y="bottom:10" w="25%" h="30" text="Close"/>
</window>
```

在 XML 中设置了控件的 name 属性，在 C 代码中就可以通过调用 widget_lookup 函数获取该控件的对象指针，进一步就可以获取控件对应的属性值了。如，获取 name=”open:dialog1” 按钮的 x 坐标：

```
widget_t* btn = widget_lookup(win, "open:dialog1", TRUE);
int32_t x = btn->x;
```

在上面的代码中，由于 widget_lookup 的返回类型是 widget_t*，且 button_t 是 widget_t 的子类控件，所以可以直接通过 btn->x 的方式获取 x 坐标。如果要获取 button_t 中的 enable_long_press 属性值，则需要将 widget_t 的类型转换为 button_t，如：

```
button_t* button = BUTTON(btn);
bool_t enable_long_press = button->enable_long_press;
```

在上面的代码中 BUTTON 是 AWTK 中定义的宏，对于其他的控件也有类似的宏，如：check_button_t 对应的宏为 CHECK_BUTTON，在代码中直接使用即可。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.6 基本控件

常见的基本控件详见下表：

控件名称	说明
button	按钮控件
label	文本控件
edit	单行编辑器控件
image	图片控件
image_base	图片控件基类（一般不直接使用）
spin_box	数值编辑器控件
combo_box	下拉列表控件
combo_box_item	下拉列表项控件（一般不直接使用）
color_tile	色块控件
dialog_title	对话框标题控件
dialog_client	对话框客户区控件
slider	滑块控件
progress_bar	进度条控件
tab_control	标签控件
tab_button	标签按钮控件
check_button	勾选控件
radio_button	单选控件
dragger	dragger 控件，可在指定范围内拖拽
digit_clock	数字时钟控件

4.6.1 button

按钮控件。点击按钮之后会触发 EVT_CLICK 事件，注册 EVT_CLICK 事件执行特定操作。按钮控件也可以作为容器使用，使用图片和文本作为其子控件，可以实现很多有趣的效果。button_t 是 widget_t 的子类控件，widget_t 的函数均适用于 button_t 控件。

1. 函数

button 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
button_create	创建 button 对象
button_cast	转换为 button 对象 (供脚本语言使用)。
button_set_repeat	设置触发 EVT_CLICK 事件的时间间隔。为 0 则不重复触发 EVT_CLICK 事件。
button_set_long_press_time	设置触发长按事件的时间。
button_set_enable_long_press	设置是否启用长按事件。
button_set_enable_preview	设置是否启用预览。
button_get_widget_vtable	获取 button 虚表。

2. 属性

button 还提供了下面属性，详见下表：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

属性名称	类型	说明
repeat	int32_t	重复触发 EVT_CLICK 事件的时间间隔。
enable_long_press	bool_t	是否启用长按事件，为 true 时才触发长按事件。
enable_preview	bool_t	是否启用预览 (主要用于软键盘)。
long_press_time	uint32_t	触发长按事件的时间 (ms)
pressed	bool_t	当前是否按下。

3. 样式

button 可设置的样式，详见下表：

样式属性	说明
bg_color	背景颜色
font_name	字体名称
font_size	字体大小
text_color	文本颜色
text_align_h	文本水平对齐的方式（取值：center|left|right）
text_align_v	文本垂直对齐的方式（取值：middle|top|bottom）
border_color	边框颜色
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称
bg_image_draw_type	背景图片的显示方式（取值：详见 image 控件一节）
icon	图标的名称
spacer	间距
margin	边距
margin_left	左边距
margin_right	右边距
margin_top	顶边距
margin_bottom	底边距
icon_at	图标的位置（取值：top|bottom|left|right）
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

例如，使用以下代码中的”default”样式，效果详见下图。

```
<!-- res/assets/default/raw/styles/default.xml -->
<button>
  <style name="default" border_color="#a0a0a0"  text_color="black" font_size="36" >
    <normal     bg_color="green" />
    <pressed     bg_color="red" x_offset="1" y_offset="1"/>
    <over       bg_color="yellow" />
    <disable    bg_color="gray" text_color="#d0d0d0" />
  </style>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  <style name="round" border_color="#a0a0a0"  text_color="black" round_radius="8">
    <normal     bg_color="#f0f0f0" />
    <pressed    bg_color="#c0c0c0" x_offset="1" y_offset="1"/>
    <over       bg_color="#e0e0e0" />
    <disable    bg_color="gray" text_color="#d0d0d0" />
  </style>
  <style name="icon" icon="earth" border_color="#a0a0a0"  text_color="black">
    <normal     bg_color="#f0f0f0" />
    <pressed    bg_color="#c0c0c0" x_offset="3" y_offset="3" />
    <over       bg_color="#e0e0e0" />
    <disable    bg_color="gray" text_color="#d0d0d0" />
  </style>
  <style name="red_btn" bg_image_draw_type="patch3_x" text_color="blue">
    <normal   bg_image="red_btn_n" />
    <pressed  bg_image="red_btn_p" x_offset="3" y_offset="3"/>
    <over     bg_image="red_btn_o" />
  </style>
</button>
```



图 4.5　button 样式示例

4. 状态

button 在样式文件中可以设置的状态，详见下表：

状态	说明
normal	正常状态
pressed	指针按下状态
over	指针悬浮状态
disable	禁用状态
focused	聚焦状态

5. 事件

button 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_CLICK	pointer_event_t	点击事件。
EVT_LONG_PRESS	pointer_event_t	长按事件。

6. 示例

- 在 xml 中，使用”button”标签创建按钮控件，代码如下，其中的样式详见上文样式一节。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
<window anim_hint="htranslate">
  <button style="round" x="c" y="10" w="128" h="30" text="Text"/>
  <button style="red_btn" x="c" y="50" w="128" h="30" text="Text"/>
  <button style="icon" x="c" y="90" w="128" h="30" text="Text"/>
</window>
```

运行效果如下图所示：

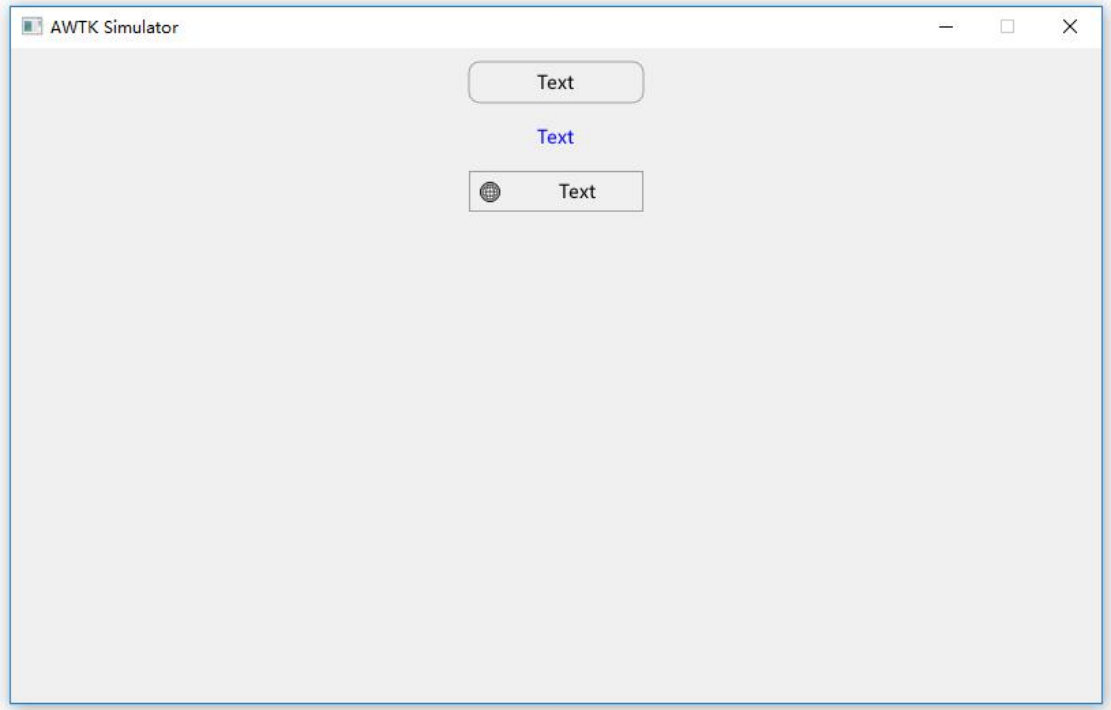


图 4.6　在 xml 中创建 button 的效果

- 在 C 代码中，使用函数 button_create 创建按钮控件，代码如下，其中的样式详见上文样式一节。

```
ret_t application_init() {
  widget_t* ok;
  widget_t* cancel;
  widget_t* win = window_create(NULL, 0, 0, 0, 0);

  ok = button_create(win, 10, 5, 80, 30);
  widget_set_text(ok, L"Inc");

  cancel = button_create(win, 100, 5, 80, 30);
  widget_set_text(cancel, L"Dec");

  return RET_OK;
}
```

创建之后，需要用 widget_set_text 或 widget_set_text_utf8 设置文本内容。运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

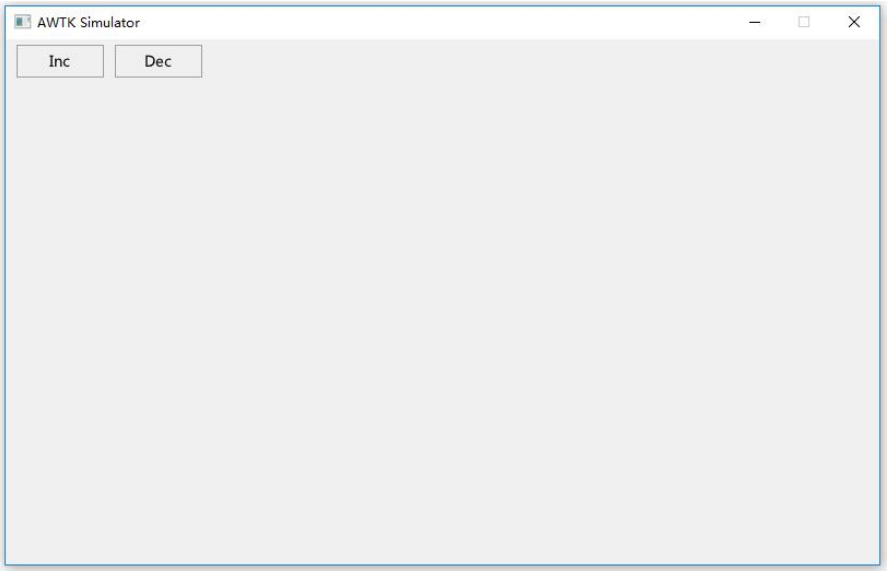


图 4.7　在 C 代码中创建 button 的效果

4.6.2　label

文本控件。用于显示一行或多行文本。文本控件设置 line_wrap 属性为 true 后，可根据文本的长度自动换行，当文本内容包含换行符时也会换行。label_t 是 widget_t 的子类控件，widget_t 的函数均适用于 label_t 控件。

1. 函数

label 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
label_create	创建 label 对象
label_set_length	设置显示字符的个数 (小余 0 时全部显示)。
label_set_max_w	设置 max_w。
label_set_line_wrap	设置是否自动换行。
label_set_word_wrap	设置是否允许整个单词换行。(需要开启自动换行才有效果)
label_resize_to_content	根据文本内容调节控件大小。
label_cast	转换为 label 对象 (供脚本语言使用)。
label_get_widget_vtable	获取 label 虚表。

2. 属性

label 还提供了下面属性，详见下表：

属性名称	类型	说明
length	int32_t	显示字符的个数 (小余 0 时全部显示)。
line_wrap	bool_t	是否自动换行 (默认 FALSE)。
word_wrap	bool_t	是否允许整个单词换行 (默认 FALSE)。
max_w	int32_t	当 auto_adjust_size 为 TRUE 时，用于控制控件的最大宽度，超出该宽度后才自动换行。

3. 样式

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

label 可设置的样式，详见 button 一节的样式小节，示例中使用到的样式，代码如下：

```
<!-- res/assets/default/raw/styles/default.xml -->
<label>
  <style name="default">
    <normal text_color="black" />
  </style>

  <style name="left">
    <normal text_color="red" text_align_h="left" border_color="#a0a0a0" margin="4"␣
↪font_size="24"/>
  </style>

  <style name="center">
    <normal text_color="green" text_align_h="center" border_color="#a0a0a0" font_
↪size="24"/>
  </style>

  <style name="right">
    <normal text_color="blue" text_align_h="right" border_color="#a0a0a0" margin="4
↪" font_size="24"/>
  </style>
</label>
```

4. 状态

label 在样式文件中可以设置的状态，详见下表：

状态	说明
normal	正常状态
disable	禁用状态
focused	聚焦状态

5. 示例

- 在 xml 中，使用”label”标签创建文本控件，代码如下，其中的样式详见上文样式一节。

```
<window anim_hint="htranslate">
  <row x="0" y="80" w="100%" h="30" children_layout="default(r=1,c=3,xm=2,s=10)">
    <label style="left" name="left" text="Left"/>
    <label style="center" name="center" text="Center"/>
    <label style="right" name="right" text="Right"/>
  </row>
</window>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

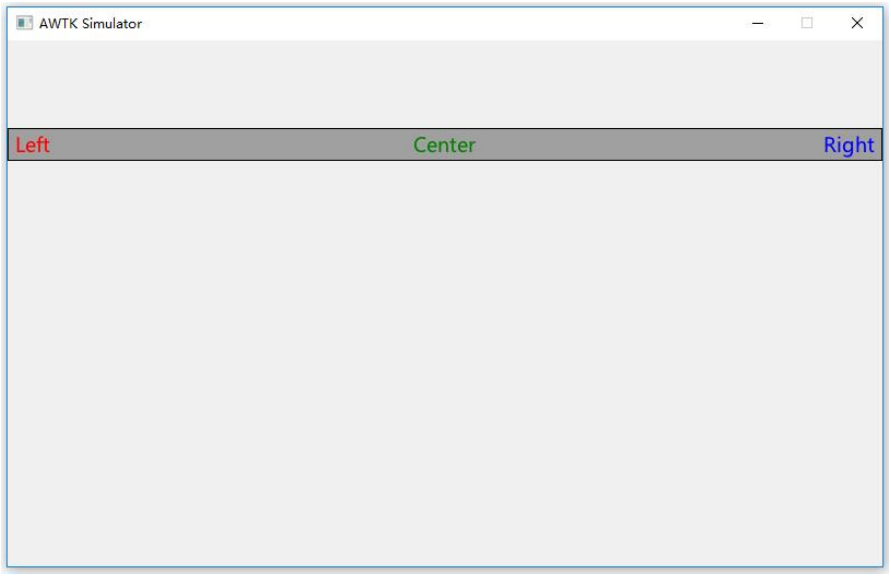


图 4.8　在 xml 创建 label 的效果

- 在 C 代码中，使用函数 label_create 创建文本控件，代码如下，其中的样式详见上文样式一节。

```
ret_t application_init() {
    widget_t* label;
    widget_t* win = window_create(NULL, 0, 0, 0, 0);
    label = label_create(win, 190, 40, 80, 30);
    widget_set_text(label, L"Right");
    widget_use_style(label, "right");
    return RET_OK;
}
```

创建之后，需要用 widget_set_text 或 widget_set_text_utf8 设置文本内容。运行效果如下图所示。



图 4.9　在 C 代码中创建 label 的效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.6.3 edit

单行编辑器控件。在基于 SDL 的平台，单行编辑器控件使用平台原生的输入法，对于嵌入式平台使用内置的输入法。在使用内置的输入法时，软键盘由输入类型决定，开发者可以自定义软键盘的界面。edit_t 是 widget_t 的子类控件，widget_t 的函数均适用于 edit_t 控件。

edit_t 控件中光标的颜色取的是样式文件中该控件的 text_color；如果没有设置 text_color，则默认是黑色。

edit_t 本身可以做为容器，放入按钮等控件。有几个特殊的子控件：

- 名为”clear”的按钮。点击时清除编辑器中的内容
- 名为”inc”的按钮。点击时增加编辑器的值，用于实现类似于 spinbox 的功能
- 名为”dec”的按钮。点击时减少编辑器的值，用于实现类似于 spinbox 的功能
- 名为”visible”的复选框。勾选时显示密码，反之不显示密码

例如，在以下代码中 edit 控件包含了 inc 和 dec 按钮，效果如下图所示。其中 edit 使用的样式详见下文样式一节。

```
<window anim_hint="htranslate" >
  <list_view x="0"  y="0" w="100%" h="-50" item_height="36" auto_hide_scroll_bar=
↪"true">
    <scroll_view name="view" x="0"  y="0" w="-12" h="100%">
     <list_item style="empty" children_layout="default(r=1,c=0,ym=1)">
      <label w="30%" text="Age"/>
      <edit w="70%" right_margin="16" tips="uint(0-150)" input_type="uint" min="0"␣
↪max="150" step="1">
        <button name="inc" repeat="300" style="spinbox_up" x="right" y="0" w="15"␣
↪h="50%" />
        <button name="dec" repeat="300" style="spinbox_down" x="right" y="bottom"␣
↪w="15" h="50%" />
      </edit>
     </list_item>
    </scroll_view>
    <scroll_bar_d name="bar" x="right" y="0" w="12" h="100%" value="0"/>
  </list_view>
  <button name="close" x="center" y="bottom:10" w="25%" h="30" text="Close"/>
</window>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

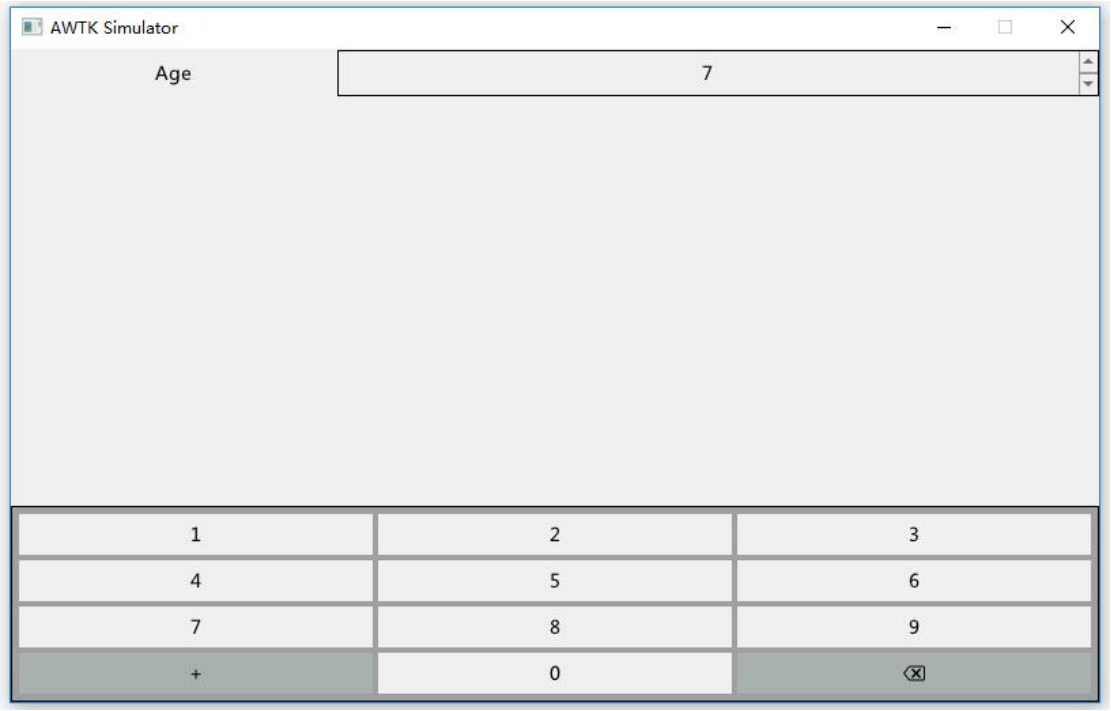


图 4.10　edit 包含 inc 和 dec 按钮

1. 函数

edit 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
edit_create	创建 edit 对象
edit_cast	转换为 edit 对象 (供脚本语言使用)。
edit_get_int	获取 int 类型的值。
edit_get_double	获取 double 类型的值。
edit_set_int	设置 int 类型的值。
edit_set_double	设置 double 类型的值。
edit_set_double_ex	设置 double 类型的值。
edit_set_text_limit	设置为文本输入及其长度限制，不允许输入超过 max 个字符，少于 min 个字符时进入 error 状态。
edit_set_int_limit	设置为整数输入及取值范围。
edit_set_float_limit	设置为浮点数输入及取值范围。
edit_set_readonly	设置编辑器是否为只读。
edit_set_cancelable	设置编辑器是否为可撤销修改。
edit_set_auto_fix	设置编辑器是否为自动改正。
edit_set_select_none_when_focused	设置编辑器是否在获得焦点时不选中文本。
edit_set_open_im_when_focused	设置编辑器是否在获得焦点时打开输入法。
edit_set_close_im_when_blured	设置编辑器是否在失去焦点时关闭输入法。
edit_set_input_type	设置编辑器的输入类型。
edit_set_action_text	设置软键盘上 action 按钮的文本。
edit_set_tips	设置编辑器的输入提示。
edit_set_tr_tips	获取翻译之后的文本，然后调用 edit_set_tips。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
edit_set_keyboard	设置自定义软键盘名称。
edit_set_password_visible	当编辑器输入类型为密码时，设置密码是否可见。
edit_set_focus	设置为焦点。
edit_set_cursor	设置输入框的光标位置。
edit_get_cursor	获取输入框的光标位置。
edit_set_is_valid_char	设置输入字符检查函数。
edit_set_is_valid_value	设置输入内容检查函数。
edit_set_fix_value	设置修正输入内容的回调函数。
edit_set_inc_value	设置增加值的回调函数。
edit_set_dec_value	设置减少值的回调函数。
edit_set_pre_input	设置预输入处理的回调函数。
edit_set_pre_delete	设置预删除处理的回调函数。
edit_set_select	选择指定范围的文本。
edit_get_selected_text	获取选中的文本。
edit_get_widget_vtable	获取 edit 虚表。

2. 属性

edit 还提供了下面属性，详见下表：

属性名称	类型	说明
tips	char*	输入提示。
tr_tips	char*	保存用于翻译的提示信息。
action_text	char*	软键盘上 action 按钮的文本。内置取值有：
keyboard	char*	自定义软键盘名称。AWTK 优先查找 keyboard 属性设置的键盘文件名（该键盘的 XML 文件需要在 default\raw\ui 目录下存在），如果没有指定 keyboard，就找 input_type 设置的键盘类型。如果指定为空字符串，则表示不需要软键盘。
min	double	最小值或最小长度。
max	double	最大值或最大长度。
step	double	步长。
input_type	input_type_t	输入类型。
readonly	bool_t	编辑器是否为只读。
password_visible	bool_t	密码是否可见。
auto_fix	bool_t	输入无效时，是否自动改正。
select_none_when_focused	bool_t	获得焦点时不选中文本。
open_im_when_focused	bool_t	获得焦点时打开输入法。
close_im_when_blured	bool_t	是否在失去焦点时关闭输入法 (默认是)。
cancelable	bool_t	是否支持撤销编辑。如果为 TRUE，在失去焦点之前可以撤销所有修改 (恢复获得焦点之前的内容)。

3. 样式

edit 可设置的样式，详见下表：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

样式属性	说明
bg_color	背景颜色
fg_color	前景颜色
font_name	字体名称
font_size	字体大小
text_color	文本颜色
tips_text_color	提示文本颜色
text_align_h	文本水平对齐的方式（取值：center|left|right）
text_align_v	文本垂直对齐的方式（取值：middle|top|bottom）
border_color	边框颜色
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称
bg_image_draw_type	背景图片的显示方式（取值：详见 image 控件一节）
fg_image	前景图片的名称
fg_image_draw_type	前景图片的显示方式（取值：详见 image 控件一节）
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
selected_bg_color	编辑器（包括 edit 和 mledit 控件）中选中区域的背景颜色
selected_fg_color	编辑器（包括 edit 和 mledit 控件）中选中区域的前景颜色
selected_text_color	编辑器（包括 edit 和 mledit 控件）中选中区域的文本颜色
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

例如，使用以下代码设置的样式，效果如下图所示。

```
<!-- res/assets/default/raw/styles/default.xml -->
<edit selected_fg_color="red" selected_text_color="blue" selected_bg_color="yellow
↪">
  <style name="default" border_color="#a0a0a0"  text_color="black" text_align_h=
↪"center" font_size="40">
    <normal      bg_color="#f0f0f0" />
    <focused     bg_color="orange" border_color="black"/>
    <disable     bg_color="gray" text_color="#d0d0d0" />
    <error       bg_color="#f0f0f0" text_color="red" />
    <empty       bg_color="green" text_color="#a0a0a0" />
    <empty_focus bg_color="black" text_color="#a0a0a0" border_color="black"/>
  </style>
  <style name="number" border_color="#a0a0a0"  text_color="black" text_align_h=
↪"right">
    <normal      bg_color="#f0f0f0" />
    <focused     bg_color="#f0f0f0" border_color="black"/>
    <disable     bg_color="gray" text_color="#d0d0d0" />
    <error       bg_color="#f0f0f0" text_color="red" />
    <empty       bg_color="#f0f0f0" text_color="#a0a0a0" />
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  </style>
</edit>
```

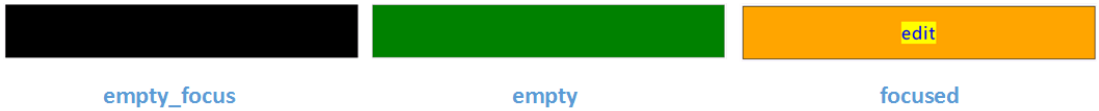


图 4.11　edit 样式示例

4. 状态

edit 在样式文件中可以设置的状态，详见下表：

状态	说明
normal	正常状态
pressed	指针按下状态
error	编辑器错误
empty	编辑器无内容状态
over	指针悬浮状态
empty_focus	编辑器无内容同时聚焦的状态
empty_over	编辑器无内容同时指针悬浮状态
disable	禁用状态
focused	聚焦状态

5. 事件

edit 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_CHANGING	value_change_event_t	文本正在改变事件 (编辑中)。
EVT_VALUE_CHANGED	value_change_event_t	文本改变事件 (编辑完成或设置文本时触发)。
EVT_IM_ACTION	event_t	软键盘 Action 点击事件。

6. 示例

- 在 xml 中，使用”edit”标签创建单行编辑器控件，代码如下，其中 edit 使用的样式详见上文样式一节。

```
<window anim_hint="htranslate" >
  <list_view x="0"  y="0" w="100%" h="-50" item_height="36" auto_hide_scroll_bar=
↪"true">
    <scroll_view name="view" x="0"  y="0" w="-12" h="100%">
     <list_item style="empty" children_layout="default(r=1,c=0,ym=1)">
      <label w="30%" text="Age"/>
      <edit w="70%" right_margin="16" tips="uint(0-150) auto_fix" input_type="uint"␣
↪min="0" max="150"
          step="1" auto_fix="true" style="number">
        <button name="inc" repeat="300" style="spinbox_up" x="right" y="0" w="15"␣
↪h="50%" />
        <button name="dec" repeat="300" style="spinbox_down" x="right" y="bottom"␣
↪w="15" h="50%" />
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
      </edit>
    </list_item>
  </list_view>
  <button name="close" x="center" y="bottom:10" w="25%" h="30" text="Close"/>
</window>
```

运行效果如下图所示：

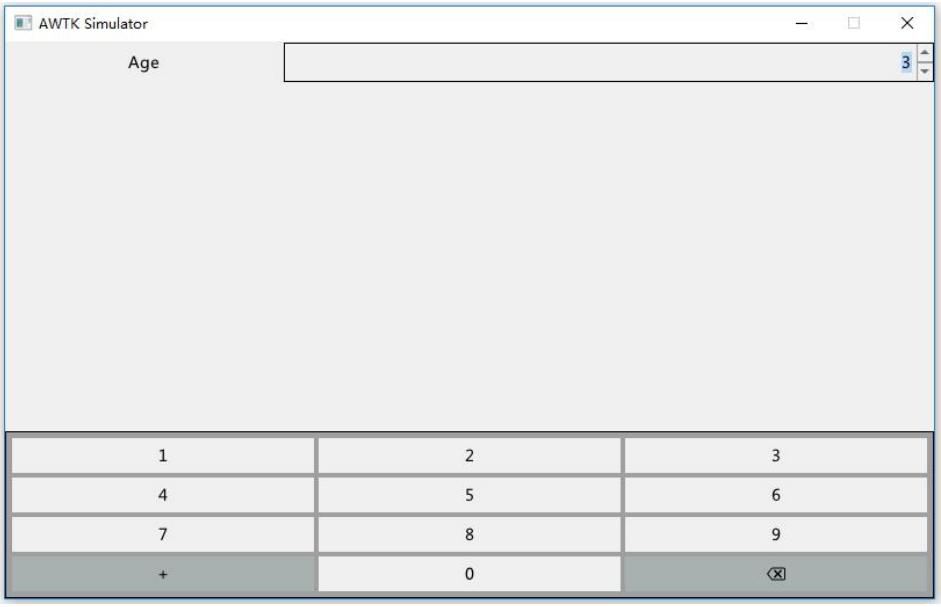


图 4.12　在 xml 中创建 edit 的效果

- 在 C 代码中，使用函数 edit_create 创建单行编辑器控件，代码如下，其中 edit 使用的样式详见上文样式一节。

```
ret_t application_init(void) {
  widget_t* win = window_create(NULL, 0, 0, 0, 0);
  widget_t* label = label_create(win, 0, 0, 0, 0);
  widget_t* edit = edit_create(win, 0, 0, 0, 0);
  widget_set_text(label, L"hello awtk!");
  widget_set_self_layout_params(label, "0", "middle", "30%", "30");
  widget_set_self_layout_params(edit, "30%", "middle", "60%", "30");
  widget_layout(win);
  return RET_OK;
}
```

创建之后，需要用 widget_set_text 或 widget_set_text_utf8 设置文本内容。运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

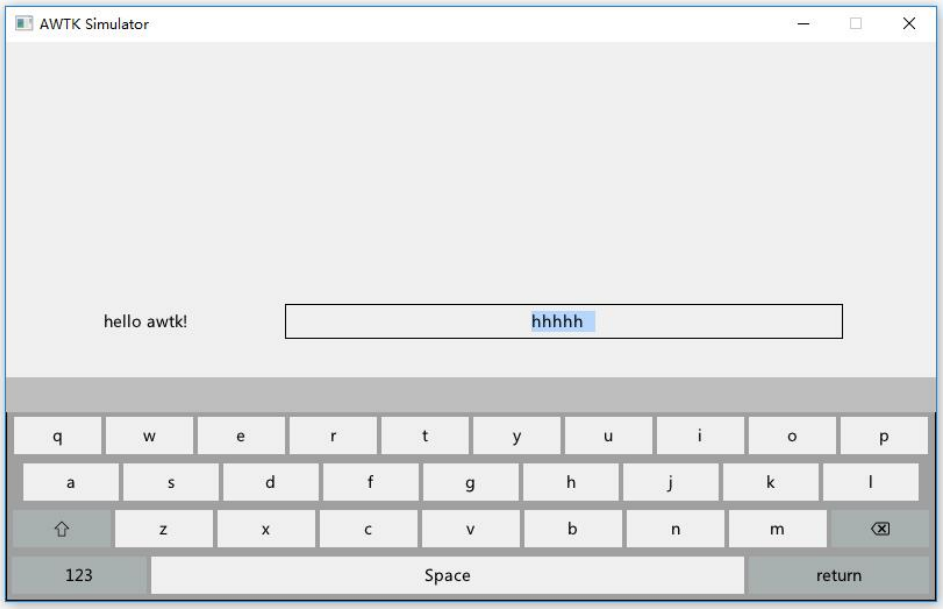


图 4.13　在 C 代码中创建 edit 的效果

4.6.4　image

图片控件，用来显示一张静态图片，目前支持 bmp/png/jpg 等格式，其中：

- 如果要显示 gif 文件，请用 gif_image
- 如果要显示 svg 文件，请用 svg_image
- 如果需要支持勾选效果，请设置 selectable 属性
- 如果需要支持点击效果，请设置 clickable 属性

image_t 是 image_base_t 的子类控件，image_base_t 的函数均适用于 image_t 控件。关于 selectable 属性的示例，请看下图。

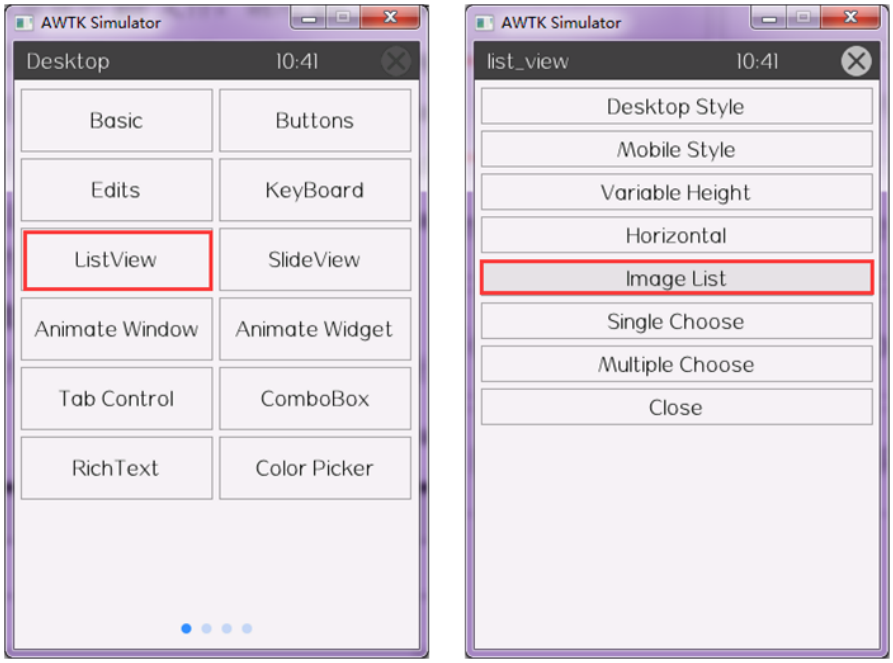


图 4.14　selectable 属性

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

1. 函数

image 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
image_create	创建 image 对象
icon_create	创建 icon 对象
image_set_draw_type	设置图片的绘制方式。
image_cast	转换为 image 对象 (供脚本语言使用)。
image_get_widget_vtable	获取 image 虚表。

2. 属性

image 还提供了下面属性，详见下表：

属性名称	类型	说明
draw_type	image_draw_type_t	图片的绘制方式 (仅在没有旋转和缩放时生效)。

draw_type 可取的值，详见下表：

draw_type 取值	说明
default	缺省显示。将图片按原大小显示在目标矩形的左上角
icon	图标显示。同居中显示，但会根据屏幕密度调整大小
center	居中显示。将图片按原大小显示在目标矩形的中央
scale	缩放显示。将图片缩放至目标矩形的大小（不保证宽高成比例）
scale_auto	自动缩放显示。将图片缩放至目标矩形宽度或高度（选取最小的比例），并居中显示
scale_w	宽度缩放显示。将图片缩放至目标矩形的宽度，高度按此比例进行缩放，超出部分不显示
scale_h	高度缩放显示。将图片缩放至目标矩形的高度，宽度按此比例进行缩放，超出部分不显示
repeat	平铺显示
repeat_x	水平方向平铺显示，垂直方向缩放
repeat_y	垂直方向平铺显示，水平方向缩放
repeat_y_inverse	垂直方向平铺显示，水平方向缩放 (从底部到顶部)。
patch9	9 宫格显示
patch3_x	水平方向 3 宫格显示，垂直方向居中显示
patch3_y	垂直方向 3 宫格显示，水平方向居中显示
patch3_x_scale_y	水平方向 3 宫格显示，垂直方向缩放显示
patch3_y_scale_x	垂直方向 3 宫格显示，水平方向缩放显示

3. 样式

image 可设置的样式，详见 button 一节的样式小节。示例中使用的样式，代码如下：

```
<!-- res/assets/default/raw/styles/default.xml -->
<image>
  <style name="border">
    <normal border_color="#000000" bg_color="#e0e0e0" text_color="black"/>
  </style>
</image>
```

4. 状态

image 在样式文件中可以设置的状态，详见下表：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

状态	说明
normal	正常状态
pressed	指针按下状态
selected	选中状态
disable	禁用状态
focused	聚焦状态

5. 示例

- 在 xml 中，使用”image”标签创建图片控件，代码如下，其中 image=”1”中的 1 是图片名，即 1.jpg。程序中 image 使用的样式，详见上文样式一节。

```
<window anim_hint="htranslate" >
  <list_view x="0"  y="0" w="100%" h="-50" item_height="128" auto_hide_scroll_bar=
↪"true">
    <scroll_view name="view" x="0"  y="0" w="-12" h="100%">
     <list_item style="empty" children_layout="default(r=1,c=3,s=2,m=2)">
      <image style="border" image="1" draw_type="scale_auto"><label w="100%" h="100
↪%" text="auto"/>
          </image>
      <image style="border" image="2" draw_type="scale_auto"><label w="100%" h="100
↪%" text="auto"/>
          </image>
      <image style="border" image="3" draw_type="scale_auto"><label w="100%" h="100
↪%" text="auto"/>
          </image>
     </list_item>
</window>
```

运行效果如下图所示：

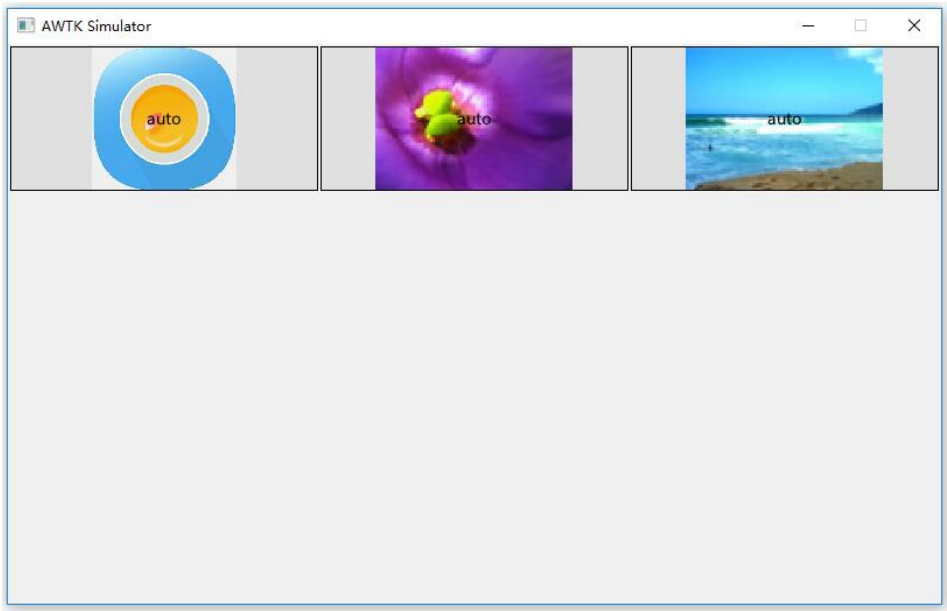


图 4.15　在 xml 中创建 image 的效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

- 在 C 代码中，使用函数 image_create 创建图片控件，代码如下：

```
ret_t application_init() {
  image = image_create(win, 10, 230, 100, 100);
  image_set_image(image, "earth");
  image_set_draw_type(image, IMAGE_DRAW_ICON);
}
```

创建之后，需要用 image_set_image 设置图片名称，用 image_set_draw_type 设置图片的绘制方式。运行效果如下图所示：

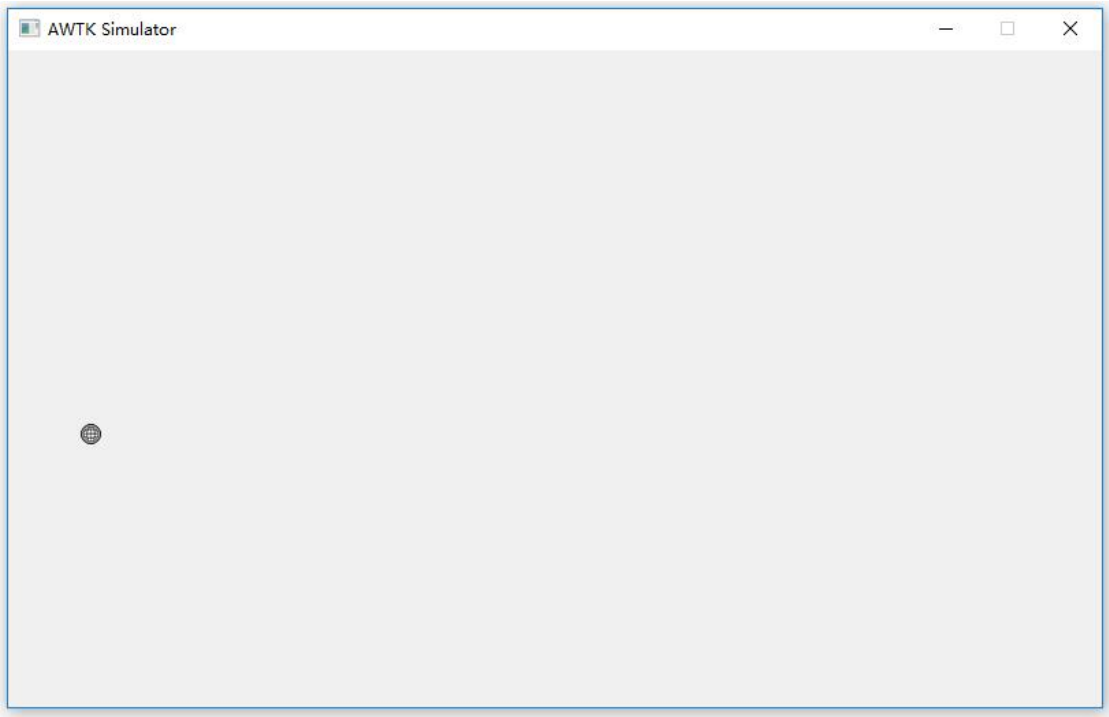


图 4.16　在 C 代码中创建 image 的效果

4.6.5　spin_box

spinbox 控件。一个特殊的数值编辑器，将 edit_t 和 button_t 进行组合，方便编辑数值。点击向上的按钮将数值增加一个 step，点击向下的按钮将数值减小一个 step。step 的值可以通过 step 属性进行设置。spin_box_t 是 edit_t 的子类控件，edit_t 的函数均适用于 spin_box_t 控件。

注：使用该控件的时候需要将 awtk/demos/assets/default/raw/images/x1 目录下：arrow_*.png 共 12 张图片复制到自己的应用目录下（如：res/assets/default/raw/images/xx），否则将不能正常显示。

1. 函数

spin_box 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
spin_box_create	创建 spin_box 对象
spin_box_cast	转换为 spin_box 对象 (供脚本语言使用)。
spin_box_set_easy_touch_mode	设置是否启用易点击模式。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
spin_box_set_button_position	设置按钮位置样式。
spin_set_repeat	设置连击的时间间隔。
spin_box_get_widget_vtable	获取 spin_box 虚表。

2. 属性

spin_box 还提供了下面属性，详见下表：

属性名称	类型	说明
easy_touch_mode	bool_t	是否启用易点击模式 (在电容屏设备上建议启用)。
button_position	char*	按钮位置样式选择，优先级高于 easy_touch_mode，各模式对应样式如下, 默认为 none。
repeat	int32_t	重复触发 EVT_CLICK 事件的时间间隔。

3. 样式

spin_box 可设置的样式，详见 edit 一节的样式小节。spin_box 的”default”样式代码如下：

```
<!-- res/assets/default/raw/styles/default.xml -->
<spin_box>
  <style name="default" border_color="#a0a0a0"  text_color="black" text_align_h=
↪"left">
    <normal     bg_color="#f0f0f0" />
    <focused    bg_color="#f0f0f0" border_color="black"/>
    <disable    bg_color="gray" text_color="#d0d0d0" />
    <error      bg_color="#f0f0f0" text_color="red" />
    <empty      bg_color="#f0f0f0" text_color="#a0a0a0" />
  </style>
</spin_box>
```

4. 示例

- 在 xml 中，使用”spin_box”标签创建 spinbox 控件，代码如下，其中使用的样式，详见上文第二小节。

```
<window anim_hint="htranslate" children_layout="default(h=30,c=1,m=2,s=2)">
  <view children_layout="default(r=1,c=0)">
    <spin_box w="70%" tips="uint(0-150) auto_fix" input_type="uint" min="0" max=
↪"150" step="1"
        auto_fix="true">
    </spin_box>
  </view>
</window>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

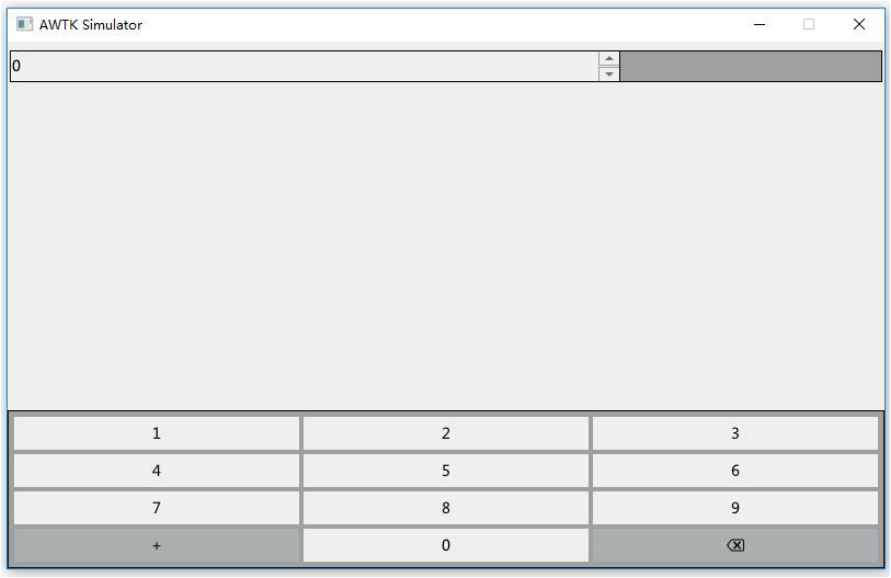


图 4.17　在 xml 中创建 spin_box 的效果

- 在 C 代码中，使用函数 spin_box_create 创建 spinbox 控件，代码如下：

```
ret_t application_init() {
  widget_t* spin_box;
  widget_t* win = window_create(NULL, 0, 0, 0, 0);

  spin_box = spin_box_create(win, 10, 10, 128, 30);
  edit_set_input_type(spin_box, INPUT_INT);

  return RET_OK;
}
```

创建之后，可以用 edit 相关函数去设置它的各种属性。运行效果如下图所示：

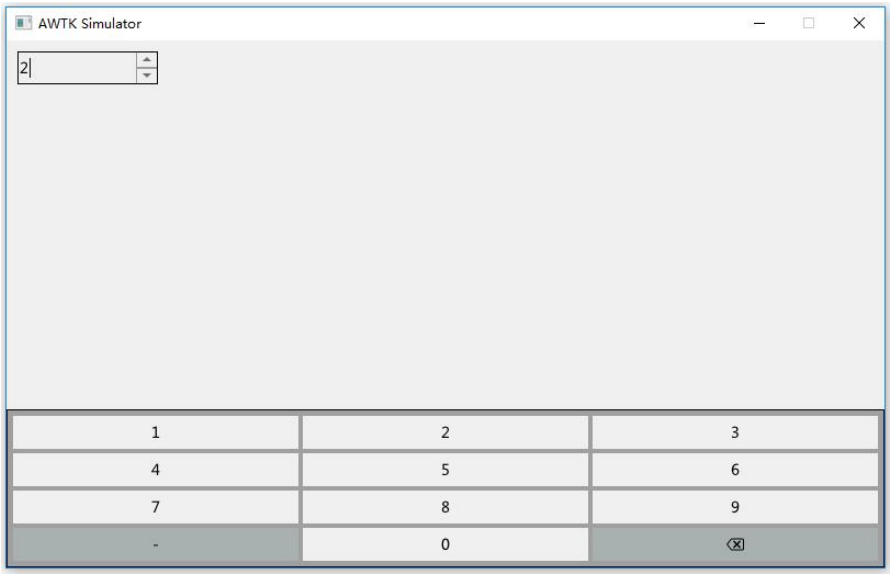


图 4.18　在 C 代码中创建 spin_box 的效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.6.6 combo_box

下拉列表控件。点击右边的按钮，可弹出一个下拉列表，从中选择一项作为当前的值。combo_box_t 是 edit_t 的子类控件，edit_t 的函数均适用于 combo_box_t 控件。另外，如果需要使用带滚动条的下拉列表控件请用 combo_box_ex_t，该控件的用法、函数、属性等都与 combo_box_t 一样。

注：使用该控件的时候需要将 awtk/demos/assets/default/raw/images/x1 目录下：arrow_down_*.png 共 3 张图片复制到自己的应用目录下（如：res/assets/default/raw/images/xx），否则将不能正常显示。

1. 函数

combo_box 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
combo_box_create	创建 combo_box 对象
combo_box_cast	转换 combo_box 对象 (供脚本语言使用)。
combo_box_set_open_window	点击按钮时可以打开 popup 窗口，本函数可设置窗口的名称。
combo_box_set_theme_of_popup	设置弹出窗口的主题。
combo_box_reset_options	重置所有选项。
combo_box_count_options	获取选项个数。
combo_box_set_selected_index	设置第 index 个选项为当前选中的选项。
combo_box_set_selected_index_by_text	根据文本设置当前选中的选项。
combo_box_set_localize_options	设置是否本地化 (翻译) 选项。
combo_box_set_value	设置值。
combo_box_set_item_height	设置 item 高度。
combo_box_append_option	追加一个选项。
combo_box_remove_option	删除选项。
combo_box_set_options	设置选项。
combo_box_set_custom_open_popup	设置自定义的打开弹出窗口的函数。
combo_box_set_on_item_click	设置弹出窗口的 item 被点击的回调函数，回调函数返回非 RET_OK，继续默认处理。
combo_box_get_option	获取第 index 个选项。
combo_box_get_value	获取 combo_box 的值。
combo_box_has_option_text	检查选项中是否存在指定的文本。
combo_box_get_text	获取 combo_box 的文本 (可能是翻译后的文本)。
combo_box_get_text_of_selected	获取 combo_box 当前选中项目的文本 (原生非翻译的文本)。
combo_box_get_widget_vtable	获取 combo_box 虚表。

2. 属性

combo_box 还提供了下面属性，详见下表：

属性名称	类型	说明
open_window	char*	为点击按钮时，要打开窗口的名称。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

属性名称	类型	说明
theme_of_popup	char*	弹出窗口的主题 (对应的 style 文件必须存在)，方便为不同 combo box 的弹出窗口指定不同的样式。
selected_index	int32_t	当前选中的选项。
value	int32_t	值。
localize_options	bool_t	是否本地化 (翻译) 选项 (缺省为 TRUE)。
options	char*	设置可选项 (冒号分隔值和文本，分号分隔选项，如:1:red;2:green;3:blue)。
item_height	int32_t	下拉选项的高度。如果 open_window 为空，则使用缺省高度。

3. 样式

combo_box 可设置的样式，详见 button 一节的样式小节。combo_box 的”default”样式，代码如下：

```
<!-- res/assets/default/raw/styles/default.xml -->
<combo_box>
  <style name="default" border_color="#a0a0a0"  text_color="black" text_align_h=
↪"left">
    <normal     bg_color="#f0f0f0" />
    <focused    bg_color="#f0f0f0" border_color="black"/>
    <disable    bg_color="gray" text_color="#d0d0d0" />
    <error      bg_color="#f0f0f0" text_color="red" />
    <empty      bg_color="#f0f0f0" text_color="#a0a0a0" />
  </style>
</combo_box>
```

4. 状态

combo_box 在样式文件中可以设置的状态，详见下表：

状态	说明
normal	正常状态
error	指针按下状态
empty	编辑器无内容状态
empty_focus	编辑器无内容同时聚焦的状态
disable	禁用状态
focused	聚焦状态

5. 事件

combo_box 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	value_change_event_t	值即将改变事件。
EVT_VALUE_CHANGED	value_change_event_t	值改变事件。

6. 示例

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

- 在 xml 中，使用”combo_box”标签创建下拉列表控件，代码如下，其中使用到的样式详见上文样式一节。

```
<window anim_hint="htranslate">
    <combo_box left_margin="6" readonly="true" x="10" y="5" w="200" h="30" text=
↪"left" options="left;center;right;"/>
    <combo_box open_window="color" readonly="true" x="10" y="50" w="200" h="30"␣
↪text="red"/>
    <combo_box readonly="true" x="10" y="bottom:5" w="200" h="30" tr_text="ok"␣
↪options="1:ok;2:cancel;"/>
    <combo_box open_window="langauge" readonly="true" x="10" y="bottom:50" w="200"␣
↪h="30" tr_text="english"/>
</window>
```

运行效果如下图所示：

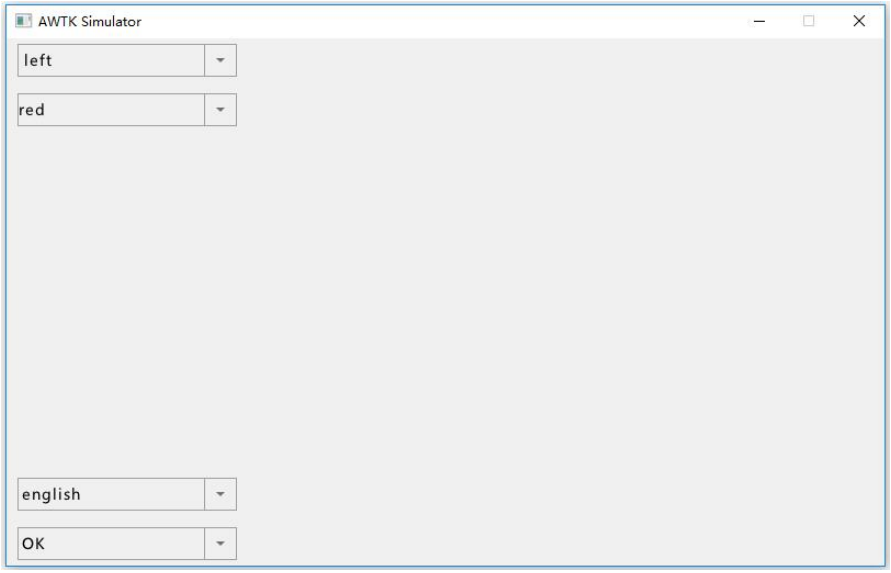


图 4.19　在 xml 中创建 combo_box 的效果

- 在 C 代码中，使用 combo_box_create 创建下拉列表控件，代码如下：

```
ret_t application_init() {
  widget_t* combo_box;
  widget_t* win = window_create(NULL, 0, 0, 0, 0);
  combo_box = combo_box_create(win, 10, 10, 128, 30);
  combo_box_set_options(combo_box, "left;center;right;");
  combo_box_set_selected_index(combo_box, 1);
  return RET_OK;
}
```

创建之后，用 combo_box_set_options 设置可选项目，用 combo_box_set_selected_index 设置缺省项。运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

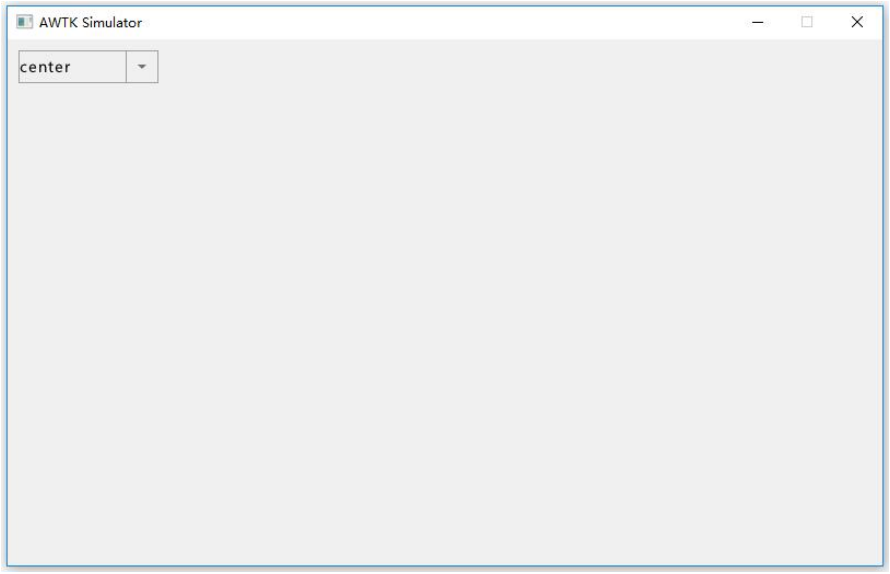


图 4.20　在 C 代码中创建 combo_box 的效果

4.6.7　combo_box_item

combo_box_item 控件，本控件仅供 combo_box 控件内部使用。combo_box_item_t 是 widget_t 的子类控件，widget_t 的函数均适用于 combo_box_item_t 控件。

1. 函数

combo_box_item 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
combo_box_item_create	创建 combo_box_item 对象
combo_box_item_cast	转换 combo_box_item 对象 (供脚本语言使用)。
combo_box_item_set_checked	设置控件是否被选中。
combo_box_item_set_value	设置控件的值。
combo_box_item_get_widget_vtable	获取 combo_box_item 虚表。

2. 属性

combo_box_item 还提供了下面属性，详见下表：

属性名称	类型	说明
value	int32_t	值。
checked	bool_t	是否被选中。

3. 事件

combo_box_item 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_CLICK	pointer_event_t	点击事件。
EVT_VALUE_WILL_CHANGE	value_change_event_t	值 (勾选状态) 即将改变事件。
EVT_VALUE_CHANGED	value_change_event_t	值 (勾选状态) 改变事件。

4. 示例

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

- 在 xml 中，使用”combo_box_item”标签创建色块控件，代码如下：

```
<!-- awtk/design/deafault/ui/language.xml -->
<popup close_when_click_outside="true" h="80" >
  <list_view x="0"  y="0" w="100%" h="100%" item_height="30">
    <scroll_view name="view" x="0"  y="0" w="-12" h="100%">
      <combo_box_item tr_text="english"/>
      <combo_box_item tr_text="chinese" />
    </scroll_view>
    <scroll_bar_d name="bar" x="right" y="0" w="12" h="100%" value="0"/>
  </list_view>
</popup>
```

4.6.8 color_tile

色块控件。用来显示一个颜色块，它通过属性而不是窗体样式来设置颜色，方便在运行时动态改变颜色。可以使用 value 属性访问背景颜色的颜色值。color_tile_t 是 widget_t 的子类控件，widget_t 的函数均适用于 color_tile_t 控件。

1. 函数

color_tile 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
color_tile_create	创建 color_tile 对象
color_tile_cast	转换为 color_tile 对象 (供脚本语言使用)。
color_tile_set_bg_color	设置背景颜色。
color_tile_set_border_color	设置边框颜色。
color_tile_set_value	设置背景颜色。
color_tile_get_bg_color	获取背景颜色。
color_tile_get_border_color	获取边框颜色。
color_tile_get_widget_vtable	获取 color_tile 虚表。

2. 属性

color_tile 还提供了下面属性，详见下表。

属性名称	类型	说明
bg_color	char*	背景颜色。
border_color	char*	边框颜色。

3. 事件

color_tile 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	value_change_event_t	值 (颜色) 即将改变事件。
EVT_VALUE_CHANGED	value_change_event_t	值 (颜色) 改变事件。

4. 示例

- 在 xml 中，使用”color_tile”标签创建色块控件，代码如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
<window>
  <color_picker x="0" y="0" w="100%" h="100%" value="orange">
    <color_component x="0" y="0" w="200" h="200" name="sv"/>
    <color_component x="210" y="0" w="20" h="200" name="h"/>
    <color_tile x="0" y="210" w="50%" h="20" name="new" bg_color="green"/>
    <color_tile x="right" y="210" w="50%" h="20" name="old" bg_color="blue"/>
  </color_picker>
</window>
```

运行效果如下图所示：

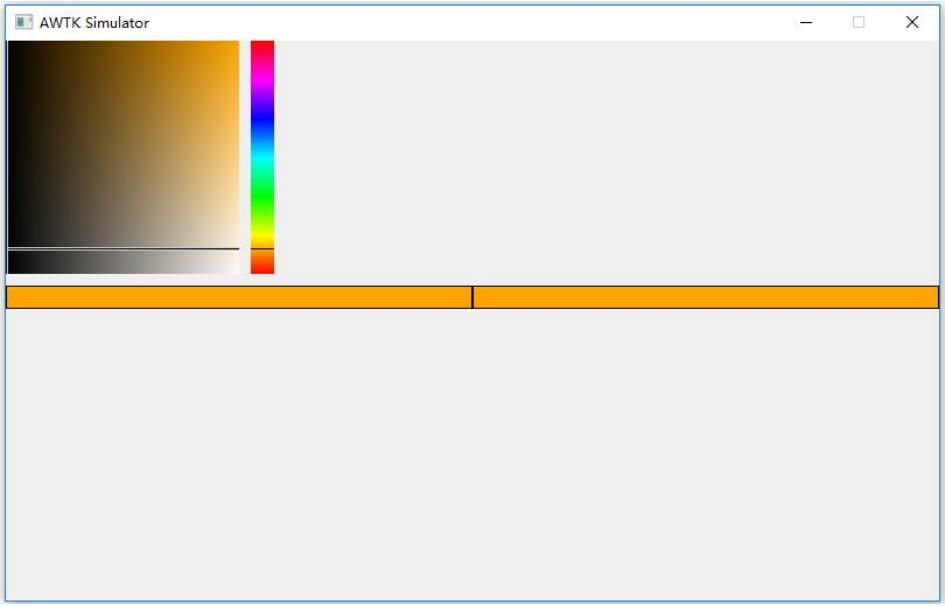


图 4.21　在 xml 中创建 color_tile 的效果

- 在 C 代码中，使用函数 color_tile_create 创建色块控件，代码如下：

```
ret_t application_init() {
  widget_t* color_tile;
  widget_t* win = window_create(NULL, 0, 0, 0, 0);

  color_tile = color_tile_create(win, 10, 10, 128, 30);
  color_tile_set_bg_color(color_tile, "red");

  return RET_OK;
}
```

创建之后，用 color_tile_set_bg_color 设置背景颜色。运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

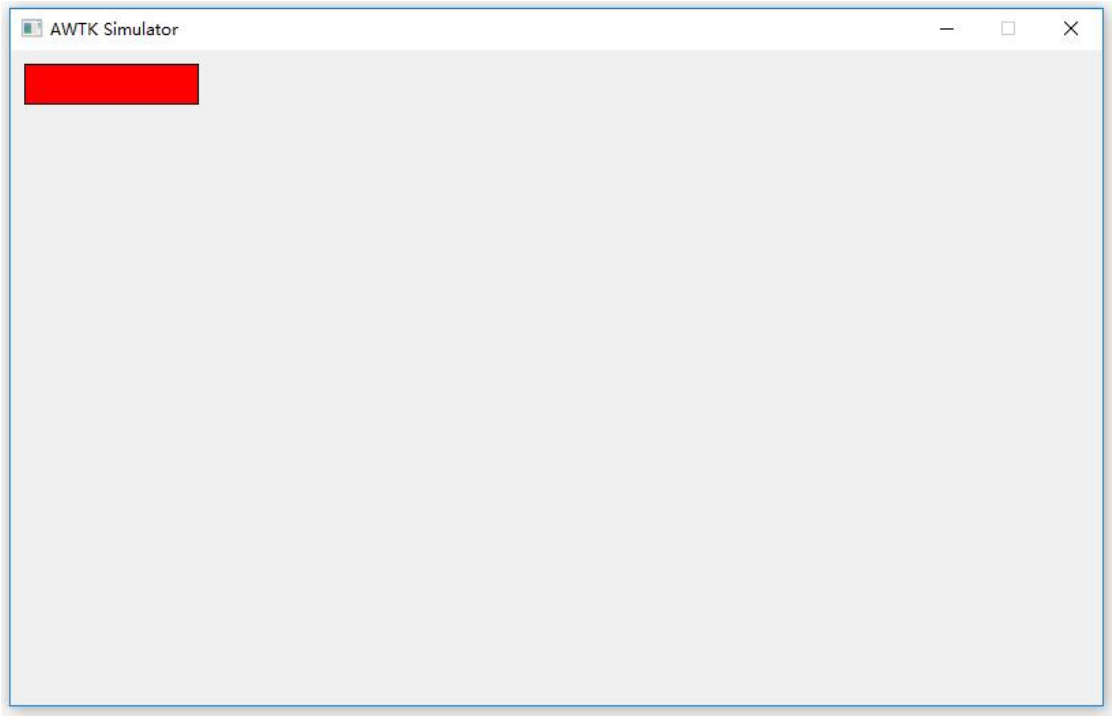


图 4.22　在 C 代码中创建 color_tile 的效果

4.6.9　dialog_title

对话框标题控件。它本身不提供布局功能，仅提供具有语义的标签，让 xml 更具有可读性。dialog_title_t 是 widget_t 的子类控件，widget_t 的函数均适用于 dialog_title_t 控件。

1. 函数

dialog_title 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
dialog_title_create	创建 dialog 对象。
dialog_title_cast	转换为 dialog_title 对象 (供脚本语言使用)。
dialog_title_get_widget_vtable	获取 dialog_title 虚表。

2. 示例

- 在 xml 中，使用”dialog_title”标签创建对话框标题控件，代码如下：

```
<dialog anim_hint="center_scale(duration=300)" x="c" y="m" w="80%" h="160" text=
↪"Dialog">
  <dialog_title x="0" y="0" w="100%" h="30" text="Hello AWTK" />
  <dialog_client x="0" y="bottom" w="100%" h="-30">
    <label name="" x="center" y="middle:-20" w="200" h="30" text="Are you ready?"/>
    <button name="quit" x="10" y="bottom:10" w="40%" h="30" text="确定"/>
    <button name="quit" x="right:10" y="bottom:10" w="40%" h="30" text="取消"/>
  </dialog_client>
</dialog>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

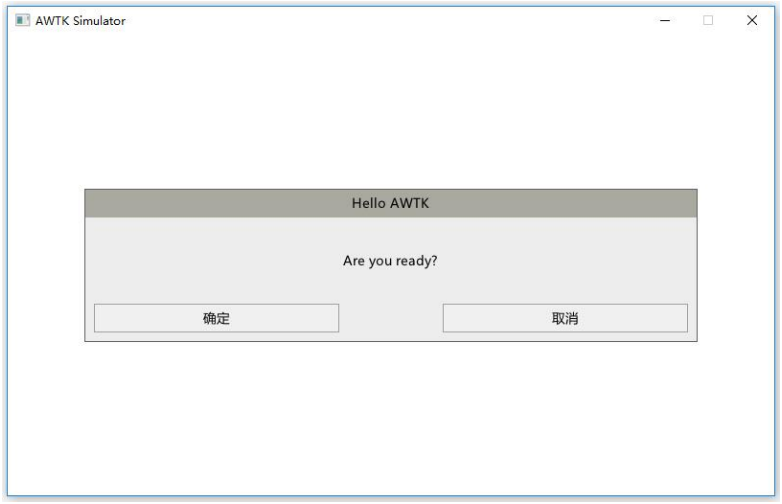


图 4.23　在 xml 中创建 dialog_title 的效果

- 在 C 代码中，使用 dialog_create_simple() 创建对话框标题控件时，会自动创建 dialog 标题对象，通过”dialog->tile”，可以对 dialog 标题进行操作，代码如下：

```
static ret_t on_show_dialog(void* ctx, event_t* e) {
  (void)ctx;
  (void)e;

  widget_t* win = dialog_create_simple(NULL, 0, 0, 240, 160);
  dialog_t* dialog = DIALOG(win);

  code = dialog_modal(win);

  return RET_OK;
}
```

运行效果如下图所示：

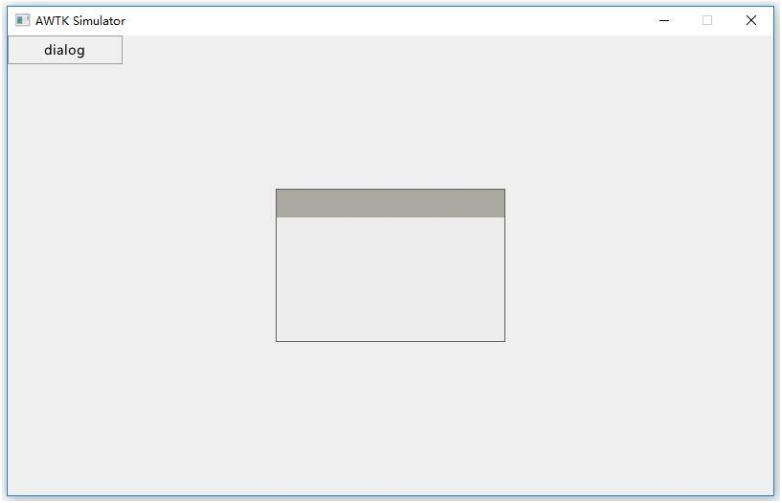


图 4.24　在 C 代码中创建 dialog_title 的效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.6.10 dialog_client

对话框客户区控件。它本身不提供布局功能，仅提供具有语义的标签，让 xml 更具有可读性。dialog_client_t 是 widget_t 的子类控件，widget_t 的函数均适用于 dialog_client_t 控件。

1. 函数

dialog_client 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
dialog_client_create	创建 dialog 客户区对象。
dialog_client_cast	转换为 dialog_client 对象 (供脚本语言使用)。
dialog_client_get_widget_vtable	获取 dialog_client 虚表。

2. 示例

- 在 xml 中，使用”dialog_client”标签创建对话框客户区控件，代码如下：

```
<dialog anim_hint="center_scale(duration=300)" x="c" y="m" w="80%" h="160" text=
↪"Dialog">
  <dialog_title x="0" y="0" w="100%" h="30" text="Hello AWTK" />
  <dialog_client x="0" y="bottom" w="100%" h="-30">
    <label name="" x="center" y="middle:-20" w="200" h="30" text="Are you ready?"/>
    <button name="quit" x="10" y="bottom:10" w="40%" h="30" text="确定"/>
    <button name="quit" x="right:10" y="bottom:10" w="40%" h="30" text="取消"/>
  </dialog_client>
</dialog>
```

运行效果如下图所示：

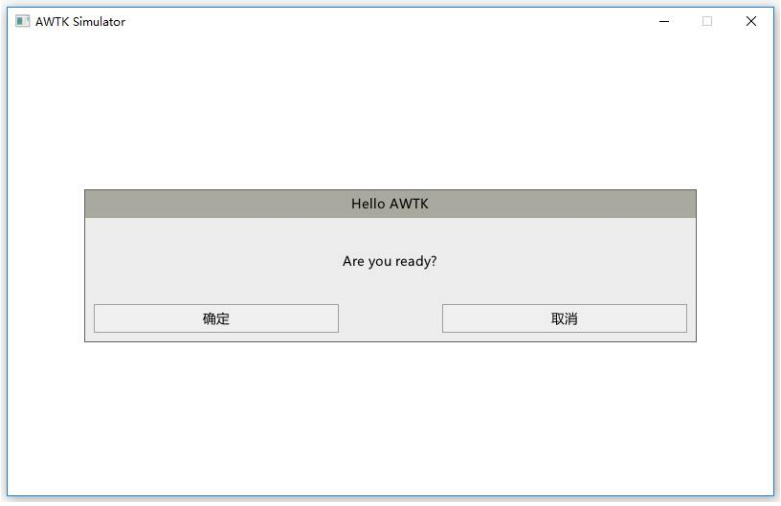


图 4.25　在 xml 中创建 dialog_client 的效果

- 在 C 代码中，使用 dialog_create_simple() 创建对话框控件时，会自动创建 dialog 客户区对象，通过”dialog->client”，可以对 dialog 客户区进行操作。代码如下：

```
static ret_t on_show_dialog(void* ctx, event_t* e) {
  (void)ctx;
  (void)e;
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  widget_t* win = dialog_create_simple(NULL, 0, 0, 240, 160);
  dialog_t* dialog = DIALOG(win);

  code = dialog_modal(win);

  return RET_OK;
}
```

运行效果如下图所示：

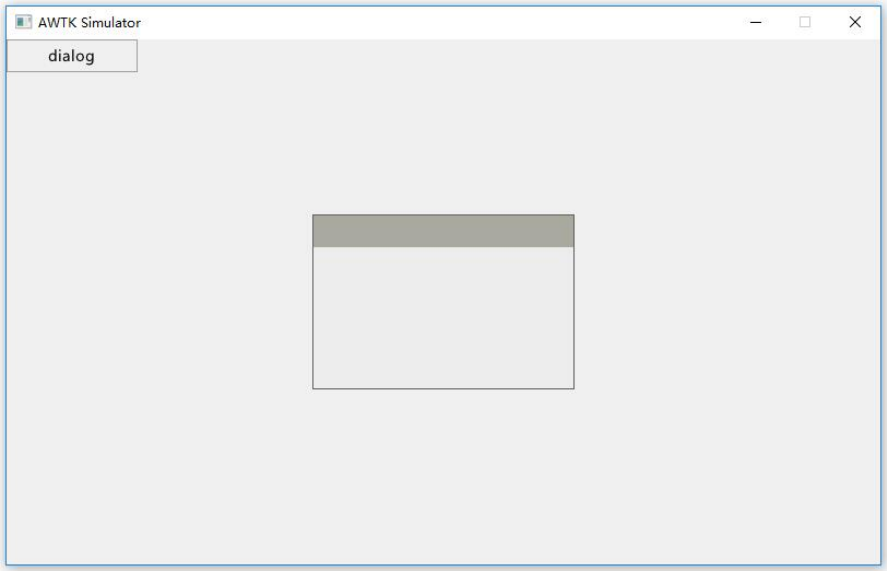


图 4.26　在 C 代码中创建 dialog_client 的效果

4.6.11　slider

滑块控件。slider_t 是 widget_t 的子类控件，widget_t 的函数均适用于 slider_t 控件。slider_t 控件可以拖拉的小方块背景色使用的是”border_color”的颜色。

1. 函数

slider 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
slider_create	创建 slider 对象
slider_cast	转换为 slider 对象 (供脚本语言使用)。
slider_set_value	设置滑块的值。
slider_set_min	设置滑块的最小值。
slider_set_max	设置滑块的最大值。
slider_set_line_cap	设置前景色的线帽形状。（默认为跟随风格的圆角设置，但是在没有设置圆角的时候无法使用“round”来设置圆角）
slider_set_step	设置滑块的拖动的最小单位。
slider_set_bar_size	设置 bar 的宽度或高度。
slider_set_vertical	设置滑块的方向。
slider_set_value_internal	设置滑块的值 (public for test)。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
slider_get_widget_vtable	获取 slider 虚表。

2. 属性

slider 还提供了下面属性，详见下表：

属性名称	类型	说明
value	double	值。
min	double	最小值。
max	double	最大值。
step	double	拖动的最小单位。
bar_size	uint32_t	轴的宽度或高度（单位：像素），为 0 表示为控件的宽度或高度的一半，缺省为 0。
dragger_size	uint32_t	滑块的宽度或高度（单位：像素），缺省为 bar_size * 1.5。
line_cap	char*	前景色的线帽形状。（取值：butt
vertical	bool_t	滑块的是否为垂直方向。
dragger_adapt_to_icon	bool_t	滑块的宽度或高度是否与 icon 适应，缺省为 true。
slide_with_bar	bool_t	是否允许在轴上滑动来改变滑块位置，缺省为 FALSE。

3. 样式

slider 可设置的样式，详见下表：

样式属性	说明
bg_color	背景颜色
fg_color	前景颜色
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)
border_color	边框颜色
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称
bg_image_draw_type	背景图片的显示方式（取值：详见 image 控件一节）
icon	图标的名称
fg_image	前景图片的名称
fg_image_draw_type	前景图片的显示方式（取值：详见 image 控件一节）
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果

例如，使用以下代码设置的样式，效果详见下图。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
<!-- res/assets/default/raw/styles/default.xml -->
<slider>
  <style name="default">
    <normal bg_color="red" fg_color="blue" border_color="green"/>
    <pressed bg_color="red" fg_color="blue" border_color="green"/>
    <over bg_color="red" fg_color="blue" border_color="green"/>
    <focused bg_color="red" fg_color="blue" border_color="green"/>
  </style>
</slider>
```



图 4.27　slider 样式示例

4. 状态

slider 在样式文件中可以设置的状态，详见下表：

状态	说明
normal	正常状态
pressed	指针按下状态
over	指针悬浮状态
disable	禁用状态
focused	聚焦状态

5. 事件

slider 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	value_change_event_t	值即将改变事件。
EVT_VALUE_CHANGING	value_change_event_t	值正在改变事件 (拖动中)。
EVT_VALUE_CHANGED	value_change_event_t	值改变事件。

6. 示例

- 在 xml 中，使用”slider”标签创建滑块控件，代码如下：

```
<window>
    <slider x="98" y="middle" w="-100" h="10%" name="g" />
</window>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

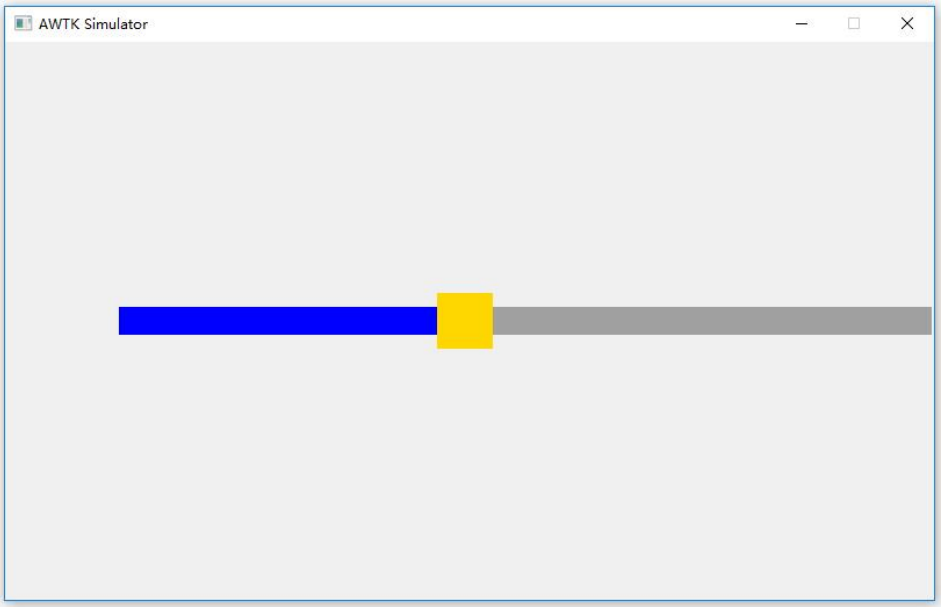


图 4.28　在 xml 中创建 slider 的效果

- 在 C 代码中，使用函数 slider_create 创建滑块控件，代码如下：

```
ret_t application_init() {
  widget_t* slider;
  widget_t* win = window_create(NULL, 0, 0, 0, 0);
  slider = slider_create(win, 10, 10, 200, 30);
  return RET_OK;
}
```

运行效果如下图所示：

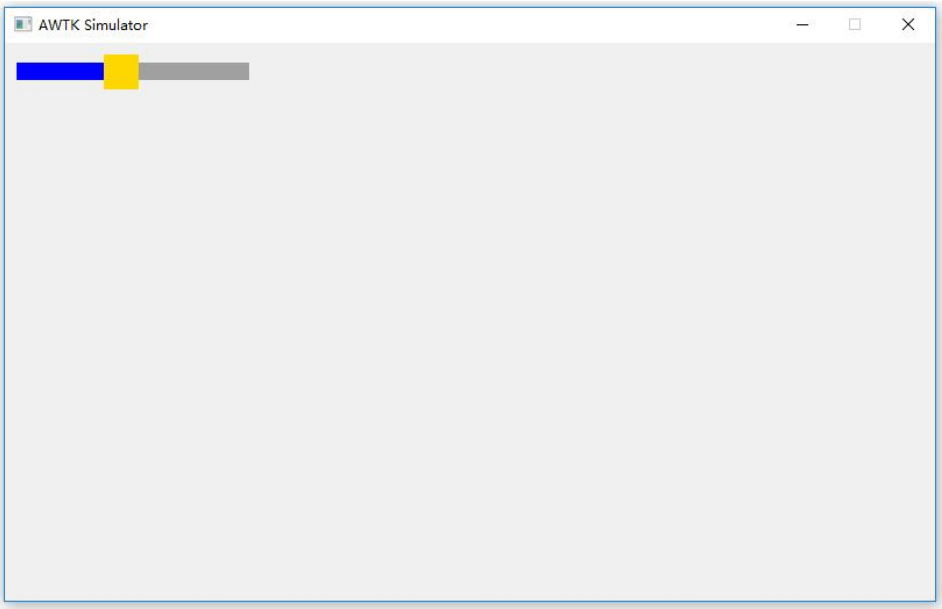


图 4.29　在 C 代码中创建 slider 的效果

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.6.12 progress_bar

进度条控件。进度条控件可以水平显示也可以垂直显示，由 vertical 属性决定。progress_bar_t 是 widget_t 的子类控件，widget_t 的函数均适用于 progress_bar_t 控件。

1. 函数

progress_bar 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
progress_bar_create	创建 progress_bar 对象
progress_bar_cast	转换为 progress_bar 对象 (供脚本语言使用)。
progress_bar_set_value	设置进度条的进度。
progress_bar_set_max	设置最大值。
progress_bar_set_format	设置格式。
progress_bar_set_vertical	设置进度条的方向。
progress_bar_set_show_text	设置进度条的是否显示文本。
progress_bar_set_reverse	设置进度条是否反向。
progress_bar_get_percent	获取进度百分比。
progress_bar_get_widget_vtable	获取 progress_bar 虚表。

2. 属性

progress_bar 还提供了下面属性，详见下表：

属性名称	类型	说明
value	double	进度条的值 [0-max]。
max	double	最大值 (缺省为 100)。
format	char*	数值到字符串转换时的格式，缺省为”%d”。
vertical	bool_t	进度条的是否为垂直方向。
show_text	bool_t	是否显示文本。
reverse	bool_t	是否反向显示。如果为 TRUE，水平方向从右向左表示增加，垂直方向从上到下表示增加。

3. 样式

progress_bar 可设置的样式，详见下表：

样式属性	说明
bg_color	背景颜色
fg_color	前景颜色
font_name	字体名称
font_size	字体大小
text_color	文本颜色
text_align_h	文本水平对齐的方式（取值：center|left|right）
text_align_v	文本垂直对齐的方式（取值：middle|top|bottom）
border_color	边框颜色
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

样式属性	说明
bg_image_draw_type	背景图片的显示方式（取值：详见 image 控件一节）
icon	图标的名称
fg_image	前景图片的名称
fg_image_draw_type	前景图片的显示方式（取值：详见 image 控件一节）
spacer	间距
margin	边距
margin_left	左边距
margin_right	右边距
margin_top	顶边距
margin_bottom	底边距
icon_at	图标的位置（取值：top|bottom|left|right）
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果

例如，使用以下代码设置的样式，效果如下图所示。

```
<!-- res/assets/default/raw/styles/default.xml -->
<progress_bar>
  <style name="default">
    <normal bg_color="red" fg_color="green" border_color="#333333"/>
  </style>
</progress_bar>
```



图 4.30　progress_bar 样式示例

4. 事件

progress_bar 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	value_change_event_t	值即将改变事件。
EVT_VALUE_CHANGED	value_change_event_t	值改变事件。

5. 示例

- 在 xml 中，使用”progress_bar”标签创建进度条控件，代码如下：

```
<window anim_hint="htranslate">
  <progress_bar name="bar1" x="10" y="142" w="240" h="16" value="40"/>
  <progress_bar name="bar2" x="280" y="128" w="30" h="118" value="20" vertical=
↪"true"/>
</window>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

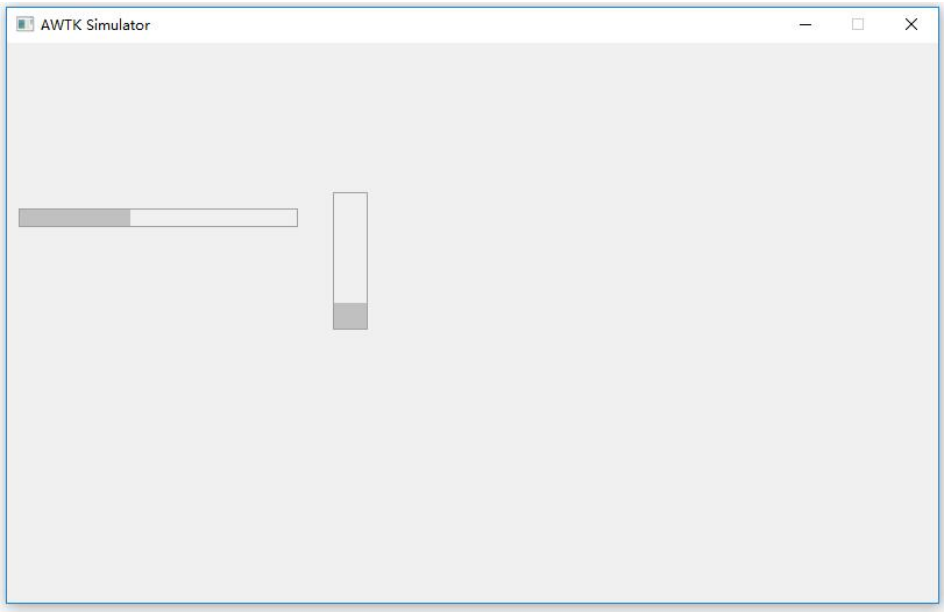


图 4.31　在 xml 中创建 progress_bar 的效果

- 在 C 代码中，使用函数 progress_bar_create 创建进度条控件，代码如下：

```
ret_t application_init() {
  widget_t* progress_bar;
  widget_t* win = window_create(NULL, 0, 0, 0, 0);

  progress_bar = progress_bar_create(win, 260, 80, 20, 118);
  widget_set_value(progress_bar, 40);
  progress_bar_set_vertical(progress_bar, TRUE);
  return RET_OK;
}
```

运行效果如下图所示：

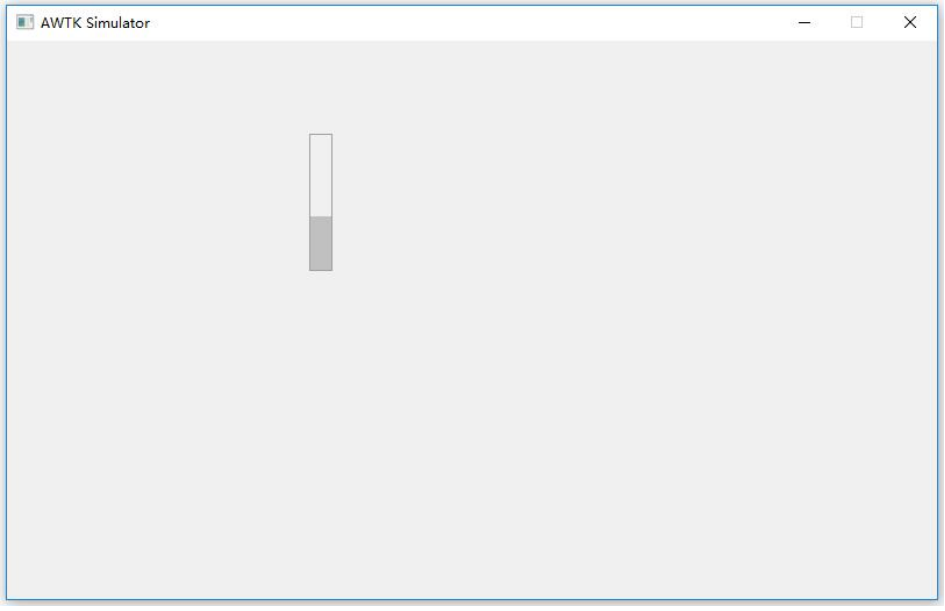


图 4.32　在 C 代码中创建 progress_bar 的效果

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.6.13 tab_control

标签控件。它本身不提供布局功能，仅提供具有语义的标签，让 xml 更具有可读性。标签控件通常会包含一个 pages 控件和一个 tab_button_group 控件，它们之间的关系如下图所示。tab_control_t 是 widget_t 的子类控件，widget_t 的函数均适用于 tab_control_t 控件。

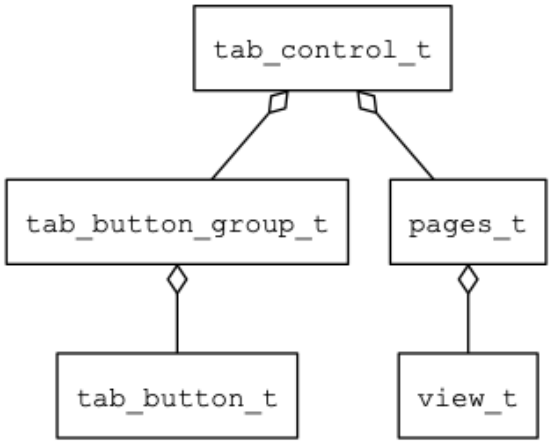


图 4.33 tab_control

1. 函数

tab_control 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
tab_control_create	创建 tab_control 对象
tab_control_cast	转换 tab_control 对象 (供脚本语言使用)。
tab_control_get_widget_vtable	获取 tab_control 虚表。

2. 样式

tab_control 可设置的样式，详见 button 一节的样式小节。tab_control 的样式示例，代码如下：

```
<!-- res/assets/default/raw/styles/default.xml -->
<tab_button>
  <style name="default" border_color="#a0a0a0"  text_color="black">
    <normal      bg_color="#d0d0d0" />
    <pressed     bg_color="#f0f0f0" />
    <over        bg_color="#e0e0e0" />
    <normal_of_active      bg_color="#f0f0f0" />
    <pressed_of_active     bg_color="#f0f0f0" />
    <over_of_active        bg_color="#f0f0f0" />
  </style>
</tab_button>

<tab_button_group>
  <style name="default">
    <normal/>
  </style>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
</tab_button_group>
```

3. 示例

- 在 xml 中，使用”tab_control”标签创建标签控件，代码如下，其中使用的样式，详见上文样式小节。

```
<window anim_hint="htranslate" theme="tab_bottom">
  <tab_control x="0" y="0" w="100%" h="100%" >
    <pages x="0" y="0" w="100%" h="-60" style="at_top">
      <?include filename="tab_views.inc" ?>
    </pages>
    <tab_button_group x="0" y="bottom" w="100%" h="60" >
      <tab_button icon="msg" active_icon="msg_active" text="Message"/>
      <tab_button icon="contact" active_icon="contact_active" text="Contact" value=
↪"true" />
      <tab_button icon="discovery" active_icon="discovery_active" text="Discovery"␣
↪/>
      <tab_button icon="me" active_icon="me_active" text="Me" />
    </tab_button_group>
  </tab_control>
</window>
```

运行效果如下图所示：

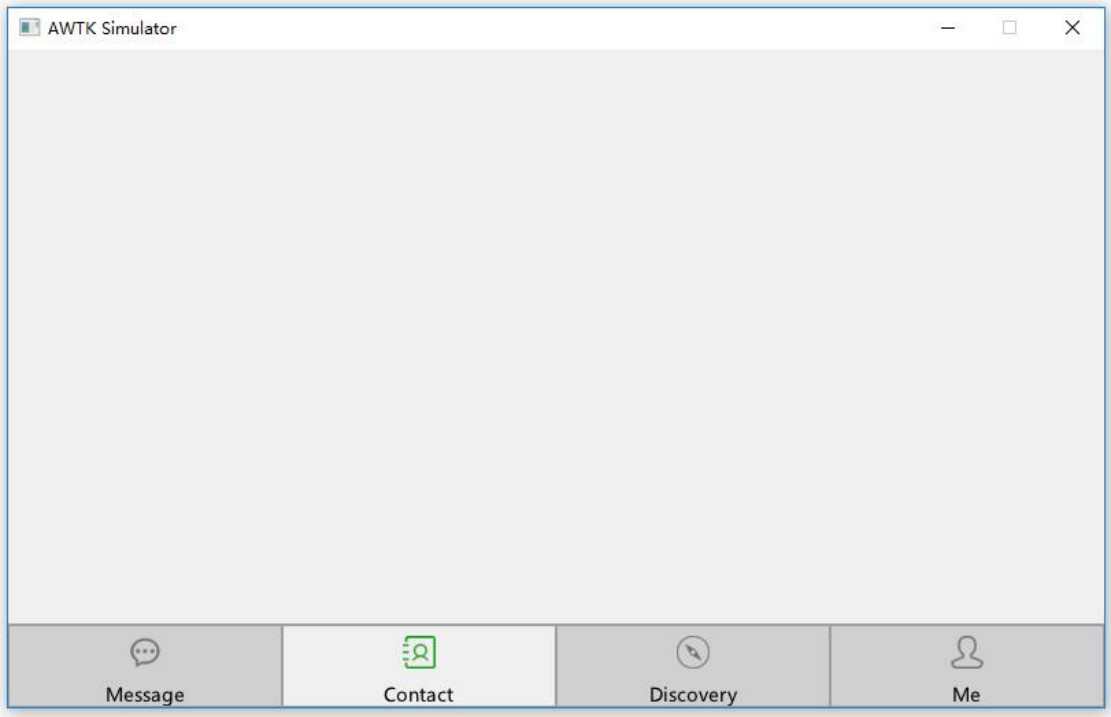


图 4.34　在 xml 中创建 tab_control 的效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.6.14 tab_button

标签按钮控件。标签按钮有点类似单选按钮，但点击标签按钮之后会自动切换当前的标签页。tab_button_t 是 widget_t 的子类控件，widget_t 的函数均适用于 tab_button_t 控件。

1. 函数

tab_button 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
tab_button_create	创建 tab_button 对象
tab_button_cast	转换 tab_button 对象 (供脚本语言使用)。
tab_button_set_value	设置为当前标签。
tab_button_set_icon	设置控件的图标。
tab_button_set_active_icon	设置控件的 active 图标。
tab_button_set_load_ui	设置控件动态加载显示 UI。
tab_button_get_widget_vtable	获取 tab_button 虚表。

2. 属性

tab_button 还提供了下面属性，详见下表：

属性名称	类型	说明
value	bool_t	值。
load_ui	char*	激活后加载的 UI 名字。
active_icon	char*	当前项的图标的名称。
icon	char*	非当前项的图标的名称。

3. 样式

tab_button 可设置的样式，详见 button 一节的样式小节。tab_button 的样式示例，代码如下：

```
<!-- res/assets/default/raw/styles/default.xml -->
<tab_button>
  <style name="default" border_color="#a0a0a0"  text_color="black">
    <normal      bg_color="#d0d0d0" />
    <pressed     bg_color="#f0f0f0" />
    <over        bg_color="#e0e0e0" />
    <normal_of_active      bg_color="#f0f0f0" />
    <pressed_of_active     bg_color="#f0f0f0" />
    <over_of_active        bg_color="#f0f0f0" />
  </style>
</tab_button>

<tab_button_group>
  <style name="default">
    <normal/>
  </style>
</tab_button_group>
```

4. 状态

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

tab_button 在样式文件中可以设置的状态，详见下表：

状态	说明
normal	正常状态
pressed	指针按下状态
over	指针悬浮状态
normal_of_active	正常状态 (当前项)
pressed_of_active	指针按下状态 (当前项)
over_of_active	指针悬浮状态 (当前项)
disable	禁用状态
focused	聚焦状态

5. 事件

tab_button 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	value_change_event_t	值 (激活状态) 即将改变事件。
EVT_VALUE_CHANGED	value_change_event_t	值 (激活状态) 改变事件。
EVT_CLICK	pointer_event_t	点击事件。

6. 示例

- 在 xml 中，使用”tab_button”标签创建标签控件，代码如下，其中使用的样式，详见上文样式一节。

```
<window anim_hint="htranslate" theme="tab_bottom_compact">
  <tab_control x="0" y="0" w="100%" h="100%" >
    <pages x="c" y="20" w="90%" h="-60" value="1">
      <view w="100%" h="100%">
        <label x="c" y="m" w="100%" h="60" text="General" />
        <button name="close" x="c" y="bottom:100" w="80" h="40" text="Close" />
      </view>
      <view w="100%" h="100%">
        <label x="c" y="m" w="100%" h="60" text="Network" />
        <button name="close" x="c" y="bottom:100" w="80" h="40" text="Close" />
      </view>
      <view w="100%" h="100%">
        <label x="c" y="m" w="100%" h="60" text="Security" />
        <button name="close" x="c" y="bottom:100" w="80" h="40" text="Close" />
      </view>
    </pages>
    <tab_button_group x="c" y="bottom:10" w="90%" h="30" compact="true" >
      <tab_button text="General"/>
      <tab_button text="Network" value="true" />
      <tab_button text="Security"/>
    </tab_button_group>
  </tab_control>
</window>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

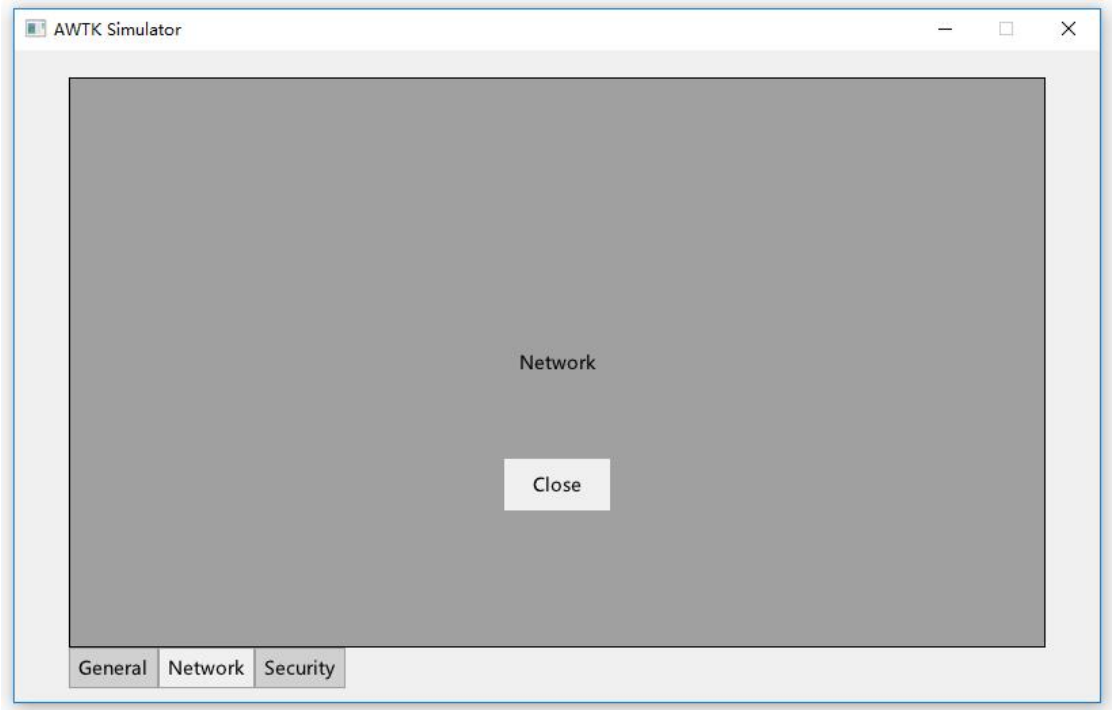


图 4.35　在 xml 中创建 tab_button 的效果

4.6.15　check_button

勾选控件。check_button_t 是 widget_t 的子类控件，widget_t 的函数均适用 check_button_t 控件。

注：使用该控件的时候需要将 awtk/demos/assets/default/raw/images/x1 目录下：unchecked.png 和 checked.png 复制到自己的应用目录下（如：res/assets/default/raw/images/xx），否则将不能正常显示。

1. 函数

check_button 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
check_button_cast	转换 check_button 对象 (供脚本语言使用)
check_button_create	创建多选按钮对象
check_button_set_value	设置控件的值

2. 属性

check_button 还提供了下面属性，详见下表：

属性名称	类型	说明
value	bool_t	值

3. 样式

check_button 可设置的样式，详见 button 一节的样式小节。check_button 的样式示例，代码如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
<!-- res/assets/default/raw/styles/default.xml -->
<check_button bg_color="#f0f0f0" text_color="black">
  <style name="default">
    <normal  icon="unchecked" />
    <pressed icon="unchecked" />
    <over    icon="unchecked" text_color="green"/>
    <normal_of_checked icon="checked" text_color="blue"/>
    <pressed_of_checked icon="checked" text_color="blue"/>
    <over_of_checked icon="checked" text_color="green"/>
  </style>
</check_button>
```

4. 状态

check_button 在样式文件中可以设置的状态，详见下表：

状态	说明
normal	正常状态
pressed	指针按下状态
over	指针悬浮状态
disable	禁用状态
normal_of_checked	正常状态 (选中项)
pressed_of_checked	指针按下状态 (选中项)
over_of_checked	指针悬浮状态 (选中项)
focused	聚焦状态
focused_of_checked	焦点状态 (选中项)

5. 事件

check_button 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	event_t	值（勾选状态）即将改变事件
EVT_VALUE_CHANGED	event_t	值（勾选状态）改变事件
EVT_CLICK	pointer_event_t	点击事件

6. 示例

- 在 xml 中，使用”check_button”标签创建勾选控件，代码如下，其中使用的样式，详见上文样式一节。

```
<window anim_hint="htranslate">
  <column x="20" y="160" w="50%" h="60" children_layout="default(r=2,c=1,ym=2,s=10)
↪" >
    <check_button name="c1" text="Book"/>
    <check_button name="c2" text="Food"/>
  </column>
</window>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

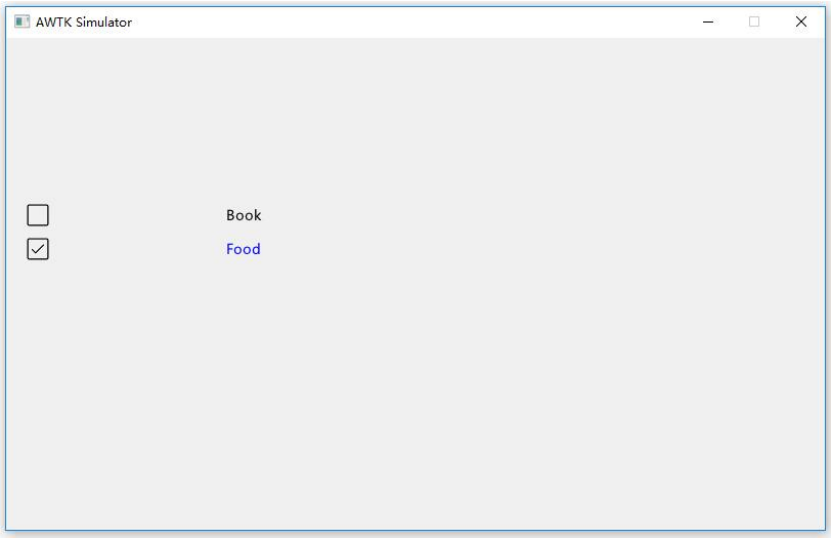


图 4.36　在 xml 中创建 check_button 的效果

- 在 C 代码中，使用函数 check_button_create 创建勾选控件，代码如下，其中使用的样式，详见上文样式一节。

```
ret_t application_init() {
  widget_t* check_button = NULL;
  widget_t* win = window_create(NULL, 0, 0, 0, 0);

  check_button = check_button_create(win, 10, 150, 80, 30);
  widget_set_text(check_button, L"Book");

  check_button = check_button_create(win, 100, 150, 80, 30);
  widget_set_text(check_button, L"Food");

  return RET_OK;
}
```

运行效果如下图所示：



图 4.37　在 C 代码中创建 check_button 的效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.6.16 radio_button

单选控件。radio_button_t 是 widget_t 的子类控件，widget_t 的函数均适用于 radio_button_t 控件。

注：使用该控件的时候需要将 awtk/demos/assets/default/raw/images/x1 目录下：radio_checked.png 和 radio_unchecked.png 复制到自己的应用目录下（如：res/assets/default/raw/images/xx），否则将不能正常显示。

1. 函数

radio_button 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
check_button_cast	转换 check_button 对象 (供脚本语言使用)
check_button_create_radio	创建单选按钮对象
check_button_get_checked_button	用于 radio button 获取同组中勾选的 radio button
check_button_set_value	设置控件的值

2. 属性

radio_button 还提供了下面属性，详见下表：

属性名称	类型	说明
value	bool_t	值

3. 样式

radio_button 可设置的样式，详见 button 一节的样式小节。radio_button 的样式示例，代码如下：

```
<!-- res/assets/default/raw/styles/default.xml -->
<radio_button>
  <style name="default" bg_color="#f0f0f0" text_color="black">
    <normal  icon="radio_unchecked" />
    <pressed icon="radio_unchecked" />
    <over    icon="radio_unchecked" text_color="green"/>
    <normal_of_checked icon="radio_checked" text_color="red"/>
    <pressed_of_checked icon="radio_checked" text_color="red"/>
    <over_of_checked icon="radio_checked" text_color="green"/>
  </style>
  <style name="left">
    <normal  icon="left_off" />
    <pressed icon="left_off" />
    <over    icon="left_off" />
    <normal_of_checked icon="left_on" />
    <pressed_of_checked icon="left_on" />
    <over_of_checked icon="left_on" />
  </style>
  <style name="middle">
    <normal  icon="middle_off" />
    <pressed icon="middle_off" />
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    <over     icon="middle_off" />
    <normal_of_checked icon="middle_on" />
    <pressed_of_checked icon="middle_on" />
    <over_of_checked icon="middle_on" />
  </style>
  <style name="right">
    <normal  icon="right_off" />
    <pressed icon="right_off" />
    <over    icon="right_off" />
    <normal_of_checked icon="right_on" />
    <pressed_of_checked icon="right_on" />
    <over_of_checked icon="right_on" />
  </style>
</radio_button>
```

4. 事件

radio_button 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	event_t	值（勾选状态）即将改变事件
EVT_VALUE_CHANGED	event_t	值（勾选状态）改变事件
EVT_CLICK	pointer_event_t	点击事件

5. 示例

- 在 xml 中，使用”radio_button”标签创建单选控件，代码如下，其中使用的样式详见上文样式一节。

```
<window anim_hint="htranslate">
  <row x="c" y="m" w="100" h="30" children_layout="default(r=1,c=3)">
    <radio_button style="left"/>
    <radio_button style="middle"/>
    <radio_button style="right"/>
  </grid>
</window>
```

运行效果如下图所示：

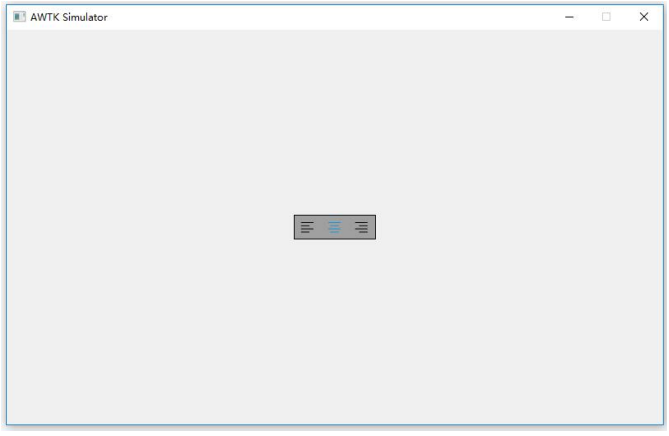


图 4.38　在 xml 中创建 radio_button 的效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

- 在 C 代码中，使用函数 check_button_create_radio 创建单选控件，代码如下：

```
ret_t application_init() {
  radio_button = check_button_create_radio(win, 10, 200, 80, 30);
  widget_set_tr_text(radio_button, "English");
  widget_set_value(radio_button, 1);

  radio_button = check_button_create_radio(win, 100, 200, 80, 30);
  widget_set_tr_text(radio_button, "Chinese");

  return RET_OK;
}
```

运行效果如下所示：



图 4.39　在 C 代码中创建 radio_button 的效果

4.6.17　dragger

dragger 控件。dragger_t 是 widget_t 的子类控件，widget_t 的函数均适用于 dragger_t 控件。

1. 函数

dragger 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
dragger_create	创建 dragger 对象。
dragger_cast	转换为 dragger 对象 (供脚本语言使用)。
dragger_set_range	设置拖动的范围。
dragger_get_widget_vtable	获取 dragger 虚表。

2. 属性

dragger 还提供了下面属性，详见下表：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

属性名称	类型	说明
x_min	xy_t	x 坐标的最小值。
y_min	xy_t	y 坐标的最小值。
x_max	xy_t	x 坐标的最大值。
y_max	xy_t	y 坐标的最大值。

3. 样式

dragger 可设置的样式，详见 button 一节的样式小节。其实 dragger 的样式使用示例，代码如下：

```
<!-- res\assets\default\raw\styles\default.xml -->
<dragger>
  <style name="scroll_bar" >
    <normal     bg_color="#d0d0d0" />
    <pressed    bg_color="#c0c0c0" />
    <over       bg_color="#c0c0c0" />
  </style>
</dragger>
```

4. 状态

dragger 在样式文件中可以设置的状态，详见下表：

状态	说明
normal	正常状态
pressed	指针按下状态
over	指针悬浮状态
disable	禁用状态
focused	聚焦状态

5. 事件

dragger 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_DRAG_START	event_t	开始拖动事件。
EVT_DRAG	event_t	拖动事件。
EVT_DRAG_END	event_t	结束拖动事件。

6. 示例

- 在 xml 中使用”dragger”标签创建 dragger 控件，代码如下，其中使用的样式，

```
<window anim_hint="htranslate">
    <dragger style="scroll_bar" x="50" y="50" w="20" h="20" x_min="0"
             x_max="200" y_min="0" y_max="200" />
  </window>
```

运行效果如下图所示，”dragger”控件可在指定范围内拖拽：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

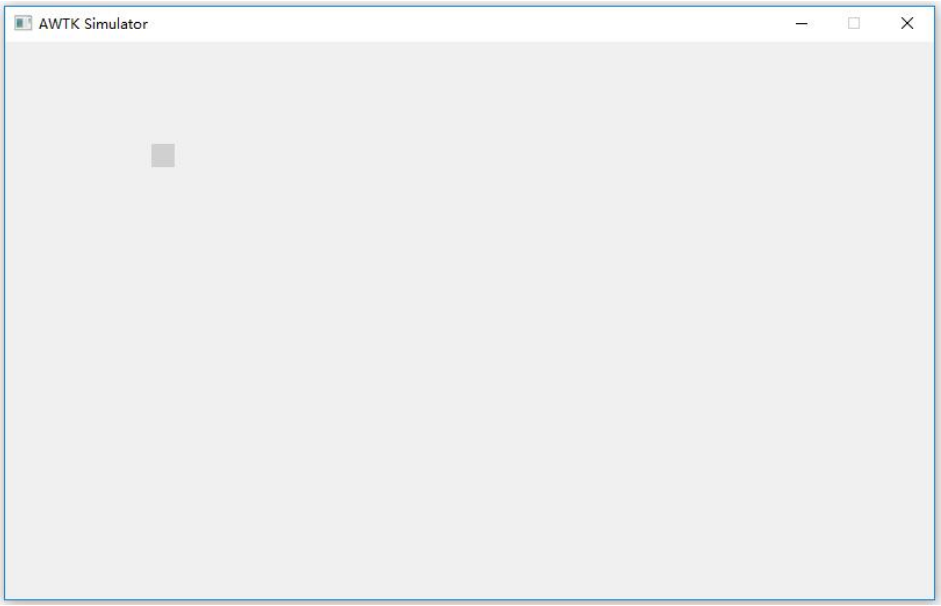


图 4.40　在 xml 中创建 dragger 的效果

- 在 C 代码中使用”dragger_create”创建 dragger 控件，代码如下，使用的样式详见上文样式一节。

```
ret_t application_init() {
  widget_t* dragger = NULL;
  widget_t* win = window_create(NULL, 0, 0, 0, 0);
  dragger = dragger_create(win, 10, 200, 80, 30);
  dragger_set_range(dragger, 20, 20, 100, 100);
  widget_use_style(dragger, "scroll_bar");
  return RET_OK;
}
```

运行效果如下图所示：

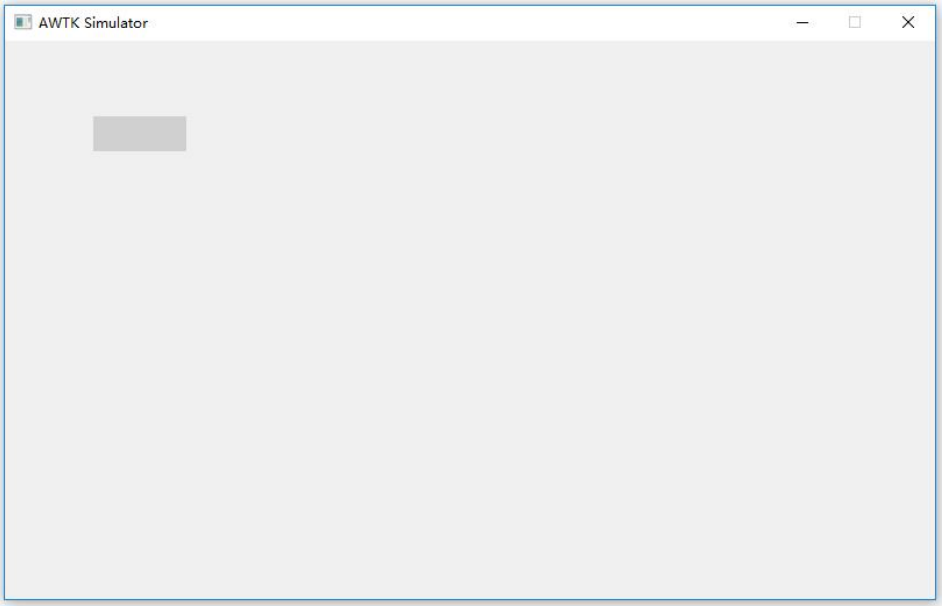


图 4.41　在 C 代码中创建 dragger 的效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.6.18 digit_clock

数字时钟控件。digit_clock_t 是 widget_t 的子类控件，widget_t 的函数均适用于 digit_clock_t 控件。

1. 函数

digit_clock 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
digit_clock_create	创建 digit_clock 对象
digit_clock_cast	转换为 digit_clock 对象 (供脚本语言使用)。
digit_clock_set_format	设置显示格式。
digit_clock_get_widget_vtable	获取 digit_clock 虚表。

2. 属性

digit_clock 还提供了下面属性，详见下表：

属性名称	类型	说明
format	char*	显示格式。

format 属性的取值详见下表：

Format 取值	说明
Y	代表年（完整显示）
M	代表月（1-12）
D	代表日（1-31）
h	代表时（0-23）
m	代表分（0-59）
s	代表秒（0-59）
w	代表星期（0-6）
W	代表星期的英文缩写（支持翻译）
YY	代表年（只显示末两位）
MM	代表月（01-12）
DD	代表日（01-31）
hh	代表时（00-23）
mm	代表分（00-59）
ss	代表秒（00-59）
MMM	代表月的英文缩写（支持翻译）

3. 样式

digit_clock 可设置的样式，详见 button 一节的样式小节。样式设置代码如下：

```
<digit_clock>
  <style name="default" bg_color="gray" font_size="30" border_color="black">
    <normal text_color="green" />
  </style>
</digit_clock>
```

4. 星期和月份中英文互译

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

digit_clock 控件中的星期和月份默认是英文显示的，如果要显示中文，如：“2019/10/12 星期六 18:30”，步骤如下：

(1) 在 UI 文件中创建 digit_clock 控件，代码如下：

```
<digit_clock  x="c" y="3.3%" w="100%" h="3.5%" format="YYYY/MM/DD W h:mm" />
```

(2) 然后，参考 8.3 章节的内容，在 strings.xml 中添加星期或者月份的中英文互译，代码如下：

```
<!-- assets/default/raw/strings/strings.xml -->
...
<string name="Mon">
    <language name="en_US">Mon</language>
    <language name="zh_CN">星期一</language>
</string>
<string name="Tues">
    <language name="en_US">Tues</language>
    <language name="zh_CN">星期二</language>
</string>
<string name="Wed">
    <language name="en_US">Wed</language>
    <language name="zh_CN">星期三</language>
</string>
<string name="Thur">
    <language name="en_US">Thur</language>
    <language name="zh_CN">星期四</language>
</string>
<string name="Fri">
    <language name="en_US">Fri</language>
    <language name="zh_CN">星期五</language>
</string>
<string name="Sat">
    <language name="en_US">Sat</language>
    <language name="zh_CN">星期六</language>
</string>
<string name="Sun">
    <language name="en_US">Sun</language>
    <language name="zh_CN">星期日</language>
</string>
<string name="Jan">
    <language name="en_US">Jan</language>
    <language name="zh_CN">1月</language>
</string>
<string name="Feb">
    <language name="en_US">Feb</language>
    <language name="zh_CN">2月</language>
</string>
<string name="Mar">
    <language name="en_US">Mar</language>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    <language name="zh_CN">3月</language>
</string>
<string name="Apr">
    <language name="en_US">Apr</language>
    <language name="zh_CN">4月</language>
</string>
...
<string name="Sept">
    <language name="en_US">Sept</language>
    <language name="zh_CN">9月</language>
</string>
<string name="Oct">
    <language name="en_US">Oct</language>
    <language name="zh_CN">10月</language>
</string>
<string name="Nov">
    <language name="en_US">Nov</language>
    <language name="zh_CN">11月</language>
</string>
<string name="Dec">
    <language name="en_US">Dec</language>
    <language name="zh_CN">12月</language>
</string>
```

5. 示例

- 在 xml 中，使用”digit_clock”标签创建数字时钟控件，代码如下，样式设置见上文第三小节。

```
<!-- awtk/demos/assets/default/raw/ui/digit_clock.xml -->
<window anim_hint="htranslate" children_layout="default(c=1,h=30,s=10)">
  <digit_clock format="h:m" />
  <digit_clock format="hh:mm" />
  <digit_clock format="h:m:s"/>
  <digit_clock format="hh:mm:ss"/>
  <digit_clock format="Y/M/D"/>
  <digit_clock format="YY-MM-DD"/>
  <digit_clock format="Y/M/D h:mm:ss"/>
  <digit_clock format="YY/MM/DD h:mm:ss"/>
</window>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

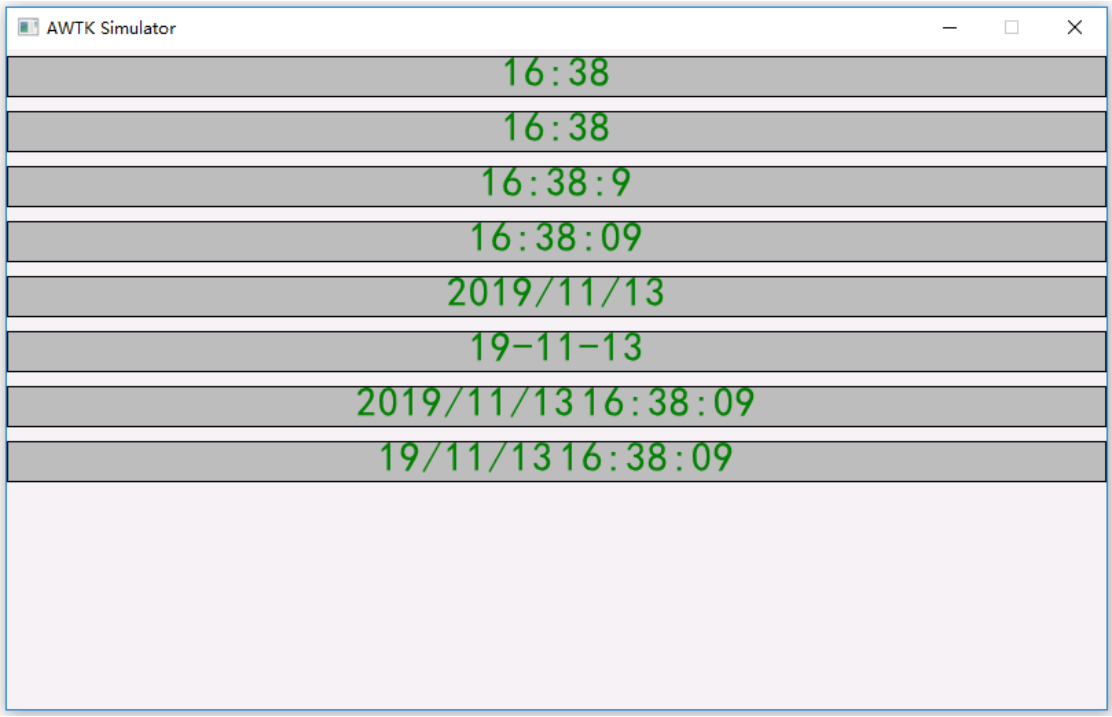


图 4.42　在 xml 中创建 digit_clock 的效果

- 在 C 代码中，使用函数 digit_clock_create 创建数字时钟控件，代码如下，样式设置见上文第三小节。

```
widget_t* tc = digit_clock_create(win, 10, 10, 240, 30);
digit_clock_set_format(tc, "YY/MM/DD h:mm:ss");
```

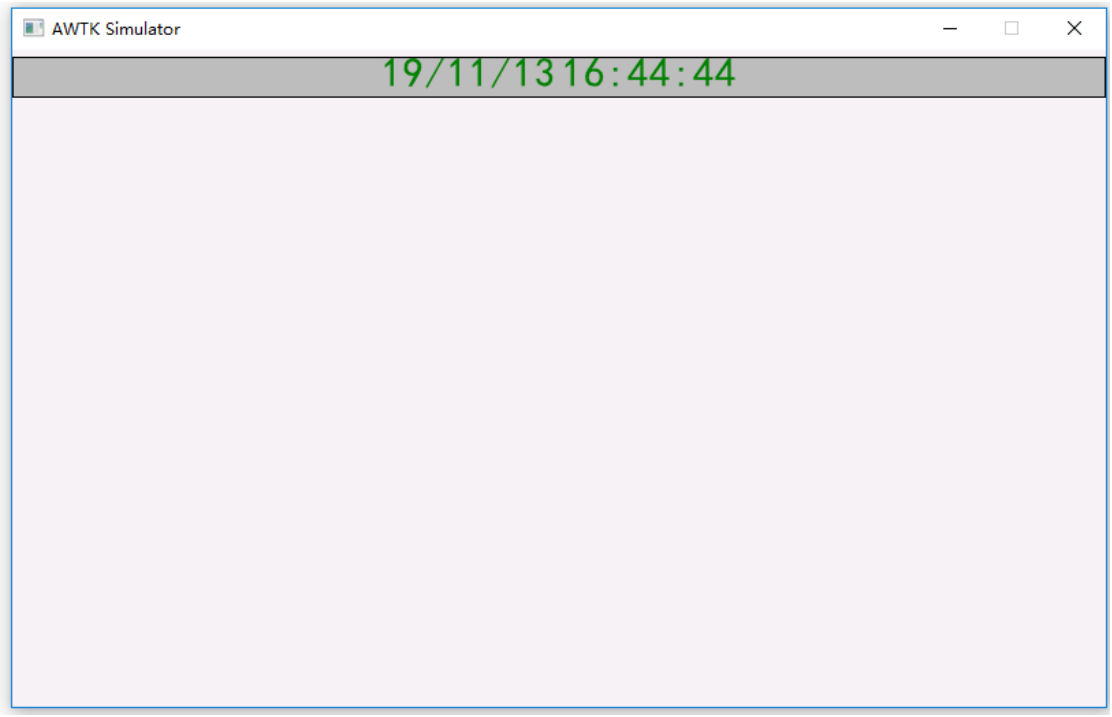


图 4.43　在 C 中创建 digit_clock 的效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.7 通用容器控件

常用的通用容器控件详见下表：

控件名称	说明
row	行控件
column	列控件
grid	网格控件
view	通用容器控件
clip_view	裁剪子控件的容器控件
grid_item	网格项控件
group_box	通用分组控件
app_bar	app_bar 控件
pages	页面管理控件
button_group	按钮分组控件
tab_button_group	标签按钮分组控件

容器可以设置的样式详见下表：

样式属性	说明
bg_color	背景颜色
border_color	边框颜色
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称
bg_image_draw_type	背景图片的显示方式（取值：详见 image 控件一节）
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

其中 group_box 和 button_group 可以设置的样式详见下表：

样式属性	说明
bg_color	背景颜色
font_name	字体名称
font_size	字体大小
text_color	文本颜色
text_align_h	文本水平对齐的方式（取值：center|left|right）
text_align_v	文本垂直对齐的方式（取值：middle|top|bottom）
border_color	边框颜色
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

样式属性	说明
bg_image_draw_type	背景图片的显示方式（取值：详见 image 控件一节）
icon	图标的名称
spacer	间距
margin	边距
margin_left	左边距
margin_right	右边距
margin_top	顶边距
margin_bottom	底边距
icon_at	图标的位置（取值：top|bottom|left|right）
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

通用容器控件在样式文件中可以设置的状态，详见下表：

状态	说明
normal	正常状态
disable	禁用状态
focused	聚焦状态

4.7.1 row

row。一个简单的容器控件，用于水平排列其子控件。它本身不提供布局功能，仅提供具有语义的标签，让 xml 更具有可读性。row_t 是 widget_t 的子类控件，widget_t 的函数均适用于 row_t 控件。

1. 函数

row 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
row_create	创建 row 对象
row_cast	转换为 row 对象 (供脚本语言使用)。
row_get_widget_vtable	获取 row 虚表。

2. 示例

- 在 xml 中，使用”row”标签创建行容器，代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/basic.xml -->
<window anim_hint="htranslate">
  ...
  <row x="0" y="40" w="100%" h="30" children_layout="default(r=1,c=3,xm=2,s=10)">
    <button name="inc_value" text="Inc"/>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    <button name="close" text="Close"/>
    <button name="dec_value" text="Dec"/>
  </row>
  ...
</window>
```

4.7.2 column

column。一个简单的容器控件，垂直排列其子控件。它本身不提供布局功能，仅提供具有语义的标签，让 xml 更具有可读性。column_t 是 widget_t 的子类控件，widget_t 的函数均适用于 column_t 控件。

1. 函数

column 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
column_create	创建 column 对象
column_cast	转换为 column 对象 (供脚本语言使用)。
column_get_widget_vtable	获取 column 虚表。

2. 示例

- 在 xml 中，使用”column”标签创建列容器，代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/basic.xml -->
<window anim_hint="htranslate">
  ...
  <column x="20" y="230" w="50%" h="90" children_layout="default(r=3,c=1,ym=2,s=10)
↪" >
    <radio_button name="r1" text="Book"/>
    <radio_button name="r2" text="Food"/>
    <radio_button name="r3" text="Pencil" value="true"/>
  </column>
  ...
</window>
```

4.7.3 grid

grid 控件。一个简单的容器控件，用于网格排列一组控件。它本身不提供布局功能，仅提供具有语义的标签，让 xml 更具有可读性。grid_t 是 widget_t 的子类控件，widget_t 的函数均适用于 grid_t 控件。

1. 函数

grid 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
grid_create	创建 grid 对象
grid_cast	转换为 grid 对象 (供脚本语言使用)。
grid_set_rows	设置行数。

ZLG ©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
grid_set_columns_definition	设置各列的参数。
grid_set_show_grid	设置是否显示网格。
grid_get_widget_vtable	获取 grid 虚表。

2. 属性

grid 还提供了下面属性，详见下表：

属性名称	类型	说明
rows	uint32_t	行数。
columns_definition	char*	各列的参数。
show_grid	bool_t	是否显示网格。

3. 示例

- 在 xml 中，使用”grid”标签创建网格容器，代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/basic.xml -->
<window anim_hint="htranslate">
  ...
  <grid x="20" y="bottom:10" w="80%" h="40" children_layout="default(r=1,c=5,x=2,
↪s=10)">
    <image draw_type="icon" image="earth"/>
    <image draw_type="icon" image="rgba" />
    <image draw_type="icon" image="rgb" />
    <image draw_type="icon" image="message"/>
    <image draw_type="icon" image="red_btn_n"/>
  </grid>
</window>
```

4.7.4 view

一个通用的容器控件。它本身不提供布局功能，仅提供具有语义的标签，让 xml 更具有可读性。view_t 是 widget_t 的子类控件，widget_t 的函数均适用于 view_t 控件。

1. 函数

view 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
view_create	创建 view 对象
view_set_default_focused_child	设置缺省获得焦点的子控件 (可用控件名或类型)。
view_cast	转换为 view 对象 (供脚本语言使用)。

2. 属性

view 还提供了下面属性，详见下表：

属性名称	类型	说明
default_focused_child	char*	缺省获得焦点的子控件 (可用控件名或类型)。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

3. 示例

- 在 xml 中，使用”view”标签创建 view 容器，代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/basic.xml -->
<window anim_hint="htranslate">
  ...
  <view x="0" y="40" w="100%" h="30" children_layout="default(r=1,c=3,xm=2,s=10)">
    <button name="inc_value" text="Inc"/>
    <button name="close" text="Close"/>
    <button name="dec_value" text="Dec"/>
  </view>
  ...
</window>
```

4.7.5 clip_view

一个裁剪子控件的容器控件。它本身不提供布局功能，仅提供具有语义的标签，让 xml 更具有可读性。在 clip_view 控件下的所有子控件都会被裁剪，无法画超出 clip_view 控件所在范围。clip_view_t 是 widget_t 的子类控件，widget_t 的函数均适用于 clip_view_t 控件。

1. 函数

clip_view 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
clip_view_create	创建 clip_view 对象
clip_view_cast	转换为 clip_view 对象 (供脚本语言使用)。

2. 示例

- 在 xml 中，使用”clip_view”标签创建 clip_view 容器，代码如下：

```
<window anim_hint="htranslate">
  ...
  <clip_view x="0" y="0" w="100" h="100">
    <button x="50" y="10" w="100" h="50" />
  </clip_view>
  ...
</window>
```

4.7.6 grid_item

grid_item。一个简单的容器控件，一般作为 grid 的子控件。它本身不提供布局功能，仅提供具有语义的标签，让 xml 更具有可读性。grid_item_t 是 widget_t 的子类控件，widget_t 的函数均适用于 grid_item_t 控件。

1. 函数

grid_item 提供的函数详见下表，具体请查看 AWTK_API 手册。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

函数名称	说明
grid_item_create	创建 grid_item 对象
grid_item_cast	转换为 grid_item 对象 (供脚本语言使用)。
grid_item_get_widget_vtable	获取 grid_item 虚表。

2. 示例

- 在 xml 中，使用”grid_item”标签创建 grid_item 容器，代码如下：

```
<grid x="0" y="0" w="100%" h="100%" children_layout="default(c=2,r=2,m=5,s=5)">
    <grid_item>
        <button x="c" y="m" w="80%" h="30" name="0" text="0"/>
    </grid_item>
    <grid_item>
        <button x="c" y="m" w="80%" h="30" name="1" text="1"/>
    </grid_item>
    <grid_item>
        <button x="c" y="m" w="80%" h="30" name="2" text="2"/>
    </grid_item>
    <grid_item>
        <button x="c" y="m" w="80%" h="30" name="3" text="3"/>
    </grid_item>
</grid>
```

4.7.7 group_box

分组控件。单选按钮在同一个父控件中是互斥的，所以通常将相关的单选按钮放在一个 group_box 中。它本身不提供布局功能，仅提供具有语义的标签，让 xml 更具有可读性。group_box_t 是 widget_t 的子类控件，widget_t 的函数均适用于 group_box_t 控件。

1. 函数

group_box 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
group_box_create	创建 group_box 对象
group_box_cast	转换为 group_box 对象 (供脚本语言使用)。
group_box_get_widget_vtable	获取 group_box 虚表。

2. 示例

- 在 xml 中，使用”group_box”标签创建分组容器，代码如下：

```
<group_box x="20" y="230" w="50%" h="90" children_layout="default(r=3,c=1,ym=2,
↪s=10)">
    <radio_button name="r1" text="Book"/>
    <radio_button name="r2" text="Food"/>
    <radio_button name="r3" text="Pencil" value="true"/>
</group_box>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.7.8 app_bar

app_bar 控件。一个简单的容器控件，一般在窗口的顶部，用于显示本窗口的状态和信息。它本身不提供布局功能，仅提供具有语义的标签，让 xml 更具有可读性。app_bar_t 是 widget_t 的子类控件，widget_t 的函数均适用于 app_bar_t 控件。

1. 函数

app_bar 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
app_bar_create	创建 app_bar 对象
app_bar_cast	转换为 app_bar 对象 (供脚本语言使用)。
app_bar_get_widget_vtable	获取 app_bar 虚表。

2. 示例

- 在 xml 中，使用”app_bar”标签创建 app_bar 容器，代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/basic.xml -->
<window anim_hint="htranslate">
  <app_bar x="0" y="0" w="100%" h="30" >
    <label x="0" y="0" w="100%" h="100%" text="Basic Controls" />
  </app_bar>
  ...
</window>
```

4.7.9 pages

页面管理控件。只有一个 Page 处于 active 状态，处于 active 状态的 Page 才能显示并接收事件。常用于实现标签控件中的页面管理。pages_t 是 widget_t 的子类控件，widget_t 的函数均适用于 pages_t 控件。

1. 函数

pages 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
pages_create	创建 pages 对象
pages_cast	转换为 pages 对象 (供脚本语言使用)。
pages_set_active	设置当前的 Page。
pages_set_auto_focused	设置切换界面时是否自动聚焦。
pages_set_active_by_name	通过页面的名字设置当前的 Page。
pages_get_widget_vtable	获取 pages 虚表。

2. 属性

pages 还提供了下面属性，详见下表：

ZLG ©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

属性名称	类型	说明
active	uint32_t	当前活跃的 page。(需要用到 MVVM 数据绑定请设置 value 属性)
auto_focused	bool_t	选择切换界面时是否自动聚焦上一次保存的焦点。（默认为 TRUE）
value	uint32_t	当前活跃的 page。

3. 事件

pages 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	value_change_event_t	值 (当前页) 即将改变事件。
EVT_VALUE_CHANGED	value_change_event_t	值 (当前页) 改变事件。

4. 示例

- 在 xml 中，使用”pages”标签创建 pages 容器，代码如下：

```
<!-- awtk/design/default/ui/tab_top_compact.xml -->
<window anim_hint="htranslate">
  ...
  <pages x="c" y="41" w="90%" h="-60" value="1">
    <view w="100%" h="100%">
      <label x="c" y="m" w="100%" h="60" text="General" />
      <button name="close" x="c" y="bottom:100" w="80" h="40" text="Close" />
    </view>
    <view w="100%" h="100%">
      <label x="c" y="m" w="100%" h="60" text="Network" />
      <button name="close" x="c" y="bottom:100" w="80" h="40" text="Close" />
    </view>
    <view w="100%" h="100%">
      <label x="c" y="m" w="100%" h="60" text="Security" />
      <button name="close" x="c" y="bottom:100" w="80" h="40" text="Close" />
    </view>
  </pages>
  ...
</window>
```

4.7.10 button_group

button group 控件。一个简单的容器控件，用于容纳一组按钮控件。它本身不提供布局功能，仅提供具有语义的标签，让 xml 更具有可读性。button_group_t 是 widget_t 的子类控件，widget_t 的函数均适用于 button_group_t 控件。

1. 函数

button_group 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
button_group_create	创建 button_group 对象
button_group_cast	转换为 button_group 对象 (供脚本语言使用)。
button_group_get_widget_vtable	获取 button_group 虚表。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

2. 示例

• 在 xml 中，使用”button_group”标签创建 button_group 容器，代码如下：

```
<button_group x="0" y="m" w="100%" h="40" children_layout="default(c=4,r=1,s=5,m=5)
↪">
    <button name="open:basic" text="Basic"/>
    <button name="open:button" text="Buttons"/>
    <button name="open:edit" text="Edits"/>
    <button name="open:keyboard" text="KeyBoard"/>
</button_group>
```

4.7.11 tab_button_group

标签按钮分组控件。一个简单的容器，主要用于对标签按钮进行布局和管理。tab_button_group_t 是 widget_t 的子类控件，widget_t 的函数均适用于 tab_button_group_t 控件。

在下图中右侧表示当前标签页的颜色取的是 fg_color，非当前标签页取的是 border_color，样式代码如下：

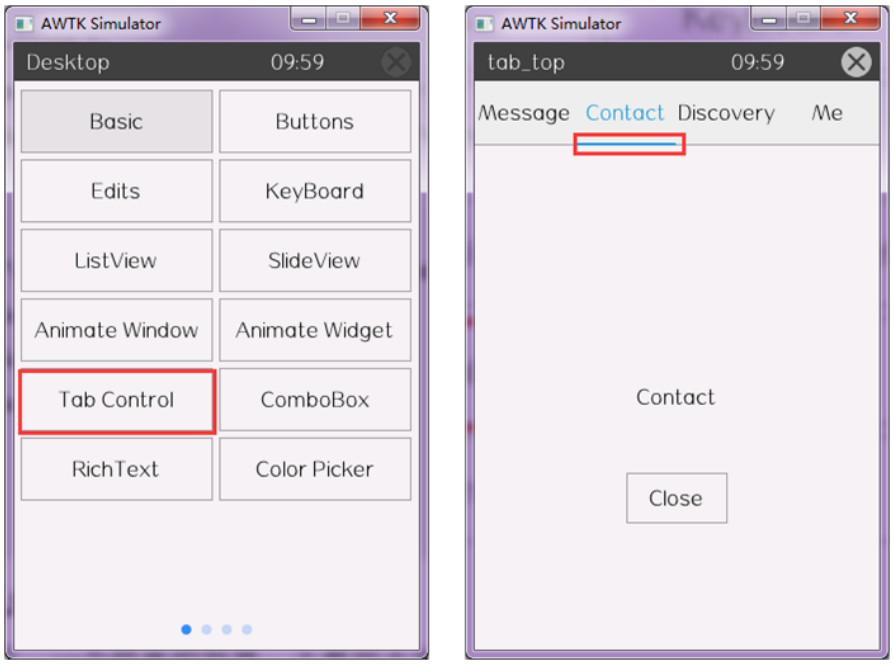


图 4.44 tab_top

```
<!-- awtk/demos/assets/default/raw/styles/tab_top.xml -->
<tab_button_group>
  <style name="highlight" border_color="#a0a0a0" fg_color="#1296db">
    <normal/>
  </style>
</tab_button_group>

<tab_button>
  <style name="default" margin="4" bg_color="#eeeeee">
    <normal     text_color="black"/>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    <pressed    text_color="black"/>
    <over       text_color="black"/>
    <normal_of_active     text_color="#1296db"/>
    <pressed_of_active    text_color="#1296db"/>
    <over_of_active       text_color="#1296db"/>
  </style>
</tab_button>

<pages>
  <style name="at_top" border_color="#a0a0a0" border="bottom">
    <normal />
  </style>
</pages>
```

1. 函数

tab_button_group 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
tab_button_group_create	创建 tab_button_group 对象
tab_button_group_set_compact	设置 compact。
tab_button_group_set_scrollable	设置 scrollable。
tab_button_group_cast	转换 tab_button_group 对象 (供脚本语言使用)。
tab_button_group_get_widget_vtable	获取 tab_button_group 虚表。

2. 属性

tab_button_group 还提供了下面属性，详见下表：

属性名称	类型	说明
compact	bool_t	紧凑型排版子控件 (缺省 FALSE)。
scrollable	bool_t	是否支持滚动 (缺省 FALSE)。
enable_hscroll_animator	bool_t	是否开启 tab_button 的左右滚动动画 (缺省 TRUE)。

3. 示例

- 在 xml 中，使用”tab_button_group”标签创建 tab_button_group 容器，代码如下：

```
<tab_button_group x="c" y="bottom:10" w="90%" h="30" compact="true" >
  <tab_button text="General"/>
  <tab_button text="Network" value="true" />
  <tab_button text="Security"/>
</tab_button_group>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.8 扩展控件

在 AWTK 中提供了两种类型的控件：内置控件和扩展控件，其中内置控件包括窗口、基本控件和通用容器。AWTK 默认加入扩展控件，如果不需要则可以定义宏 WITHOUT_EXT_WIDGETS。

常见的扩展控件详见下表：

控件名称	说明
canvas_widget	画布控件
color_picker	颜色选择器控件
color_component	颜色选择器的颜色分量
combo_box_ex	扩展 combo_box 控件
draggable	draggable 控件
gif_image	gif_image 控件
gauge	仪表控件
gauge_pointer	仪表指针控件
image_animation	图片动画控件
image_value	图片值控件
keyboard	软键盘控件
candidates	输入法候选者字词控件
lang_indicator	输入法语言指示器
progress_circle	进度圆环控件
rich_text	图文混排控件
slide_menu	左右滑动菜单控件
slide_view	滑动视图控件
svg_image	SVG 图片控件
switch	开关控件
text_selector	文本选择器控件
time_clock	模拟时钟控件
digit_clock	数字时钟控件
list_view	列表视图控件
list_view_h	水平列表视图控件
list_item	列表项控件
list_item_seperator	属性页分组控件
scroll_bar	滚动条控件
scroll_view	滚动视图控件
mutable_image	mutable 图片控件（用于辅助实现 camera/video）
mledit	多行编辑器控件
line_number	多行编辑器的行号控件
slide_indicator	滑动视图的指示器控件
hscroll_label	可水平滚动的文本
serial_widget	用于串口通信的控件
timer_widget	定时器控件
vpage	虚拟页面

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.8.1 canvas_widget

画布控件。画布控件让开发者可以自己在控件上绘制需要的内容。canvas_widget_t 是 widget_t 的子类控件，widget_t 的函数均适用于 canvas_widget_t 控件。

1. 函数

canvas_widget 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
canvas_widget_create	创建 canvas_widget 对象
canvas_widget_cast	转换为 canvas_widget 对象 (供脚本语言使用)。
canvas_widget_get_widget_vtable	获取 canvas_widget 虚表。

2. 样式

canvas_widget 可设置的样式，详见下表：

样式属性	说明
bg_color	背景颜色
border_color	边框颜色
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称
bg_image_draw_type	背景图片的显示方式（取值：详见 image 控件一节）
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

- canvas 样式应用示例见以下代码，效果如下图所示：

```
<canvas>
<style name="default" border_color="black" border_width="3" border="all" >
<normal  bg_color="gray" />
<disable  bg_color="gray" round_radius="6" x_offset="3" y_offset="3"/>
<focused  bg_color="gray" bg_image_draw_type="center" bg_image="1"/>
</style>
</canvas>
```

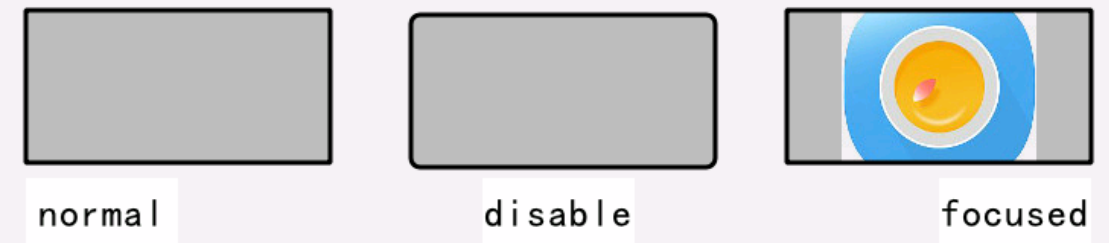


图 4.45　canvas 样式效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

3. 状态

canvas_widget 在样式文件中可以设置的状态，详见下表：

状态	说明
normal	正常状态
disable	禁用状态
focused	聚焦状态

4. 示例

- 在 xml 中，使用”canvas_widget”标签创建画布控件，代码如下，控件样式设置见上文样式一节。

```
<!-- awtk/demos/assets/default/raw/ui/vgcanvas.xml -->
<window anim_hint="htranslate">
  <canvas name="paint_vgcanvas" x="0" y="0" w="100%" h="100%" />
</window>
```

运行效果如下图所示：

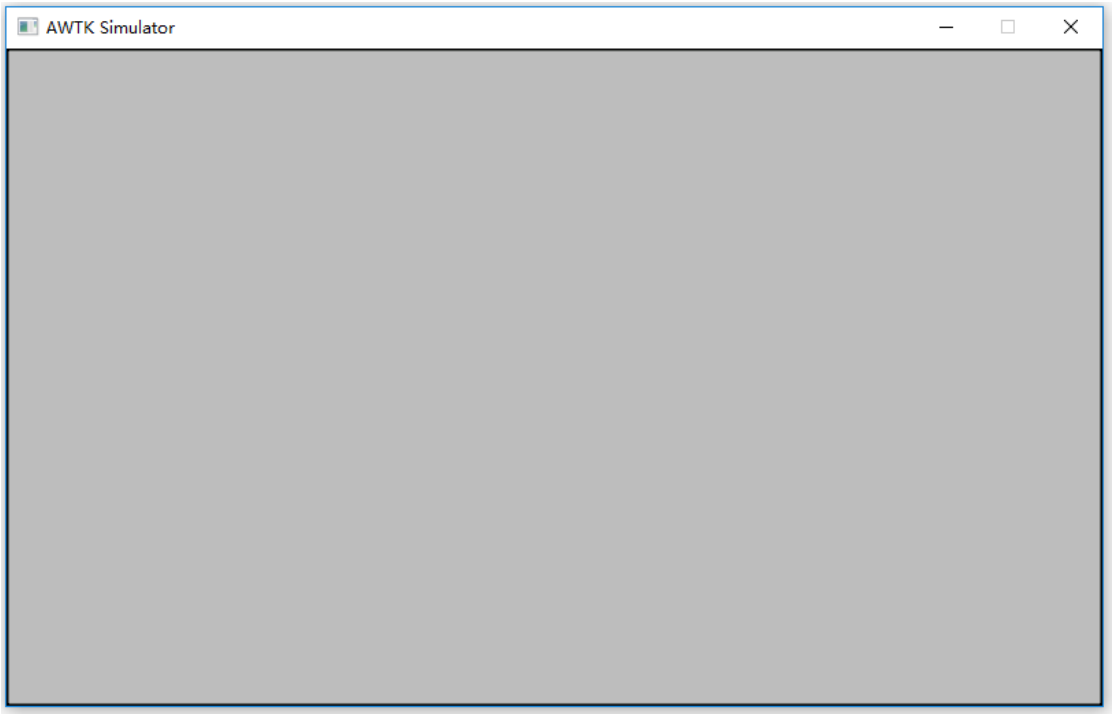


图 4.46　在 xml 中创建 canvas 的效果

- 在 C 代码中，使用函数 canvas_widget_create 创建画布控件，代码如下，控件样式设置见上文样式一节。

```
/* awtk/demos/demo_vg_app.c */
ret_t application_init() {
  widget_t* win = window_create(NULL, 0, 0, 0, 0);
  widget_t* canvas = canvas_widget_create(win, 0, 0, win->w, win->h);

  // widget_on(canvas, EVT_PAINT, on_paint_vg_simple, NULL);
```

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  widget_on(canvas, EVT_PAINT, on_paint_vg, NULL);

  timer_add(on_timer, win, 500);

  return RET_OK;
}
```

创建之后，需要用 widget_on 注册 EVT_PAINT 事件，并在 EVT_PAINT 事件处理函数中绘制，运行效果如下图所示：

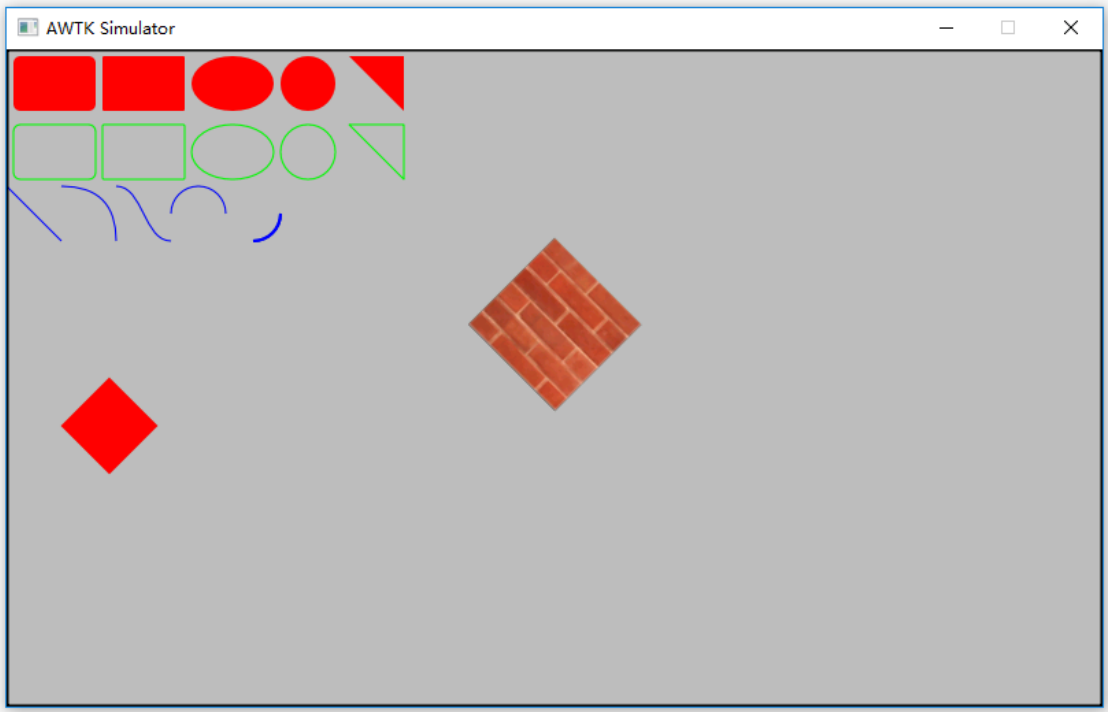


图 4.47　在 C 中创建 canvas 的效果

4.8.2　color_picker

颜色选择器。color_picker_t 是 widget_t 的子类控件，widget_t 函数均适用于 color_picker_t 控件。

1. 函数

color_picker 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
color_picker_create	创建 color_picker 对象
color_picker_set_color	设置颜色。
color_picker_cast	转换为 color_picker 对象 (供脚本语言使用)。
color_picker_get_widget_vtable	获取 color_picker 虚表。

2. 属性

color_picker 还提供了下面属性，详见下表：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

属性名称	类型	说明
value	char*	颜色。

3. 样式

color_picker 可设置的样式，详见 canvas_widget 一节的样式小节。color_picker 控件样式设置示例代码如下：

```
<style name="default" border_color="black" border_width="3" border="all" >
  <normal  bg_color="gray" />
  <disable  bg_color="gray" round_radius="6" x_offset="3" y_offset="3"/>
  <focused  bg_color="gray" bg_image_draw_type="center" bg_image="1"/>
</style>
```

运行效果如下图所示：

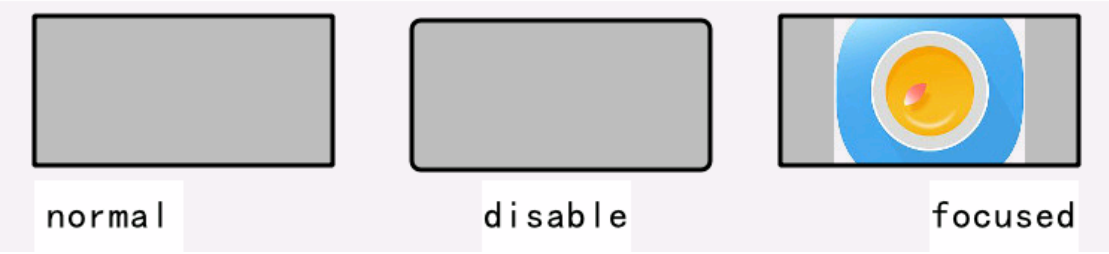


图 4.48　color_picker 样式效果

4. 事件

color_picker 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	value_change_event_t	值 (颜色) 即将改变事件。
EVT_VALUE_CHANGED	value_change_event_t	值 (颜色) 改变事件。

5. 示例

- 在 xml 中，使用”color_picker”标签创建颜色选择器控件，代码如下，样式设置见上文样式一节。

```
<!-- awtk/demos/assets/default/raw/ui/color_picker_full.xml -->
<window>
  <color_picker x="0" y="0" w="100%" h="100%" value="orange">
   <color_component x="0" y="0" w="200" h="200" name="sv"/>
   <color_component x="210" y="0" w="20" h="200" name="h"/>
   <color_tile x="0" y="210" w="50%" h="20" name="new" bg_color="green"/>
   <color_tile x="right" y="210" w="50%" h="20" name="old" bg_color="blue"/>
  </color_picker>
  <button name="close" x="center" y="bottom:5" w="25%" h="50" text="Close"/>
</window>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

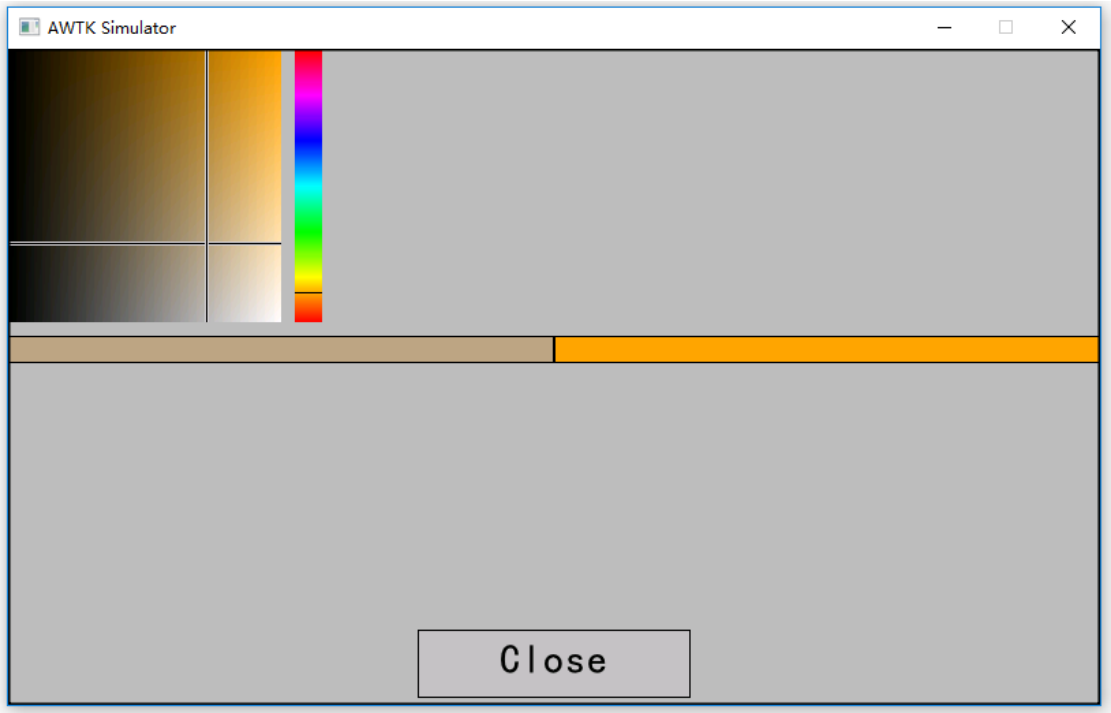


图 4.49　在 xml 中创建 color_picker 及子控件的效果

其中的子控件的 name 必须按规则命名，详见下表：

取值	说明
r	红色分量可以是 spin_box、edit 和 slider
g	绿色分量可以是 spin_box、edit 和 slider
b	蓝色分量可以是 spin_box、edit 和 slider
h	Hue 分量可以是 spin_box、edit、slider 和 color_component
s	Saturation 分量可以是 spin_box、edit 和 slider
v	Value/Brightness 分量可以是 spin_box、edit 和 slider
sv	Saturation 和 Value/Brightness 分量可以是 color_component
old	旧的值可以是 spin_box、edit 和 color_tile
new	新的值可以是 spin_box、edit 和 color_tile

以上面取值”r”为例，表示 name=”r”可以在 spin_box、edit 和 slider 这几个控件上使用，代码如下，控件样式设置见上文样式一节。

```
<color_picker name="c" x="0" y="0" w="100%" h="100%" value="orange">
        <color_tile name="new" x="0"  y="210" w="50%" h="20"/>
        <color_tile name="old" x="right" y="210" w="50%" h="20"/>
        <edit name="new" x="0"     y="240" w="50%" h="30"  input_type="hex"/>
        <edit name="old" x="right" y="240" w="50%" h="30"  readonly="true"/>
        <color_component name="sv" x="0"  y="0" w="200" h="200" />
        <color_component name="h" x="210" y="0" w="20"  h="200" />

        <view x="0" y="300" w="100%" h="90" children_layout="default( r=3,c=3,s=2)
↪">
                <label    name="r" x="0" y="middle" w="100%" h="100%" text="r"/>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
                <spin_box name="r" x="0" y="middle" w="100%" h="100%" min="0"␣
↪input_type="int" max="255"
        step="1" auto_fix="true"  />
                <slider   name="r" x="0" y="middle" w="100%" h="20"  />

                <label    name="g" x="0" y="middle" w="100%" h="100%" text="g"/>
                <spin_box name="g" x="0" y="middle" w="100%" h="100%" min="0"␣
↪input_type="int" max="255"
        step="1" auto_fix="true"  />
                <slider   name="g" x="0" y="middle" w="100%" h="20"/>

                <label    name="b" x="0" y="middle" w="100%" h="100%" text="b"/>
                <spin_box name="b" x="0" y="middle" w="100%" h="100%" min="0"␣
↪input_type="int" max="255"
        step="1" auto_fix="true"  />
                <slider   name="b" x="0" y="middle" w="100%" h="20"  />
        </view>
</color_picker>
```

运行效果如下图所示：

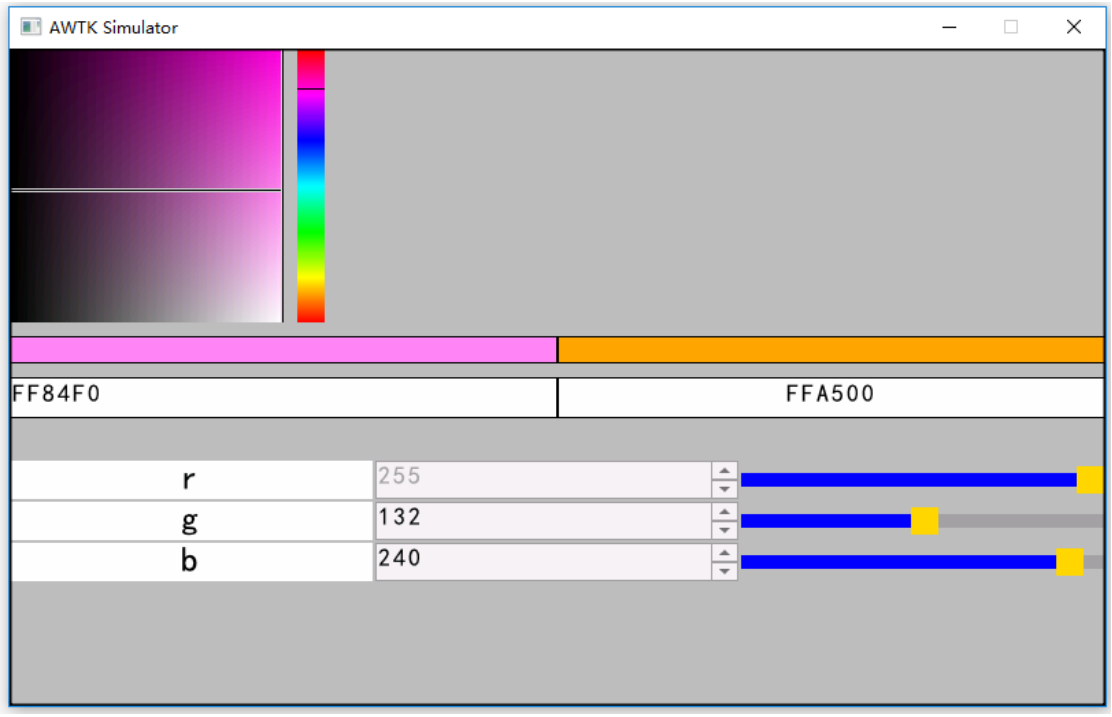


图 4.50　color_picker_rgb 运行效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.8.3 color_component

颜色选择器的颜色分量。控件名称的命名规则可以查看 color_picker 控件中的示例。color_component_t 是 widget_t 的子类控件，widget_t 函数均适用于 color_component_t 控件。

1. 函数

color_component 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
color_component_create	创建 color_component 对象
color_component_set_color	设置颜色。
color_component_set_hsv	设置颜色。
color_component_get_h	获取 h 分量。
color_component_get_s	获取 s 分量。
color_component_get_v	获取 v 分量。
color_component_cast	转换为 color_component 对象 (供脚本语言使用)。
color_component_get_widget_vtable	获取 color_component 虚表。

2. 事件

color_component 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_CHANGING	value_change_event_t	颜色正在改变事件。

3. 示例

- 在 xml 中，使用”color_component”标签创建 color_component 控件，代码如下。

```
<color_picker name="c" x="0" y="0" w="100%" h="100%" value="orange">
  ...
        <color_component name="sv" x="0"  y="0" w="200" h="200" />
        <color_component name="h" x="210" y="0" w="20"  h="200" />
  ...
</color_picker>
```

4.8.4 combo_box_ex

扩展 combo_box 控件。支持以下功能：

1. 支持滚动。项目比较多时显示滚动条。
2. 自动调整弹出窗口的宽度。根据最长文本自动调整弹出窗口的宽度。
3. 支持分组显示。如果 item 的文本以”seperator.”开头，视为一个分组开始，其后的文本为分组的标题。比如:“seperator.basic”，会创建一个 basic 为标题的分组。combo_box_ex_t 是 combo_box_t 的子类控件，combo_box_t 函数均适用于 combo_box_ex_t 控件。

1. 函数

combo_box_ex 提供的函数详见下表，具体请查看 AWTK_API 手册。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

函数名称	说明
combo_box_ex_create	创建 combo_box_ex 对象

2. 示例

```
<!-- awtk/design/default/ui/combo_box.xml -->
<window anim_hint="htranslate">
  ...
  <combo_box_ex x="10" y="100" w="200" h="30" options="1;2;3;4;5;6;7;8;9;0;11;22;
↪33;44;55" theme_of_popup="number"/>
  <combo_box_ex x="0" y="middle:-60" w="100" h="30" options="seperator.basic;33;44;
↪55;seperator.user;hello;world;it is long option" text="33" readonly="true"/>
  <combo_box_ex name="combo_box_ex_for_clone" readonly="true" x="0" y="middle:100"␣
↪w="100" h="30" options="1;2;3;4;5;6;7;8" theme_of_popup="number"/>
  ...
</window>
```

4.8.5 gif_image

GIF 图片控件。需要注意的是 GIF 图片的尺寸大于控件大小时会自动缩小图片，但一般的嵌入式系统的硬件加速都不支持图片缩放，所以缩放图片会导致性能明显下降。如果性能不满意时，请确认一下 GIF 图片的尺寸是否小余控件大小。gif_image_t 是 image_base_t 的子类控件，image_base_t 的函数均适用于 gif_image_t 控件。

1. 函数

gif_image 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
gif_image_create	创建 gif_image 对象
gif_image_play	播放。
gif_image_stop	停止 (并重置 index 为 0)。
gif_image_pause	暂停。
gif_image_set_loop	设置循环播放次数。
gif_image_cast	转换为 gif_image 对象 (供脚本语言使用)。
gif_image_get_widget_vtable	获取 gif_image 虚表。

2. 属性

gif_image 还提供了下面属性，详见下表：

属性名称	类型	说明
running	bool_t	是否正在运行。
loop	uint32_t	循环播放的次数。

3. 示例

- 在 xml 中，使用”gif_image”标签创建 GIF 图片控件，代码如下：

```
<!-- awtk\demos\assets\default\raw\ui\gif_image.xml -->
<window anim_hint="htranslate">
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  <view x="0" y="0" w="100%" h="-60" children_layout="default(c=2,r=2)">
    <gif image="bee"/>
    <gif image="bee"/>
    <gif image="bee"/>
    <gif image="bee"/>
  </view>
</window>
```

4.8.6 gauge

表盘控件。表盘控件就是一张图片。gauge_t 是 widget_t 的子类控件，widget_t 的函数均适用于 gauge_t 控件。

1. 函数

gauge 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
gauge_create	创建 gauge 对象
gauge_cast	转换为 gauge 对象 (供脚本语言使用)。
gauge_set_image	设置背景图片的名称。
gauge_set_draw_type	设置图片的显示方式。
gauge_get_widget_vtable	获取 gauge 虚表。

2. 属性

gauge 还提供了下面属性，详见下表：

属性名称	类型	说明
image	char*	背景图片。
draw_type	image_draw_type_t	图片的绘制方式。

3. 样式

gauge 可设置的样式，详见 canvas_widget 一节的样式小节。样式设置示例代码如下：

```
<guage>
  <style name="default" border_color="black" border_width="3" border="all">
    <normal  bg_color="gray" />
        <disable  bg_color="gray" round_radius="6" x_offset="3" y_offset="3"/>
        <focused  bg_color="gray" bg_image_draw_type="center" bg_image="1"/>
  </style>
</guage>
```

运行效果如下图所示：

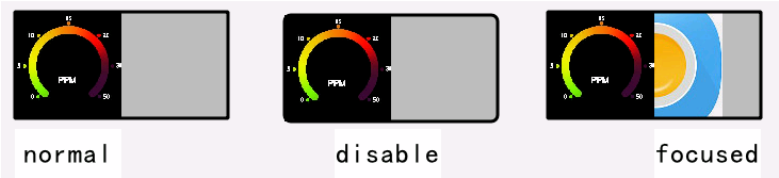


图 4.51　gauge 样式设置效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4. 示例

- 在 xml 中，使用”gauge”标签创建表盘控件，代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/guage.xml -->
<window style="dark" anim_hint="htranslate" >
  <guage x="c" y="10" w="240" h="240" image="guage_bg" >
    <gauge_pointer x="c" y="50" w="24" h="140" value="-128" image="gauge_pointer"
        animation="value(from=-128, to=128, yoyo_times=1000, duration=3000,
↪delay=1000)"/>
  </guage>
  <button name="close" x="center" y="bottom:10" w="25%" h="30" text="Close"/>
</window>
```

运行效果如下图所示：

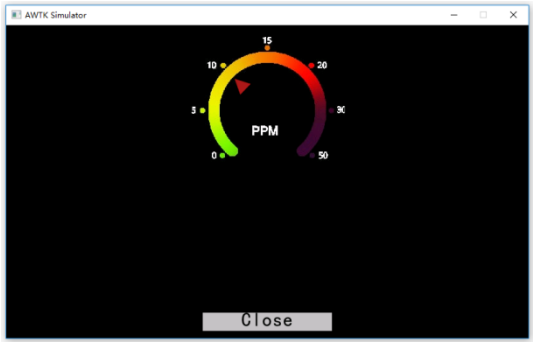


图 4.52　在 xml 中创建 gauge 的效果

- 在 C 代码中，使用 gauge_create 创建表盘控件，代码如下：

```
widget_t* guage = guage_create(win, 10, 10, 200, 200);
guage_set_image(guage, "guage_bg");
```

运行效果见如下图所示：

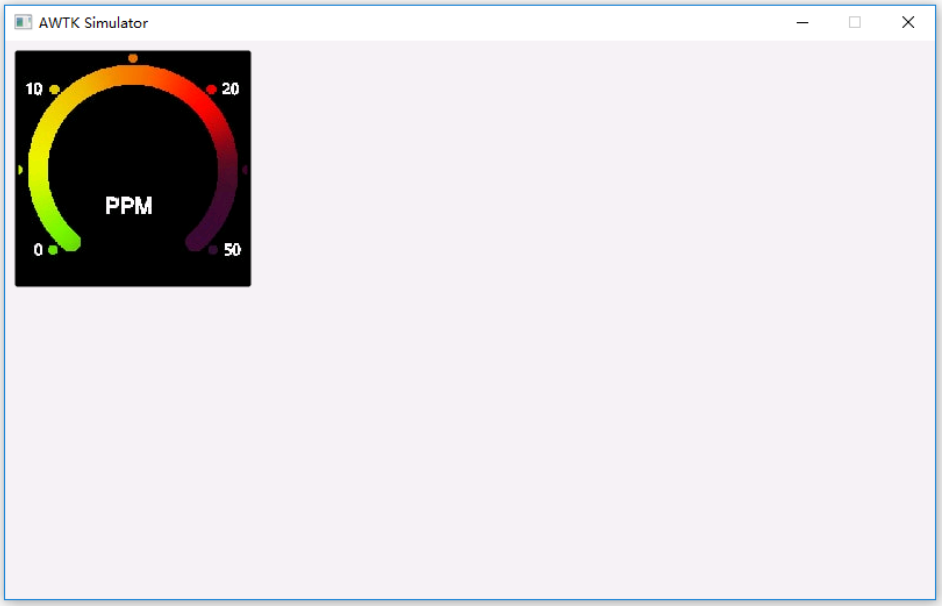


图 4.53　在 C 中创建 gauge 的效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.8.7 gauge_pointer

仪表指针控件。仪表指针就是一张旋转的图片，图片可以是普通图片也可以是 SVG 图片。在嵌入式平台上对于旋转图片，SVG 图片的效率比位图高数倍，所以推荐使用 SVG 图片。gauge_pointer_t 是 widget_t 的子类控件，widget_t 的函数均适用于 gauge_pointer_t 控件。

1. 函数

gauge_pointer 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
gauge_pointer_create	创建 gauge_pointer 对象
gauge_pointer_cast	转换为 gauge_pointer 对象 (供脚本语言使用)。
gauge_pointer_set_angle	设置指针角度。12 点钟方向为 0 度，顺时钟方向为正，单位为度。
gauge_pointer_set_image	设置指针的图片。
gauge_pointer_set_anchor	设置指针的旋转锚点。
gauge_pointer_get_widget_vtable	获取 gauge_pointer 虚表。

2. 属性

gauge_pointer 还提供了下面属性，详见下表：

属性名称	类型	说明
angle	float_t	指针角度。12 点钟方向为 0 度，顺时钟方向为正，单位为度。
image	char*	指针图片。
anchor_x	char*	图片旋转锚点 x 坐标。(后面加上 px 为像素点，不加 px 为相对百分比坐标 0.0f 到 1.0f)
anchor_y	char*	图片旋转锚点 x 坐标。(后面加上 px 为像素点，不加 px 为相对百分比坐标 0.0f 到 1.0f)

3. 样式

gauge_pointer 可设置的样式，详见下表：

样式属性	说明
bg_color	背景颜色
border_color	边框颜色
border_width	边框线宽
Border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称
bg_image_draw_type	背景图片的显示方式（取值：详见 image 控件一节）
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
fg_color	前景颜色
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

4. 事件

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

gauge_pointer 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	value_change_event_t	值 (角度) 即将改变事件。
EVT_VALUE_CHANGED	value_change_event_t	值 (角度) 改变事件。

5. 示例

- 在 xml 中，使用”gauge_pointer”标签创建仪表指针控件，代码如下：

```
<window style="dark" anim_hint="htranslate" >
  <guage x="c" y="10" w="240" h="240" image="guage_bg" >
    <gauge_pointer x="c" y="50" w="24" h="140" value="-128" image="gauge_pointer"
        animation="value(from=-128, to=128, yoyo_times=1000, duration=3000,
↪delay=1000)"/>
  </guage>
  <button name="close" x="center" y="bottom:10" w="25%" h="30" text="Close"/>
</window>
```

运行效果如下图所示：



图 4.54　在 xml 中创建 gauge_pointer 的效果

- 在 C 代码中，使用 gauge_pointer_create 创建仪表指针控件，代码如下：

```
widget_t* gauge_pointer = gauge_pointer_create(guage, 10, 10, 100, 30);
gauge_pointer_set_image(gauge_pointer, "gauge_pointer");
```

创建之后，需要用 gauge_pointer_set_image 设置仪表指针图片。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.8.8 image_animation

图片动画控件，指定一个图片前缀，依次显示指定序列的图片，从而形成动画效果。image_animation_t 是 widget_t 的子类控件，widget_t 的函数均适用于 image_animation_t 控件。

1. 函数

image_animation 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
image_animation_create	创建 image_animation 对象
image_animation_set_loop	设置是否循环播放。
image_animation_set_image	设置图片前缀。
image_animation_set_interval	设置播放间隔时间。
image_animation_set_delay	设置延迟播放时间 (仅适用于自动播放)。
image_animation_set_auto_play	设置是否自动播放。
image_animation_set_sequence	设置播放序列。比如 image 为”fire”，sequence 为”12223”, 将依次播放”fire1”,“fire2”,“fire2”,“fire2”,“fire3”。
image_animation_set_range_sequence	设置播放序列。比如 image 为”fire”，start_index 为 0, end_index 为 99, 将依次播放”fire0”, …,“fire99”。
image_animation_play	播放。
image_animation_stop	停止 (并重置 index 为-1)。
image_animation_pause	暂停。
image_animation_next	手动切换到下一张图片。
image_animation_set_format	设置生成图片名的格式。
image_animation_set_unload_after_paint	设置绘制完成后 unload 图片，以释放内存空间。
image_animation_set_reverse	设置是否倒序播放。
image_animation_set_show_when_done	设置结束播放后是否保持显示最后一帧。
image_animation_cast	转换为 image_animation 对象 (供脚本语言使用)。
image_animation_is_playing	判断是否在播放。
image_animation_get_widget_vtable	获取 image_animation 虚表。

2. 属性

image_animation 还提供了下面属性，详见下表：

属性名称	类型	说明
image	char*	图片名称的前缀。
sequence	char*	播放的序列，字符可选值为数字和英文大小写字母，字符可以重复。如：0123456789 或者 123123abcd。
start_index	uint32_t	图片起始序数。
end_index	uint32_t	图片结束序数。
reverse	bool_t	是否倒序播放。
loop	bool_t	是否循环播放。
auto_play	bool_t	是否自动播放。
unload_after_paint	bool_t	绘制完成后 unload 图片，以释放内存空间。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

属性名称	类型	说明
format	char*	索引到图片名转换时的格式，缺省为”%s%d”。
interval	uint32_t	每张图片播放的时间 (毫秒)。
delay	uint32_t	自动播放时延迟播放的时间 (毫秒)。
show_when_done	bool_t	结束后是否继续显示最后一帧。

3. 样式

image_animation 可设置的样式，详见 canvas_widget 一节的样式小节。样式设置代码如下：

```
<image_animation>
  <style name="default" border_color="black" border_width="3" border="all" >
    <normal  bg_color="gray" />
        <disable  bg_color="gray" round_radius="6" x_offset="3" y_offset="3"/>
        <focused  bg_color="gray" bg_image_draw_type="center" bg_image="1"/>
  </style>
</image_animation>
```

运行效果如下图所示：

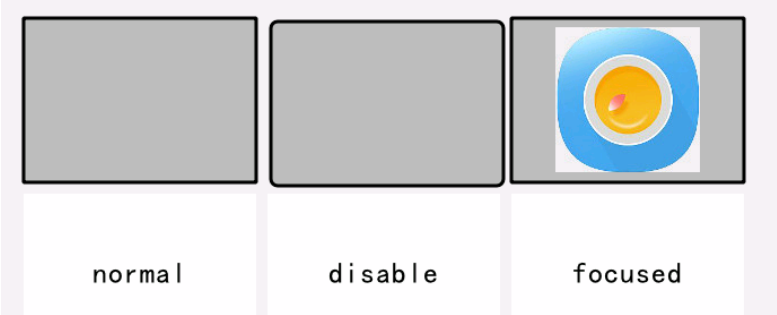


图 4.55　image_animation 样式设置效果

4. 事件

image_animation 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_ANIM_ONCE	event_t	使能循环播放时，控件动画完成一次事件。
EVT_ANIM_END	event_t	控件动画完成事件。

5. 示例

- 在 xml 中，使用”image_animation”标签创建图片动画控件，代码如下，其中样式设置见上文样式一节。

```
<!-- awtk/demos/assets/default/raw/ui/image_animation.xml -->
<window anim_hint="htranslate">
  ...
    <image_animation image="ani" sequence="123456789abc" auto_play="true" interval=
↪"50" delay="100"/>
    <image_animation image="ani" sequence="123456789abc" auto_play="true" interval=
↪"50" delay="200"/>
```

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    ......
  <button name="close" x="center" y="bottom:50" w="50%" h="50" text="close"/>
</window>
```

运行效果如下图所示：

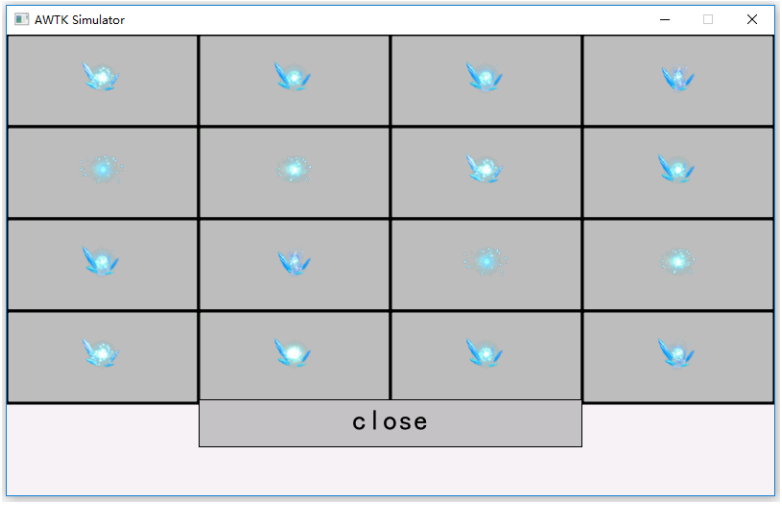


图 4.56　在 xml 中创建 image_animation 的效果

- 在 C 代码中，使用 image_animation_create 创建图片动画控件，代码如下，其中的样式设置见上文样式一节。

```
image_animation = image_animation_create(win, 10, 10, 200, 200);
image_animation_set_image(image_animation, "ani");
image_animation_set_interval(image_animation, 50);
image_animation_set_sequence(image_animation, "123456789abc");
image_animation_play(image_animation);
```

运行效果如下图所示：

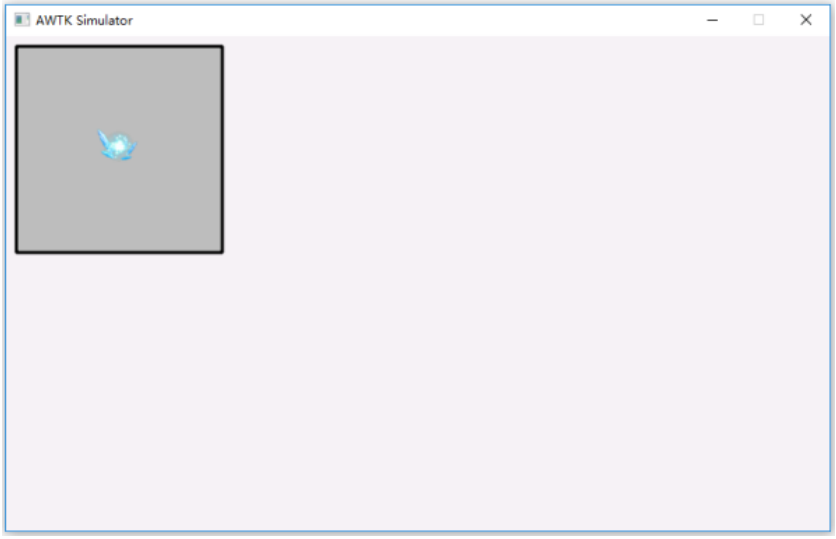


图 4.57　在 C 中创建 image_animation 的效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.8.9 image_value

图片值控件。可以用图片来表示如电池电量、WIFI 信号强度和其它各种数值的值。image_value_t 是 widget_t 的子类控件，widget_t 的函数均适用于 image_value_t 控件。其原理如下：

1. 把 value 以 format 为格式转换成字符串。
2. 把每个字符与 image（图片文件名前缀）映射成一个图片名。
3. 最后把这些图片显示出来。

1. 函数

image_value 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
image_value_create	创建 image_value 对象
image_value_set_image	设置图片前缀。
image_value_set_format	设置格式。
image_value_set_click_add_delta	设置点击时加上的增量。
image_value_set_value	设置值。
image_value_set_min	设置最小值。
image_value_set_max	设置最大值。
image_value_cast	转换为 image_value 对象 (供脚本语言使用)。
image_value_get_widget_vtable	获取 image_value 虚表。

2. 属性

image_value 还提供了下面属性，详见下表：

属性名称	类型	说明
image	char*	图片名称的前缀。
format	char*	数值到字符串转换时的格式，缺省为”%d”。
click_add_delta	double	点击时加上一个增量。
value	double	值。
min	double	最小值 (如果设置了 click_add_delta，到达最小值后回到最大值)。
max	double	最大值 (如果设置了 click_add_delta，到达最大值后回到最小值)。

3. 样式

image_value 可设置的样式，详见 canvas_widget 一节的样式小节。image_value 样式设置代码如下：

```
<image_value>
  <style name="default" border_color="black" border_width="3" border="all" >
    <normal  bg_color="gray" />
        <disable  bg_color="gray" round_radius="6" x_offset="3" y_offset="3"/>
        <focused  bg_color="gray" bg_image_draw_type="center" bg_image="1"/>
  </style>
</image_value>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

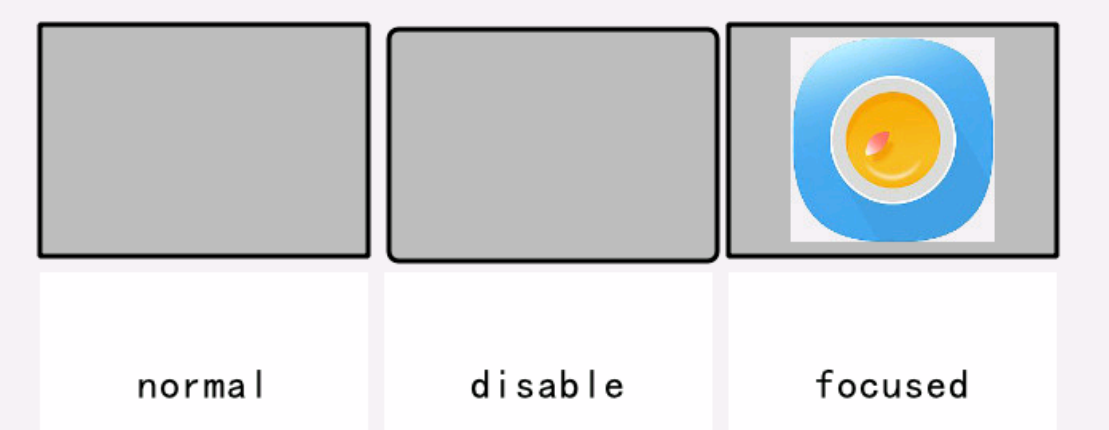


图 4.58　image_value 样式设置效果

4. 事件

image_value 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	value_change_event_t	值即将改变事件 (click_add_delta 为非 0 时，点击触发)。
EVT_VALUE_CHANGED	value_change_event_t	值改变事件 (click_add_delta 为非 0 时，点击触发)。

5. 示例

- 在 xml 中，使用”image_value”标签创建图片值控件，代码如下，其中的样式设置见上文样式一节。

```
<!-- awtk/demos/assets/default/raw/ui/image_value.xml -->
<window anim_hint="htranslate">
  <view x="0" y="0" w="100%" h="100%" children_layout="default(c=2,r=3)">
    <image_value  value="0" image="battery_"
      animation="value(from=0, to=5, duration=10000)"/>
    <image_value  value="0" image="battery_"
      animation="value(from=0, to=5, duration=10000)"/>
    <image_value  value="0" format="%02d" image="num_"
      animation="value(from=10, to=100, duration=100000)"/>
    <image_value  value="0" format="%04d" image="num_"
      animation="value(from=10, to=10000, duration=100000)"/>
    ...
  </view>
</window>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

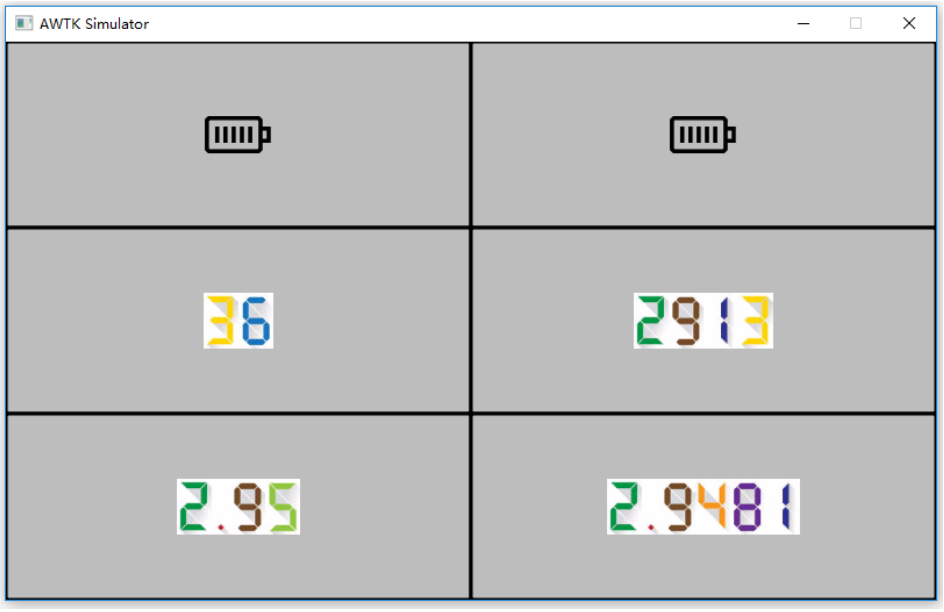


图 4.59　在 xml 中创建 image_value 的效果

- 在 C 代码中，使用 image_value_create 创建图片值控件，代码如下，其中的样式设置见上文样式一节。

```
image_value = image_value_create(win, 10, 10, 200, 200);
image_value_set_image(image_value, "num_");
image_value_set_value(image_value, 100);
```

运行效果如下图所示：

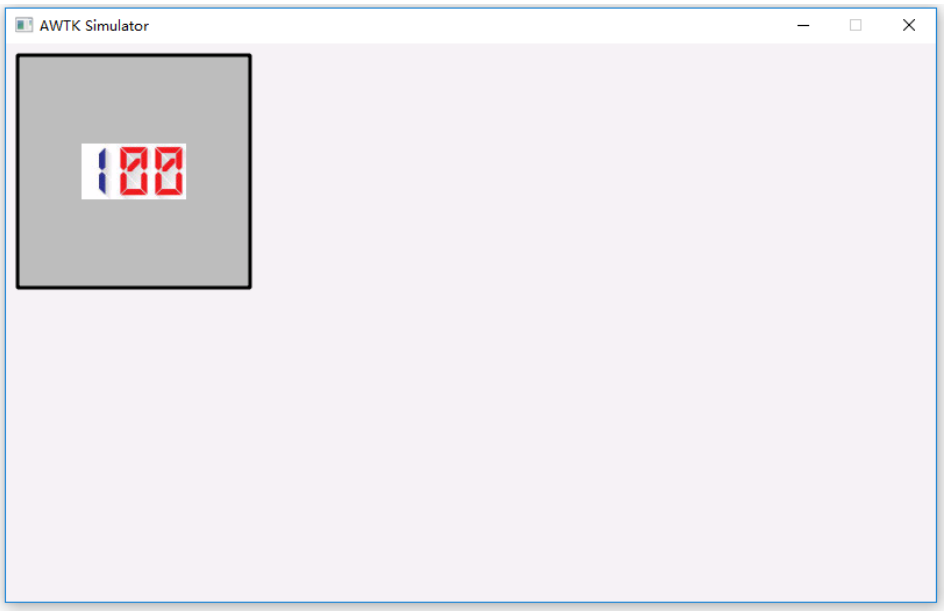


图 4.60　在 C 中创建 image_value 的效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.8.10 keyboard

软键盘。软键盘是一个特殊的窗口，由编辑器通过输入法自动打开和关闭。编辑器输入类型和软键盘 UI 资源文件的对应关系，详见下表：

输入类型	软键盘 UI 资源文件
INPUT_PHONE	kb_phone.xml
INPUT_INT	kb_int.xml
INPUT_FLOAT	kb_float.xml
INPUT_UINT	kb_uint.xml
INPUT_UFLOAT	kb_ufloat.xml
INPUT_HEX	kb_hex.xml
INPUT_EMAIL	kb_ascii.xml
INPUT_PASSWORD	kb_ascii.xml
INPUT_CUSTOM	使用自定义的键盘
其它	kb_default.xml

keyboard 中按钮子控件的名称功能说明，详见下表：

名称	功能
return	回车键
action	定制按钮
backspace	删除键
tab	tab 键
space	空格键
close	关闭软键盘
clear	清除编辑器的内容
cancel	恢复编辑器的内容
back	关闭当前窗口
back_to_home	返回 home 窗口
前缀 key:	键值
前缀 hard_key:	模拟物理键盘
前缀 page:	切换到页面
前缀 opt:	多个字符选择一个，点击切换到下一个，超时提交字符 (用于实现九宫格输入)

1. 函数

keyboard 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
keyboard_create	创建 keyboard 对象
keyboard_close	关闭 keyboard 窗口。
keyboard_cast	转换为 keyboard 对象 (供脚本语言使用)。
keyboard_get_widget_vtable	获取 keyboard 虚表。

2. 示例

- 创建 edit 输入类型 hex，xml 程序代码如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
<edit name="new" x="c" y="m" w="80%" h="50"  input_type="hex"/>
```

运行效果如下下图所示：

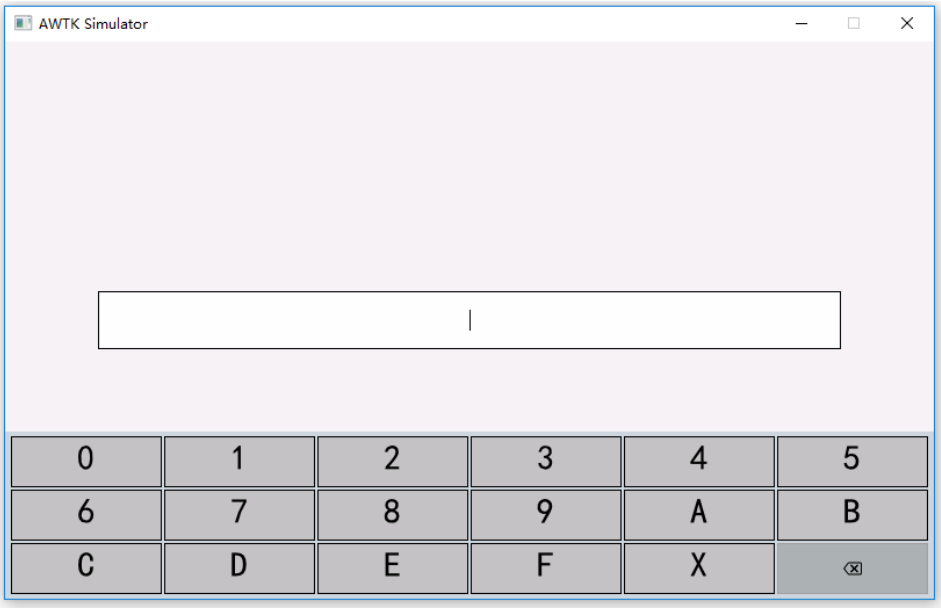


图 4.61　输入类型 hex 的键盘效果

4.8.11　candidates

输入法候选字词控件。candidates_t 是 widget_t 的子类控件，widget_t 的函数均适用于 candidates_t 控件。

1. 函数

candidates 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
candidates_create	创建 candidates 对象
candidates_cast	转换为 candidates 对象 (供脚本语言使用)。
candidates_set_pre	设置是否为预候选字列表。
candidates_set_select_by_num	设置是否启用用数字选择候选字。
candidates_set_auto_hide	设置是否自动隐藏。
candidates_set_button_style	设置按钮的 style 名称。
candidates_get_widget_vtable	获取 candidates 虚表。

2. 属性

candidates 还提供了下面属性，详见下表：

属性名称	类型	说明
pre	bool_t	是否为预候选字。
select_by_num	bool_t	是否启用用数字选择候选字。比如按下 1 选择第 1 个候选字，按下 2 选择第 2 个候选字。
auto_hide	bool_t	没有候选字时，是否自动隐藏控件。
button_style	char*	按钮的 style 名称。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

属性名称	类型	说明
enable_preview	bool_t	是否启用候选字预览。

3. 状态

candidates 在样式文件中可以设置的状态，详见下表：

状态	说明
normal	正常状态
pressed	按下状态
over	悬浮状态
focused	聚焦状态

4. 示例

- 在 xml 中，使用”candidates”标签创建输入法候选字词控件，代码如下，如果希望启用用数字选择对应的候选字，请设置属性 grab_keys=“true”

```
<!-- awtk/demos/assets/default/raw/ui/progress_circle.xml -->
<keyboard theme="keyboard" x="0" y="bottom" w="100%" h="40%">
  ...
  <candidates x="0" y="0" w="100%" h="30" grab_keys="true"/>
  ...
</keyboard>
```

4.8.12 lang_indicator

输入法语言指示器。用于显示输入法的输入语言或类型，主要用于 T9 输入法。lang_indicator_t 是 widget_t 的子类控件，widget_t 的函数均适用于 lang_indicator_t 控件。

1. 函数

lang_indicator 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
lang_indicator_create	创建 lang_indicator 对象
lang_indicator_set_image	设置缺省获得焦点的子控件 (可用控件名或类型)。
lang_indicator_cast	转换为 lang_indicator 对象 (供脚本语言使用)。

2. 属性

lang_indicator 还提供了下面属性，详见下表：

属性名称	类型	说明
image	char*	如果希望用图片格式显示，本属性用于指定图片的前缀。

3. 示例

```
<!-- awtk/design/default/ui/kb_default_t9ext.xml-->
<keyboard disable_anim="true" theme="keyboard" style="transparent" x="0" y="bottom"␣
↪w="100%" h="78"
    move_focus_up_key="up"
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    move_focus_down_key="down"
    move_focus_left_key="left"
    move_focus_right_key="right"
    grab_keys="true">
   ...
   <lang_indicator self_layout="default(x=r:2, y=0, w=48, h=20)" />
   ...
</keyboard>
```

4.8.13 progress_circle

进度圆环控件。progress_circle_t 是 widget_t 的子类控件，widget_t 的函数均适用于 progress_circle_t 控件。

1. 函数

progress_circle 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
progress_circle_create	创建 progress_circle 对象
progress_circle_cast	转换为 progress_circle 对象 (供脚本语言使用)。
progress_circle_set_value	设置值。
progress_circle_set_max	设置最大值。
progress_circle_set_format	设置格式。
progress_circle_set_line_width	设置环线的厚度。
progress_circle_set_start_angle	设置起始角度。
progress_circle_set_line_cap	设置线帽类型。
progress_circle_set_show_text	设置是否显示文本。
progress_circle_set_counter_clock_wise	设置是否为逆时针方向。
progress_circle_calc_line_dirty_rect	获取 progress_circle 的新值和旧值之间的绘制脏矩形。
progress_circle_get_widget_vtable	获取 progress_circle 虚表。

2. 属性

progress_circle 还提供了下面属性，详见下表：

属性名称	类型	说明
value	float_t	值 (缺省为 0)。
max	float_t	最大值 (缺省为 100)。
format	char*	数值到字符串转换时的格式，缺省为”%d”。
start_angle	int32_t	起始角度 (单位为度，缺省-90)。
line_width	uint32_t	环线的厚度 (缺省为 8)。
line_cap	char*	线帽类型 (round: 圆头，square: 方头，butt: 平头)。
counter_clock_wise	bool_t	是否为逆时针方向 (缺省为 FALSE)。
show_text	bool_t	是否显示文本 (缺省为 TRUE)。

3. 样式

progress_circle 可设置的样式，详见下表：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

样式属性	说明
bg_color	背景颜色
font_name	字体名称
font_size	字体大小
text_color	文本颜色
text_align_h	文本水平对齐的方式（取值：center|left|right）
text_align_v	文本垂直对齐的方式（取值：middle|top|bottom）
border_color	边框颜色
border_width	边框线宽
Border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称
bg_image_draw_type	背景图片的显示方式（取值：详见 image 控件一节）
fg_image	前景图片的名称
fg_image_draw_type	前景图片的显示方式（取值：详见 image 控件一节）
Icon	图标的名称
Spacer	间距
Margin	边距
margin_left	左边距
margin_right	右边距
margin_top	顶边距
margin_bottom	底边距
icon_at	图标的位置（取值：top|bottom|left|right）
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

样式设置代码如下：

```
<progress_circle>
  <style name="default" border_color="black" border_width="3" border="all" text_
↪color="green">
    <normal  bg_color="gray" fg_color="red"/>
        <disable  bg_color="gray" fg_color="red" round_radius="6" x_offset="3" y_
↪offset="3"/>
        <focused  bg_color="gray" fg_image="progress_circle"/>
  </style>
</progress_circle>
```

样式设置效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

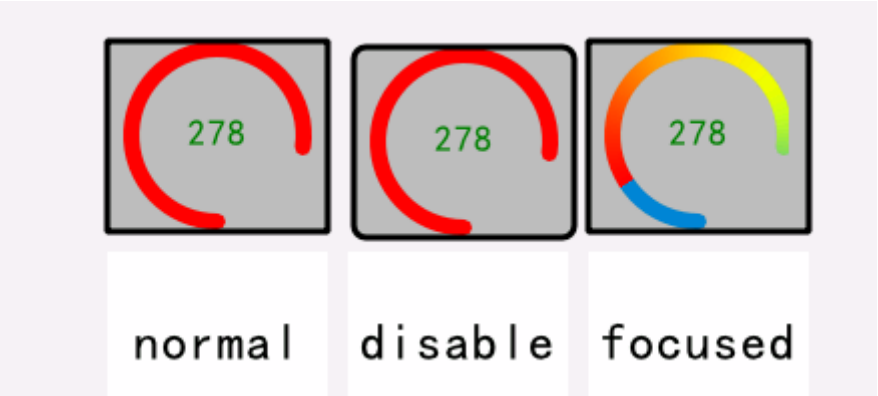


图 4.62　process_circle 样式设置效果

4. 事件

progress_circle 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	value_change_event_t	值即将改变事件。
EVT_VALUE_CHANGED	value_change_event_t	值改变事件。

1. 示例

- 在 xml 中，使用”progress_circle”标签创建进度圆环控件，代码如下，其中的样式设置见上文样式一节。

```
<!-- awtk/demos/assets/default/raw/ui/progress_circle.xml -->
<window >
  <view x="c" y="m" w="200" h="200" children_layout="default(r=2,c=2,s=5)">
    <progress_circle max="360" show_text="true"
      animation="value(from=0, to=360, yoyo_times=1000, duration=3000, easing=sin_
↪inout)"/>

    <progress_circle max="360" show_text="true" start_angle="90"
      animation="value(from=0, to=300, yoyo_times=1000, duration=3000, easing=sin_
↪inout)"/>

    <progress_circle style="image" max="360" show_text="true" start_angle="-213"
      animation="value(from=0, to=300, yoyo_times=1000, duration=3000, easing=sin_
↪inout)"/>

    <progress_circle style="image" max="100" show_text="true" start_angle="90"␣
↪start_angle="-225"
      counter_clock_wise="true" unit="%"
      animation="value(from=10, to=100, yoyo_times=1000, duration=3000, easing=sin_
↪inout)"/>
   </view>

   <button name="close" x="center" y="bottom:10" w="25%" h="30" text="Close"/>
</window>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

运行效果如下图所示：

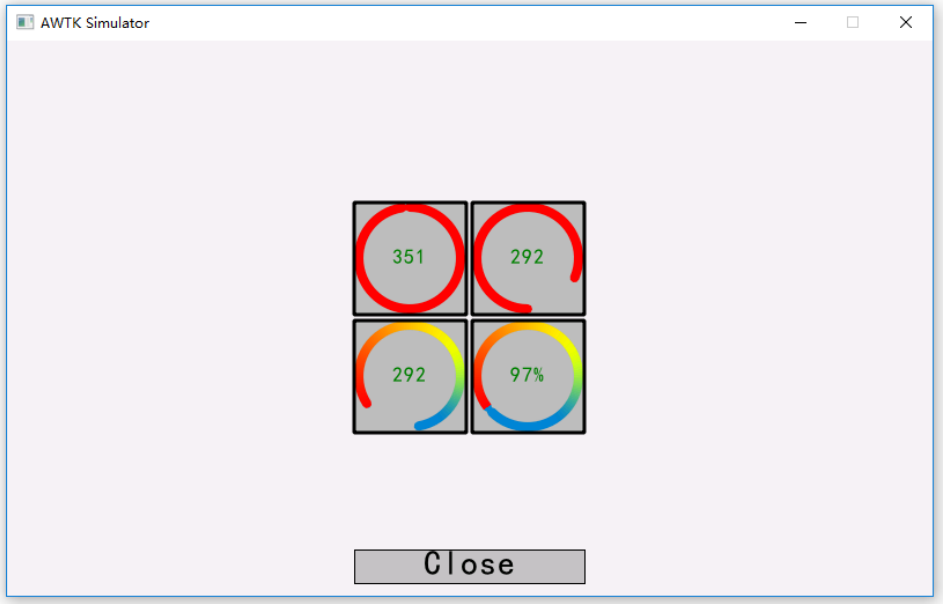


图 4.63　在 xml 中创建 progress_circle 的效果

- 在 C 代码中，使用函数 progress_circle_create 创建进度圆环控件，代码如下，其中的样式设置见上文样式一节。

```
progress_circle = progress_circle_create(win, 10, 10, 200, 200);
progress_circle_set_max(progress_circle, 360);
widget_set_value(progress_circle, 128);
```

运行效果如下图所示：

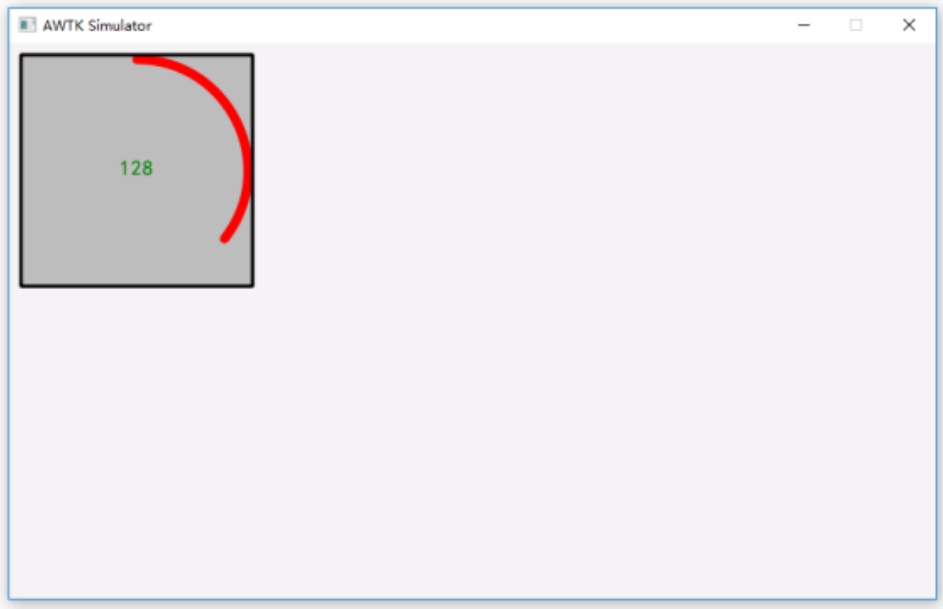


图 4.64　在 C 中创建 progress_circle 的效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.8.14 rich_text

图文混排控件，实现简单的图文混排。rich_text_t 是 widget_t 的子类控件，widget_t 的函数均适用于 rich_text_t 控件。

1. 函数

rich_text 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
rich_text_create	创建 rich_text 对象
rich_text_set_text	设置文本。
rich_text_set_yslidable	设置是否允许 y 方向滑动。
rich_text_cast	转换为 rich_text 对象 (供脚本语言使用)。
rich_text_get_widget_vtable	获取 rich_text 虚表。

2. 属性

rich_text 还提供了下面属性，详见下表：

属性名称	类型	说明
line_gap	uint32_t	行间距。
yoffset	int32_t	y 偏移。
yslidable	bool_t	标识控件是否允许上下拖动。
need_reset	bool_t	标识控件是否需要重新绘图。

3. 样式

rich_text 可设置的样式，详见下表：

样式属性	说明
bg_color	背景颜色
font_name	字体名称
font_size	字体大小
text_color	文本颜色
text_align_h	文本水平对齐的方式（取值：center|left|right）
text_align_v	文本垂直对齐的方式（取值：middle|top|bottom）
border_color	边框颜色
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称
bg_image_draw_type	背景图片的显示方式（取值：详见 image 控件一节）
icon	图标的名称
spacer	间距
margin	边距
margin_left	左边距
margin_right	右边距
margin_top	顶边距
margin_bottom	底边距
icon_at	图标的位置（取值：top|bottom|left|right）

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

样式属性	说明
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

样式设置代码如下：

```
<rich_text>
  <style name="default" font_name="default" font_size="30" border_color="black"
↪border_width="3"
  border="all" text_color="green">
    <normal  bg_color="gray"  text_align_h="left" fg_color="red"/>
        <disable  bg_color="gray" text_align_v="bottom" fg_color="red" round_
↪radius="6" x_offset="3"
  y_offset="3"/>
        <focused  bg_color="gray"  bg_image="1" bg_image_draw_type="scale"/>
  </style>
</rich_text>
```

运行效果如下图所示：

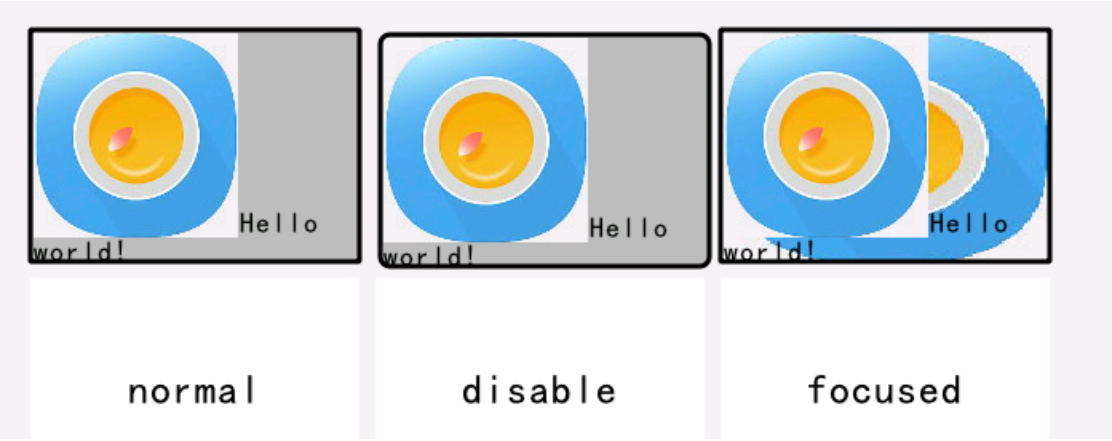


图 4.65　rich_text 样式设置效果

4. 示例

- 在 xml 中，使用”rich_text”标签创建图文混排控件，代码如下，其中的样式设置见上文样式一节。

```
<!-- awtk\demos\assets\default\raw\ui\rich_text1.xml -->
<window anim_hint="htranslate" >
    <rich_text line_gap="5" x="0" y="20" w="100%" h="400"
    text="<image draw_type=&quota;auto&quota; name=&quota;1&quota;/>
          <image name=&quota;2&quota;/>
          <font color=&quota;red&quota; size==&quota;18&quota;>Hello world!</font>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
          <image name=&quota;earth&quota;/>
        <font color=&quota;blue&quota; size==&quota;18&quota;>ProTip!</font>"/>

  <button name="close" text="Close" x="c" y="bottom:10" w="25%" h="40"/>
</window>
```

运行效果如下图所示：

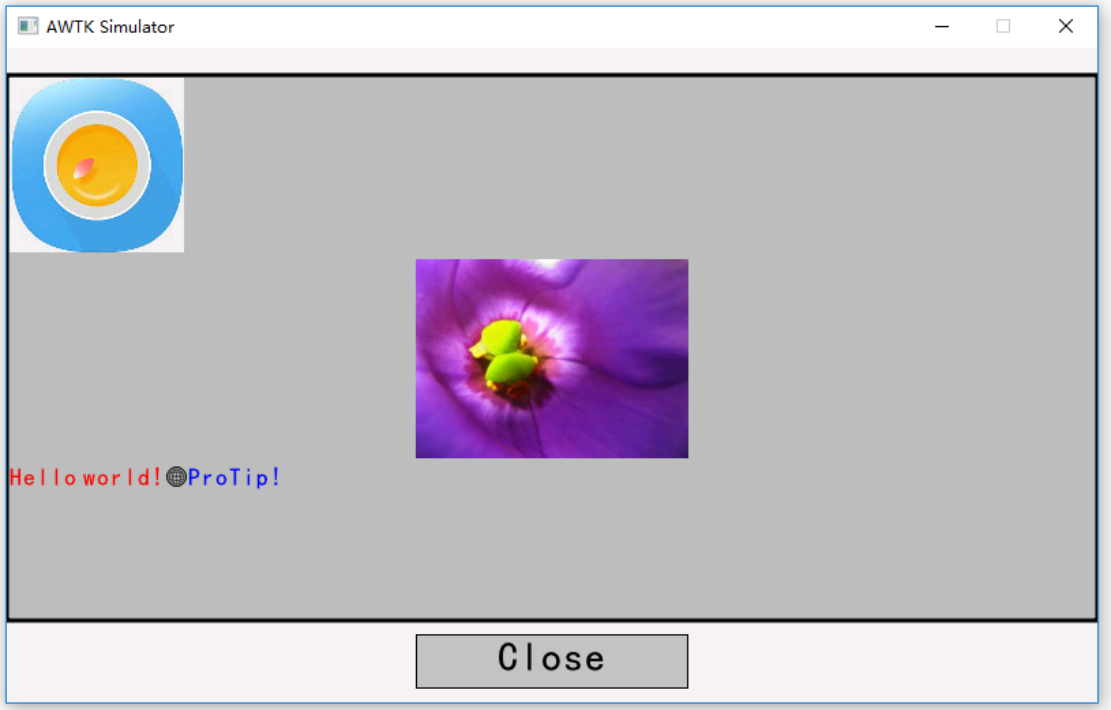


图 4.66　在 xml 中创建 rich_text 的效果

- 在 C 代码中，使用 rich_text_create 创建图文混排控件，代码如下，其中的样式设置见上文样式一节。

```
widget_t* rich_text = rich_text_create(win, 0, 0, 0, 0);
widget_set_text_utf8(rich_text,
    "<image name=\"bricks\"/>                                                    \
    <font color=\"gold\" align_v=\"bottom\" size=\"24\">hello awtk!</font>       \
    <font color=\"green\" size=\"20\">ProTip!\
    The feed shows you events from people you follow and repositories you watch.␣
↪nhello world. </font> \
    <font color=\"red\" size=\"20\">确定取消中文字符测试。确定。取消。中文字符测</
↪font>");
widget_set_self_layout_params(rich_text, "center", "middle", "100%", "100%");
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

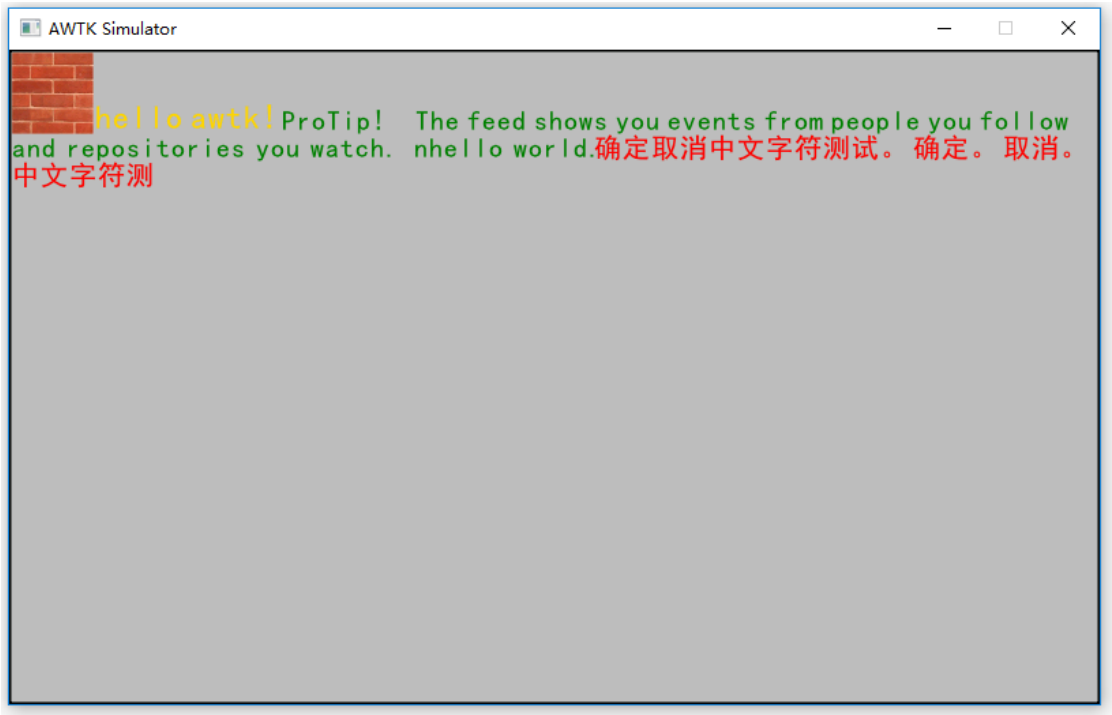


图 4.67　在 C 中创建 rich_text 的效果

4.8.15　slide_menu

左右滑动菜单控件。一般用一组按钮作为子控件，通过左右滑动改变当前的项。除了当菜单使用外，也可以用来切换页面。slide_menu_t 是 widget_t 的子类控件，widget_t 的函数均适用于 slide_menu_t 控件。

1. 函数

slide_menu 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
slide_menu_create	创建 slide_menu 对象
slide_menu_cast	转换 slide_menu 对象 (供脚本语言使用)。
slide_menu_set_value	设置当前项。
slide_menu_set_align_v	设置垂直对齐方式。
slide_menu_set_min_scale	设置最小缩放比例。
slide_menu_set_spacer	设置菜单项之间的间距。
slide_menu_set_menu_w	设置菜单项的宽度。
slide_menu_set_clip	设置是否动态裁剪菜单项。
slide_menu_scroll_to_prev	切换至上一项。
slide_menu_scroll_to_next	切换至下一项。
slide_menu_get_widget_vtable	获取 slide_menu 虚表。

2. 属性

slide_menu 还提供了下面属性，详见下表：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

属性名称	类型	说明
value	int32_t	值。代表当前选中项的索引。
align_v	align_v_t	垂直对齐方式。
min_scale	float_t	最小缩放比例。
spacer	int32_t	菜单项之间的间距。
menu_w	char*	菜单项的宽度 (后面加上 px 为像素点，不加 px 为相对百分比坐标 0.0f 到 1.0f)(空字符串则使用控件高度)。
clip	bool_t	是否动态裁剪菜单项 (默认裁剪，不裁剪时，如果显示偶数项，左边会多一项)。

3. 样式

slide_menu 可设置的样式，详见下表：

样式属性	说明
bg_color	背景颜色
border_color	边框颜色
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称
bg_image_draw_type	背景图片的显示方式（取值：详见 image 控件一节）
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
mask_color	蒙版颜色
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

样式设置代码如下：

```
<style name="default" border_color="black" border_width="3" border="all">
  <normal bg_color="gray"  mask_color="red" />
  <disable bg_color="gray" round_radius="6" x_offset="3" y_offset="3"/>
  <focused  bg_image="1" bg_image_draw_type="scale"/>
</style>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

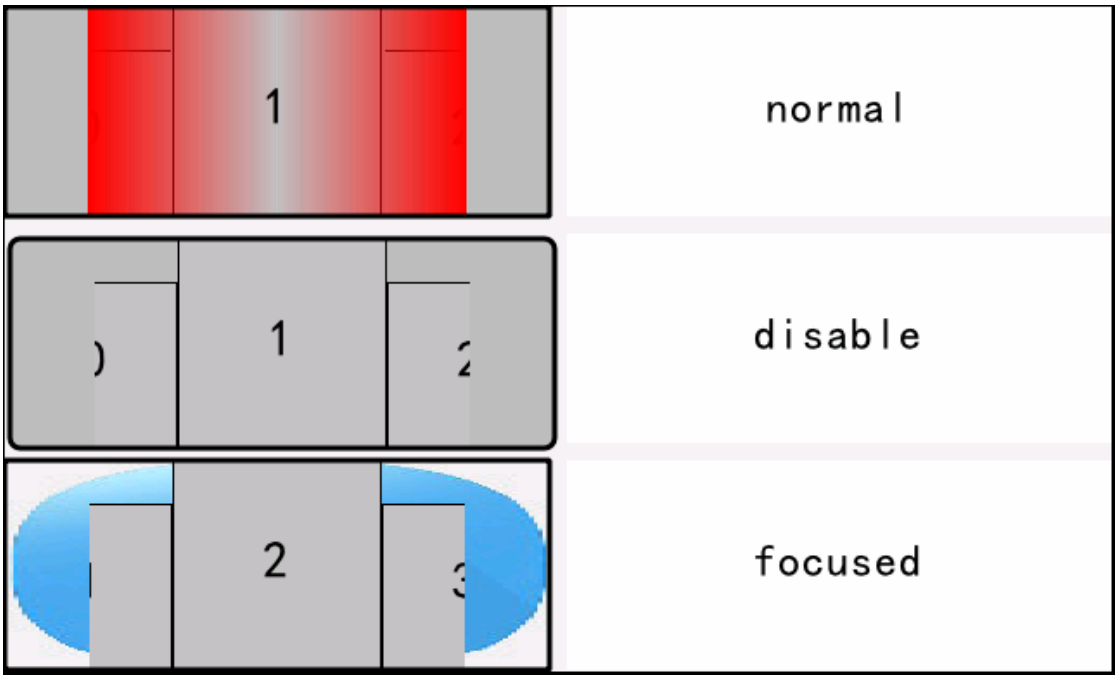


图 4.68　slide_menu 样式效果

4. 事件

slide_menu 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	value_change_event_t	值 (当前项) 即将改变事件。
EVT_VALUE_CHANGED	value_change_event_t	值 (当前项) 改变事件。

5. 示例

- 在 xml 中，使用”slide_menu”标签创建左右滑动菜单，代码如下，其中的样式设置见上文样式一节。

```
<!-- awtk/demos/assets/default/raw/ui/slide_menu.xml -->
<window theme="slide_menu" children_layout="default(c=1,h=60,s=10)">
  <slide_menu style="mask" align_v="top">
    <button style="slide_button" text="0"/>
    <button style="slide_button" text="1"/>
    <button style="slide_button" text="2"/>
    <button style="slide_button" text="3"/>
    <button style="slide_button" text="4"/>
  </slide_menu>
</window>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

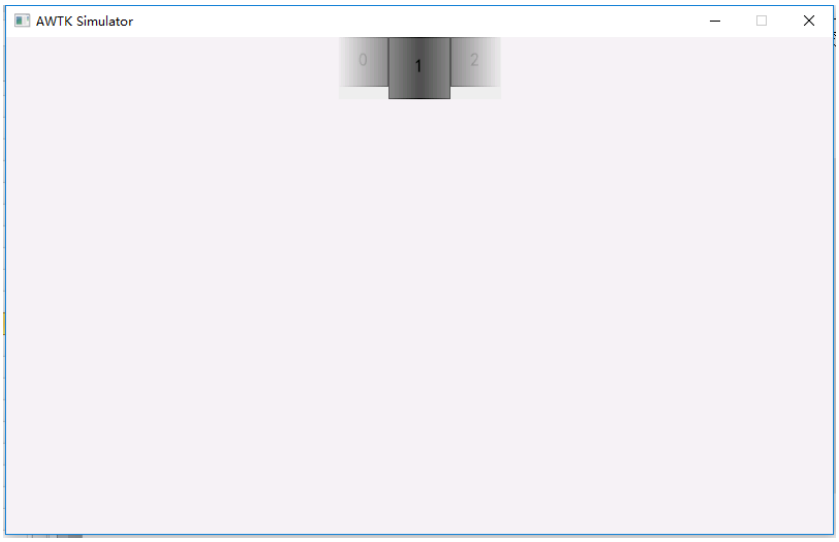


图 4.69　在 xml 中创建 slide_menu 的效果

- 在 C 代码中，使用函数 slide_menu_create 创建左右滑动菜单，代码如下，样式设置见上文样式一节。

```
slide_menu = slide_menu_create(win, 10, 10, 300, 60);
b = button_create(slide_menu, 0, 0, 0, 0);
widget_set_text_utf8(b, "1");
b = button_create(slide_menu, 0, 0, 0, 0);
widget_set_text_utf8(b, "2");
b = button_create(slide_menu, 0, 0, 0, 0);
widget_set_text_utf8(b, "3");
b = button_create(slide_menu, 0, 0, 0, 0);
widget_set_text_utf8(b, "4");
```

运行效果如下图所示：

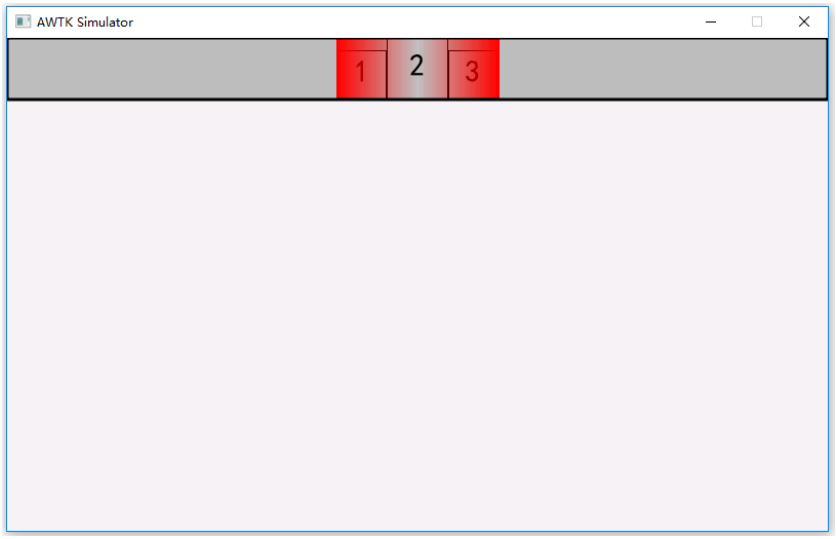


图 4.70　在 C 中创建 slide_menu 的效果

可按下面的方法关注当前项改变的事件：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
widget_on(slide_menu, EVT_VALUE_CHANGED, on_current_changed, slide_menu);
```

可按下面的方法关注当前按钮被点击的事件：

```
widget_on(b, EVT_CLICK, on_button_click, b);
```

4.8.16 slide_view

滑动视图。滑动视图可以管理多个页面，并通过滑动来切换当前页面。也可以管理多张图片，让它们自动切换。slide_view_t 是 widget_t 的子类控件，widget_t 的函数均适用于 slide_view_t 控件。

1. 函数

slide_view 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
slide_view_create	创建 slide_view 对象
slide_view_cast	转换为 slide_view 对象 (供脚本语言使用)。
slide_view_set_auto_play	设置为自动播放模式。
slide_view_set_active	设置当前页的序号 (默认启用动画)。
slide_view_set_active_ex	设置当前页的序号。
slide_view_set_vertical	设置为上下滑动 (缺省为左右滑动)。
slide_view_set_anim_hint	设置页面切换动画。
slide_view_set_loop	设置循环切换模式。
slide_view_set_drag_threshold	设置拖拽临界值。
slide_view_set_animating_time	设置动画时间。
slide_view_remove_index	删除指定序号页面。
slide_view_get_widget_vtable	获取 slide_view 虚表。

2. 属性

slide_view 还提供了下面属性，详见下表：

属性名称	类型	说明
vertical	bool_t	是否为上下滑动模式。
auto_play	uint16_t	自动播放。0 表示禁止自动播放，非 0 表示自动播放时每一页播放的时间。
loop	bool_t	循环切换模式。
anim_hint	char*	页面切换效果。
drag_threshold	uint32_t	拖动临界值。
animating_time	uint32_t	动画时间（单位：毫秒）。

3. 样式

slide_view 可设置的样式，详见 canvas_widget 一节的样式小节。样式设置代码如下：

```
<slide_view>
  <style name="default" border_color="black" border_width="3" border="all" >
    <normal  bg_color="gray" />
          <disable  bg_color="gray" round_radius="6" x_offset="3" y_offset="3"/>
         <focused  bg_color="gray" bg_image_draw_type="center" bg_image="1"/>
  </style>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
</slide_view>
```



图 4.71　slide_view 样式效果

4. 事件

slide_view 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	value_change_event_t	值 (当前页) 即将改变事件。
EVT_VALUE_CHANGED	value_change_event_t	值 (当前页) 改变事件。
EVT_PAGE_CHANGED	event_t	页面改变事件。
EVT_PAGE_CHANGING	event_t	页面正在改变。

5. 示例

- 在 xml 中，使用”slide_view”标签创建滑动视图控件，代码如下，样式设置见上文样式一节。

```
<slide_view x="0" y="0" w="100%" h="100%" loop="true">
        <view x="0" y="0" w="100%" h="100%" children_layout="default(w=60,h=60,m=5,
↪s=10)">
        </view>
        <view x="0" y="0" w="100%" h="100%" children_layout="default(w=60,h=60,m=5,
↪s=10)">
        </view>
</slide_view>
<slide_indicator x="0" y="b" w="100%" h="20" style="blue"/>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.



图 4.72　在 xml 中创建 slide_view 的效果

- 在 C 代码中，使用函数 slide_view_create 创建滑动视图控件，代码如下，样式设置见上文样式一节，运行效果见上图。

```
slide_view = slide_view_create(win, 0, 0, win->w, win->h);
```

4.8.17　svg_image

SVG 图片控件。svg_image_t 是 image_base_t 的子类控件，image_base_t 的函数均适用于 svg_image_t 控件。

1. 函数

svg_image 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
svg_image_create	创建 svg_image 对象
svg_image_set_image	设置控件的图片名称。
svg_image_set_cache_mode	控件设置是否开启离线缓存渲染模式。
svg_image_set_draw_type	控件设置 svg 图片绘制模式。
svg_image_cast	转换为 svg_image 对象 (供脚本语言使用)。
svg_image_get_widget_vtable	获取 svg_image 虚表。

2. 属性

svg_image 还提供了下面属性，详见下表：

属性名称	类型	说明
is_cache_mode	bool_t	离线缓存渲染模式。
draw_type	image_draw_type_t	svg 图片的绘制方式 (支持旋转缩放，目前仅支持 scale、scale_auto 模式)。

3. 样式

svg_image 可设置的样式，详见下表：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

样式属性	说明
bg_color	背景颜色
font_name	字体名称
font_size	字体大小
text_color	文本颜色
text_align_h	文本水平对齐的方式（取值：center|left|right）
text_align_v	文本垂直对齐的方式（取值：middle|top|bottom）
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）
icon	图标的名称
spacer	间距
margin	边距
margin_left	左边距
margin_right	右边距
margin_top	顶边距
margin_bottom	底边距
icon_at	图标的位置（取值：top|bottom|left|right）
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

样式设置代码如下：

```
<svg>
  <style name="default">
    <normal border_color="#000000" fg_color="red" />
  </style>
</svg>
```

4. 示例

- 在 xml 中，使用”svg_image”标签创建 SVG 图片控件，代码如下，其中的样式设置见上文样式一节。

```
<!-- awtk/demos/assets/default/raw/ui/svg_image.xml -->
<window anim_hint="htranslate" children_layout="default(c=4,r=4)">
  <svg image="china" />
  <svg image="pointer_1" />
  <svg image="girl" />
  <svg image="pointer_4" />
  <svg image="china" animation="rotation(from=0, to=6.28, yoyo_times=1000,
↪duration=5000);scale(x_from=0, x_to=2, y_from=0, y_to=2, yoyo_times=1000,
↪duration=5000);"/>
  <svg image="pointer_1" animation="rotation(from=0, to=6.28, yoyo_times=1000,
↪duration=5000);scale(x_from=0, x_to=2, y_from=0, y_to=2, yoyo_times=1000,
↪duration=5000);"/>
```

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  <svg image="girl" animation="rotation(from=0, to=6.28, yoyo_times=1000,␣
↪duration=5000);scale(x_from=0, x_to=2, y_from=0, y_to=2, yoyo_times=1000,␣
↪duration=5000);"/>
  <svg image="pointer_4" animation="rotation(from=0, to=6.28, yoyo_times=1000,␣
↪duration=5000);scale(x_from=0, x_to=2, y_from=0, y_to=2, yoyo_times=1000,␣
↪duration=5000);"/>
</window>
```

运行效果如下图所示：

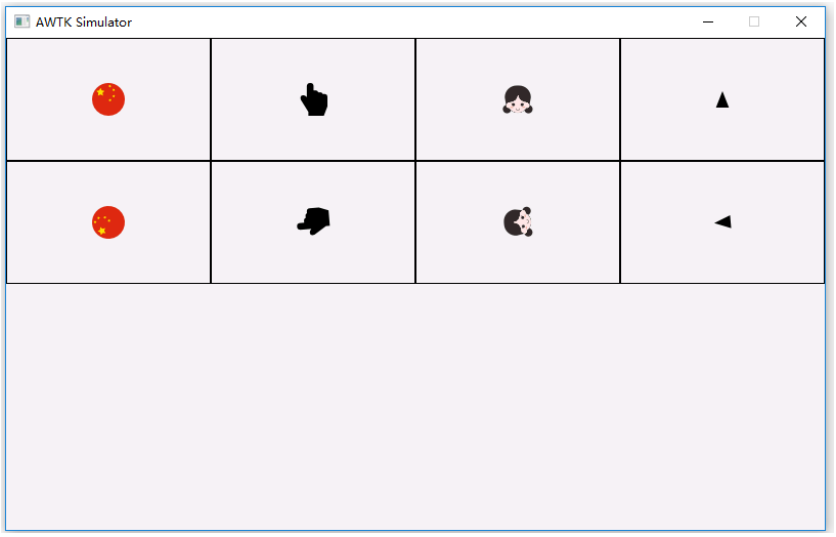


图 4.73　在 xml 中创建 svg 的效果

- 在 C 代码中，使用函数 svg_image_create 创建 SVG 图片控件，代码如下，其中的样式设置见上文样式一节。

```
widget_t* image = svg_image_create(win, 10, 10, 200, 200);
image_set_image(image, "girl");
```

创建之后需要用 widget_set_image 设置图片名称。

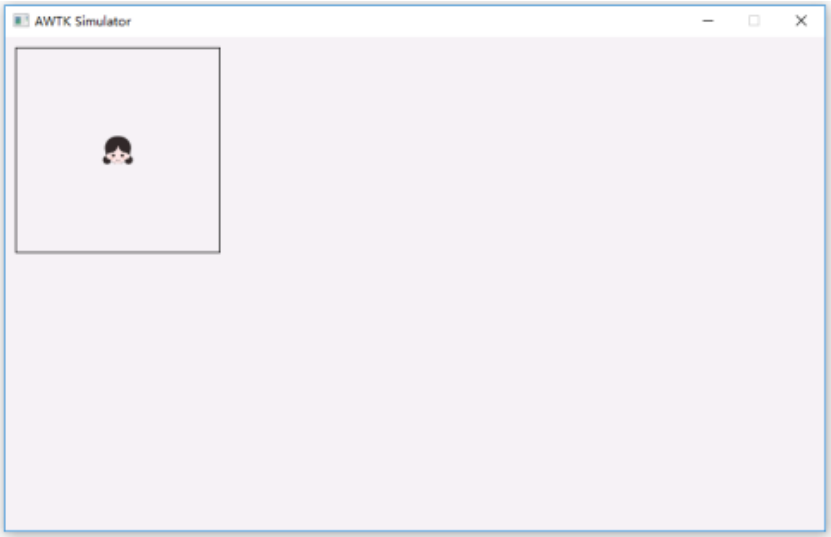


图 4.74　在 C 中创建 svg 的效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.8.18 switch

开关控件。switch_t 是 widget_t 的子类控件，widget_t 的函数均适用于 switch_t 控件。

注: 使用该控件的时候需要将 awtk/demos/assets/default/raw/images/x1 目录下：switch.png 复制到自己的应用目录下（如：res/assets/default/raw/images/xx），否则将不能正常显示。

1. 函数

switch 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
switch_create	创建 switch 对象
switch_set_value	设置控件的值。
switch_cast	转换 switch 对象 (供脚本语言使用)。
switch_get_widget_vtable	获取 switch 虚表。

2. 属性

switch 还提供了下面属性，详见下表：

属性名称	类型	说明
value	bool_t	值。
max_xoffset_ratio	float_t	当开关处于关闭时，图片偏移相对于图片宽度的比例 (缺省为 1/3)。

3. 样式

switch 可设置的样式，详见下表：

样式属性	说明
bg_color	背景颜色
fg_color	前景颜色
border_color	边框颜色
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称
bg_image_draw_type	背景图片的显示方式（取值：详见 image 控件一节）
fg_image	前景图片的名称
fg_image_draw_type	前景图片的显示方式（取值：详见 image 控件一节）
margin	边距
margin_left	左边距
margin_right	右边距
margin_top	顶边距
margin_bottom	底边距
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
selected_bg_color	编辑器（包括 edit 和 mledit 控件）中选中区域的背景颜色
selected_fg_color	编辑器（包括 edit 和 mledit 控件）中选中区域的前景颜色
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

样式属性	说明
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

样式设置代码如下：

```
<switch>
  <style name="default" border_color="black" >
    <normal  bg_image="switch" />
  </style>
  <style name="no_image" border_color="black" >
    <normal  bg_color="#828282" selected_bg_color="#338fff" fg_color="#ffffff"
↪round_radius="11"/>
  </style>
  <style name="image" border_color="black" >
    <normal  bg_image="switch" round_radius="11"/>
  </style>
</switch>
```

运行效果如下图所示：

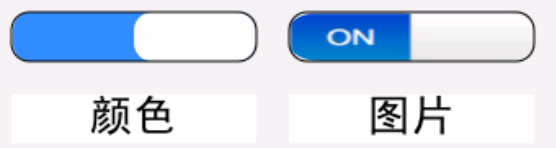


图 4.75　switch 样式设置效果

4. 事件

switch 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_CLICK	pointer_event_t	点击事件。
EVT_VALUE_WILL_CHANGE	value_change_event_t	值 (开关状态) 即将改变事件。
EVT_VALUE_CHANGED	value_change_event_t	值 (开关状态) 改变事件。

5. 示例

- 在 xml 中，使用”switch”标签创建开关控件，代码如下，样式设置见上文样式一节。

```
<!-- awtk/demos/assets/default/raw/ui/switch.xml -->
<window anim_hint="htranslate">
  <switch x="10" y="60" w="60" h="22" />
  <switch x="200" y="60" w="60" h="22" round_radius="10" />
  <button name="close" x="center" y="middle" w="50%" h="30" text="close"/>
</window>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

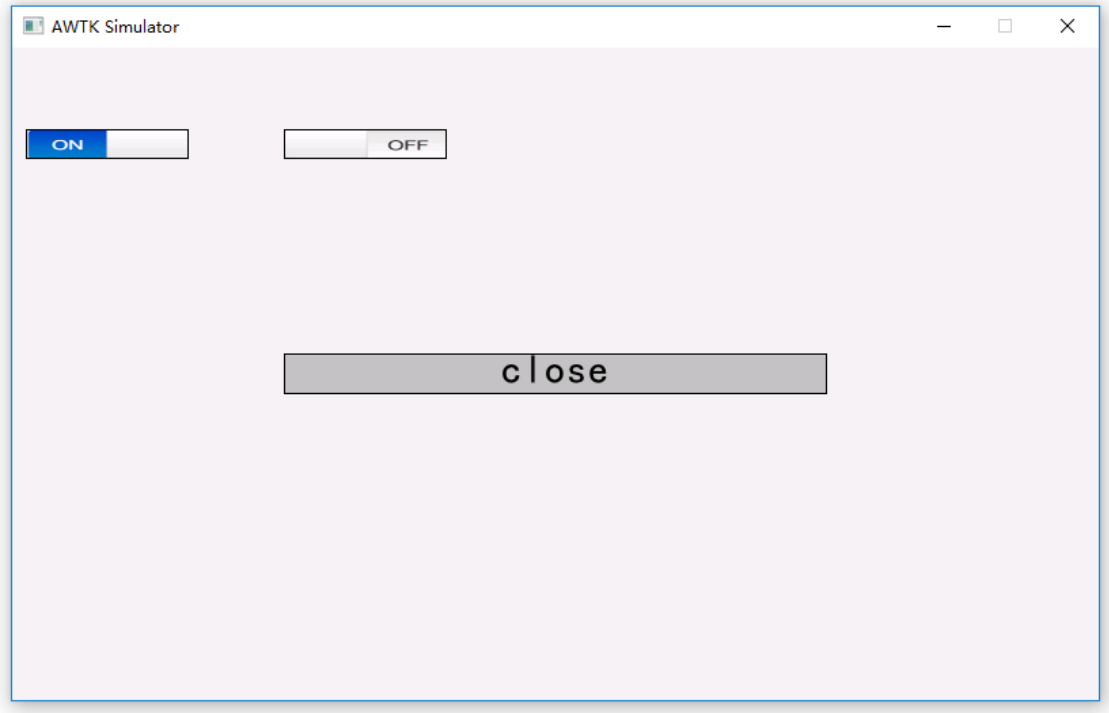


图 4.76　在 xml 中创建 switch 的效果

- 在 C 代码中，使用函数 switch_create 创建开关控件，代码如下，样式设置详见上文样式一节。效果见上图。

```
widget_t* sw = switch_create(win, 10, 10, 80, 30);
widget_on(sw, EVT_VALUE_CHANGED, on_changed, NULL);
```

4.8.19　text_selector

文本选择器控件，通常用于选择日期和时间等。目前需要先设置 options 和 visible_nr，再设置其它参数（在 XML 中也需要按此顺序）。text_selector_t 是 widget_t 的子类控件，widget_t 的函数均适用于 text_selector_t 控件。

1. 函数

text_selector 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
text_selector_create	创建 text_selector 对象
text_selector_cast	转换 text_selector 对象 (供脚本语言使用)。
text_selector_reset_options	重置所有选项。
text_selector_count_options	获取选项个数。
text_selector_append_option	追加一个选项。
text_selector_set_options	设置选项。
text_selector_set_range_options_ex	设置一系列的整数选项。
text_selector_set_range_options	设置一系列的整数选项。
text_selector_get_option	获取第 index 个选项。
text_selector_get_value	获取 text_selector 的值。
text_selector_set_value	设置 text_selector 的值。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
text_selector_get_text	获取 text_selector 的文本。
text_selector_set_text	设置 text_selector 的文本。
text_selector_set_selected_index	设置第 index 个选项为当前选中的选项。
text_selector_set_visible_nr	设置可见的选项数。
text_selector_set_localize_options	设置是否本地化 (翻译) 选项。
text_selector_set_loop_options	设置是否循环选项。
text_selector_set_yspeed_scale	设置 Y 轴偏移速度比例。
text_selector_set_animating_time	设置滚动动画播放时间。
text_selector_set_enable_value_animator	设置是否修改值时启用动画。
text_selector_set_mask_easing	设置绘制蒙版的变化趋势。
text_selector_set_mask_area_scale	设置绘制蒙版的区域占比（范围 0~1）。
text_selector_get_widget_vtable	获取 text_selector 虚表。

2. 属性

text_selector 还提供了下面属性，详见下表：

属性名称	类型	说明
visible_nr	uint32_t	可见的选项数量 (只能是 1 或者 3 或者 5，缺省为 5)。
selected_index	int32_t	当前选中的选项。
options	char*	设置可选项 (英文冒号 (:) 分隔值和文本，英文分号 (;) 分隔选项，如:1:red;2:green;3:blue)。
yspeed_scale	float_t	y 偏移速度比例。
animating_time	uint32_t	滚动动画播放时间。(单位毫秒)
localize_options	bool_t	是否本地化 (翻译) 选项 (缺省为 FALSE)。
loop_options	bool_t	是否循环选项 (缺省为 FALSE)。
enable_value_animator	bool_t	是否修改值时启用动画。
mask_easing	easing_type_t	绘制蒙版的变化趋势。
mask_area_scale	float_t	绘制蒙版的区域占比（范围 0~1）。

3. 样式

text_selector 可设置的样式，详见下表。

样式属性	说明
bg_color	背景颜色
fg_color	前景颜色
mask_color	蒙版颜色
font_name	字体名称
font_size	字体大小
text_color	文本颜色
text_align_h	文本水平对齐的方式（取值：center|left|right）
text_align_v	文本垂直对齐的方式（取值：middle|top|bottom）
border_color	边框颜色
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

样式属性	说明
bg_image	背景图片的名称
bg_image_draw_type	背景图片的显示方式
margin	边距
margin_left	左边距
margin_right	右边距
margin_top	顶边距
margin_bottom	底边距
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

样式设置代码如下：

```
<text_selector>
  <style name="default" fg_color="#a0a0a0" text_color="black" text_align_h="center
↪">
    <normal bg_color="#f0f0f0"/>
  </style>
  <style name="dark" fg_color="#a0a0a0" text_color="black" text_align_h="center">
    <normal bg_color="#ffffff" mask_color="#404040" border_color="#404040"/>
  </style>
</text_selector>
```

4. 事件

text_selector 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	value_change_event_t	值 (当前项) 即将改变事件。
EVT_VALUE_CHANGED	value_change_event_t	值 (当前项) 改变事件。

5. 示例

- 在 xml 中，使用”text_selector”标签创建文本选择器控件，代码如下，样式设置见上文样式一节。

```
<!-- awtk/demos/assets/default/raw/ui/text_selector.xml -->
<window anim_hint="htranslate">
  <row  x="10" y="30" w="100%" h="150" children_layout="default(row=1,col=3)">
    <text_selector style="dark" options="2000-2050" text="2018"/>
    <text_selector options="1-12" text="1"/>
    <text_selector options="1-31" text="1"/>
  </row>
  <button name="close" x="center" y="middle" w="50%" h="30" text="close"/>
  <row  x="10" y="bottom:30" w="100%" h="90" children_layout="default(row=1,col=3)
↪">
```

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    <text_selector style="dark" options="2000-2050" visible_nr="3" text="2018"/>
    <text_selector options="1-12" visible_nr="3" text="1"/>
    <text_selector options="red;green;blue;gold;orange" visible_nr="3" text="red"/>
  </row>
</window>
```

运行效果如下图所示：

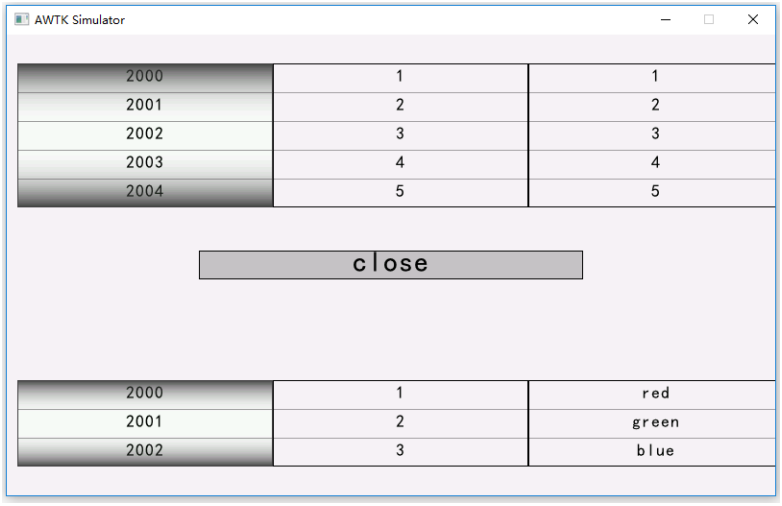


图 4.77　在 xml 中创建 text_selector 的效果

- 在 C 代码中，使用函数 text_selector_create 创建文本选择器控件，代码如下，样式设置见上文样式一节。

```
widget_t* ts = text_selector_create(win, 10, 10, 80, 150);
text_selector_set_options(ts, "1:red;2:green;3:blue;4:orange;5:gold");
text_selector_set_value(ts, 1);
widget_use_style(ts, "dark");
```

运行效果如下图所示：

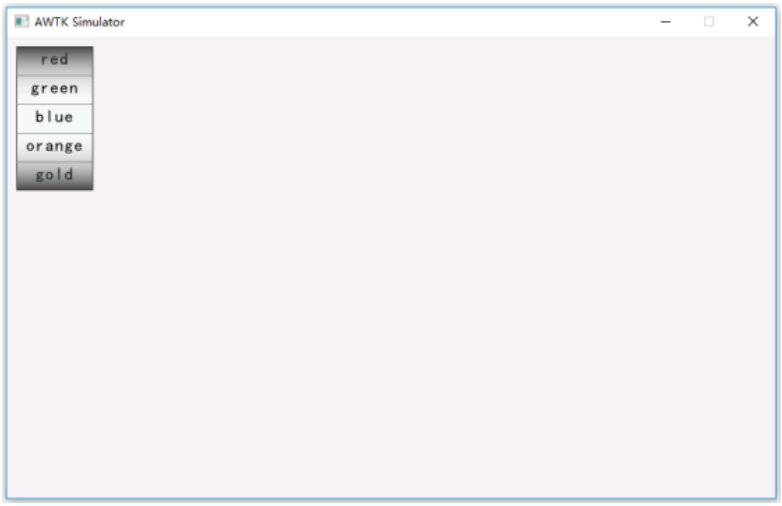


图 4.78　在 C 中创建 text_selector 的效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.8.20 time_clock

模拟时钟控件。time_clock_t 是 widget_t 的子类控件，widget_t 的函数均适用于 time_clock_t 控件。

1. 函数

time_clock 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
time_clock_create	创建 time_clock 对象
time_clock_cast	转换为 time_clock 对象 (供脚本语言使用)。
time_clock_set_hour	设置小时的值。
time_clock_set_minute	设置分钟的值。
time_clock_set_second	设置秒的值。
time_clock_set_hour_image	设置小时的图片。
time_clock_set_minute_image	设置分钟的图片。
time_clock_set_second_image	设置秒的图片。
time_clock_set_bg_image	设置背景图片。
time_clock_set_image	设置中心图片。
time_clock_set_hour_anchor	设置小时指针的旋转锚点。
time_clock_set_minute_anchor	设置分钟指针的旋转锚点。
time_clock_set_second_anchor	设置秒钟指针的旋转锚点。
time_clock_get_widget_vtable	获取 time_clock 虚表。

2. 属性

time_clock 还提供了下面属性，详见下表：

属性名称	类型	说明
hour	int32_t	小时。
minute	int32_t	分钟。
second	int32_t	秒。
image	char*	中心图片。
bg_image	char*	背景图片。
hour_image	char*	时针图片。
minute_image	char*	分针图片。
second_image	char*	秒针图片。
hour_anchor_x	char*	时针图片旋转锚点 x 坐标。(后面加上 px 为像素点，不加 px 为相对百分比坐标 0.0f 到 1.0f)
hour_anchor_y	char*	时针图片旋转锚点 y 坐标。(后面加上 px 为像素点，不加 px 为相对百分比坐标 0.0f 到 1.0f)
minute_anchor_x	char*	分针图片旋转锚点 x 坐标。(后面加上 px 为像素点，不加 px 为相对百分比坐标 0.0f 到 1.0f)
minute_anchor_y	char*	分针图片旋转锚点 y 坐标。(后面加上 px 为像素点，不加 px 为相对百分比坐标 0.0f 到 1.0f)
second_anchor_x	char*	秒针图片旋转锚点 x 坐标。(后面加上 px 为像素点，不加 px 为相对百分比坐标 0.0f 到 1.0f)

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

属性名称	类型	说明
second_anchor_y	char*	秒针图片旋转锚点 y 坐标。(后面加上 px 为像素点，不加 px 为相对百分比坐标 0.0f 到 1.0f)

3. 示例

- 在 xml 中，使用”time_clock”标签创建模拟时钟控件，代码如下：

```
<!-- awtk\demos\assets\default\raw\ui\time_clock.xml -->
<window anim_hint="htranslate" >
  <time_clock x="c" y="m" w="300" h="300" bg_image="clock_bg" image="clock" hour_
↪image="clock_hour" minute_image="clock_minute" second_image="clock_second"/>
</window>
```

运行效果如下：

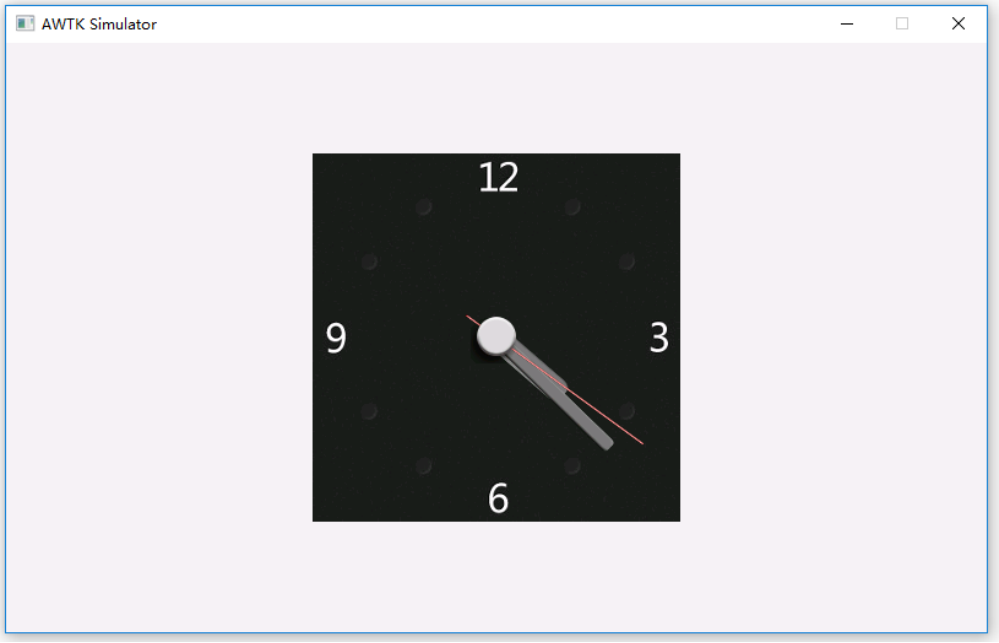


图 4.79　在 xml 中创建 time_clock 的效果

- 在 C 代码中，使用函数 time_clock_create 创建模拟时钟控件，代码如下：

```
widget_t* tc = time_clock_create(win, 10, 10, 300, 300);
time_clock_set_image(tc, "clock");
time_clock_set_bg_image(tc, "clock_bg");
time_clock_set_hour_image(tc, "clock_hour");
time_clock_set_minute_image(tc, "clock_minute");
time_clock_set_second_image(tc, "clock_second");
```

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

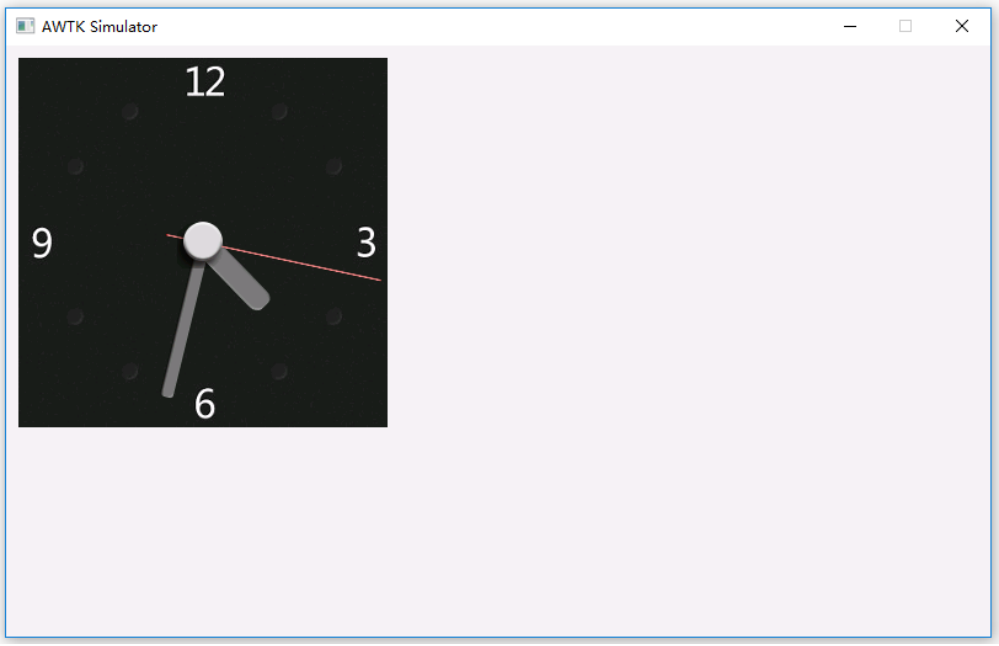


图 4.80　在 C 中创建 time_clock 的效果

4.8.21　list_view

列表视图控件。列表视图控件是一个可以垂直滚动的列表控件。如果不需要滚动，可以用 view 控件配置适当的 layout 参数作为列表控件。列表视图中的列表项可以固定高度，也可以使用不同高，请参考：awtk/demos/assets/default/raw/ui/list_view_vh.xml

列表视图中的列表项可以固定高度，也可以使用不同高度。列表视图控件中可以有滚动条，也可以没有滚动条。既可以使用移动设备风格的滚动条，也可以使用桌面风格的滚动条。list_view_t 是 widget_t 的子类控件，widget_t 的函数均适用于 list_view_t 控件。

1. 函数

list_view 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
list_view_create	创建 list_view 对象
list_view_set_item_height	设置列表项的高度。
list_view_set_default_item_height	设置列表项的缺省高度。
list_view_set_auto_hide_scroll_bar	设置是否自动隐藏滚动条。
list_view_set_floating_scroll_bar	设置滚动条是否悬浮在 scroll_view 上面。
list_view_cast	转换为 list_view 对象 (供脚本语言使用)。
list_view_reinit	list_view 重新初始化。
list_view_get_widget_vtable	获取 list_view 虚表。

2. 属性

list_view 还提供了下面属性，详见下表：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

属性名称	类型	说明
item_height	int32_t	列表项的高度。如果 item_height > 0，所有列表项使用固定高度，否则使用列表项自身的高度。
default_item_height	int32_t	列表项的缺省高度。如果 item_height <= 0 而且列表项自身的高度 <= 0，则使用缺省高度。
auto_hide_scroll_bar	bool_t	如果不需要滚动条时，自动隐藏滚动条。
floating_scroll_bar	bool_t	滚动条是否悬浮在 scroll_view 上面

设置列表项高度的优先级：item_height > 列表项自身高度 > default_item_height，具体代码如下：

```
<list_view x="0" y="0" w="30%" h="100%" item_height="40" default_item_height="80" >
    <scroll_view name="view" x="0"  y="0" w="-12" h="100%">
      <tab_button text="page1" value="true" h="120" />
      <tab_button text="page2" />
      ...
    </scroll_view>
    <scroll_bar_d name="bar" x="right" y="0" w="12" h="100%" value="0"/>
</list_view>
```

3. 样式

list_view 可设置的样式，详见 canvas_widget 一节的样式小节。样式设置代码如下：

```
<list_view>
    <style name="default" text_color="white" round_radius="10">
        <normal bg_color="#292929" />
    </style>
</list_view>
```

4. 示例

- 在 xml 中，使用”list_view”标签创建列表视图控件，代码如下，list_view 样式设置见上文样式一节，list_item 样式设置详见 list_item 一节，scroll_view 样式设置详见 scroll_view 一节。

```
<window anim_hint="htranslate">
<list_view x="0"  y="30" w="100%" h="-80"       item_height="60">
  <scroll_view name="view" x="0"  y="0" w="100%" h="100%">
    <list_item style="odd" children_layout="default(rows=1,cols=0)">
      <image draw_type="icon" w="30" image="earth"/>
      <label w="-30" text="1.Hello AWTK !">
        <switch x="r:10" y="m" w="60" h="20"/>
      </label>
    </list_item>
    <list_item style="odd" children_layout="default(rows=1,cols=0)">
      <label w="100%" text="1.Hello AWTK !"/>
    </list_item>
    <list_item style="odd" children_layout="default(rows=1,cols=0)">
      <label w="100%" text="1.Hello AWTK !"/>
    </list_item>
```

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    <list_item style="odd" children_layout="default(rows=1,cols=0)">
      <label w="100%" text="1.Hello AWTK !"/>
    </list_item>
    <list_item style="odd" children_layout="default(rows=1,cols=0)">
      <label w="100%" text="1.Hello AWTK !"/>
    </list_item>
    <list_item style="odd" children_layout="default(rows=1,cols=0)">
      <label w="100%" text="1.Hello AWTK !"/>
    </list_item>
  </scroll_view>
 </list_view>
</window>
```

运行效果如下图所示：



图 4.81　list_view、scroll_view、list_item 效果

4.8.22　list_view_h

水平列表视图控件。list_view_h_t 是 widget_t 的子类控件，widget_t 的函数均适用于 list_view_h_t 控件。

1. 函数

list_view_h 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
list_view_h_create	创建 list_view_h 对象
list_view_h_set_item_width	设置列表项的宽度。
list_view_h_set_spacing	设置列表项的间距。
list_view_h_cast	转换为 list_view_h 对象 (供脚本语言使用)。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
list_view_h_get_widget_vtable	获取 list_view_h 虚表。

2. 属性

list_view_h 还提供了下面属性，详见下表：

属性名称	类型	说明
item_width	int32_t	列表项的宽度。
spacing	int32_t	间距。

3. 样式

list_view_h 可设置的样式，详见 canvas_widget 一节的样式小节。样式设置代码如下：

```
<list_view_h>
    <style name="default" text_color="black" border_color="black">
        <normal bg_color="gray"/>
    </style>
</list_view_h>
```

4. 示例

- 在 xml 中，使用”list_view_h”标签创建水平列表视图控件，代码如下，样式设置见上文样式一节。

```
<!-- awtk/demos/assets/default/raw/ui/list_view_h.xml -->
<window anim_hint="htranslate">
  <list_view_h x="center"  y="10" w="90%" h="100" item_width="200" spacing="5">
    <scroll_view name="view" w="100%" h="100%">
      <image style="border" draw_type="auto" image="1" text="1"/>
      <image style="border" draw_type="auto" image="2" text="2"/>
      <image style="border" draw_type="auto" image="3" text="3"/>
      <image style="border" draw_type="auto" image="1" text="4"/>
      <image style="border" draw_type="auto" image="2" text="5"/>
      <image style="border" draw_type="auto" image="3" text="6"/>
      <image style="border" draw_type="auto" image="1" text="7"/>
      ...
    </scroll_view>
  </list_view_h>
  <button name="close" x="center" y="bottom:10" w="50%" h="30" text="close"/>
</window>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

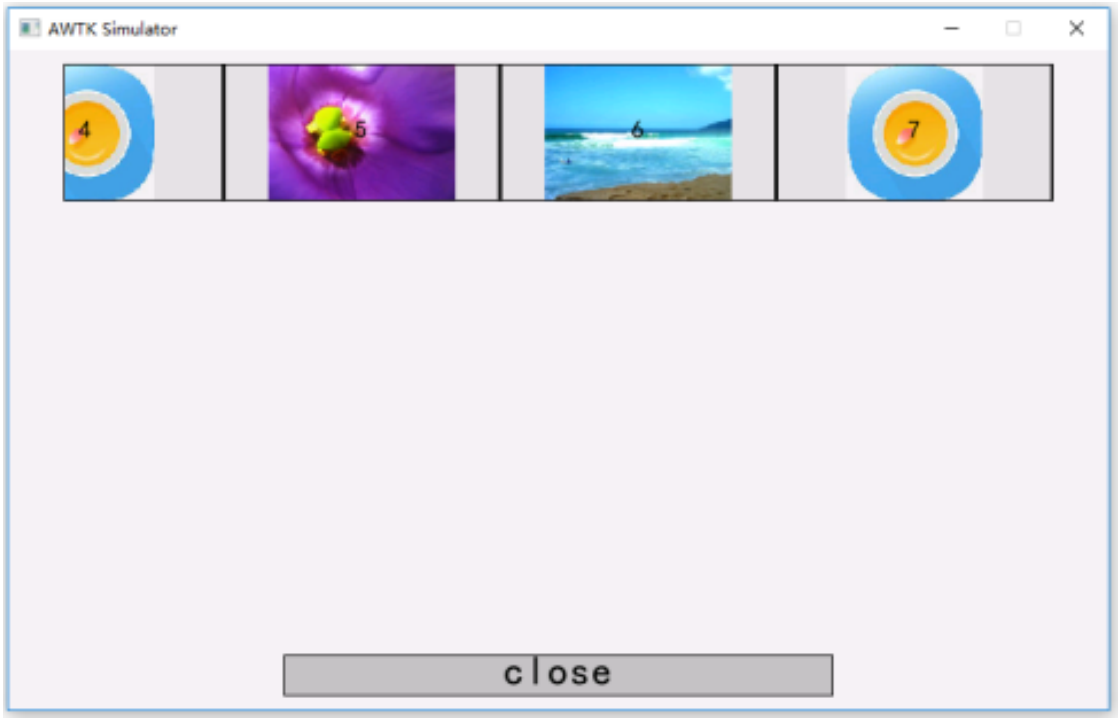


图 4.82　在 xml 中创建 list_view_h 的效果

4.8.23　list_item

列表项控件。列表项控件是一个简单的容器控件，一般作为列表视图中滚动视图的子控件。list_item_t 是 widget_t 的子类控件，widget_t 的函数均适用于 list_item_t 控件。

1. 函数

list_item 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
list_item_create	创建 list_item 对象
list_item_cast	转换为 list_item 对象 (供脚本语言使用)。
list_item_get_widget_vtable	获取 list_item 虚表。

2. 样式

list_item 可设置的样式，详见 canvas_widget 一节的样式小节。样式设置代码如下：

```
<list_item>
  <style name="default" border_color="#a0a0a0"  border="bottom" text_color="black">
    <normal     bg_color="#f0f0f0" />
    <pressed    bg_color="#c0c0c0" />
    <over       bg_color="#f0f0f0" />
  </style>
<list_item>
```

3. 事件

list_item 还提供了下面事件，详见下表：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

事件名称	类型	说明
EVT_CLICK	pointer_event_t	点击事件。

4. 示例

- 在 xml 中，使用”list_item”标签创建列表项控件，代码如下，样式设置见上文样式一节。

```
<!-- awtk/demos/assets/default/raw/ui/list_view_m.xml -->
<window anim_hint="htranslate">
  <list_view x="0"  y="30" w="100%" h="-80" item_height="60">
    <scroll_view name="view" x="0"  y="0" w="100%" h="100%">
      <list_item style="odd" children_layout="default(rows=1,cols=0)">
        <image draw_type="icon" w="30" image="earth"/>
        <label w="-30" text="1.Hello AWTK !">
          <switch x="r:10" y="m" w="60" h="20"/>
        </label>
      </list_item>
  </list_view>
  <button name="close" x="center" y="bottom:10" w="50%" h="30" text="close"/>
</window>
```

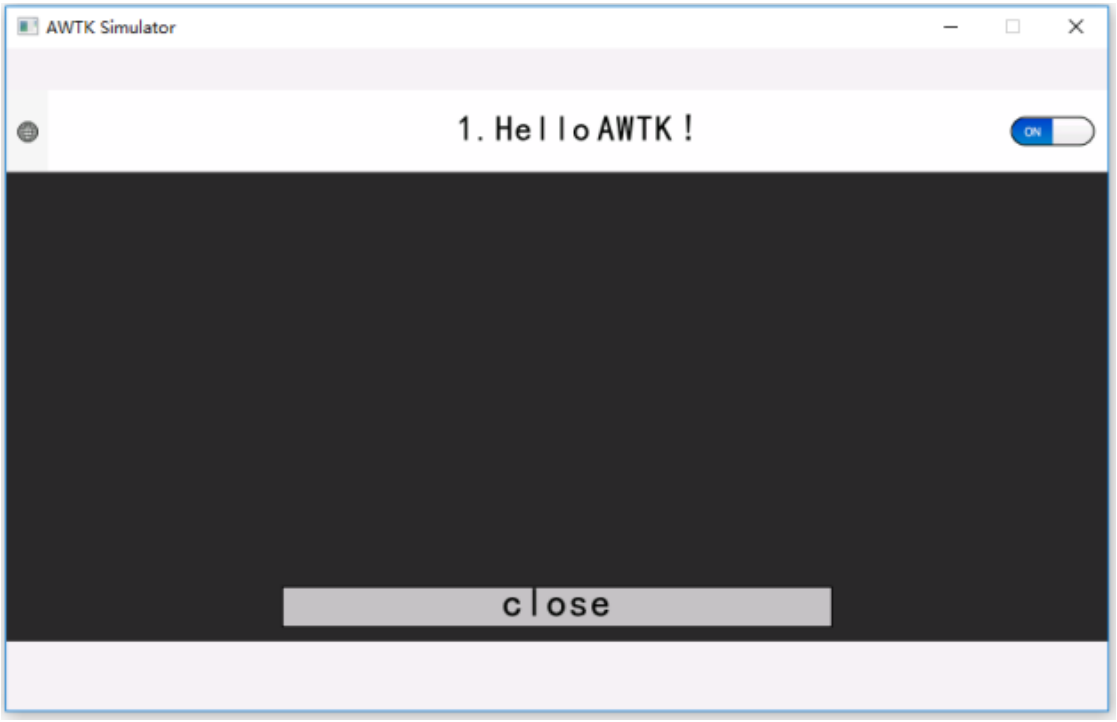


图 4.83　在 xml 中创建 list_item 的效果

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.8.24 list_item_seperator

用来模拟实现风琴控件 (accordion) 和属性页分组控件。当前控件被点击时，显示/隐藏当前控件到下一个分隔符控件之间的控件。list_item_seperator_t 是 widget_t 的子类控件，widget_t 的函数均适用于 list_item_seperator_t 控件。

1. 函数

list_item_seperator 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
list_item_seperator_create	创建 list_item_seperator 对象
list_item_seperator_cast	转换为 list_item_seperator 对象 (供脚本语言使用)。

2. 状态

list_item_seperator 在样式文件中可以设置的状态。

状态	说明
normal	正常状态
over	指针悬浮状态
pressed	按下状态
focused	聚焦状态
normal_of_checked	展开后正常状态
pressed_of_checked	展开后按下状态
over_of_checked	展开后指针悬浮状态
focused_of_checked	展开后聚焦状态

3. 示例

- 在 xml 中使用”list_item_seperator”标签创建 list_item_seperator。如：

```
<window>
  ...
  <list_item_seperator radio="true" text="Group2" h="32"/>
  <list_item style="empty" children_layout="default(r=1,c=0,ym=1)">
    <label w="30%" text="ASCII"/>
    <edit w="70%" text="" tips="ascii" input_type="ascii" focused="true" action_
↪text="next"/>
  </list_item>
  <list_item style="empty" children_layout="default(r=1,c=0,ym=1)">
    <label w="30%" text="Int"/>
    <edit w="70%" text="" tips="int" input_type="int"/>
  </list_item>
  <list_item_seperator radio="true" text="Group3" h="32"/>
  <list_item style="empty" children_layout="default(r=1,c=0,ym=1)">
    <label w="30%" text="Float"/>
    <edit w="70%" text="" tips="float" input_type="float"/>
  </list_item>
  <list_item style="empty" children_layout="default(r=1,c=0,ym=1)">
    <label w="30%" text="UFloat"/>
    <edit w="70%" text="" tips="unsigned float" input_type="ufloat"/>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  </list_item>
  ...
</window>
```

4.8.25 scroll_bar

滚动条控件（目前只支持垂直滚动）分为：scroll_bar_d（桌面风格）和 scroll_bar_m（手机风格）两种滚动条，所以在 UI 界面 XML 文件中写 scroll_bar_d 或者 scroll_bar_m，而不是 scroll_bar。scroll_bar_t 是 widget_t 的子类控件，widget_t 的函数均适用于 scroll_bar_t 控件。

如果需要修改滚动条中滑块中的样式，请修改 dragger 控件下 scroll_bar 样式，代码如下：

```
<!-- awtk/demos/assets/default/raw/styles/default.xml -->
...
<dragger>
  <style name="scroll_bar" >
    <normal     bg_color="#d0d0d0" />
    <pressed    bg_color="#c0c0c0" />
    <over       bg_color="#c0c0c0" />
  </style>
</dragger>
...
```

注：使用该控件的时候需要将 awtk/demos/assets/default/raw/images/x1 目录下：arrow_*.png 共 12 张图片复制到自己的应用目录下（如：res/assets/default/raw/images/xx），否则将不能正常显示。

1. 函数

scroll_bar 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
scroll_bar_create	创建 scroll_bar 对象
scroll_bar_cast	转换为 scroll_bar 对象 (供脚本语言使用)。
scroll_bar_create_mobile	创建 mobile 风格的 scroll_bar 对象
scroll_bar_create_desktop	创建 desktop 风格的 scroll_bar 对象
scroll_bar_set_params	设置参数。
scroll_bar_scroll_to	滚动到指定的值。
scroll_bar_set_value	设置值，并触发 EVT_VALUE_CHANGED 事件。
scroll_bar_add_delta	在当前的值上增加一个值，并触发 EVT_VALUE_CHANGED 事件。
scroll_bar_scroll_delta	在当前的值上增加一个值，并滚动到新的值，并触发 EVT_VALUE_CHANGED 事件。
scroll_bar_set_value_only	设置值，但不触发 EVT_VALUE_CHANGED 事件。
scroll_bar_set_auto_hide	设置 auto_hide 属性。
scroll_bar_is_mobile	判断是否是 mobile 风格的滚动条。
scroll_bar_set_animator_time	设置翻页滚动动画时间。
scroll_bar_hide_by_opacity_animation	通过动画隐藏滚动条。
scroll_bar_show_by_opacity_animation	通过动画显示滚动条。

ZLG ©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

2. 属性

scroll_bar 还提供了下面属性，详见下表：

属性名称	类型	说明
virtual_size	int32_t	虚拟宽度或高度。
value	int32_t	当前的值。
row	int32_t	行的高度。
animator_time	uint32_t	翻页滚动动画时间。
animatable	bool_t	滚动时是否启用动画。
auto_hide	bool_t	是否自动隐藏 (仅对 mobile 风格的滚动条有效)。

3. 状态

scroll_bar 在样式文件中可以设置的状态，详见下表，其中 scroll_bar_m 没有 over 状态。

状态	说明
normal	正常状态
over	指针悬浮状态
disable	禁用状态

4. 事件

scroll_bar 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	value_change_event_t	值 (滚动值) 即将改变事件。
EVT_VALUE_CHANGED	value_change_event_t	值 (滚动值) 改变事件。

5. 示例

- 在 xml 中，使用”scroll_bar”或”scroll_bar_d”或”scroll_bar_m”标签创建滚动条控件，代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/selectm.xml -->
<dialog anim_hint="bottom_to_top" x="0" y="bottom" w="100%" h="160">
  <dialog_title style="empty" x="0" y="0" w="100%" h="30">
    <button name="quit" x="5" y="middle" w="60" h="-4" text="Done"/>
    <label x="center" y="middle" w="128" h="-4" text="Select multiple"/>
    <button name="quit" x="right:5" y="middle" w="60" h="-4" text="Cancel"/>
  </dialog_title>
  <dialog_client x="0" y="bottom" w="100%" h="-30">
  <list_view x="0"  y="0" w="100%" h="100%" item_height="30">
    <scroll_view name="view" x="0"  y="0" w="100%" h="100%">
     <check_button style="odd_icon_right" name="c1" text="Book"/>
     <check_button style="even_icon_right" name="c2" text="Pencil"/>
     <check_button style="odd_icon_right" name="c1" text="Food"/>
     <check_button style="even_icon_right" name="c2" text="Bike"/>
    </scroll_view>
    <scroll_bar_m name="bar" x="right" y="0" w="6" h="100%" value="0"/>
  </list_view>
  </dialog_client>
```

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
</dialog>
```

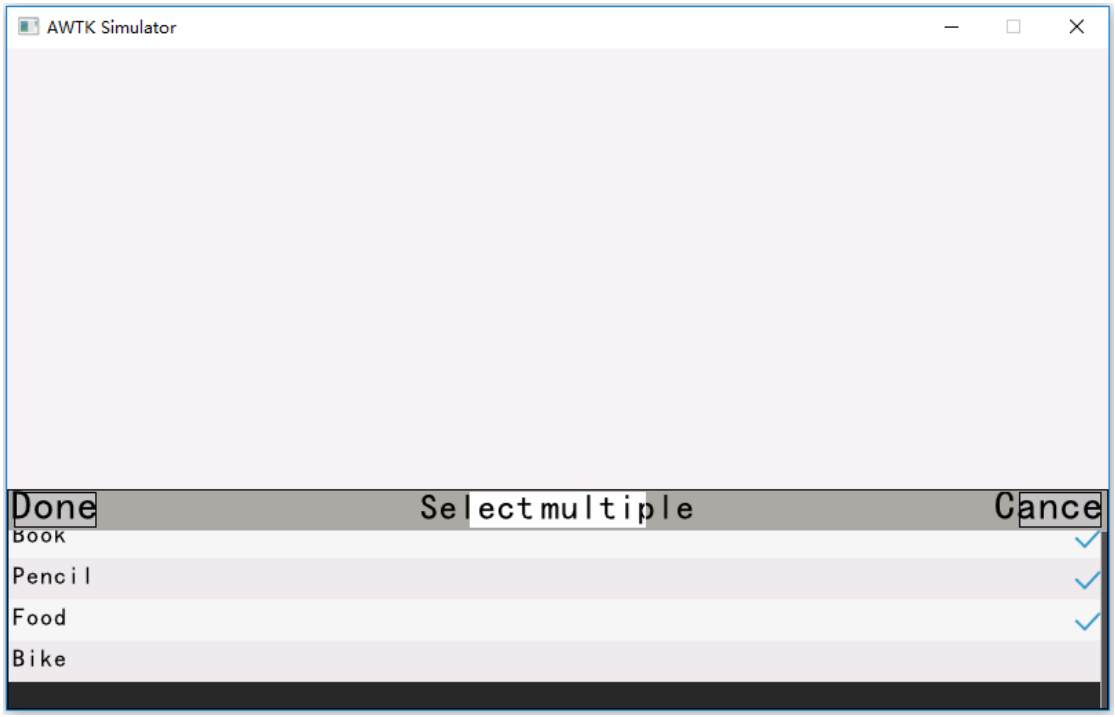


图 4.84　在 xml 中创建 scroll_bar 的效果

4.8.26　scroll_view

滚动视图。scroll_view_t 是 widget_t 的子类控件，widget_t 的函数均适用于 scroll_view_t 控件。

1. 函数

scroll_view 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
scroll_view_create	创建 scroll_view 对象
scroll_view_cast	转换为 scroll_view 对象 (供脚本语言使用)。
scroll_view_set_virtual_w	设置虚拟宽度。
scroll_view_set_virtual_h	设置虚拟高度。
scroll_view_set_xslidable	设置是否允许 x 方向滑动。
scroll_view_set_yslidable	设置是否允许 y 方向滑动。
scroll_view_set_snap_to_page	设置滚动时 offset 是否按页面对齐。
scroll_view_set_move_to_page	设置滚动时是否每次翻一页
scroll_view_set_recursive	设置是否递归查找全部子控件。
scroll_view_set_recursive_only	设置是否递归查找全部子控件。(不触发 repaint 和 relayout)。
scroll_view_set_offset	设置偏移量。
scroll_view_set_speed_scale	设置偏移速度比例。
scroll_view_set_slide_limit_ratio	设置滑动到极限时可继续滑动区域的占比。
scroll_view_scroll_to	滚动到指定的偏移量。
scroll_view_scroll_delta_to	滚动到指定的偏移量。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
scroll_view_get_widget_vtable	获取 scroll_view 虚表。

2. 属性

scroll_view 还提供了下面属性，详见下表：

属性名称	类型	说明
virtual_w	wh_t	虚拟宽度。
virtual_h	wh_t	虚拟高度。
xoffset	int32_t	x 偏移量。
yoffset	int32_t	y 偏移量。
xspeed_scale	float_t	x 偏移速度比例。
yspeed_scale	float_t	y 偏移速度比例。
xslidable	bool_t	是否允许 x 方向滑动。
yslidable	bool_t	是否允许 y 方向滑动。
snap_to_page	bool_t	滚动时 offset 是否按页面对齐。
move_to_page	bool_t	是否每次翻一页（当 move_to_page 为 ture 的时候才有效果，主要用于区分一次翻一页还是一次翻多页）。
recursive	bool_t	是否递归查找全部子控件。
slide_limit_ratio	float_t	滑动到极限时可继续滑动区域的占比。

3. 样式

scroll_view 可设置的样式，详见 canvas_widget 一节的样式小节。样式设置代码如下：

```
<scroll_view>
    <style name="default" text_color="white">
        <normal bg_color="#292929"/>
    </style>
</scroll_view>
```

4. 事件

scroll_view 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_SCROLL_START	event_t	开始滚动事件。
EVT_SCROLL_END	event_t	结束滚动事件。
EVT_SCROLL	event_t	滚动事件。
EVT_PAGE_CHANGED	event_t	页面改变事件。
EVT_PAGE_CHANGING	event_t	页面正在改变。

5. 示例

- 在 xml 中，使用”scroll_view”标签创建滚动视图，代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/selectm.xml -->
<dialog anim_hint="bottom_to_top" x="0" y="bottom" w="100%" h="160">
  <dialog_title style="empty" x="0" y="0" w="100%" h="30">
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    <button name="quit" x="5" y="middle" w="60" h="-4" text="Done"/>
    <label x="center" y="middle" w="128" h="-4" text="Select multiple"/>
    <button name="quit" x="right:5" y="middle" w="60" h="-4" text="Cancel"/>
  </dialog_title>
  <dialog_client x="0" y="bottom" w="100%" h="-30">
  <list_view x="0"  y="0" w="100%" h="100%" item_height="30">
    <scroll_view name="view" x="0"  y="0" w="100%" h="100%">
     <check_button style="odd_icon_right" name="c1" text="Book"/>
     <check_button style="even_icon_right" name="c2" text="Pencil"/>
     <check_button style="odd_icon_right" name="c1" text="Food"/>
     <check_button style="even_icon_right" name="c2" text="Bike"/>
    </scroll_view>
    <scroll_bar_m name="bar" x="right" y="0" w="6" h="100%" value="0"/>
  </list_view>
  </dialog_client>
</dialog>
```

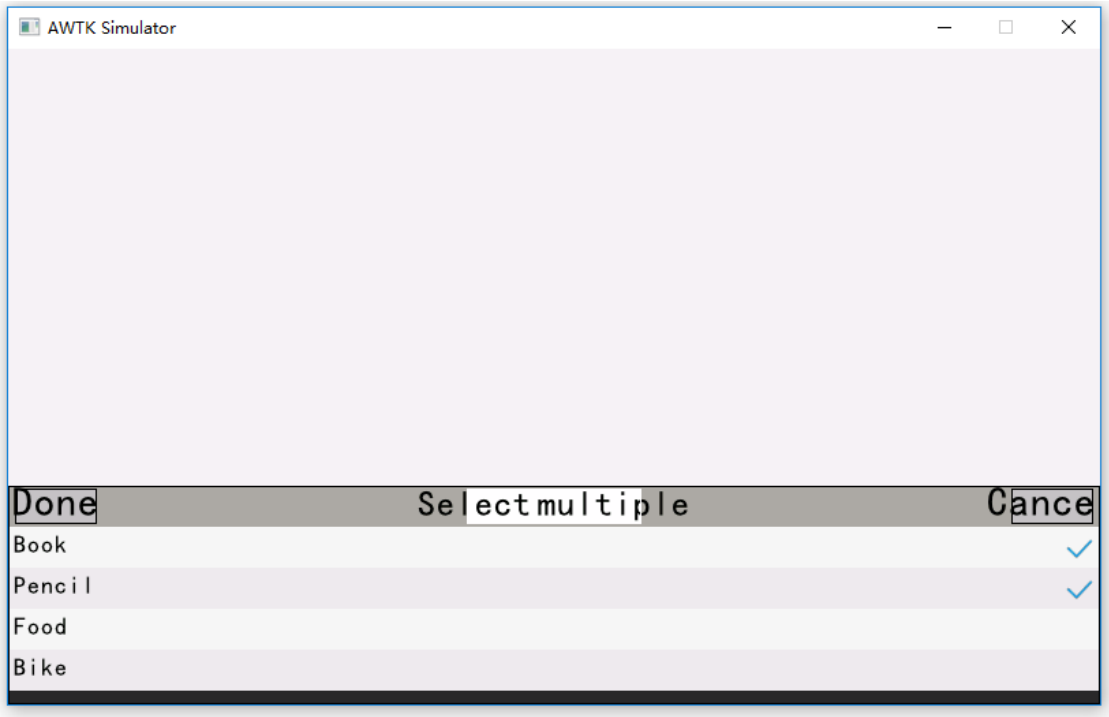


图 4.85　在 xml 中创建 scroll_view 的效果

4.8.27　mutable_image

mutable 图片控件。像摄像头和视频的图像是变化的，每一帧都不同，把这类图片称为 mutable image。本控件辅助实现摄像头和视频的显示功能。mutable_image_t 是 image_base_t 的子类控件，image_base_t 的函数均适用于 mutable_image_t 控件。

1. 函数

mutable_image 提供的函数详见下表，具体请查看 AWTK_API 手册。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

函数名称	说明
mutable_image_create	创建 mutable_image 对象
mutable_image_set_need_redraw	设置 need_redraw 回调函数。
mutable_image_set_prepare_image	设置 prepare_image 回调函数。
mutable_image_set_create_image	设置 create_image 回调函数。
mutable_image_set_framebuffer	设置 framebuffer(当硬件支持多层合成时才用)。
mutable_image_cast	转换为 mutable_image 对象 (供脚本语言使用)。
mutable_image_init	初始化 mutable_image （提供给继承的子类使用的）
mutable_image_on_destroy	释放 mutable_image （提供给继承的子类使用的）
mutable_image_on_paint_self	mutable_image 的绘制函数（提供给继承的子类使用的）
mutable_image_get_widget_vtable	获取 mutable_image 虚表。

2. 示例

- 在 xml 中，使用”mutable_image”标签创建 mutable 图片控件，代码如下：

```
<mutable_image w="100%" h="100%"/>
```

- 在 C 代码中，使用函数 mutable_image_create 创建 mutable 图片控件，代码如下：

```
widget_t* image = mutable_image_create(win, 10, 10, 200, 200);
mutable_image_set_prepare_image(image, get_camera_image, camera);
```

4.8.28 mledit

多行编辑器控件。mledit_t 是 widget_t 的子类控件，widget_t 的函数均适用于 mledit_t 控件。

1. 函数

mledit 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
mledit_create	创建 mledit 对象
mledit_set_readonly	设置编辑器是否为只读。
mledit_set_cancelable	设置编辑器是否为可撤销修改。
mledit_set_focus	设置为焦点。
mledit_set_wrap_word	设置编辑器是否自动折行。
mledit_set_overwrite	设置编辑器是否启用覆盖行（在行数达到最大行数时，可继续新增行，但最早的行将会消失）。
mledit_set_max_lines	设置编辑器的最大行数。
mledit_set_max_chars	设置编辑器的最大字符数（0 为不限制字符数）。
mledit_set_tips	设置编辑器的输入提示。
mledit_set_tr_tips	获取翻译之后的文本，然后调用 mledit_set_tips。
mledit_set_keyboard	设置自定义软键盘名称。
mledit_set_cursor	设置编辑器光标位置。
mledit_get_cursor	获取编辑器光标位置。
mledit_set_scroll_line	设置编辑器滚动速度。
mledit_scroll_to_offset	设置编辑器滚动到指定偏移位置。
mledit_set_open_im_when_focused	设置编辑器是否在获得焦点时打开输入法。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
mledit_set_close_im_when_blured	设置编辑器是否在失去焦点时关闭输入法。
mledit_set_select	选择编辑器中指定范围的文本。
mledit_get_selected_text	获取选中的文本。
mledit_insert_text	插入一段文本。
mledit_cast	转换为 mledit 对象 (供脚本语言使用)。
mledit_get_widget_vtable	获取 mledit 虚表。

2. 属性

mledit 还提供了下面属性，详见下表：

属性名称	类型	说明
tips	char*	输入提示。
tr_tips	char*	保存用于翻译的提示信息。
keyboard	char*	自定义软键盘名称。
max_lines	uint32_t	最大行数。
max_chars	uint32_t	最大字符数。
scroll_line	uint32_t	鼠标一次滚动行数。
overwrite	bool_t	是否启用覆盖行。
wrap_word	bool_t	是否自动折行。
readonly	bool_t	编辑器是否为只读。
cancelable	bool_t	是否支持撤销编辑。如果为 TRUE，在失去焦点之前可以撤销所有修改 (恢复获得焦点之前的内容)。
open_im_when_focused	bool_t	获得焦点时打开输入法。
close_im_when_blured	bool_t	是否在失去焦点时关闭输入法 (默认是)。

3. 样式

mledit 可设置的样式，详见 edit 一节的样式小节，样式设置代码如下：

```
<mledit selected_fg_color="#020202" selected_text_color="#010101" selected_bg_
↪color="#b5d7fd">
  <style name="default" border_color="#a0a0a0"  text_color="black" text_align_h=
↪"left">
    <normal     bg_color="#f0f0f0" />
    <focused    bg_color="#f0f0f0" border_color="black"/>
    <disable    bg_color="gray" text_color="#d0d0d0" />
    <error      bg_color="#f0f0f0" text_color="red" />
    <empty      bg_color="#f0f0f0" text_color="#a0a0a0" />
    <empty_focus bg_color="#f0f0f0" text_color="#a0a0a0" border_color="black"/>
  </style>
</mledit>
```

4. 事件

mledit 还提供了下面事件，详见下表：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

事件名称	类型	说明
EVT_VALUE_CHANGING	value_change_event_t	文本正在改变事件 (编辑中)。
EVT_VALUE_CHANGED	value_change_event_t	文本改变事件 (编辑完成或设置文本时触发)。

5. 示例

- 在 xml 中，使用”mledit”标签创建多行编辑器控件，代码如下。其中使用的样式详见上文样式一节。

```
<window anim_hint="htranslate" >
  <mledit x="c" y="10" h="40%" w="90%" focus="true" left_margin="33" top_margin="2"
↪bottom_margin="2" right_margin="16" wrap_word="true" max_lines="100">
   <line_number x="0" y="0" w="32" h="100%" value="0"/>
   <scroll_bar_d x="right" y="0" w="14" h="100%" value="0"/>
 </mledit>
 <mledit x="c" y="b:10" h="40%" w="90%" wrap_word="false">
 </mledit>
</window>
```



图 4.86　在 xml 中实现 mledit 的效果

4.8.29　line_number

行号。多行编辑器的行号。line_number_t 是 widget_t 的子类控件，widget_t 的函数均适用于 line_number_t 控件。

1. 函数

line_number 提供的函数详见下表，具体请查看 AWTK_API 手册。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

函数名称	说明
line_number_create	创建 line_number 对象
line_number_set_top_margin	设置顶部边距。
line_number_set_bottom_margin	设置顶部边距。
line_number_set_line_height	设置行高。
line_number_set_yoffset	设置 y 偏移。
line_number_set_lines_of_each_row	设置每一个逻辑行 (row) 占几个物理行 (line)。
line_number_cast	转换为 line_number 对象 (供脚本语言使用)。
line_number_add_highlight_line	增加高亮行。
line_number_set_active_line	设置 active 行。
line_number_clear_highlight	清除高亮行。
line_number_is_highlight_line	判断指定行是否是高亮行。
line_number_get_widget_vtable	获取 line_number 虚表。

2. 样式

line_number 可设置的样式，详见下表：

样式属性	说明
bg_color	背景颜色
fg_color	前景颜色
font_name	字体名称
font_size	字体大小
text_color	文本颜色
text_align_h	文本水平对齐的方式（取值：center|left|right）
text_align_v	文本垂直对齐的方式（取值：middle|top|bottom）
border_color	边框颜色
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称
bg_image_draw_type	背景图片的显示方式（取值：详见 image 控件一节）
margin	边距
margin_left	左边距
margin_right	右边距
margin_top	顶边距
margin_bottom	底边距
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

样式设置代码如下：

```
<line_number>
  <style name="default">
    <normal text_color="black" bg_color="#d0d0d0" text_align_h="right"/>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  </style>
</line_number>
```

3. 示例

- 在 xml 中，使用”line_number”标签创建 line_number 控件，代码如下，运行效果详见 mledit 一节中的示例。

```
<!-- awtk/demos/assets/default/raw/ui/mledit.xml -->
<window anim_hint="htranslate" >
  <mledit x="c" y="10" h="40%" w="90%" focus="true"
    left_margin="33" top_margin="2" bottom_margin="2" right_margin="16" wrap_word=
↪"true"
        max_lines="100">
   <line_number x="0" y="0" w="32" h="100%" value="0"/>
   <scroll_bar_d x="right" y="0" w="14" h="100%" value="0"/>
 </mledit>
 <mledit x="c" y="b:10" h="40%" w="90%" wrap_word="false">
 </mledit>
</window>
```

4.8.30 slide_indicator

slide_view 的指示器控件。支持直线、弧线排布，默认有 4 种绘制样式，若设置了 icon/active_icon，则优先使用 icon/active_icon。slide_indicator_t 是 widget_t 的子类控件，widget_t 的函数均适用于 slide_indicator_t 控件。

1. 函数

slide_indicator 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
slide_indicator_create	创建 slide_indicator 对象
slide_indicator_create_linear	创建 slide_indicator 对象（线性显示）
slide_indicator_create_arc	创建 slide_indicator 对象（圆弧显示）
slide_indicator_cast	转换为 slide_indicator 对象 (供脚本语言使用)。
slide_indicator_set_value	设置当前页的序号。
slide_indicator_set_max	设置指示器的数量。
slide_indicator_set_default_paint	设置指示器的默认绘制类型。
slide_indicator_set_auto_hide	设置指示器是否自动隐藏。
slide_indicator_set_margin	设置指示器的边距 (默认为 10 像素)。
slide_indicator_set_spacing	设置指示器的间距 (指示器有弧度时为角度值，否则为直线间距)。
slide_indicator_set_size	设置指示器的大小 (默认为 8)。
slide_indicator_set_anchor	设置旋转锚点。
slide_indicator_set_indicated_target	设置指示器指示的目标。
slide_indicator_set_transition	设置是否启用过渡效果。

2. 属性

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

slide_indicator 还提供了下面属性，详见下表：

属性名称	类型	说明
value	uint32_t	值 (缺省为 0)。
max	uint32_t	最大值 (缺省为 100)。
default_paint	indicator_default_paint_t	指示器的类型。
auto_hide	uint16_t	自动隐藏。0 表示禁止，非 0 表示无操作后延迟多久隐藏。
margin	int32_t	指示器与边缘的边距。
spacing	float_t	指示器的中心之间的间距（圆弧显示时，间距的单位为弧度，否则为像素）。
size	uint32_t	指示器的大小。
anchor_x	char*	锚点 x 坐标。(后面加上 px 为像素点，不加 px 为相对百分比坐标 0.0f 到 1.0f)
anchor_y	char*	锚点 y 坐标。(后面加上 px 为像素点，不加 px 为相对百分比坐标 0.0f 到 1.0f)
indicated_target	char*	指示器指示的目标控件的名称。
transition	bool_t	是否启用过渡效果。

default_paint 属性指示器的类型的取值详见下表：

指示器的类型	说明
stroke_dot	镂空圆点指示器，当前项填充 selected_fg_color，其他镂空 fg_color
fill_dot	实心圆点指示器，当前项填充 selected_fg_color，其他填充 fg_color
stroke_rect	镂空矩形指示器，当前项填充 selected_fg_color，其他镂空 fg_color
fill_rect	实心矩形指示器，当前项填充 selected_fg_color，其他填充 fg_color

3. 样式

slide_indicator 可设置的样式，详见下表：

样式属性	说明
bg_color	背景颜色
border_color	边框颜色
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称
bg_image_draw_type	背景图片的显示方式（取值：详见 image 控件一节）
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
fg_color	前景颜色
selected_fg_color	编辑器（包括 edit 和 mledit 控件）中选中区域的背景颜色
icon	图标的名称
active_icon	active 图标的名称
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

样式属性	说明
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

slide_indicator 的样式示例代码如下：

```
<slide_indicator>
  <style name="default">
    <normal  fg_color="#ff000040" selected_fg_color="#ff0000"/>
  </style>
  <style name="blue">
    <normal  fg_color="#338fff40" selected_fg_color="#338fff"/>
  </style>
  <style name="icon">
    <normal  icon="dot" active_icon="active_dot"/>
  </style>
</slide_indicator>
```

4. 事件

slide_indicator 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_VALUE_WILL_CHANGE	value_change_event_t	值 (当前页的序号) 即将改变事件。
EVT_VALUE_CHANGED	value_change_event_t	值 (当前页的序号) 改变事件。
EVT_PAGE_CHANGED	event_t	页面改变事件。
EVT_PAGE_CHANGING	event_t	页面正在改变。

5. 示例

- 在 xml 中，使用”slide_indicator”标签创建 slide_indicator 控件，代码如下，其中使用的样式详见上文样式一节。

```
<window anim_hint="htranslate" text="Landscape">
  <slide_view x="0" y="0" w="100%" h="100%" vertical="true">
    <view x="0" y="0" w="100%" h="100%">
        <label x="c" y="m" w="20%" h="10%" text = "first"/>
    </view>
    <view x="0" y="0" w="100%" h="100%">
        <label x="c" y="m" w="20%" h="10%" text = "secend"/>
    </view>
    <view x="0" y="0" w="100%" h="100%">
        <label x="c" y="m" w="20%" h="10%" text = "third"/>
    </view>
  </slide_view>
  <slide_indicator x="0" y="b" w="100%" h="20"/>
</window>
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

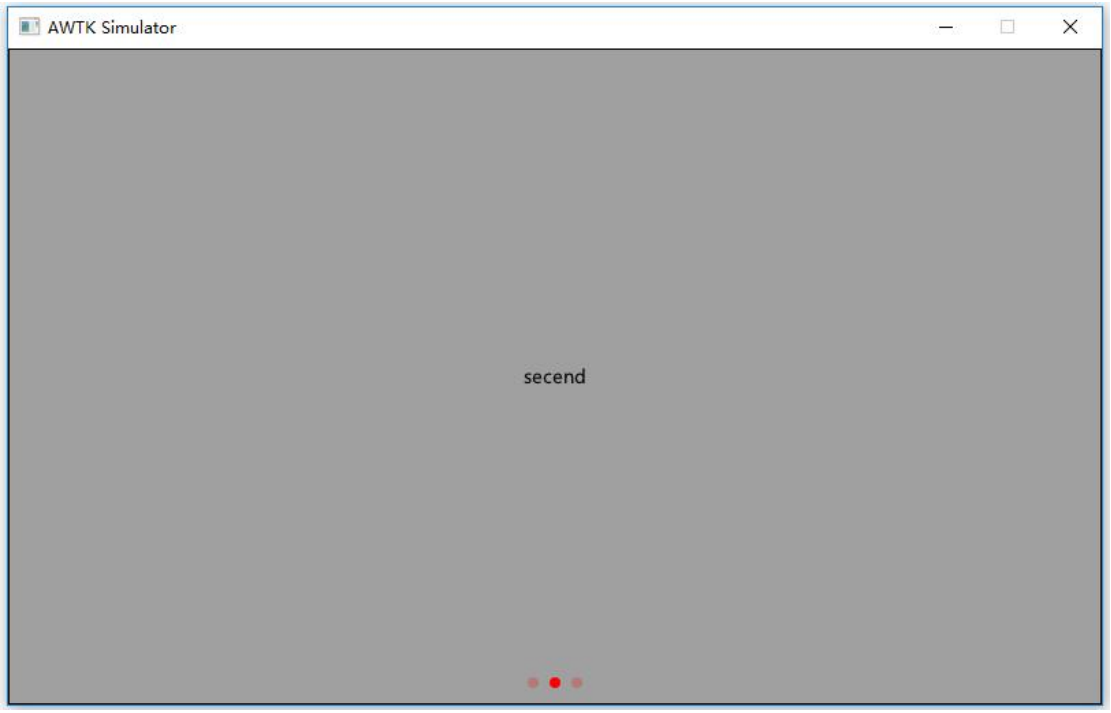


图 4.87　在 xml 中创建 slide_indicator 的效果

4.8.31　hscroll_label

可水平滚动的文本控件，方便实现长文本滚动。hscroll_label_t 是 widget_t 的子类控件，widget_t 的函数均适用于 hscroll_label_t 控件。

1. 函数

hscroll_label 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
hscroll_label_create	创建 hscroll_label 对象
hscroll_label_set_lull	设置 lull。
hscroll_label_set_duration	设置 duration。
hscroll_label_set_speed	设置 speed（设置后 duration 不生效）。
hscroll_label_set_only_focus	设置 only_focus。
hscroll_label_set_only_parent_focus	设置 only_parent_focus。
hscroll_label_set_loop	设置 loop。
hscroll_label_set_yoyo	设置 yoyo。
hscroll_label_set_ellipses	设置 ellipses。
hscroll_label_set_stop_at_begin	设置 stop_at_begin。
hscroll_label_set_xoffset	设置 x 偏移 (一般无需用户调用)。。
hscroll_label_start	启动 (一般无需用户调用)。
hscroll_label_stop	停止 (一般无需用户调用)。
hscroll_label_cast	转换为 hscroll_label 对象 (供脚本语言使用)。
hscroll_label_get_widget_vtable	获取 hscroll_label 虚表。

2. 属性

hscroll_label 还提供了下面属性，详见下表：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

属性名称	类型	说明
only_focus	bool_t	只有处于 focus 时才滚动 (缺省否)。
only_parent_focus	bool_t	只有父控件处于 focus 时才滚动 (缺省否)。
loop	bool_t	loop 是否循环滚动 (缺省 FALSE)。
yoyo	bool_t	是否往返滚动 (缺省 FALSE)。
ellipses	bool_t	文本显示不下时，在行尾显示省略号 (缺省 FALSE)。
lull	int32_t	滚动之间的间歇时间 (ms)，缺省 3000ms。
duration	int32_t	完整的滚动一次需要的时间 (ms)，缺省 5000ms。
speed	float_t	滚动速度 (px/ms)（设置后 duration 不生效）。
xoffset	int32_t	偏移量。
text_w	int32_t	文本的宽度。
stop_at_begin	bool_t	滚动完毕后停在文本开头 (缺省 FALSE)。

3. 样式

hscroll_label 可设置的样式，详见下表：

样式属性	说明
bg_color	背景颜色
font_name	字体名称
font_size	字体大小
text_color	文本颜色
text_align_h	文本水平对齐的方式（取值：center|left|right）
text_align_v	文本垂直对齐的方式（取值：middle|top|bottom）
border_color	边框颜色
border_width	边框线宽
border	边框类型（取值：all|top|bottom|left|right）
bg_image	背景图片的名称
bg_image_draw_type	背景图片的显示方式（取值：详见表 4.27）
margin	边距
margin_left	左边距
margin_right	右边距
margin_top	顶边距
margin_bottom	底边距
x_offset	x 方向的偏移，方便实现按下的效果
y_offset	y 方向的偏移，方便实现按下的效果
round_radius	圆角半径 (仅在 with_vgcanvas 定义时生效)
round_radius_top_left	左上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_top_right	右上角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_left	左下角圆角半径 (仅 with_vgcanvas 定义时生效)
round_radius_bottom_right	右下角圆角半径 (仅 with_vgcanvas 定义时生效)

4. 示例

- 在 xml 中，使用”hscroll_label”标签创建 hscroll_label 控件，代码如下：

```
<window anim_hint="htranslate" text="hscroll_label" children_layout="default(c=1,
↪h=30,xm=10,s=5)">
  <hscroll_label text="炫酷的 GUI 引擎。" />
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  <hscroll_label
    lull="1000"
    loop="true"
    yoyo="true"
    ellipses="true"
    focusable="true"
    text="(always ellipses loop yoyo)为用户提供一个功能强大、高效可靠、简单易用、
          可轻松做出炫酷效果的 GUI 引擎。文字滚动效果呢。。。" />
</window>
```

运行效果如下所示：



图 4.88　在 xml 中创建 hscroll_label 的效果

4.8.32　serial_widget

用于串口通信的控件，可以通过该控件简单地控制串口，serial_widget_t 是 widget_t 的子类控件，widget_t 的函数均适用于 serial_widget_t 控件。

1. 函数

serial_widget 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
serial_widget_create	创建 serial_widget 对象
serial_widget_cast	转换为 serial_widget 对象 (供脚本语言使用)。
serial_widget_set_baudrate	设置波特率。
serial_widget_set_device	设置设备。
serial_widget_set_bytesize	设置字节位数。
serial_widget_set_parity	设置奇偶校验。
serial_widget_set_stopbits	设置停止位。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
serial_widget_set_flowcontrol	设置流控。
serial_widget_set_check_interval	设置轮询时间。
serial_widget_get_widget_vtable	获取 serial_widget 虚表。

2. 属性

serial_widget 还提供了下面属性，详见下表：

属性名称	类型	说明
device	char*	设备 (文件) 名。
baudrate	uint32_t	波特率。
bytesize	uint32_t	字节位数。
parity	uint32_t	奇偶校验。
stopbits	uint32_t	停止位。
flowcontrol	uint32_t	流控。
istream	tk_istream_t*	输入流。
ostream	tk_ostream_t*	输出流。
iostream	tk_iostream_t*	输入/出流。
check_interval	uint32_t	轮询时间（单位：ms）。

3. 事件

serial_widget 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_DATA	event_t	数据到来事件。

4. 示例

- 在 xml 中，使用”serial”标签创建 serial_widget 控件，代码如下：

```
<window text="serial">
  <serial device="COM1" baudrate="115200"/>
</window>
```

4.8.33 timer_widget

定时器。主要目的是方便以拖拽的方式创建定时器。timer_widget_t 是 widget_t 的子类控件，widget_t 的函数均适用于 timer_widget_t 控件。

1. 函数

timer_widget 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
timer_widget_create	创建 timer_widget 对象
timer_widget_cast	转换为 timer_widget 对象 (供脚本语言使用)。
timer_widget_set_duration	设置时长 (ms)。
timer_widget_get_widget_vtable	获取 timer_widget 虚表。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

2. 属性

timer_widget 还提供了下面属性，详见下表：

属性名称	类型	说明
duration	uint32_t	时长 (ms)。

3. 事件

timer_widget 还提供了下面事件，详见下表：

事件名称	类型	说明
EVT_TIMER	event_t	定时器事件。

4. 示例

- 在 xml 中，使用”timer”标签创建 timer_widget 控件，代码如下：

```
<window text="timer">
  <timer x="c" y="50" w="100" h="100" duration="1000"/>
</window>
```

4.8.34 vpage

虚拟页面 (根据情况自动加载/卸载页面，并提供入场/出场动画)。虚拟页面只能作为 pages 的直接子控件使用。如果指定了 ui_asset:

1. 当页面切换到后台时自动卸载，并触发 EVT_VPAGE_CLOSE 消息。
2. 当页面切换到前台时自动加载，在动画前出发 EVT_VPAGE_WILL_OPEN 消息，在动画完成时触发 EVT_VPAGE_CLOSE 消息。

1. 函数

vpage 提供的函数详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
vpage_create	创建 vpage 对象
vpage_cast	转换为 vpage 对象 (供脚本语言使用)。
vpage_set_ui_asset	设置 UI 资源名称。
vpage_set_anim_hint	设置动画类型 (vtranslate: 垂直平移，htranslate: 水平平移)。
vpage_get_widget_vtable	获取 vpage 虚表。

2. 属性

vpage 还提供了下面属性，详见下表：

属性名称	类型	说明
ui_asset	char*	UI 资源名称。
anim_hint	char*	动画类型 (目前支持：vtranslate: 垂直平移，htranslate: 水平平移)。

3. 事件

vpage 还提供了下面事件，详见下表：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

事件名称	类型	说明
EVT_VPAGE_WILL_OPEN	event_t	页面即将打开 (动画前)。
EVT_VPAGE_OPEN	event_t	页面打开完成 (动画后)。
EVT_VPAGE_CLOSE	event_t	页面已经关闭 (动画后)。

4. 示例

- 在 xml 中，使用”vpage”标签创建 vpage 控件，代码如下：

```
<!-- awtk/design/default/ui/vpage_vtrans.xml -->
<window text="VPage V Translate">
  <pages name="pages" x="31%" y="0" w="68%" h="100%" style="at_top">
    <vpage name="page1" w="100%" h="100%" ui_asset="tab_button_view_page1" anim_
↪hint="vtranslate" />
    <vpage name="page2" w="100%" h="100%" ui_asset="tab_button_view_page2" anim_
↪hint="vtranslate" />
    <vpage name="page3" w="100%" h="100%" anim_hint="vtranslate" >
      <label x="c" y="m" w="200" h="30" text="page3" />
      <button x="c" y="b:10" w="128" h="30" text="close" on:click="back()"/>
    </vpage>
</window>
```

4.9 自定义控件

除了使用 AWTK 内置的控件外，还可以自定义控件，其步骤如下：

1. 注册控件
2. 创建控件
3. 使用控件

注: 如果想在 AWTK Designer 中导入自定义控件使用，那么就需要在 Designer 中新建自定义控件项目，新建完成后可以得到一个空白控件的框架，用户需要遵循 AWTK 自定义控件规范[17] 实现控件代码。

下面以 Chart-Demo 演示例子中的饼图为例，看看如何实现自定义控件的。代码可以从 GitHub[18] 下载，效果详见下图：

[17] https://github.com/zlgopen/awtk/tree/master/docs/custom_widget_rules.md
[18] https://github.com/zlgopen/awtk-examples/tree/master/Chart-Demo

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

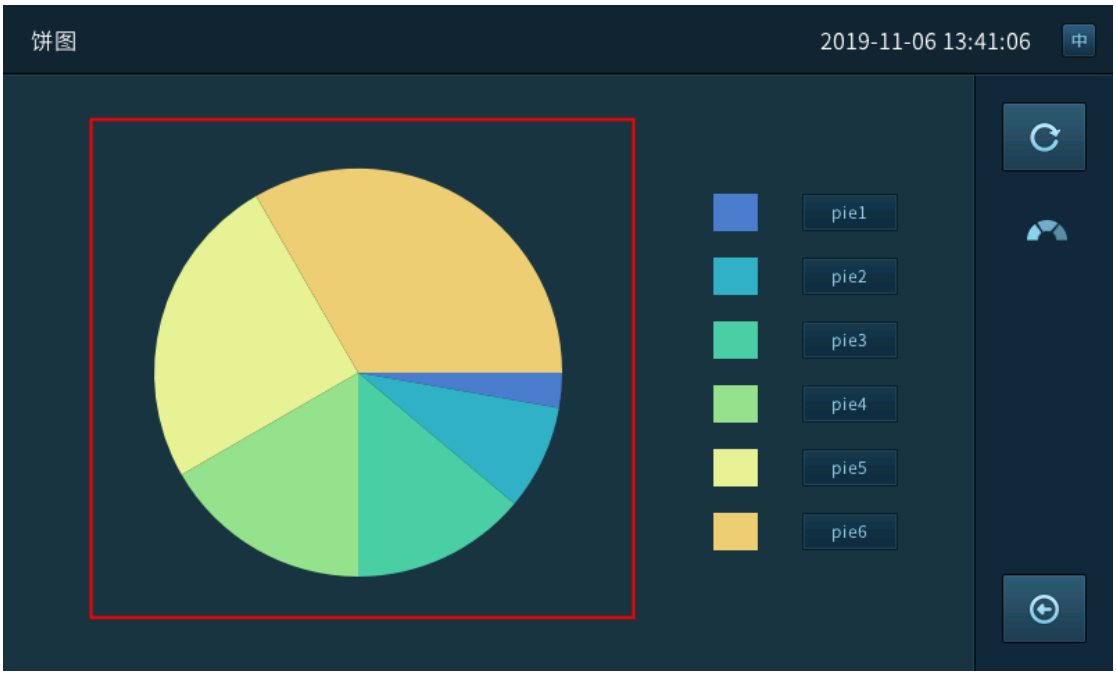


图 4.89　饼图

4.9.1　注册控件

使用 widget_factory_register 函数注册控件，控件的类型为 WIDGET_TYPE_PIE_SLICE，控件的创建函数是 pie_slice_create，代码如下：

```
/* Chart-Demo/src/custom_widgets/custom_widgets.c */
ret_t custom_widgets_init() {
  ...
  widget_factory_register(widget_factory(), WIDGET_TYPE_PIE_SLICE, pie_slice_
↪create);

  return RET_OK;
}
```

例如，在编写的 UI 文件中（res_800_480/assets/default/raw/ui/window_pie.xml）使用了 pie_slice 控件，那么 AWTK 在解析该 XML 的时候根据 WIDGET_TYPE_PIE_SLICE 判断是不是 AWTK 内置的控件。如果不是，就会根据 WIDGET_TYPE_PIE_SLICE 找到注册的 pie_slice_create 函数，然后调用该函数创建控件。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

4.9.2 创建控件

要创建控件就需要了解一下 struct widget_vtable_t 结构体，该结构体中的成员主要完成控件的创建、响应事件以及子控件的布局等。

AWTK 提供了非常便捷和灵活的方式，只需重新实现 struct widget_vtable_t 结构体中的几个成员函数就可以完成创建控件，通常需要实现的函数如下：

- create 控件创建函数
- on_paint_self 控件自绘函数
- on_event 控件响应键盘、鼠标等事件函数
- get_prop 获取控件属性函数
- set_prop 设置控件属性函数

其他的根据需要实现 struct widget_vtable_t 结构体中的成员，代码详见下文。

创建控件主要有两个步骤：

步骤一：定义 struct widget_vtable_t 变量

```
/* awtk/src/base/widget.h */
struct _widget_vtable_t {
  ...
  widget_create_t create;
  widget_get_prop_t get_prop;
  widget_get_prop_default_value_t get_prop_default_value;
  widget_set_prop_t set_prop;
  widget_on_keyup_t on_keyup;
  widget_on_keydown_t on_keydown;
  widget_on_paint_background_t on_paint_background;
  widget_on_paint_self_t on_paint_self;
  widget_on_paint_children_t on_paint_children;
  widget_on_paint_border_t on_paint_border;
  widget_on_paint_begin_t on_paint_begin;
  widget_on_paint_end_t on_paint_end;
  widget_on_pointer_down_t on_pointer_down;
  widget_on_pointer_move_t on_pointer_move;
  widget_on_pointer_up_t on_pointer_up;
  widget_on_layout_children_t on_layout_children;
  widget_invalidate_t invalidate;
  widget_on_add_child_t on_add_child;
  widget_on_remove_child_t on_remove_child;
  widget_on_event_t on_event;
  widget_on_event_before_children_t on_event_before_children;
  widget_find_target_t find_target;
  widget_on_destroy_t on_destroy;
};
```

饼图 pie_slice 实现的结构成员，代码如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
/* Chart-Demo/src/custom_widgets/pie_slice/pie_slice.c */
static const widget_vtable_t s_pie_slice_vtable = {.size = sizeof(pie_slice_t),
                                                   .type = WIDGET_TYPE_PIE_SLICE,
                                                   .clone_properties = s_pie_slice_clone_
↪properties,
                                                   .create = pie_slice_create,
                                                   .on_paint_self = pie_slice_on_paint_self,
                                                   .on_event = pie_slice_on_event,
                                                   .get_prop = pie_slice_get_prop,
                                                   .set_prop = pie_slice_set_prop};
```

步骤二：实现 struct widget_vtable_t 中的结构成员

上个步骤中定义了 s_pie_slice_vtable 变量，它的各个结构成员所代表的含义请看下文。

(1) size

表示的是饼图 pie_slice_t 的大小，此时就需要定义 struct pie_slice_t 了，代码如下：

```
/* Chart-Demo/src/custom_widgets/pie_slice/pie_slice.h */
typedef struct _pie_slice_t {
  widget_t widget;
  /**
   * @property {float_t} value
   * @annotation ["set_prop","get_prop","readable","persitent","design","scriptable
↪"]
   * 值(缺省为0)。
   */
  float_t value;
  /**
   * @property {uint32_t} max
   * @annotation ["set_prop","get_prop","readable","persitent","design","scriptable
↪"]
   * 最大值(缺省为100)。
   */
  uint32_t max;
  /* ... */
} pie_slice_t;
```

(2) type

控件的类型，这个类型在注册控件的时候也会用到。

(3) clone_properties

克隆 widget 时需要复制的属性。

(4) create

创建控件函数，这个函数在注册控件的时候也会用到。在这个函数中主要为 pie_slice_t 分配内存以及初始化，还有就是将 s_pie_slice_vtable 赋值给 pie_slice_t，代码如下：

```
/* Chart-Demo/src/custom_widgets/pie_slice/pie_slice.c */
widget_t* pie_slice_create(widget_t* parent, xy_t x, xy_t y, wh_t w, wh_t h) {
  pie_slice_t* pie_slice = TKMEM_ZALLOC(pie_slice_t);
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  widget_t* widget = WIDGET(pie_slice);
  return_value_if_fail(pie_slice != NULL, NULL);

  widget_init(widget, parent, &s_pie_slice_vtable, x, y, w, h);

  pie_slice->max = 100;
  pie_slice->inner_radius = 8;
  pie_slice->start_angle = 0;
  pie_slice->show_text = FALSE;
  pie_slice->counter_clock_wise = FALSE;
  pie_slice->is_exploded = FALSE;
  pie_slice->explode_distancefactor = 25;
  pie_slice->is_semicircle = FALSE;

  return widget;
}
```

(5) on_paint_self

控件的自绘函数，在这函数中通过调用 vgcanvas 或者 canvas 接口绘制相应的图形。在饼图这个例子中调用了 vgcanvas 接口完成了饼图的绘制，代码如下：

```
/* Chart-Demo/src/custom_widgets/pie_slice/pie_slice.c */
static ret_t pie_slice_on_paint_self(widget_t* widget, canvas_t* c) {
  bitmap_t img;
  style_t* style = widget->astyle;
  color_t trans = color_init(0, 0, 0, 0);
  vgcanvas_t* vg = canvas_get_vgcanvas(c);
  pie_slice_t* pie_slice = PIE_SLICE(widget);
  color_t color = style_get_color(style, STYLE_ID_FG_COLOR, trans);
  const char* image_name = style_get_str(style, STYLE_ID_FG_IMAGE, NULL);
  bool_t has_image = image_name && widget_load_image(widget, image_name, &img) ==␣
↪RET_OK;

  if (pie_slice->value > pie_slice->max) {
    pie_slice->value = pie_slice->max;
  }

  if (vg != NULL && (has_image || color.rgba.a)) {
    xy_t cx = widget->w / 2;
    xy_t cy = widget->h / 2;
    float_t end_angle = 0;
    float_t r = 0;
    bool_t ccw = pie_slice->counter_clock_wise;
    float_t start_angle = TK_D2R(pie_slice->start_angle);
    float_t angle = (M_PI * 2 * pie_slice->value) / pie_slice->max;

    if (ccw) {
      end_angle = start_angle - angle + M_PI * 2;
    } else {
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    end_angle = start_angle + angle;
  }

  vgcanvas_save(vg);
  vgcanvas_translate(vg, c->ox, c->oy);
  if (end_angle > start_angle) {
    vgcanvas_set_fill_color(vg, color);
    vgcanvas_begin_path(vg);
    r = tk_min(cx, cy);
    r -= pie_slice->explode_distancefactor;

    if (pie_slice->is_semicircle) {
      if (ccw) {
        start_angle = M_PI * 2 - (start_angle + angle) / 2;
        end_angle = start_angle + angle / 2;
        cy = cy + r * 0.5;
        r += pie_slice->explode_distancefactor * 1.5;
        vgcanvas_arc(vg, cx, cy, r, start_angle, end_angle, !ccw);
        r -= pie_slice->inner_radius;
        vgcanvas_arc(vg, cx, cy, r, end_angle, start_angle, ccw);
      }
    } else {
      vgcanvas_arc(vg, cx, cy, r, start_angle, end_angle, ccw);
      if (r - pie_slice->inner_radius <= 0) {
        vgcanvas_line_to(vg, cx, cy);
      } else {
        r -= pie_slice->inner_radius;
        vgcanvas_arc(vg, cx, cy, r, end_angle, start_angle, !ccw);
      }
    }

    vgcanvas_close_path(vg);
    if (has_image) {
      vgcanvas_paint(vg, FALSE, &img);
    } else {
      vgcanvas_fill(vg);
    }
  }
  vgcanvas_restore(vg);
}

color = style_get_color(style, STYLE_ID_TEXT_COLOR, trans);
if (pie_slice->show_text && color.rgba.a) {
  char s[TK_NUM_MAX_LEN + TK_NUM_MAX_LEN + 1];
  const char* unit = pie_slice->unit != NULL ? pie_slice->unit : "";
  tk_snprintf(s, sizeof(s), "%u%s", (uint32_t)(pie_slice->value), unit);

  widget_set_text_utf8(widget, s);
  widget_paint_helper(widget, c, NULL, NULL);
```

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  }

  return RET_OK;
}
```

(6) on_event

键盘和鼠标响应事件，如果想通过键盘或者鼠标完成某些特定的功能，就可以在这个函数中实现。例如在本例中实现了点击某个扇形的时候展开，代码如下：

```
/* Chart-Demo/src/custom_widgets/pie_slice/pie_slice.c */
static ret_t pie_slice_on_event(widget_t* widget, event_t* e) {
  uint16_t type = e->type;
  pie_slice_t* pie_slice = PIE_SLICE(widget);
  switch (type) {
    case EVT_POINTER_DOWN: {
      pie_slice->pressed = TRUE;
      xy_t cx = widget->w / 2;
      xy_t cy = widget->h / 2;
      float_t r = tk_min(cx, cy) - pie_slice->explode_distancefactor;
      pointer_event_t* pointer_event = (pointer_event_t*)e;
      point_t p = {pointer_event->x, pointer_event->y};
      widget_to_local(widget, &p);
      int32_t x_f = p.x - widget->w * 0.5;
      int32_t y_f = p.y - widget->h * 0.5;
      pie_slice_t* iter_result = pie_slice_get_clicked_pie(widget, pie_slice, r, &
↪x_f, &y_f);
      if (iter_result == NULL) {
        break;
      }
      pie_slice_on_event_4_pointer_down(widget, pie_slice, r, x_f, y_f, iter_
↪result);
      break;
    }
    case EVT_POINTER_UP: {
      pointer_event_t evt = *(pointer_event_t*)e;
      if (pie_slice->pressed && widget_is_point_in(widget, evt.x, evt.y, FALSE)) {
        evt.e = event_init(EVT_CLICK, widget->parent);
        widget_dispatch(widget, (event_t*)&evt);
        pie_slice->pressed = FALSE;
      }
      break;
    }
    default:
      break;
  }

  return RET_OK;
}
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

(7) get_prop

获取属性值，可以调用 widget_get_prop（最终会调用 pie_slice_get_prop 函数）函数获取属性值，例如想要获取 struct pie_slice_t 中 is_exploded 成员的值，代码如下：

```
/* Chart-Demo/src/window_pie.c */
static ret_t create_animator_to_zero(widget_t* win) {
  set_btn_enable(win, FALSE);

  widget_t* pie_view = widget_lookup(win, "pie_view", TRUE);
  if (pie_view) {
    WIDGET_FOR_EACH_CHILD_BEGIN_R(pie_view, iter, i)
    value_t v;
    widget_get_prop(iter, PIE_SLICE_PROP_IS_EXPLODED, &v);
    save_pie_exploded[i] = value_bool(&v);

    pie_slice_t* pie_slice = PIE_SLICE(iter);
    if (pie_slice->is_exploded) {
      pie_slice_set_exploded(iter);
    }
    int32_t delay = 80;
    delay = delay * (nr - 1 - i);
    char param[100];
    tk_snprintf(param, sizeof(param), "value(to=0, duration=50, delay=%d,␣
↪easing=sin_out)", delay);
    widget_create_animator(iter, param);
    WIDGET_FOR_EACH_CHILD_END();
  }

  return RET_OK;
}
```

(8) set_prop

设置属性值。可以调用 widget_set_prop（最终会调用 pie_slice_set_prop 函数）函数设置属性值，例如想要设置 struct pie_slice_t 中 is_exploded 成员的值，代码如下：

```
/* Chart-Demo/src/window_pie.c */
static ret_t on_save_exploded_timer(const timer_info_t* timer) {
  widget_t* win = WIDGET(timer->ctx);

  uint32_t count = widget_animator_manager_count(widget_animator_manager());
  if (count == 0) {
    widget_t* pie_view = widget_lookup(win, "pie_view", TRUE);
    if (pie_view) {
      WIDGET_FOR_EACH_CHILD_BEGIN_R(pie_view, iter, i)
      value_t v;
      value_set_bool(&v, !save_pie_exploded[i]);
      widget_set_prop(iter, PIE_SLICE_PROP_IS_EXPLODED, &v);
      pie_slice_set_exploded(iter);
      WIDGET_FOR_EACH_CHILD_END();
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    }
    set_btn_enable(win, TRUE);

    return RET_REMOVE;
  }

  return RET_REPEAT;
}
```

4.9.3 使用控件

注册和创建好控件就可以使用了，使用的方法和 AWTK 内置的控件一样。例如通过编写 XML 文件使用 pie_slice 饼图控件，代码如下：

```
<!-- Chart-Demo/res_800_480/assets/default/raw/ui/window_pie.xml -->
<window anim_hint="htranslate" tr_text="pie_title" x="0%" y="50" w="800" h="427" >
    <view name="pie_view" x="2%" y="m" w="60%" h="80%">
        <pie_slice name="pie1_slice" w="100%" h="100%" max="360" start_angle="0"␣
↪value="0" show_text="false" inner_radius="550" style="pie1"/>
        <pie_slice name="pie2_slice" w="100%" h="100%" max="360" start_angle="0"␣
↪value="0" show_text="false" inner_radius="550" style="pie2"/>
        <pie_slice name="pie3_slice" w="100%" h="100%" max="360" start_angle="0"␣
↪value="0" show_text="false" inner_radius="550" style="pie3"/>
        <pie_slice name="pie4_slice" w="100%" h="100%" max="360" start_angle="0"␣
↪value="0" show_text="false" inner_radius="550" style="pie4"/>
        <pie_slice name="pie5_slice" w="100%" h="100%" max="360" start_angle="0"␣
↪value="0" show_text="false" inner_radius="550" style="pie5"/>
        <pie_slice name="pie6_slice" w="100%" h="100%" max="360" start_angle="0"␣
↪value="0" show_text="false" inner_radius="550" style="pie6"/>
    </view>
    ...
</window>
```

XML 文件中 pie_slice 里面的 start_angle、show_text 等设置的值，会调用 pie_slice_set_prop 函数，给对应的对象设置相应的值。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

5. 动画

本章导读：

一个炫酷的 *GUI* 界面是离不开动画的，通过使用动画给人用户眼前一亮的感觉。*AWTK* 提供了多种动画机制，可以让 *GUI* 变得更加的形象、流畅。

5.1 简介

AWTK 提供了两大类动画：窗口动画和控件动画，下面介绍如何使用这两种动画。

5.2 窗口动画

窗口动画是现代 GUI 最基本的功能之一，在窗口打开或关闭时，引入一个过渡动画，让用户感觉这个过程是流畅的。窗口动画的基本原理很简单：在打开或关闭窗口时，把前后两个窗口预先绘制到两张内存图片上，按照指定规则显示两张图片，形成动画效果。

5.2.1 动画类型

窗口本身只需指定期望的动画类型，由窗口管理器负责在适当的时候（如打开和关闭窗口时），创建窗口动画并让窗口动画绘制到屏幕上。在窗口动画期间，窗口管理器会禁止窗口本身的绘制，并忽略所有输入事件。目前支持的动画有：普通窗口动画和对话框动画。

- 普通窗口动画详见下表：

窗口动画类型	说明
htranslate	左右平移动画（速度最快，嵌入式平台推荐使用）
vtranslate	上下平移动画
slide_up	向上弹出
slide_down	向下弹出
slide_left	向左弹出
slide_right	向右弹出

- 对话框动画详见下表：

窗口动画类型	说明
popup	向上弹出（速度最快，嵌入式平台推荐使用）
popdown	向下弹出
fade	淡入淡出动画
center_scale	缩放动画（没有硬件时加速慎用）

注: 以前的 bottom_to_top 相当于 popup，仍然可以使用，但建议用 popup 代替。以前的 top_to_bottom 相当于 popdown，仍然可以使用，但建议用 popdown 代替。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

5.2.2 使用方法

给窗口或对话框指定动画效果，只需设置窗口或对话框的 anim_hint 属性即可，代码如下：

```
<window name="main" anim_hint="vtranslate">
...
</window>
```

5.2.3 动画参数

可以为动画指定参数，其格式为类似函数调用的参数格式。

（1）所有动画都支持下面的参数：

- duration 动画持续时间（毫秒）
- easing 插值算法名称

如：

```
anim_hint="center_scale(duration=300)"
```

（2）slide_up、slide_down、slide_left 和 slide_right 动画特有参数：

- start_alpha 起始 alpha 值（0-255）
- end_alpha 结束 alpha 值（0-255）
- alpha 相当于将 start_alpha 和 end_alpha 设置为同一个值

注：alpha 值用于在背景上画一层透明的颜色，这能够实现让背景变暗的效果。如果 start_alpha 和 end_alpha 不相同，则能实现动态变暗的效果，这有额外的运行开销，如果性能不佳，建议设置 alpha 即可。

5.2.4 示例

给窗口或对话框指定动画效果，只需设置窗口或对话框的 anim_hint 属性即可，下面以 awtk/bin/demoui.exe 中的主界面 main.xml 为例，代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/main.xml -->
<window closable="no" text="Desktop" anim_hint="htranslate">
  <slide_view x="0" y="0" w="100%" h="100%">
    <view x="0" y="0" w="100%" h="100%" children_layout="default(c=2,r=8,m=5,s=5)">
      <button name="open:basic" text="Basic"/>
      <button name="open:button" text="Buttons"/>
      <button name="open:edit" text="Edits"/>
      <button name="open:keyboard" text="KeyBoard"/>
      <button name="open:list_view" text="ListView"/>
      <button name="open:slide_view" text="SlideView"/>
      <button name="open:animation" text="Animate Window"/>
      <button name="open:animator" text="Animate Widget"/>
      <button name="open:tab_control" text="Tab Control"/>
      <button name="open:combo_box" text="ComboBox"/>
      <button name="open:rich_text" text="RichText"/>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
      <button name="open:color_picker" text="Color Picker"/>
    </view>
    ...
</window>
```

还可以指定动画时长（duration，单位为毫秒），格式与函数调用类似，不过参数用 name=value 的形式，如：

```
anim_hint="center_scale(duration=300)"
```

5.2.5 自定义动画

开发者可以自定义动画，自定义的动画可以像内置动画一样，在 XML 文件中启用。

自定义动画非常简单，通常只需要几行代码即可。步骤如下：

（1）实现 window_animator_t 接口。如：

```
static const window_animator_vtable_t s_window_animator_popup_vt = {
.overlap = TRUE,
.type = "popup",
.desc = "popup",
.size = sizeof(window_animator_t),
.draw_prev_window = window_animator_overlap_default_draw_prev,
.draw_curr_window = window_animator_to_top_draw_curr};
window_animator_t* window_animator_popup_create(bool_t open, object_t* args) {
return window_animator_create(open, &s_window_animator_popup_vt);
}
```

（2）注册到 window_animator 工厂。如：

```
window_animator_factory_register(factory, WINDOW_ANIMATOR_POPUP, window_animator_
↪popup_create);
```

5.3 控件动画

控件动画是一种很常见的动画，常用于入场动画、离场动画、装饰用户界面和吸引用户注意力等。

5.3.1 动画类型

AWTK 目前支持的控件动画类型详见下表：

控件动画类型	说明
move	通过改变控件位置形成动画效果
value	通过改变控件值形成动画效果
opacity	通过改变控件透明度形成动画效果
scale	通过改变控件缩放比例形成动画效果（目前需要 vgcanvas）
rotation	通过改变控件旋转角度形成动画效果（目前需要 vgcanvas，draw_type 为 icon 才能旋转）
其它任何数值型的属性	如 x/y/w/h 等等

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

5.3.2 特色

- 支持逆向模式
- 支持停止和暂停
- 支持定时（延迟）执行
- 支持重复模式（可指定次数）
- 支持往返模式（可指定次数）
- 支持多种插值算法（如加速和减速等）
- 支持同一控件多个动画并行执行
- 支持同一控件多个动画串行执行
- 支持时间倍率，让时间变快和变慢
- 支持按名称去开始、暂停、停止和销毁动画

5.3.3 函数

AWTK 提供了的函数接口详见下表，具体请查看 AWTK_API 手册。

函数名称	说明
widget_animator_destroy	销毁 animator 对象
widget_animator_init	初始化仅供子类内部使用
widget_animator_off	注销指定事件的处理函数
widget_animator_on	注册指定事件的处理函数
widget_animator_pause	暂停动画
widget_animator_set_destroy_when_done	设置完成时是否自动销毁动画对象（缺省销毁）
widget_animator_set_name	设置名称
widget_animator_set_repeat	设置为重复模式
widget_animator_set_reversed	设置为逆向模式
widget_animator_set_time_scale	设置时间倍率，用于实现时间加速减速和停滞的功能
widget_animator_set_yoyo	设置为 yoyo 模式
widget_animator_start	启动动画
widget_animator_stop	停止动画

5.3.4 XML 参数

1、公共参数，详见下表：

参数	说明
name	动画名称（缺省为动画的类型如 move）
delay	延迟启动时间（毫秒）
duration	时长（毫秒）
easing	插值算法（见后面描述）
yoyo_times	往返的次数（x2），为 0 视为永久播放
repeat_times	重复的次数，为 0 视为永久播放
auto_start	创建后自动启动（缺省为 true）
auto_destroy	完成后自动销毁（缺省为 true）

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

2、widget_animator_move 动画的参数，详见下表：

参数	说明
x_from	x 起始位置
y_from	y 起始位置
x_to	x 结束位置
y_to	y 结束位置

3、widget_animator_value 动画的参数，详见下表：

参数	说明
from	起始值
to	结束值

4、widget_animator_opacity 动画的参数，详见下表：

参数	说明
from	起始值（0-255）
to	结束值（0-255）

5、widget_animator_scale 动画的参数，详见下表：

参数	说明
x_from	x 方向缩放起始值
y_from	y 方向缩放起始值
x_to	x 方向缩放结束值
y_to	y 方向缩放结束值

6、widget_animator_rotation 动画的参数，详见下表：

参数	说明
from	起始值（弧度）
to	结束值（弧度）

7、其它数值型属性动画的参数，详见下表：

参数	说明
from	起始值
to	结束值

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

5.3.5 插值算法名称（easing）

easing 的类型和可取的值如下：

1. linear：
    - linear
2. quadratic：
    - quadratic_out
    - quadratic_inout
3. cubic：
    - cubic_in
    - cubic_out
4. sin：
    - sin_in
    - sin_out
    - sin_inout
5. pow：
    - pow_in
    - pow_out
    - pow_inout
6. circular：
    - circular_in
    - circular_out
    - circular_inout
7. elastic：
    - elastic_in
    - elastic_out
    - elastic_inout
8. back：
    - back_in
    - back_out
    - back_inout
9. bounce
    - bounce_inout
    - bounce_out
    - bounce_inout

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

5.3.6 示例

既可以在 UI 文件的 XML 中，也可以使用 C 代码中创建动画，接下来将分别介绍如何在 XML 和 C 代码中创建动画。

1. 在 XML 中

在缺省情况下，动画创建后自动启动，完成后自动销毁。可以指定 auto_start=false 禁止创建后自动启动，指定参数 auto_destroy=false 禁止完成时自动销毁（控件销毁时仍然会自动销毁）。

在 XML 中可以使用 animation 参数创建动画，多个参数可以用”;”分隔，此方法最为简单，后续可以在程序中通过动画名字，对动画进行启动和暂停等控制。下面以 awtk/bin、demoui.exe 中的 AnimateWidget 页面为例，代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/animator.xml -->
<window>
  <image name="fade" image="logo" x="center" y="10" w="200" h="60" opacity="0"
    animation="opacity(from=50, to=255, yoyo_times=1000, duration=1000)" />
  <image name="fade" image="logo" x="center" y="80" w="200" h="60" opacity="0"
    animation="opacity(from=50, to=255, yoyo_times=1000, duration=1000, delay=1000)
↪" />
  <image name="fade" image="logo" x="center" y="150" w="200" h="60" opacity="0"
    animation="opacity(from=50, to=255, yoyo_times=1000, duration=1000, delay=2000)
↪" />
  <image name="move" image="logo" x="0" y="middle:60" w="200" h="60"
    animation="move(x_from=0, x_to=200, yoyo_times=1000, duration=1000, delay=3000)
↪"/>
  <progress_bar name="value" x="center" y="middle" w="90%" h="40"
    animation="value(from=50, to=100, yoyo_times=1000, duration=1000, delay=4000)"/
↪>
  <button name="close" x="center" y="bottom:10" w="25%" h="30" text="Close"/>
</window>
```

2. 在 C 代码中

在 C 代码中有两种方式创建动画，下面以 awtk/bin/demo_animator.exe 为例：

(1) 采用 widget_animator_xxx_create，其中 xxx 可取值 move/value/opacity/scale/rotation，此方法比较麻烦，不再推荐使用，代码如下：

```
/* awtk/demos/demo_animator_app.c */
ret_t application_init() {
  ...
  animator = widget_animator_move_create(image, 1000, delay, EASING_SIN_INOUT);
  widget_animator_move_set_params(animator, image->x, image->y, image->x + 100,␣
↪image->y + 100);
  widget_animator_set_destroy_when_done(animator, FALSE);
  widget_animator_set_repeat(animator, times);
  widget_animator_start(animator);
  ...
}
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

(2) 通过函数 widget_create_animator 创建动画，推荐使用，代码如下：

```
/* awtk/demos/demo_animator_app.c */
ret_t application_init() {
  ...
  widget_create_animator(progress_bar, "value(from=50, to=100, duration=500, yoyo_
↪times=1000, delay=1000)");
  widget_create_animator(opacity, "opacity(to=50, duration=500, yoyo_times=1000)");
  widget_create_animator(image6, "opacity(to=50, duration=500, yoyo_times=1000,␣
↪delay=1000)");
  widget_create_animator(image2, "rotation(to=6.28, yoyo_times=0, duration=1000,␣
↪easing=sin_out)");
  widget_create_animator(image3, "scale(x_to=-1, yoyo_times=1000, duration=1000,␣
↪easing=sin_out)");
  widget_create_animator(image4, "scale(x_to=2, y_to=2, yoyo_times=1000,␣
↪duration=1000, easing=sin_out)");
  widget_create_animator(image5, "y(to=400, duration=1000, easing=sin_out)");
  widget_create_animator(image5, "opacity(to=0, duration=500, yoyo_times=1000,␣
↪delay=1000)");
  ...
}
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

6. 画布

本章导读：

AWTK 提供了各种绘制图像的 *API* 接口，方便程序员绘制直线、矩形、椭圆等形状。也可以根据 *AWTK* 提供的 *API* 接口绘制自己想要的控件，例如饼图、柱状图等。*AWTK* 提供了两种画布：普通和矢量图画布，通过调用这两种画布提供的画图接口，实现不同的绘图功能。

6.1 普通画布

6.1.1 简介

canvas 类。普通画布可以实现一些简单的绘制功能，如：绘制水平或垂直直线、文本、图片、矩形等。如果需要绘制比较复杂一点的图形就需要使用矢量图画布，如：绘制椭圆、圆弧等。

6.1.2 函数

canvas 提供的函数详见下表，具体的函数 API 请查看 AWTK_API 手册。

函数名称	说明
canvas_begin_frame	绘制开始
canvas_cast	转换为 canvas 对象 (供脚本语言使用)
canvas_clear_rect	用填充颜色填充指定矩形
canvas_draw_hline	画水平线
canvas_draw_icon	绘制图标
canvas_draw_image	绘制图片
canvas_draw_image_at	在指定位置画图
canvas_draw_image_ex	绘制图片
canvas_draw_image_ex2	绘制图片
canvas_draw_line	画直线
canvas_draw_points	画多个点
canvas_draw_text	绘制文本
canvas_draw_text_bidi_in_rect	绘制文本
canvas_draw_text_in_rect	绘制文本
canvas_draw_utf8	绘制文本
canvas_draw_utf8_in_rect	绘制文本
canvas_draw_vline	画垂直线
canvas_end_frame	绘制结束
canvas_fill_rect	绘制矩形
canvas_fill_rect_gradient	绘制矩形
canvas_fill_rounded_rect	填充区域
canvas_fill_rounded_rect_ex	填充圆角矩形区域
canvas_fill_rounded_rect_gradient	填充圆角矩形区域
canvas_fill_rounded_rect_gradient_ex	填充圆角矩形区域

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
canvas_get_clip_rect	获取裁剪区
canvas_get_font_height	获取字体的高度
canvas_get_height	获取画布的高度
canvas_get_text_metrics	获取当前字体的度量信息
canvas_get_vgcanvas	获取 vgcanvas 对象
canvas_get_width	获取画布的宽度
canvas_init	初始化，系统内部调用
canvas_measure_text	计算文本所占的宽度
canvas_measure_utf8	计算文本所占的宽度
canvas_reset	释放相关资源
canvas_set_assets_manager	设置 canvas 的 assets_manager 对象
canvas_set_clip_rect	设置裁剪区
canvas_set_clip_rect_ex	设置裁剪区
canvas_set_fill_color	设置填充颜色
canvas_set_fill_color_str	设置填充颜色
canvas_set_font	设置字体
canvas_set_font_manager	设置 canvas 的 font_manager 对象
canvas_set_fps	设置 FPS
canvas_set_global_alpha	设置全局 alpha 值
canvas_set_stroke_color	设置线条颜色
canvas_set_stroke_color_str	设置线条颜色
canvas_set_text_align	设置文本对齐方式
canvas_set_text_color	设置文本颜色
canvas_set_text_color_str	设置文本颜色
canvas_stroke_rect	绘制矩形
canvas_stroke_rounded_rect	绘制边框
canvas_stroke_rounded_rect_ex	绘制边框
canvas_translate	平移原点坐标
canvas_untranslate	反向平移原点坐标

6.1.3 属性

canvas 还提供了下面属性，详见下表：

属性名称	类型	说明
assets_manager	assets_manager_t*	资源管理器对象
clip_bottom	xy_t	当前裁剪矩形的底部位置
clip_left	xy_t	当前裁剪矩形的左边位置
clip_right	xy_t	当前裁剪矩形的右边位置
clip_top	xy_t	当前裁剪矩形的顶部位置
font	font_t*	字体对象
font_manager	font_manager_t*	字体管理器对象
font_name	char*	当前字体名称
font_size	uint16_t	当前字体大小
fps	uint32_t	当前的帧率

ZLG
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

属性名称	类型	说明
global_alpha	uint8_t	当前全局 alpha
lcd	lcd_t*	lcd 对象
ox	xy_t	x 坐标偏移
oy	xy_t	y 坐标偏移
show_fps	bool_t	是否显示帧率
text_align_h	align_h_t	文本水平对齐方式
text_align_v	align_v_t	文本垂直对齐方式

6.1.4 示例

canvas 的用法与 vgcanvas 的类似，请参考下文 6.2.4 章节。

6.2 矢量图画布

6.2.1 简介

vgcanvas_t 矢量图画布抽象基类。与 canvas 相比 vgcanvas 的效率要低一些，但功能也丰富些。绘制一些简单的图形，如线条、矩形等可以用 canvas；复杂一点图形，如圆角矩形就用 vgcanvas。

具体实现时，vgcanvas 内部调用了 nanovg、cairo 的功能进行绘图。目前只提供了基于 nanovg 的实现，支持软件渲染和硬件渲染。AWTK 对 nanovg 进行了一些改进：

- 可以用 agg/agge 实现软件渲染（暂时不支持文本绘制）
- 可以用 bgfx 使用 DirectX（Windows 平台）、Metal（iOS 平台）和使用 OpenGL，Vulkan（Linux 平台）硬件加速

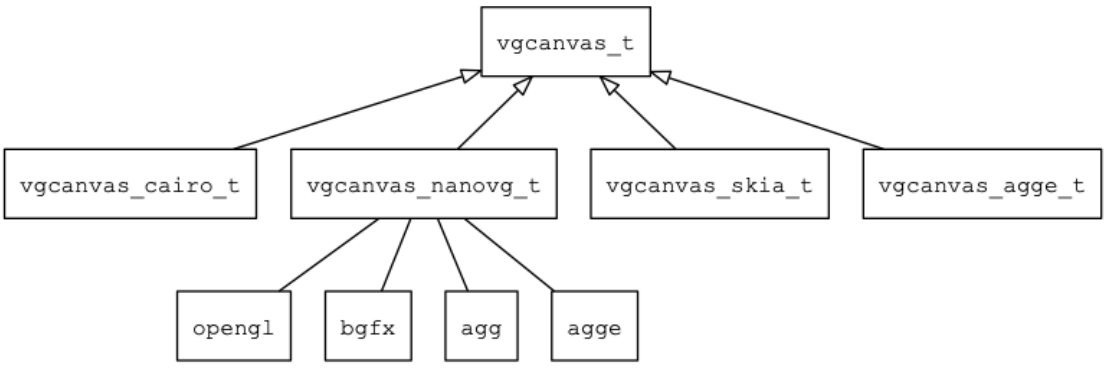


图 6.1 vgcanvas 的 UML 图

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

6.2.2 函数

vgcanvas 提供的函数详见下表，具体的函数 API 请查看 AWTK_API 手册。

函数名称	说明
vgcanvas_arc	生成一条圆弧
vgcanvas_arc_to	生成一条圆弧路径到指定点
vgcanvas_begin_frame	开始绘制，系统内部调用
vgcanvas_begin_path	清除之前的路径，并重新开始一条路径
vgcanvas_bezier_to	生成一条三次贝塞尔曲线
vgcanvas_bind_fbo	绑定 fbo 对象
vgcanvas_cast	转换为 vgcanvas 对象 (供脚本语言使用)
vgcanvas_clear_cache	释放 vgcanvas 对象的缓冲数据
vgcanvas_clear_rect	用颜色清除指定矩形区域
vgcanvas_clip_path	使用当前的 path 裁剪
vgcanvas_clip_rect	矩形裁剪
vgcanvas_close_path	闭合路径
vgcanvas_create	创建 vgcanvas
vgcanvas_create_fbo	创建 fbo 对象
vgcanvas_destroy	销毁 vgcanvas 对象
vgcanvas_destroy_fbo	销毁 fbo 对象
vgcanvas_draw_icon	绘制图标
vgcanvas_draw_image	绘制图片
vgcanvas_ellipse	生成一个椭圆路径
vgcanvas_end_frame	结束绘制系统内部调用
vgcanvas_fbo_to_bitmap	把 fbo 对象的数据拷贝到 bitmap 中
vgcanvas_fill	填充多边形
vgcanvas_fill_text	绘制文本
vgcanvas_flush	flush
vgcanvas_get_height	获取高度
vgcanvas_get_text_metrics	获取当前字体的度量信息
vgcanvas_get_width	获取宽度
vgcanvas_intersect_clip_rect	设置一个与前一个裁剪区做交集的矩形裁剪区
vgcanvas_is_point_in_path	检查点是否在当前路径中
vgcanvas_line_to	生成一条线段 (从当前点到目标点)
vgcanvas_measure_text	测量文本的宽度
vgcanvas_move_to	移动当前点到指定点
vgcanvas_paint	用图片填充/画多边形 (可能存在可移植性问题，除非必要请勿使用)
vgcanvas_path_winding	设置路径填充实心与否
vgcanvas_quad_to	生成一条二次贝塞尔曲线
vgcanvas_rect	生成一个矩形路径
vgcanvas_reinit	重新初始化，系统内部调用
vgcanvas_reset	重置所有状态
vgcanvas_reset_curr_state	重置当前状态
vgcanvas_restore	恢复上次 save 的状态
vgcanvas_rotate	旋转

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

函数名称	说明
vgcanvas_rounded_rect	生成一个圆角矩形路径
vgcanvas_save	保存当前的状态如颜色和矩阵等信息
vgcanvas_scale	缩放
vgcanvas_set_antialias	设置是否启用反走样
vgcanvas_set_fill_color	设置填充颜色
vgcanvas_set_fill_color_str	设置填充颜色
vgcanvas_set_fill_gradient	设置填充颜色为渐变色
vgcanvas_set_fill_linear_gradient	设置填充颜色为线性渐变色
vgcanvas_set_fill_radial_gradient	设置填充颜色为径向渐变色
vgcanvas_set_font	设置字体的名称
vgcanvas_set_font_size	设置字体的大小
vgcanvas_set_global_alpha	设置全局透明度
vgcanvas_set_line_cap	设置 line cap
vgcanvas_set_line_join	设置 line join
vgcanvas_set_line_width	设置线条的宽度
vgcanvas_set_miter_limit	设置 miter limit
vgcanvas_set_stroke_color	设置线条颜色
vgcanvas_set_stroke_color_str	设置线条颜色
vgcanvas_set_stroke_gradient	设置线条颜色为渐变色
vgcanvas_set_stroke_linear_gradient	设置线条颜色为线性渐变色
vgcanvas_set_stroke_radial_gradient	设置线条颜色为径向渐变色
vgcanvas_set_text_align	设置文本水平对齐的方式
vgcanvas_set_text_baseline	设置文本垂直对齐的方式
vgcanvas_set_transform	设置变换矩阵
vgcanvas_stroke	画线
vgcanvas_transform	变换矩阵
vgcanvas_translate	平移
vgcanvas_unbind_fbo	解开绑定 fbo 对象

6.2.3 属性

vgcanvas 还提供了下面属性，详见下表：

属性名称	类型	说明
anti_alias	bool_t	是否启用反走样功能
fill_color	color_t	填充颜色
font	char*	字体
font_size	float_t	字体大小
global_alpha	float_t	全局 alpha
h	wh_t	canvas 的高度
line_cap	const char*	line_cap
line_join	const char*	line_join
line_width	float_t	线宽
miter_limit	float_t	miter_limit
ratio	float_t	显示比例

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

续上表

属性名称	类型	说明
stride	uint32_t	一行占的字节
stroke_color	color_t	线条颜色
text_align	const char*	文本对齐方式
text_baseline	const char*	文本基线
w	wh_t	canvas 的宽度

6.2.4 示例

例如：要实现下图所示的效果，步骤如下：



图 6.2 vgcanvas 示例

步骤一：创建 canvas_widget 控件，并为该控件注册 EVT_PAINT 绘制事件的回调函数 on_paint_vg。

步骤二：在 on_paint_vg 回调函数中调用 canvas 或者 vgcanvas 画布接口提供的函数进行重绘。

具体代码如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
/* awtk/demos/demo_vg_app.c */
static ret_t on_paint_vg(void* ctx, event_t* e) {
  paint_event_t* evt = (paint_event_t*)e;
  canvas_t* c = evt->c;
  vgcanvas_t* vg = canvas_get_vgcanvas(c);

  vgcanvas_save(vg);
  vgcanvas_set_line_width(vg, 1);
  vgcanvas_set_stroke_color(vg, color_init(0, 0xff, 0, 0xff));
  vgcanvas_set_fill_color(vg, color_init(0xff, 0, 0, 0xff));
  ...
  return RET_OK;
}

ret_t application_init() {
  widget_t* win = window_create(NULL, 0, 0, 0, 0);
  widget_t* canvas = canvas_widget_create(win, 0, 0, win->w, win->h);
  widget_on(canvas, EVT_PAINT, on_paint_vg, NULL);

  timer_add(on_timer, win, 500);

  return RET_OK;
}
```

注: 完整示例请参考 demo_vg_app.c[19]。

6.3 离线画布

6.3.1 简介

1. 在线画布

通常可以调用 canvas 或者 vgcanvas 画布接口提供的函数，将要绘制的内容显示到屏幕上，称这种方式为在线画布，并且可以通过下面几种方式实现：

（1）方式一：创建 canvas_widget 控件，并为该控件注册 EVT_PAINT 绘制事件的回调函数，在该回调函数中调用 canvas 或者 vgcanvas 画布接口提供的函数进行绘图。

注: 完整示例请参考 demo_vg_app.c[20]。

（2）方式二：在控件中的 on_paint_self 自绘函数中，调用 canvas 或者 vgcanvas 画布接口提供的函数进行绘图。

注: 完整示例请参考 guage.c[21]。

2. 离线画布

将 canvas 绘制到内存 bitmap 对象中，然后可以对 bitmap 对象做进一步处理，比如将 bitmap 对象保存到文件中，实现截图效果，称这种方式为离线画布。

为了解决用户离线画布的需求，在 AWTK 中提供了 canvas_offline 类供用户使用。

[19] https://github.com/zlgopen/awtk/blob/master/demos/demo_vg_app.c
[20] https://github.com/zlgopen/awtk/blob/master/demos/demo_vg_app.c
[21] https://github.com/zlgopen/awtk/tree/master/src/ext_widgets/guage.c

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

6.3.2 函数

canvas_offline 类提供了下表中的函数，具体的函数 API 请查看 AWTK_API 手册。

函数名称	说明
canvas_offline_begin_draw	设置离线 canvas 开始绘图
canvas_offline_bitmap_move_to_new_bitmap	把离线 canvas 的离线 bitmap 移动赋值给新的 bitmap
canvas_offline_clear_canvas	清除离线 canvas 所有数据，并把背景设置为全透明
canvas_offline_create	创建一个离线的 canvas
canvas_offline_custom_begin_draw	用户自定义 canvas_offline_custom_begin_draw
canvas_offline_custom_bitmap_move_to_new_bitmap	用户自定义 canvas_offline_custom_bitmap_move_to_new_bitmap
canvas_offline_custom_clear_canvas	用户自定义 canvas_offline_custom_clear_canvas
canvas_offline_custom_create	用户自定义 canvas_offline_create
canvas_offline_custom_destroy	用户自定义 canvas_offline_custom_destroy
canvas_offline_custom_end_draw	用户自定义 canvas_offline_custom_end_draw
canvas_offline_custom_flush_bitmap	用户自定义 canvas_offline_custom_flush_bitmap
canvas_offline_custom_get_bitmap	用户自定义 canvas_offline_custom_get_bitmap
canvas_offline_destroy	释放离线 canvas 和离线 bitmap
canvas_offline_end_draw	设置离线 canvas 结束绘图
canvas_offline_flush_bitmap	把离线 canvas 的数据放到绑定的 bitmap 中
canvas_offline_get_bitmap	获取离线 canvas 的离线 bitmap

6.3.3 属性

canvas_offline 还提供了下面属性，详见下表：

属性名称	类型	说明
bitmap	bitmap_t*	绑定的离线 bitmap

6.3.4 示例

例如，使用离线画布完成一个截屏的功能，其步骤如下：

步骤一：创建离线画布。调用 canvas_offline_create 函数创建离线画布，在离线画布内置了一个离线的 bitmap 位图，提供给用户使用。创建离线画布，代码如下：

```
canvas_t* canvas_offline = canvas_offline = canvas_offline_create(320, 480, BITMAP_
↪FMT_RGBA8888);
```

注：在 OpenGL 模式下离线画布 canvas 格式只能为 RGBA8888；在 AGGE 模式下可以为：RGBA8888、BGRA8888、RGB565 和 BGR5656。（其他的格式暂时不支持）

步骤二：绘图。离线画布的绘图过程和平时使用 canvas 或者 vgcanvas 绘图是一样的，只不过在绘制之前需要先调用 canvas_offline_begin_draw 函数开始绘图，并在绘制结束的时候调用 canvas_offline_end_draw 函数结束绘图。例如，把整个窗口绘制到离线画布上，代码如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
bitmap_t* offline_bitmap = NULL;
canvas_offline_begin_draw(canvas_offline);       /* 开始离线 canvas 绘图 */
widget_paint(window_manager(), canvas_offline);  /* 把整个窗口绘制到离线的 canvas
↪中 */
canvas_offline_end_draw(canvas_offline);         /* 结束离线 canvas 绘图 */

/* 由于需要把 bitmap 保存为 png 文件，所以需要把显存中的数据回传到内存中 */
canvas_offline_flush_bitmap(canvas_offline);
offline_bitmap = canvas_offline_get_bitmap(canvas_offline); /* 获取离线 bitmap */
bitmap_save_png(offline_bitmap, "test.png");               /* 把 bitmap 保存为 png
↪文件 */
```

步骤三：释放离线画布和绑定的离线 bitmap，代码如下：

```
canvas_offline_destroy(canvas_offline);
```

注: 完整示例请参考 demo_canvas_offline.c[22]。

[22] https://github.com/zlgopen/awtk/blob/master/demos/demo_canvas_offline.c

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

7. 输入法

本章导读：

AWTK 提供了多种输入键盘，可以根据应用的具体情况灵活搭配选择一种或者多种键盘，还可以根据自己的喜好设置不同的窗体样式。

7.1 简介

输入法是 GUI 重要的组件之一，虽然实现起来并不是太复杂，但其涉及的组件比较多，理解起来还是比较困难的，这里介绍一下 AWTK 中输入法的内部架构，详见下图：

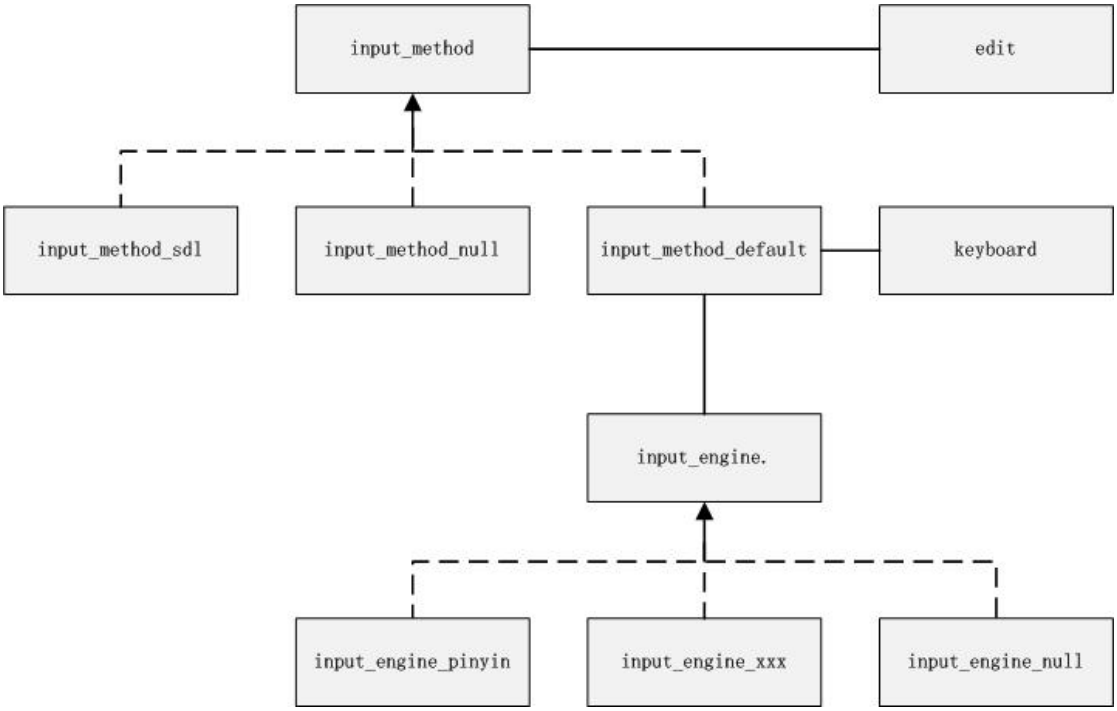


图 7.1 输入法内部架构图

- input_method sdl 实现包装了原生输入法 (SDL 的实现有些问题，还需要进一步完善)
- input_method_null 实现只是提供了空的实现，不启用输入法和软键盘
- input_method default 提供了 AWTK 自己的实现，负责软键盘的打开关闭和输入法引擎的创建

7.2 软键盘

在嵌入式系统中，通常没有物理键盘，所以需要在屏幕上实现软键盘。AWTK 中的软键盘是一个普通的窗口，其中的按钮和候选字控件，都是用 AWTK 的 UI 描述文件定义的，可以方便实现各种不同的软键盘。软键盘的描述文件放在 awtk/demos/assets/default/raw/ui 目录下，文件名以 kb_ 打头。与普通窗口相比，软键盘有以下不同：

- 被点击时不会影响原有编辑器的焦点
- 不接受按键事件和输入文本事件

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

在 AWTK 给的演示示例 awtk-examples 目录下的 demo 默认都启动了软键盘功能。例如，在 UI 界面的 XML 文件中添加 edit 编辑器控件，并设置好 input_type 输入类型，就可以弹出软键盘了，具体示例请看 7.5 章节。如果不想启动键盘请添加宏 WITH_NULL_IM。

7.3 键盘类型

AWTK 为了开发方便，提供了多种类型的键盘，详见下表。其中”输入类型”对应 edit 编辑控件的 input_type 属性，代码详见 7.5 章节。

输入类型	键盘描述文件	说明
phone	kb_phone	电话号码
int	kb_int	整数
float	kb_float	浮点数
uint	kb_uint	非负整数
ufloat	kb_ufloat	非负浮点数
hex	kb_hex	16 进制
email	kb_ascii	邮件地址
password	kb_ascii	密码
custom	自定义的键盘的 XML 文件	使用自定义的键盘
custom_password	自定义的密码软键盘的 XML 文件	使用自定义的密码软键盘
ipv4	kb_uint	IP V4 地址 (如：192.168.1.1)
date	kb_uint	日期 (如：2020/02/20)
time	kb_uint	时间 (时分，如：12:00)
time_full	kb_uint	时间 (时分秒，如：12:00:00)
不输入	kb_default	默认

7.4 键盘映射

AWTK 提供了一套自己的键盘映射关系，详见下表，因为内容过多该表没有列出全部，完整的映射关系请看：在 AWTK_API 手册根据关键字 key_code_t 查看。

例如，在键盘上按下 F2 键对应在 AWTK 的是 TK_KEY_F2，相关示例请看 3.5.4 章节的第一小节。

名称	说明
TK_KEY_RETURN	回车键
TK_KEY_BACKSPACE	删除键
TK_KEY_TAB	Tab 键
TK_KEY_SPACE	空格键
TK_KEY_0	数字键 0
TK_KEY_1	数字键 1
TK_KEY_2	数字键 2
TK_KEY_a	字母键 a
TK_KEY_b	字母键 b
TK_KEY_F1	F1 键
TK_KEY_F2	F2 键
…	…

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

7.5 示例

因为程序不一定需要键盘，如果程序需要使用键盘需要做下面两件事：

- 拷贝 UI 文件：根据需要将 awtk/demos/assets/default/raw/ui 目录下文件名以 kb_ 打头 UI 文件拷贝到应用程序 assets/default/raw/ui 目录下。
- 拷贝图片：因为有些键盘还需要使用到图片（使用到的图片可以打开 kb_ 开头的 UI 文件看看），所以还需要将 awtk/demos/assets/default/raw/images 下对应的图片拷贝到应用程序 assets/default/raw/images 目录下。

下面以 awtk/bin/demoui.exe 下的 KeyBoard 页面为例。当点击 edit 编辑框控件的时候，AWTK 会根据 edit 设置的 input_type 键盘类型，弹出不同的键盘，以 kb_ 开头的键盘 UI 文件还可以根据实际的屏幕大小适当的修改它的高度以及样式等。如果 edit 输入类型不填，将默认使用 kb_default 键盘，效果如下：

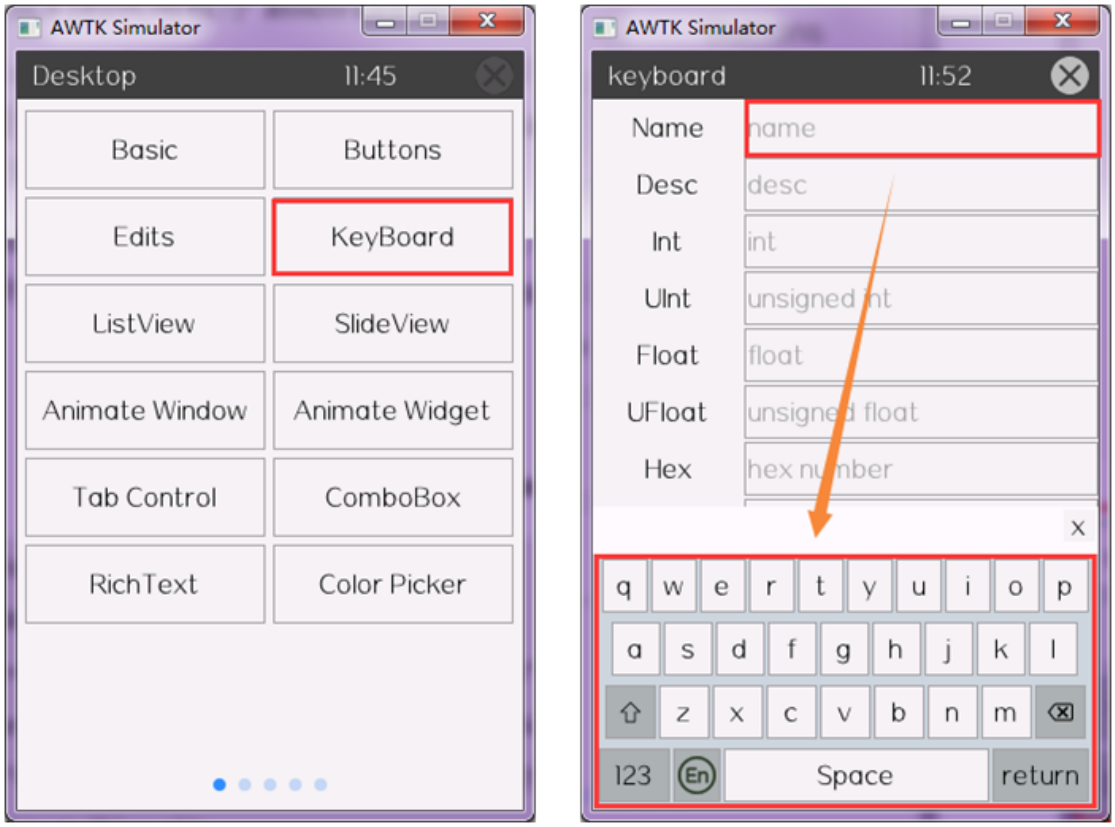


图 7.2 键盘

UI 文件具体代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/keyboard.xml -->
<window anim_hint="htranslate">
  <list_view x="0"  y="0" w="100%" h="-50" item_height="36" auto_hide_scroll_bar=
↪"true">
    <scroll_view name="view" x="0"  y="0" w="-12" h="100%">
     <list_item style="empty" children_layout="default(r=1,c=0)">
      <label w="30%" text="Name"/>
      <edit w="70%" text="" tips="name"/>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    </list_item>
    <list_item style="empty" children_layout="default(r=1,c=0)">
     <label w="30%" text="Desc"/>
     <edit w="70%" text="" tips="desc"/>
    </list_item>
    <list_item style="empty" children_layout="default(r=1,c=0)">
     <label w="30%" text="Int"/>
     <edit w="70%" text="" tips="int" input_type="int"/>
    </list_item>
    <list_item style="empty" children_layout="default(r=1,c=0)">
     <label w="30%" text="UInt"/>
     <edit w="70%" text="" tips="unsigned int" input_type="uint"/>
    </list_item>
    <list_item style="empty" children_layout="default(r=1,c=0)">
     <label w="30%" text="Float"/>
     <edit w="70%" text="" tips="float" input_type="float"/>
    </list_item>
    ...
</window>
```

7.6 自定义键盘

有时 AWTK 自带的键盘，可能不能满足应用程序的需要。例如，需要将软键盘嵌入到窗口内部（比如计算器和密码输入等），这时可以使用自定义软键盘，这里以 AWTK 中的 demoui 为例，详见下图：

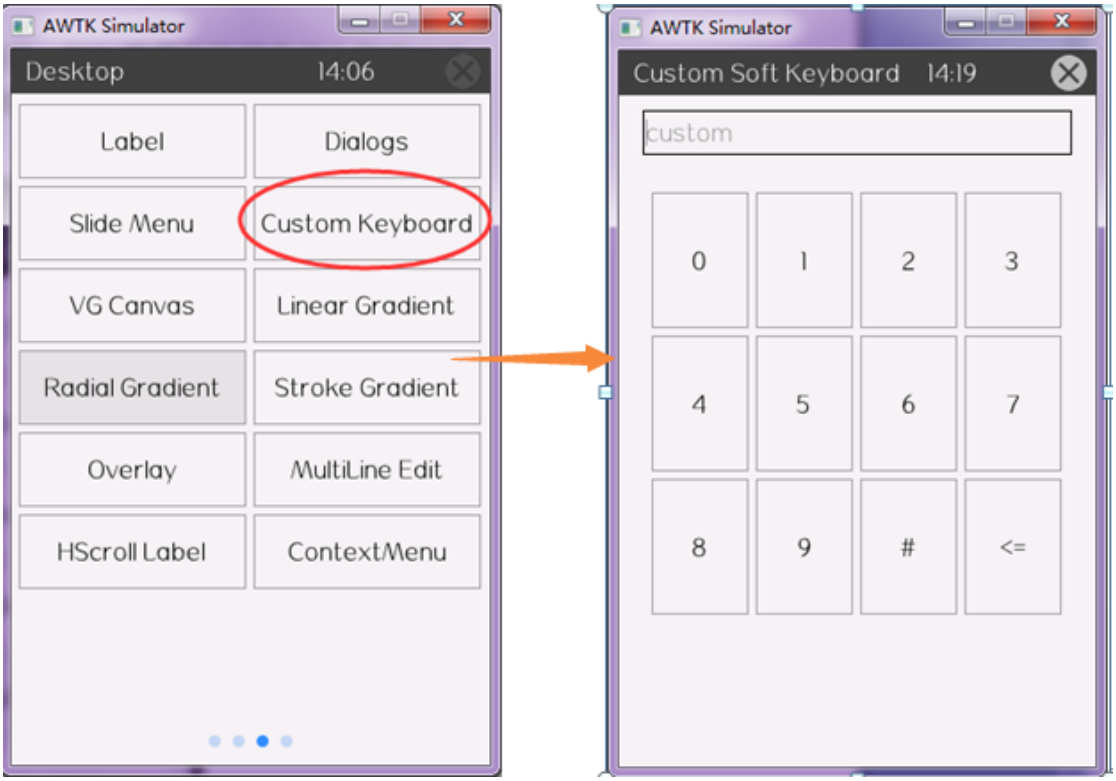


图 7.3 自定义键盘

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

实现自定义键盘的步骤如下：

(1) 设置 edit 的 input_type 为”custom”（它会禁止启用内置的软键盘），代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/soft_keyboard.xml -->
<window text="Custom Soft Keyboard" anim_hint="htranslate" >
  <edit x="c" y="10" w="90%" h="30" focus="true" input_type="custom" text="" tips=
↪"custom"/>
  ...
</window>
```

如果希望初始化时编辑器自动获的焦点，可以设置 focus 为 true。

(2) 软键盘的按钮放入一个 view（任何容器控件均可）中，并将 view 的 is_keyboard 设置为 true，代码如下：

```
<!-- awtk/demos/assets/default/raw/ui/soft_keyboard.xml -->
<window text="Custom Soft Keyboard" anim_hint="htranslate" >
  ...
  <view y="60" x="c" w="90%" h="-60" is_keyboard="true"
    children_layout="default(r=4,c=4,m=5,s=5)" >
    <button name="key" text="0" />
    <button name="key" text="1" />
    <button name="key" text="2" />
    <button name="key" text="3" />
    <button name="key" text="4" />
    <button name="key" text="5" />
    <button name="key" text="6" />
    <button name="key" text="7" />
    <button name="key" text="8" />
    <button name="key" text="9" />
    <button name="key" text="#" />
    <button name="backspace" text="<=" />
  </view>
</window>
```

(3) 处理按钮事件

处理正常按键，代码如下：

```
/* awtk/demos/demo_ui_app.c */
static ret_t on_send_key(void* ctx, event_t* e) {
  widget_t* button = WIDGET(e->target);
  char text[2];
  text[0] = (char)button->text.str[0];
  text[1] = '\0';

  input_method_commit_text(input_method(), text);

  return RET_OK;
}
```

处理删除键，代码如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
/* awtk/demos/demo_ui_app.c */

static ret_t on_backspace(void* ctx, event_t* e) {
  input_method_dispatch_key(input_method(), TK_KEY_BACKSPACE);

  return RET_OK;
}
```

如果你不希望出现编辑器的光标，可以使用 label 控件代替 edit 控件，输入和删除时直接操作 label 的 text。

7.7 软键盘的 action 按钮

在 Android、iPhone 等手机的软键盘上有一个特殊的按钮，这个按钮的功能和显示的文本与当前的编辑操作密切相关，在不同的编辑器（edit 控件）中，可能显示”发送”、“下一个”、”回车”等。

AWTK 的软键盘也有这样一个特殊的按钮，称为软键盘的 action 按钮。在 AWTK 内置的软键盘中，该 action 按钮的默认文本为”Return”，例如 kb_default 软键盘。

下面以 demoui 为例，实现 Text 编辑器唤醒 kb_default 软键盘时，action 按钮显示文本为”Next”，如下图所示，实现步骤请看下文。

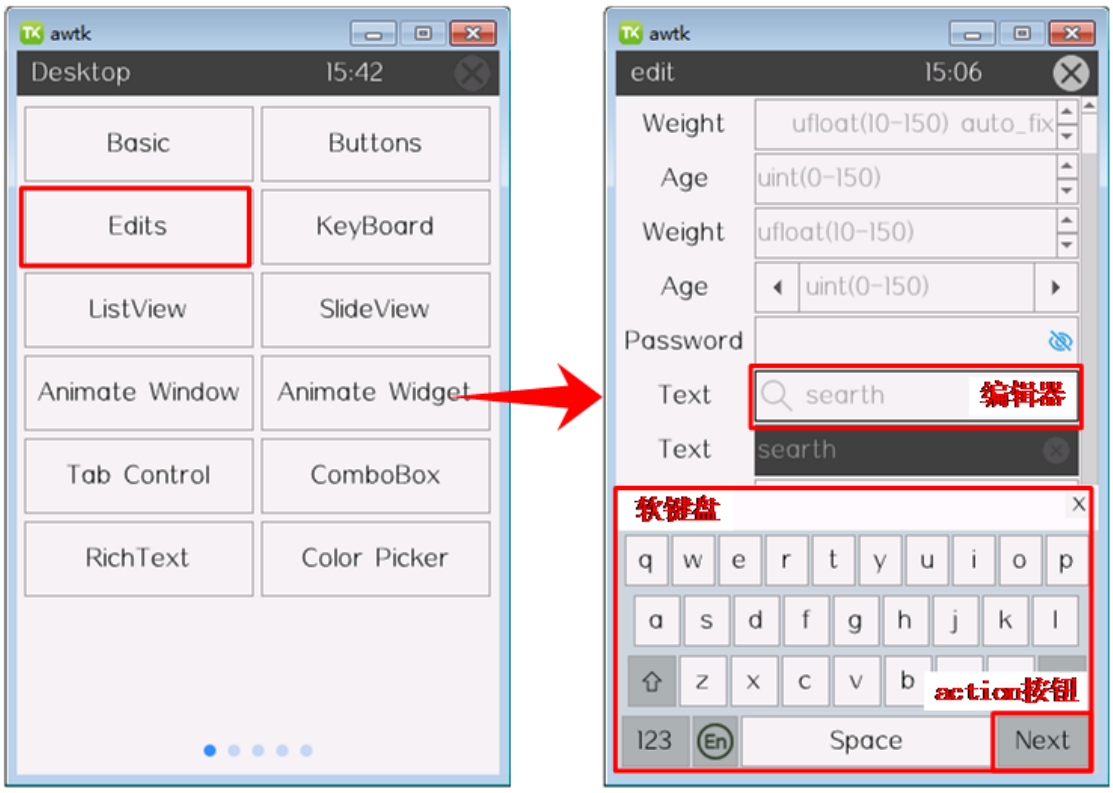


图 7.4 Text 编辑器唤醒 kb_default 软键盘

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

7.7.1 添加 action 按钮

在软键盘的 UI 文件（kb_default.xml）中定义 action 按钮，即将 button 的 name 属性设置为 action，代码如下：

```
<!-- awtk/design/default/ui/kb_default.xml -->
<keyboard theme="keyboard" x="0" y="bottom" w="100%" h="40%">
  <pages x="0" y="bottom" w="100%" h="-28" active="4">
    <view name="upper" x="0" y="0" w="100%" h="100%" children_layout="default(r=4,
↪c=1,s=2,m=2)">
      ...
      <group_box children_layout="default(r=1,c=0,s=2,m=2)">
        ...
        <button name="action" style="highlight" w="20%" text="Return"/>
      </group_box>
    </view>
    ...
</pages>
  ...
</keyboard>
```

7.7.2 修改 action 按钮显示文本

将 Text 编辑器（edit 控件）的 action_text 属性设置为 Next，当该编辑器唤醒软键盘时会根据 action_text 属性设置软键盘中 action 按钮的文本，代码如下：

```
<!-- awtk/design/default/ui/edit.xml -->
<window anim_hint="htranslate" >
  <list_view x="0"  y="0" w="100%" h="-50" item_height="36" auto_hide_scroll_bar=
↪"true">
    <scroll_view name="view" x="0"  y="0" w="-12" h="100%">
     ...
      <list_item style="empty" children_layout="default(r=1,c=0,ym=1)">
      <label w="30%" text="Text"/>
      <edit name="edit" w="70%" left_margin="34" tips="searth" min="0" max="150"
            step="0.1" action_text="Next">
        <image draw_type="icon" image="find" x="0" y="0" w="30" h="100%" />
      </edit>
     </list_item>
    ...
    </scroll_view>
    <scroll_bar_d name="bar" x="right" y="0" w="12" h="100%" value="0"/>
  </list_view>
  <button name="close" x="center" y="bottom:10" w="25%" h="30" text="Close"/>
</window>
```

需要注意的是：以上代码中设置了 kb_default 软键盘 action 按钮的 text 为 Return，因此该软键盘的 action 按钮默认显示为”Return”。

如果没有设置编辑框的 action_text 属性，那么被唤醒的软键盘的 action 按钮的显示其

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

text 属性的值，此处为”Return”；如果设置了编辑框的 action_text 属性，那么被唤醒的软键盘的 action 按钮优先显示编辑框 action_text 属性的值。

7.7.3 实现 action 按钮功能

实现 action 按钮的自定义功能，可以注册对应编辑器 **EVT_IM_ACTION** 事件，例如，在指定编辑器唤醒的软键盘中点击 action 按钮，打印该编辑器设置的 action_text，代码如下：

```
/* EVT_IM_ACTION事件的回调函数 */
static ret_t on_action_event(void* ctx, event_t* evt) {
  widget_t* target = WIDGET(evt->target);
  edit_t* edit = EDIT(target);
  log_debug("edit action_text: %s \n", edit->action_text);

  return RET_OK;
}

/* 注册编辑器EVT_IM_ACTION事件 */
ret_t application_init(void) {
  widget_t* win = window_open("home_page");
  widget_t* edit = widget_lookup(win, "edit", TRUE);
  widget_on(edit, EVT_IM_ACTION, on_action_event, NULL);

  return RET_OK;
}
```

注：完整示例请参考 edit.c[23]。

7.8 加入中文输入法

在有些示例项目中，没有加入输入法，主要是开发板的 flash 不够。如果 flash 够大（不小于 4M 时），可以按下面步骤加入中文输入法，步骤如下：

（1）添加 AWTK 目录下的相关源代码

- 加入 3rd/gpinyin/src 中的代码
- 加入 src/input_engines/input_engine_pinyin.cpp
- 去掉 src/input_engines/input_engine_null.cpp
- include 路径加入 3rd/gpinyin/include
- 把 3rd/gpinyin/data 目录下的 gpinyin.dat 文件（拼音输入法字典）拷贝到项目的资源目录中，例如：res/assets/default/raw/data。
- 把 default_full.ttf 拷贝到 default.tff，重新生成资源并编译（defualt.ttf 没有汉字，default_full.ttf 里才有，否则输入的字符无法显示）。

（2）拼音输入法需要：设置 INPUT_ENGINE=’pinyin’ 以及定义宏 WITH_IME_PINYIN，具体可修改 awtk/awtk_config.py 文件，代码如下：

[23] https://github.com/zlgopen/awtk-c-demos/blob/master/demos/edit.c

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
# awtk/awtk_config.py
...
#INPUT_ENGINE='null'
#INPUT_ENGINE='spinyin'
#INPUT_ENGINE='t9'
#INPUT_ENGINE='t9ext'
INPUT_ENGINE='pinyin'
...
if INPUT_ENGINE == 't9':
    COMMON_CCFLAGS = COMMON_CCFLAGS + ' -DWITH_IME_T9 '
elif INPUT_ENGINE == 't9ext' :
    COMMON_CCFLAGS = COMMON_CCFLAGS + ' -DWITH_IME_T9EXT'
elif INPUT_ENGINE == 'pinyin' :
    COMMON_CCFLAGS = COMMON_CCFLAGS + ' -DWITH_IME_PINYIN '
elif INPUT_ENGINE == 'spinyin' :
    COMMON_CCFLAGS = COMMON_CCFLAGS + ' -DWITH_IME_SPINYIN '
elif INPUT_ENGINE == 'null' :
    COMMON_CCFLAGS = COMMON_CCFLAGS + ' -DWITH_IME_NULL '
  ...
```

注: 关于输入法的其他说明，请看：awtk/src/input_engines/README.md 文档。

（3）在 UI 界面 XML 中添加 edit 编辑器控件，不要设置 input_type 属性，点击下图中右侧 edit 编辑器控件可以弹出默认中文输入法键盘。

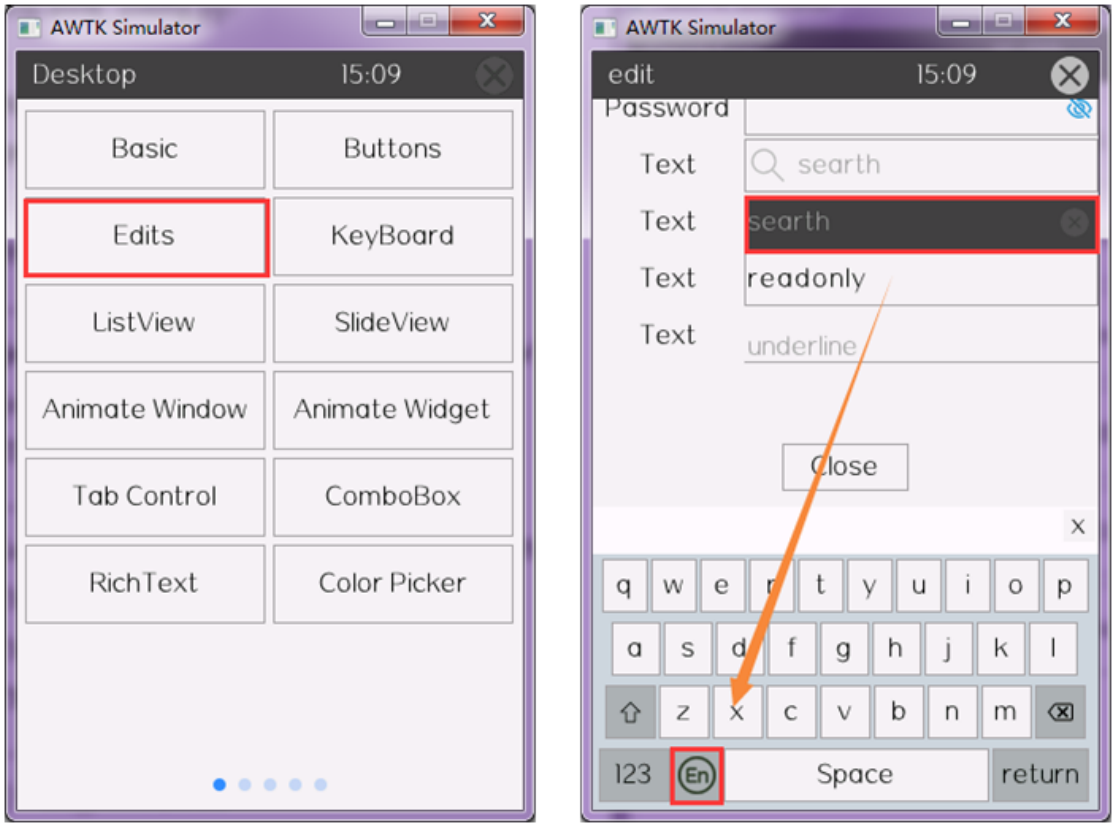


图 7.5　弹出中文输入法

具体代码如下：

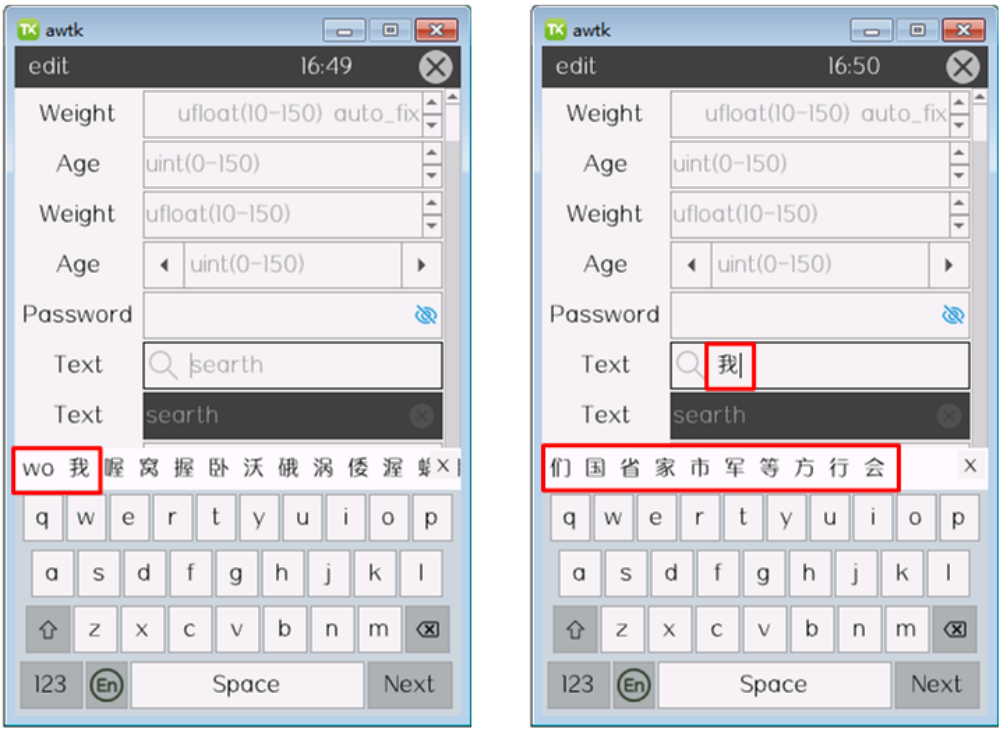
©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
<!-- awtk/demos/assets/default/raw/ui/edit.xml -->
<window anim_hint="htranslate" >
  <list_view x="0"  y="0" w="100%" h="-50" item_height="36" auto_hide_scroll_bar=
↪"true">
    <scroll_view name="view" x="0"  y="0" w="-12" h="100%">
        ...
     <list_item style="empty" children_layout="default(r=1,c=0,ym=1)">
      <label w="30%" text="Text"/>
      <edit w="70%" left_margin="34" tips="searth" min="0" max="150" step="0.1">
        <image draw_type="icon" image="find" x="0" y="0" w="30" h="100%" />
      </edit>
     </list_item>
     ...
    </scroll_view>
    <scroll_bar_d name="bar" x="right" y="0" w="12" h="100%" value="0"/>
  </list_view>
  <button name="close" x="center" y="bottom:10" w="25%" h="30" text="Close"/>
</window>
```

7.9 更新拼音输入法字典和联想字库

拼音输入法字典保存了输入的拼音和出现的汉字之间的对应关系，例如输入拼音”wo”，会出现汉字”我”，如下图左侧所示。

联想字库是指在输入了某个汉字或词组后，输入法根据该汉字或词组提供其常用的组词，例如输入汉字”我”后，根据常用组词”我们”、“我国”、“我省”等，输入法会提供”们”、“国”、”省”等汉字，如下图右侧所示。



输入拼音“wo”的结果　　输入汉字“我”的结果

图 7.6　拼音输入法字典（左侧）和联想字库（右侧）

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

在某些情况下，开发者需要自己更新拼音输入法字典和联想字库。比如：使用更好的输入法字典、去掉一些不需要的汉字或者使用更完善的联想字库，具体方法详见下文。

7.9.1 更新拼音输入法字典

更新拼音输入法字典的步骤如下：

1. 修改配置文件

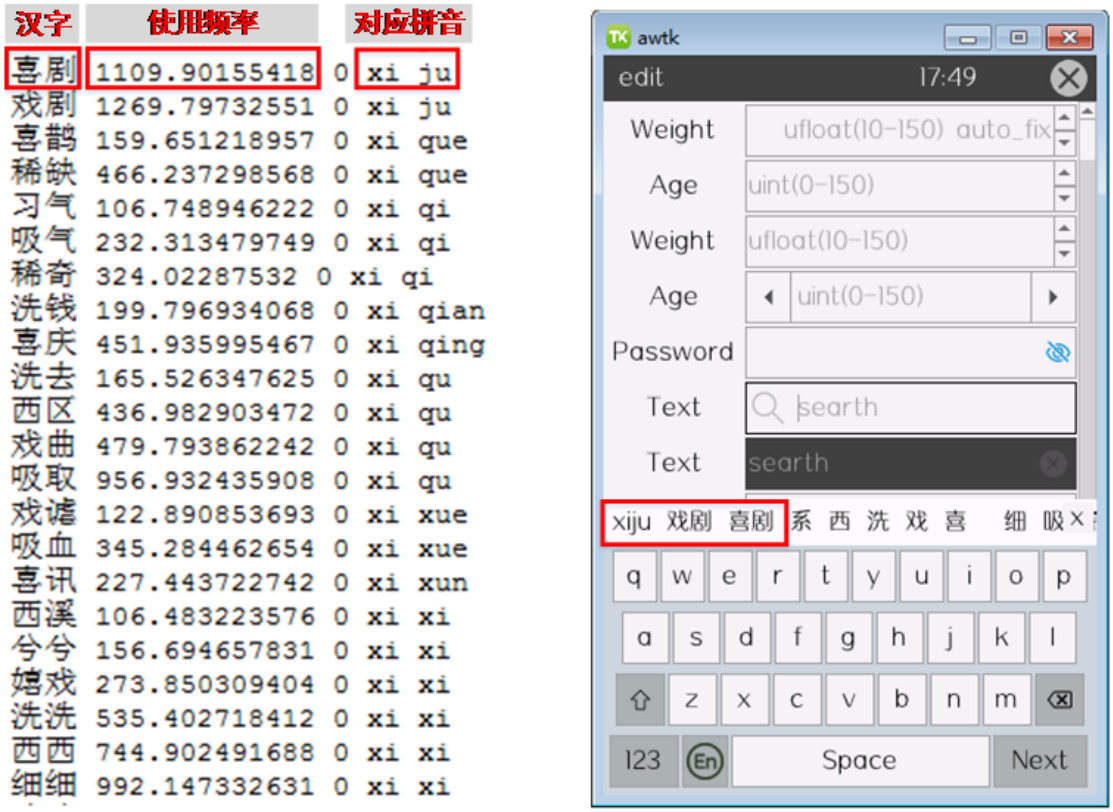
（1）配置文件 awtk/3rd/gpinyin/data/valid_utf16.txt：

该文件中存放了有效的 utf16 编码的汉字。汉字要正常显示，则必须要存在于该文件中，当需要去掉一些不必要的汉字时，可以删除该文件中对应的汉字，但该文件通常不做修改。

（2）配置文件 awtk/3rd/gpinyin/data/rawdict_utf16_65105_freq.txt：

该文件中存放了拼音对应的汉字以及该汉字的使用频率，使用频率越高，输入拼音时对应汉字的排名就越前，比如配置文件中”戏剧”的使用频率比”喜剧”高，那么在输入拼音”xiju”时，”戏剧”就排在”喜剧”之前，如下图所示。

如果想要输入拼音”xiju”时，”喜剧”排在”戏剧”之前，只需让”喜剧”的使用频率高于”戏剧”即可，例如此处将”喜剧”的使用频率改为 1300.00000000 即可。



rawdict_utf16_65105_freq.txt文件　　输入“xiju”的结果

图 7.7　rawdict_utf16_65105_freq.txt 文件

2. 重新生成字典数据

修改完配置文件后，需要重新生成字典数据，在 awtk 目录下打开终端，运行 awtk/bin 目录下的 gpinyingen 程序，执行以下命令：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
./bin/gpinyingen
```

该程序通过上面两个配置文件生成 awtk/3rd/gpinyin/data/gpinyin.dat，该文件就是更新后的拼音输入法字典。

3. 在项目中使用 gpinyin.dat

将生成的 awtk/3rd/gpinyin/data/gpinyin.dat 文件拷贝到项目资源目录的 data 目录下，例如：res/assets/default/raw/data。

注: 若项目是用 Designer 创建的，那么可以将 gpinyin.dat 文件拷贝到项目的 design/default/data 目录中，再使用 Designer 打包项目资源。

7.9.2 更新联想字库

更新拼音输入法的联想字库有两种方式，一种是抓取网页数据生成 words.json 文件，另一种是直接使用现有数据生成 words.json 文件，然后通过 words.json 生成 words.bin 文件，最终通过 words.bin 文件更新联想字库。

由于更新联想字库需要执行 js 脚本文件，所以开发者需要先安装 nodejs，安装过程可自行上网搜索。更新联想字库的步骤如下：

（1）生成 words.json 文件

方式一：通过抓取网页数据生成 words.json，在 awtk/tools/word_gen 目录中打开终端，执行脚本 gen_words_json.js，抓取网页，生成 words.json 文件，命令如下：

```
node gen_words_json.js
```

注: 可修改脚本 gen_words_json.js 中的 maxURLS 改变最大网页数量。

方式二：通过现有数据生成 words.json，在 awtk/tools/word_gen 目录中打开终端，执行脚本 to_json.js，使用现有的数据，即 chinese_with_freq.txt，生成 words.json 文件，命令如下：

```
node to_json.js
```

注: chinese_with_freq.txt 是从 GitHub[24] 下载的，开发者可自行下载需要的联想字库配置文件，若想手动修改该文件，修改方式可参考 7.9.1 章节更新拼音输入法字典中修改配置文件。

（2）通过 words.json 生成二进制的 words.bin 文件，在 awtk/tools/word_gen 目录中打开终端，执行脚本 to_words_bin.js，命令如下：

```
node to_words_bin.js
```

（3）通过生成的 words_bin 更新 demoui 的联想字库，将 words_bin 拷贝到 demoui 的资源目录，例如 demos/assets/default/raw/data，然后重命名为 suggest_words_zh_cn.dat。

此处以 Windows 平台为例，在 awtk 根目录下打开终端，执行以下命令：

```
copy tools/word_gen/words.bin demos/assets/default/raw/data/suggest_words_zh_cn.dat
```

（4）如果开发者在 awtk/awtk_config.py 文件中没有定义宏 **WITH_FS_RES**，即不支持文件系统，那么还需要重新打包资源，在 awtk 根目录打开终端，执行以下命令：

[24] https://github.com/ling0322/webdict

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
python scripts/update_res.py all
```

注: AWTK 默认定义宏 WITH_FS_RES，即支持文件系统。

需要注意的是，基于 nodejs 的中文分词模块（segment）有可能出现 OOM 异常（内存溢出），这是 nodejs 安装目录下 node_modules/segment/lib/module/DictTokenizer.js 脚本文件中 getChunks 函数导致的。

如果以上遇到问题，可以通过限制 chunks.length 的大小解决，例如限制 chunks.length 为 5000，代码如下：

```
var getChunks = function (wordpos, pos, text) {
  ...
  for (var i = 0; i < words.length; i++) {
    var word = words[i];
    var nextcur = word.c + word.w.length;
    if (!wordpos[nextcur]) {
      ret.push([word]);
    } else  {
      var chunks = getChunks(wordpos, nextcur);
      for (var j = 0; j < chunks.length && j < 5000; j++) {
        ret.push([word].concat(chunks[j]));
      }
    }
  }
  ...
  return ret;
};
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

8. 多国语言互译

本章导读：

一款应用可能会有不同国家的人使用，为了让应用走向国际化的平台，一个框架可以提供多国语言互译就显得非常重要了。同一个应用要想在不同的国家显示该地的语言，只需要一个配置文件就可以完成，*AWTK* 就提供了这样的功能。

8.1 简介

支持多国语言的存储（Unicode）和显示（字体）是 GUI 必需要做的，但字符串的翻译完全可以用 gettext 等第三方库函数来实现。AWTK 之所以选择自己实现，主要出于以下几点考虑：

- 减少不必要的第三方库的依赖。运行时需要的代码也就几十行，自己实现更简单代码更少
- 方便支持实时切换当前语言。自己实现字符串的翻译，不要应用程序做额外的工作，即可实现实时切换当前语言

8.2 语言文件介绍

采用 XML 文件（UTF-8）保存字符串的各个语言的对应关系，方便程序员和翻译人员进行编辑，在 AWTK 中要实现多国语言互译需要编写 strings.xml 文件，并将该文件放到 assets/default/raw/strings 目录下，代码如下：

```
<!-- awtk-examples/Chart-Demo/res_800_480/assets/default/raw/strings/strings.xml --
↪>
...
<string name="main_title">
    <language name="en_US">Chart Demo</language>
    <language name="zh_CN">AWTK功能演示</language>
</string>
...
```

8.3 文本翻译

要实现多国语言互译，其步骤如下：

1. 在应用程序中设置要翻译的文本。
2. 编写 strings.xml 文件。
3. C 代码中调用 locale_info_change 函数实时切换语言。
4. 调用脚本重新生成资源文件和重新编译应用程序。

具体的说明，请看以下章节。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

8.3.1 设置需要翻译的文本

在 AWTK 中，有两种方式可以设置需要翻译的文本：XML 方式和 C 代码方式，用户可以任选其中的一种，推荐使用 XML 方式。

1. XML 方式

采用 XML 方式时，在编写 UI 界面时使用 tr_text 表示要翻译的文本。

下面以 awtk-examples/Chart-Demo/bin/demo.exe 中的主页面为例。例如，点击下图中右上角的”中”按钮，实现中英文切换。

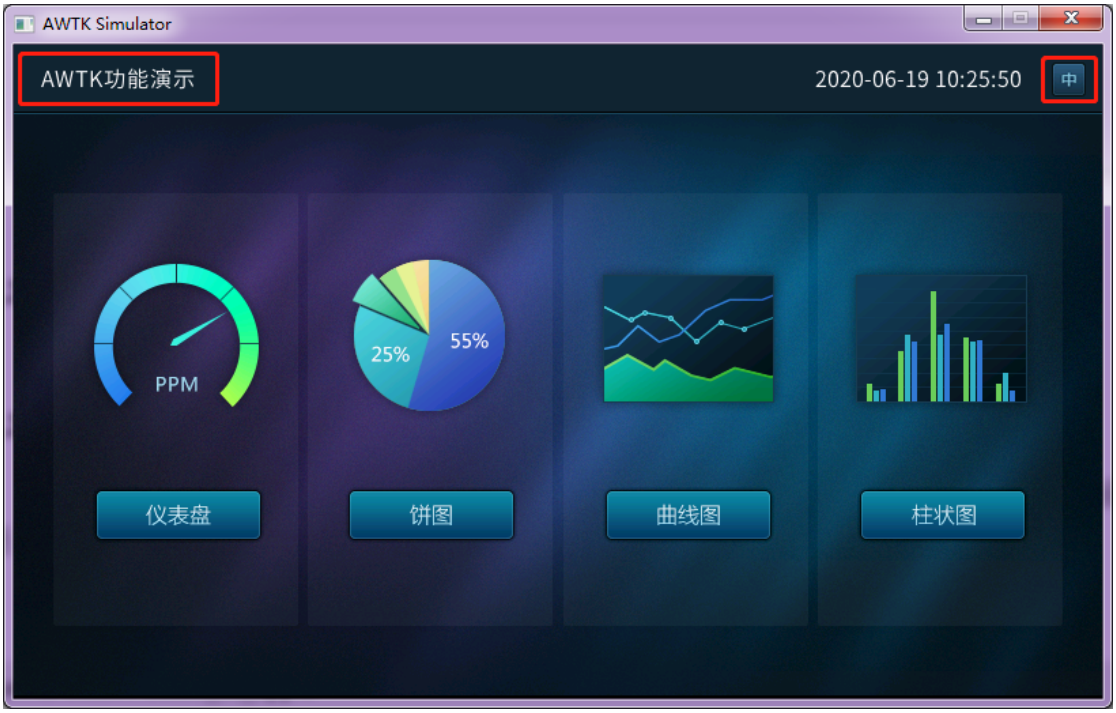


图 8.1 Chart-Demo

在以下代码中 tr_text=“main_title”表示要翻译的文本：

```
<!-- awtk-examples/Chart-Demo/res_800_480/assets/default/raw/ui/main.xml -->
<window anim_hint="htranslate" tr_text="main_title" style="main" x="0%" y="50" w=
↪"800" h="430" >
    <image name="meter_image" x="30" y="57" w="180" h="318" clickable="true" />
    <image name="pie_image" x="218" y="57" w="180" h="318" clickable="true" />
    <image name="graph_image" x="406" y="57" w="180" h="318" clickable="true" />
    <image name="histogram_image" x="594" y="57" w="180" h="318" clickable="true" /
↪>
    <button name="meter" x="58" y="275" w="126" h="41" text="" style="main_meter"/>
    <button name="pie" x="244" y="275" w="126" h="41" text="" style="main_pie"/>
    <button name="window_line_series" x="433" y="275" w="126" h="41" text="" style=
↪"main_graph"/>
    <button name="window_bar_series" x="620" y="275" w="126" h="41" text="" style=
↪"main_histogram"/>
</window>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

2. C 代码方式

采用 C 代码方式时，可以在应用程序的 C 代码中调用 widget_set_tr_text 函数设置要翻译的文本。

下面以 awt/bin/demotr.exe 为例，在以下代码中”ok”字符串表示要翻译的文本。

```
/* awtk/demos/demo_tr_app.c */
ret_t application_init() {
  ...
  ok = button_create(win, 10, 5, 80, 30);
  widget_set_tr_text(ok, "ok");

  cancel = button_create(win, 100, 5, 80, 30);
  widget_set_tr_text(cancel, "cancel");
  ...
  return RET_OK;
}
```

8.3.2 编写 strings.xml 文件

要实现点击 Chart-Demo 界面右上角的”中”按钮，将中文“AWTK 功能演示”翻译为“Chart Demo”，需要编写对应的 strings.xml 文件，代码详见 8.3.1 章节的第一小节。

8.3.3 实时切换语言

在应用程序中设置好了要翻译的文本后，需要在 C 代码中使用 locale_info_change 函数设置当前的国家和语言来实现实时切换语言，其中 change_locale(“zh_CN”) 函数中的传参是重新生成资源后的多国语言文件名，具体详见下文。示例代码如下：

```
/* awtk-examples/Chart-Demo/src/window_main.c */
static ret_t change_locale(const char* str) {
  char country[3];
  char language[3];
  strncpy(language, str, 2);
  strncpy(country, str + 3, 2);
  locale_info_change(locale_info(), language, country);
  return RET_OK;
}

static ret_t on_language(void* ctx, event_t* e) {
  ...
  if (strstr(style, "zh") != NULL) {
    widget_use_style(lang_btn, "language_en");
    change_locale("en_US");
    change_func(lang_btn, TRUE);
  } else {
    widget_use_style(lang_btn, "language_zh");
    change_locale("zh_CN");
    change_func(lang_btn, FALSE);
  }
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  return RET_OK;
}

ret_t application_init(void) {
  ...
  widget_t* sys_bar = window_open("system_bar");
  if (sys_bar) {
    change_locale("zh_CN");
    widget_t* lang_btn = widget_lookup(sys_bar, "language_btn", TRUE);
    widget_on(lang_btn, EVT_CLICK, on_language, sys_bar);
  }
  ...
}
```

8.3.4 重新生成资源和编译应用程序

完成了上面步骤后，需要执行脚本重新生成资源，Windows 平台执行 assets_gen.bat 脚本（Ubuntu 平台执行 assets_gen.sh），此处以 Windows 平台为例，在 Chart-Demo 目录下打开终端，执行以下命令：

```
assets_gen.bat
```

资源生成成功后，将会根据 strings.xml 文件生成对应的语言文件：en_US.bin 和 zh_CN.bin，如下图所示：



图 8.2 多国语言文件

重新生成资源后，就可以调用 scons 命令，编译应用程序了，在 Chart-Demo 目录下打开终端，执行以下命令：

```
scons
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

8.3.5 重新设置缺省字体

此步骤非实现多国语言互译的必要步骤，只有出现下文提到的场景，才需要重新设置 AWTK 的缺省字体（AWTK 中默认的缺省字体是 default.ttf）。

在做切换多国语言的时候，在应用程序的界面上可能会出现不能正常显示某国语言的文字。上面提到 Chart-Demo 示例做中英文切换时候，在应用程序界面上能正常显示中文和英文。是因为 res_xxx/assets/default/raw/fonts 目录下的 default.ttf 字库中包含了中文和英文。如果 default.ttf 字库只包含中文，那么在切换为英文的时候，在 Chart-Demo 示例的界面上将不能正常显示英文，如果需要正常显示英文，其步骤如下：

步骤一：在 res_xxx/assets/default/raw/fonts 目录添加包含英文的字库文件（如：english.ttf）。

步骤二：重新生成资源文件。

步骤三：在切换英文时，在 C 代码中调用 system_info_set_default_font() 函数设置缺省字体为”english”，代码如下：

```
/* Chart-Demo/src/window_main.c */
static ret_t on_language(void* ctx, event_t* e) {
  ...
  if (strstr(style, "zh") != NULL) {
    system_info_set_default_font(system_info(), "english");
    widget_use_style(lang_btn, "language_en");
    change_locale("en_US");
    change_func(lang_btn, TRUE);
  }
  ...
  return RET_OK;
}
```

注：在多国语言互译中，调用 system_info_set_default_font() 函数设置缺省字体后，不要在控件的样式中设置 font_name 属性。如果设置了 font_name 属性，AWTK 会根据 font_name 设置的值，寻找对应的字库，对于该控件调用 system_info_set_default_font() 函数就不会生效了。

8.4 图片翻译

在一些应用程序中，有些文字是直接绘制在图片上的。所以在切换到不同的语言时，需要加载不同的图片。这时只要在图片名称中包含”$locale$”即可，加载时自动替换成当前的语言。如：有两张图片 language_en.png 和 language_zh.png 分别用在英文和中文环境下，那么可以将在样式文件 default.xml 这样写：

```
<button>
   ...
    <style name="language_btn" >
        <normal     icon="language_$locale$" />
        <pressed    icon="language_$locale$"/>
        <over       icon="language_$locale$" />
        <disable     />
    </style>
</button>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

在英文环境下，加载图片时会按下列顺序查找（中文环境下也类似）：

- flag_en_US
- flag_en
- flag_

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

9. 经典案例

本章导读：

使用 *AWTK* 可以高效开发出漂亮的 *GUI* 应用。

9.1 简介

本章以实际的应用场景为案例，来阐述使用 AWTK 做项目的开发过程及其代码实现过程，帮助开发者积累开发应用的实战经验。经典案例如下：

- 洁净新风系统
- 炫酷图表
- 音乐播放器
- 智能手表

以上案例可以安装 AWStudio[25] 后下载查看。

9.2 洁净新风系统

9.2.1 功能详解

洁净新风系统实现的功能点主要有下面几点：

- 点击”开关”按钮，启动/停止 PM2.5、二氧化碳浓度、送风室内外温度、排风室内外温度等模拟读数，以及模拟报警
- 点击”自动”按钮，启动/停止背景（淡入淡出）切换
- 点击”定时”按钮，弹出定时设置对话框，模拟开/关定时功能（即时钟图标的显示/隐藏）
- 点击”记录”按钮，弹出记录列表（模拟数据）页面
- 点击”设置”按钮，弹出设置页面，设置控制、报警参数
- 点击三角形按钮，可以加/减频率、温度、湿度
- 实时更新系统时间

9.2.2 应用实现

本案例使用目录结构、编译和运行步骤与第 2 章介绍的 hello world 应用类似，这里就不重复介绍了。

本案例使用 XML 设计界面，业务逻辑使用 C 语言实现。主界面下方的五个按钮，通过注册点击事件进入相应的界面。主界面右边的按钮，通过注册点击事件实现参数的递增或递减。主界面中间部分通过创建 image_animation 和 opacity 动画实现炫酷的效果。运行的效果详见下图：

[25] https://awtk.zlg.cn

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.



图 9.1　洁净新风系统

本案例使用到的代码均可以在 CleanAir-Demo 目录找到。

1. 如何打开窗口

(1) 在 AWTK 中，可以使用一个 xml 文件来描述一个窗口的界面结构。比如描述记录列表窗口的 record.xml 代码如下：

```
<!-- assets/default/raw/ui/record.xml -->
<window anim_hint="htranslate">
  <button name="close" x="4" y="4" w="60" h="30" text="返回"/>
  <view x="4" y="36" w="100%" h="40" layout="r:1 c:5">
      <label text="状态" style="header"/>
      <label text="室内温度" style="header"/>
      <label text="室外温度" style="header"/>
      <label text="室内湿度" style="header"/>
      <label text="室外湿度" style="header"/>
  </view>
  <list_view x="4"  y="76" w="-8" h="400" item_height="40">
    <scroll_view name="view" x="0"  y="0" w="100%" h="100%">
      <list_item style="odd_clickable" layout="r1 c0">
        <image draw_type="icon" w="20%" image="bell"/>
        <label w="20%" text="24.4℃"/>
        <label w="20%" text="26.4℃"/>
        <label w="20%" text="20%"/>
        <label w="20%" text="30%"/>
      </list_item>
      ......
    </scroll_view>
    <scroll_bar_m name="bar" x="right" y="0" w="6" h="100%" value="0"/>
  </list_view>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
</window>
```

(2) 然后在代码中使用 window_open() 即可打开该窗口，代码如下：

```
/* src/window_record.c */
widget_t* win = window_open("record");
```

其中，”record”为 xml 的文件名，运行效果如下图所示：



图 9.2　记录界面

(3) 如果需要手动关闭窗口使用 window_close() 即可，代码如下：

```
/* src/window_record.c */
window_close(win);
```

2. 如何打开对话框

(1) 比如创建一个定时按钮窗口，代码如下：

```
<!-- assets/default/raw/ui/timing.xml -->
<dialog anim_hint="center_scale" h="200" w="580" x="c" y="m">
        <dialog_title h="35" text="定时设置" w="100%" x="0" y="0"/>
        <dialog_client h="-35" w="100%" x="0" y="bottom">
                <row h="30" layout="row:1 col:3" w="200" x="30" y="10">
                        <label style="timing" text="时"/>
                        <label style="timing" text="分"/>
                        <label style="timing" text="秒"/>
                </row>
                <row h="120" layout="row:1 col:3" w="200" x="30" y="40">
                        <text_selector options="1-24" text="15" visible_nr="3"/>
                        <text_selector options="1-60" text="10" visible_nr="3"/>
                        <text_selector options="1-60" text="16" visible_nr="3"/>
                </row>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
                <button h="30" name="ok" style="timing_switch_btn" text="开" w="150
↪" x="right:30" y="33"/>
                <button h="30" name="cancle" style="timing_switch_btn"
                text="关" w="150" x="right:30" y="96"/>
        </dialog_client>
</dialog>
```

(2) 然后在代码中使用 window_open() 即可打开该窗口，代码如下：

```
/* src/window_timing.c*/
widget_t* win = window_open("timing");
```

其中，”timing”为 xml 的文件名，运行效果如下图所示：

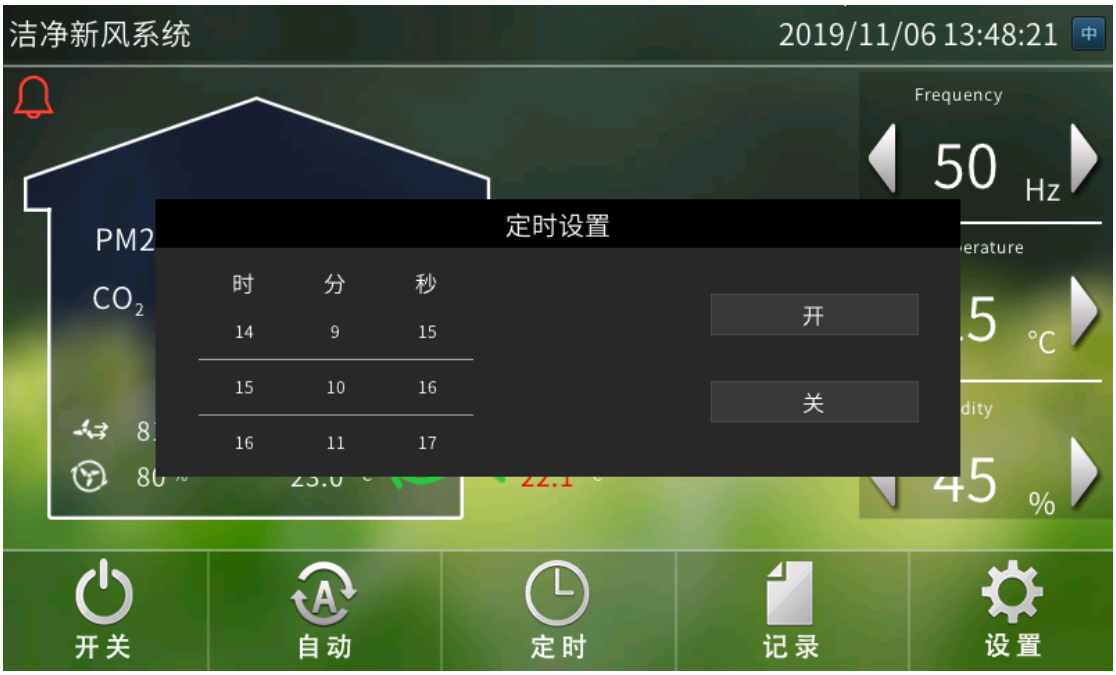


图 9.3　定时界面

(3) 如果需要手动关闭窗口使用 dialog_quit() 即可，代码如下：

```
/* src/window_timing.c */
dialog_quit(dialog, RET_OK);
```

3. 如何查找控件

在 main.xml 文件中设置控件的名字为”timing_status”，然后 C 代码中调用 widget_lookup() 查找控件，代码如下：

```
<!-- src/assets/raw/ui/main.xml -->
<image name="timing_status" image="clock" draw_type="icon"/>
```

```
/* src/window_main.c */
widget_t* timing = widget_lookup(win, "timing_status", TRUE);
```

4. 如何响应界面事件

比如在主界面上设置开关按钮的点击事件，代码如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
<!-- assets/default/raw/ui/main.xml -->
<button name="switch" x="0" w="147" h="100%" style="switch_btn"/>
```

```
/* src/window_main.c */
widget_t* win = widget_get_window(widget);
widget_on(widget, EVT_CLICK, on_switch, win);
```

5. 如何设置控件是否可见

AWTK 中可通过控件的 visible 属性控制该控件是否可见。比如控制定时图标是否可见，代码如下：

```
/* src/window_main.c */
widget_t* timing = widget_lookup(win, "timing_status", TRUE);
if (open_timing_window(&t) == RET_OK) {
  widget_set_visible(timing, TRUE, FALSE);
} else {
  widget_set_visible(timing, FALSE, FALSE);
}
```

6. 如何设置窗口顶部容器

比如在主界面上显示”洁净新风系统”，代码如下：

```
<!-- assets/default/raw/ui/main.xml -->
<app_bar x="0" y="0" w="100%" h="40">
    <label name="dev_name" x="0" y="0" w="200" h="100%" text="洁净新风系统" style=
↪"title_left"/>
    <label name="sys_time" x="200" y="0" w="-200" h="100%" style="title_right"/>
</app_bar>
```

7. 如何显示文本

比如在主界面上显示”PM2.5”对应的值 18，代码如下：

```
<!-- assets/default/raw/ui/main.xml -->
<label name="PM2_5" x="80" w="108" h="100%" text="18" style="reading_pm_co"/>
```

```
/* src/window_main.c */
widget_t* pm2_5 = widget_lookup(win, "PM2_5", TRUE);
...
widget_set_text_utf8(label, tk_itoa(text, sizeof(text), val));
```

8. 如何显示图片

比如在主界面上显示排风图片（fan_2.png），代码如下：

```
<!-- assets/default/raw/ui/main.xml -->
<image name="fan_2" image="fan_2" draw_type="default" y="m"/>
```

9. 如何显示富文本

比如在主界面上显示 μg/m^3^ 分两部分处理：显示 μg/m 和显示上标数字 3，代码如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
<!-- assets/default/raw/ui/main.xml -->
<rich_text x="188" w="-188" h="100%" text="
  <font color=&quota;white&quota; align_v=&quota;top&quota; size=&quota;18&quota;>
↪μg/m</font>
  <font color=&quota;white&quota; align_v=&quota;top&quota; size=&quota;10&quota;>3
↪</font>" />
```

10. 如何使用布局

比如在主界面上以一行两列的方式显示时钟（clock.png）和报警图标（bell.png），代码如下：

```
<!-- assets/default/raw/ui/main.xml -->
<view x="0" y="44" w="88" h="44" layout="r:1 c:2">
  <image name="timing_status" image="clock" draw_type="icon"/>
  <image name="alarm_status"  image="bell"  draw_type="icon"/>
</view>
```

11. 如何实现动画

(1) 控件动画 API

AWTK 中，可以通过该控件绑定一个动画对象来实现动画。比如实现左下角风扇图标（fan_2.png）旋转，代码如下：

```
/* src/window_main.c */
widget_animator_t* widget_animator_get(widget_t* widget, const char* name,
                                       uint32_t duration, bool_t opacity) {
  widget_animator_t* animator = NULL;
  value_t val;
  value_set_pointer(&val, NULL);
  if (widget_get_prop(widget, name, &val) != RET_OK || value_pointer(&val) == NULL)␣
↪{
    if (opacity) {
      animator = widget_animator_opacity_create(widget, duration, 0, EASING_SIN_
↪OUT);
    } else {
      animator = widget_animator_rotation_create(widget, duration, 0, EASING_
↪LINEAR);
    }
    value_set_pointer(&val, animator);
    widget_set_prop(widget, name, &val);
  } else {
    animator = (widget_animator_t*)value_pointer(&val);
  }
  return animator;
}
...
animator = widget_animator_get(fan_2, "animator", 4000, FALSE);
widget_animator_rotation_set_params(animator, 0, 2*3.14159265);
widget_animator_set_repeat(animator, 0);
widget_animator_start(animator);
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

又或者，实现风向箭头的淡入淡出动画 (wind_in.png)，代码如下：

```
/* src/window_main.c */
animator = widget_animator_get(wind_out, "animator", 1000, TRUE);
widget_animator_opacity_set_params(animator, 50, 255);
widget_animator_set_yoyo(animator, 0);
widget_animator_start(animator);
```

(2)image_animation 组件

在 AWTK 中，可以使用 image_animation 组件来实现一组图片顺序播放的动画。比如设置主界面上 fan1a.png 和 fan_1b.png 图片动画，代码如下：

```
<!-- src/assets/raw/ui/main.xml -->
<image_animation name="fan_1" image="fan_1" sequence="ab" auto_play="false"
                 interval="500" delay="100" y="m"/>
```

```
/* src/window_main.c */
image_animation_play(fan_1);
```

12. 如何定时刷新界面数据

比如实时更新主界面右上角的系统时间，代码如下：

```
/* src/window_main.c */
timer_add(on_systime_update, win, 1000);
```

13. 如何使用时分秒控件

比如点击主界面中的”定时”按钮弹出”定时设置”界面显示时分秒控件，代码如下：

```
<!-- assets/default/raw/ui/timing.xml -->
<row h="120" layout="row:1 col:3" w="200" x="30" y="40">
        <text_selector options="1-24" text="15" visible_nr="3"/>
        <text_selector options="1-60" text="10" visible_nr="3"/>
        <text_selector options="1-60" text="16" visible_nr="3"/>
</row>
```

14. 如何显示列表

比如点击主界面中的”记录”按钮弹出的列表视图，代码如下：

```
<!-- assets/default/raw/ui/record.xml -->
<list_view x="4"  y="76" w="-8" h="400" item_height="40">
    <scroll_view name="view" x="0"  y="0" w="100%" h="100%">
      <list_item style="odd_clickable" layout="r1 c0">
        <image draw_type="icon" w="20%" image="bell"/>
        <label w="20%" text="24.4℃"/>
        <label w="20%" text="26.4℃"/>
        <label w="20%" text="20%"/>
        <label w="20%" text="30%"/>
      </list_item>
     ...
    </scroll_view>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    <scroll_bar_m name="bar" x="right" y="0" w="6" h="100%" value="0"/>
  </list_view>
```

15. 如何实现分页

(1) 比如点击主界面中的”设置”按钮弹出分页视图，代码如下：

```
<!-- assets/default/raw/ui/setting.xml -->
<window anim_hint="htranslate" name="setting_page">
  <button h="30" name="close" text="返回" w="60" x="4" y="4"/>
  <pages h="440" w="80%" x="right" y="40">
    <view h="60%" layout="r:4 c:3" w="100%">
      <label style="setting_page" text="室内温度"/>

     ...
  </pages>
  <list_view auto_hide_scroll_bar="true" h="440" item_height="40" w="20%" x="left"␣
↪y="40">
    <scroll_view h="100%" name="view" w="-12" x="0" y="0">
      <tab_button text="控制参数设定" value="true"/>
      <tab_button text="报警设置"/>
    </scroll_view>
    <scroll_bar_d h="100%" name="bar" value="0" w="12" x="right" y="0"/>
  </list_view>
</window>
```

(2) 运行效果如下图所示：

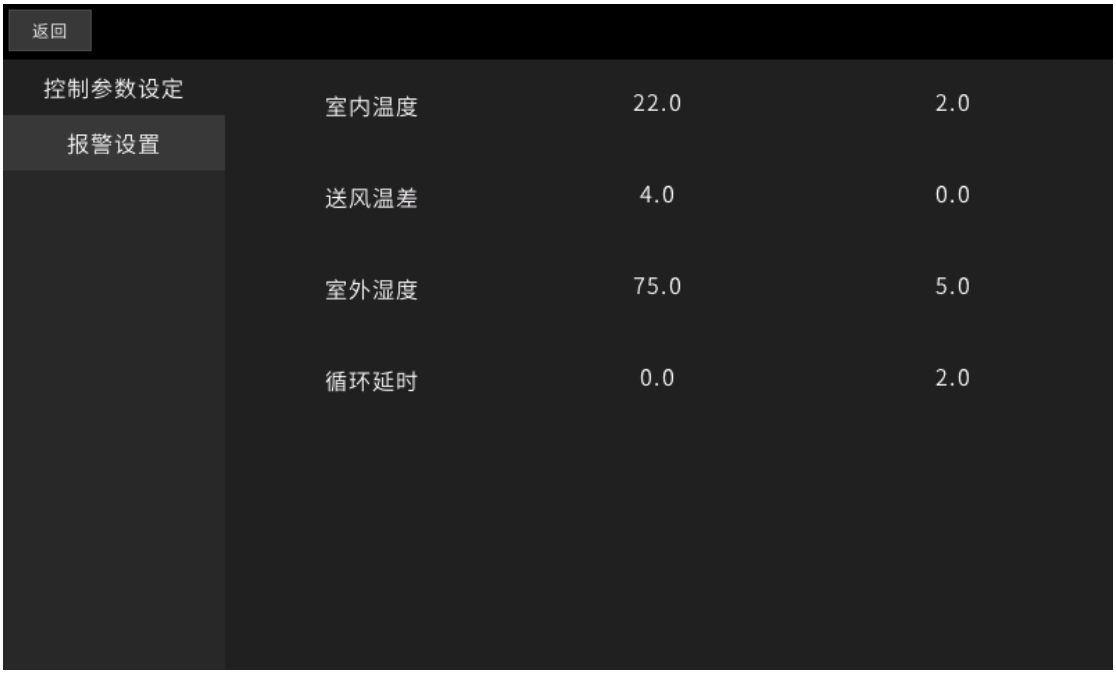


图 9.4　设置页面

16. 如何设置控件样式

比如设置主界面中的”开关”按钮的样式，代码如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
<!-- assets/default/raw/ui/main.xml -->
<button name="switch" x="0" w="147" h="100%" style="switch_btn"/>
```

```
<!-- assets/default/raw/styles/default.xml -->
<button>
    ...
    <style name="switch_btn">
        <normal icon="btn_1"/>
        <pressed icon="btn_1_push"/>
        <over icon="btn_1_hover"/>
      </style>
    ...
</button>
```

```
/* src/window_main.c */
widget_t* btn = widget_lookup(win, "auto", TRUE);
widget_use_style(btn, "auto_btn_2a");
```

17. 如何实现中英文互译

比如点击主界面右上角的”中”按钮实现中英文环境下文本和图片的切换。接下来将介绍如何实现该功能。

(1) 中英环境下文本互译

比如要实现”洁净新风系统”和”CleanAir Demo”文本之间的互译，步骤如下：

首先，在 strings.xml 文件中配置要互译的文本，代码如下：

```
<!-- CleanAir-Demo/res_800_480/assets/default/raw/strings/strings.xml -->
...
<string name="CleanAir Demo">
    <language name="en_US">CleanAir Demo</language>
    <language name="zh_CN">洁净新风系统</language>
</string>
...
```

然后，在 main.xml 中使用 tr_text 设置要翻译的文本，代码如下：

```
<!-- CleanAir-Demo/res_800_480/assets/default/raw/ui/main.xml -->
<window anim_hint="htranslate" style="black">
    <label name="bkgnd" x="0" y="0" w="100%" h="100%" style="bg0"/>
    <app_bar x="0" y="0" w="100%" h="40">
        <label name="dev_name" x="0" y="0" w="200" h="100%"
                  tr_text="CleanAir Demo" style="title_left"/>
        <label name="sys_time" x="right:35" y="0" w="-200" h="100%" style="title_
↪right"/>
                <button name="language_btn" x="right:5" y="0" w="25" h="100%"␣
↪style="language_btn"/>
    </app_bar>
   ...
</window>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

最后，在 window_main.c 调用 locale_info_change 函数实现互译，代码如下：

```
/* CleanAir-Demo/src/window_main.c */
/* 中英文互译 */
static ret_t change_locale(const char *str) {
  char country[3];
  char language[3];

  strncpy(language, str, 2);
  strncpy(country, str + 3, 2);
  locale_info_change(locale_info(), language, country);

  return RET_OK;
}

/* 点击中英文互译按钮 */
static ret_t on_language(void *ctx, event_t *e) {
  (void)ctx;
  (void)e;

  const char *language = locale_info()->language;
  if (tk_str_eq(language, "en")) {
    change_locale("zh_CN");
  } else {
    change_locale("en_US");
  }

  return RET_OK;
}
```

(2) 中英环境下图片切换

比如要实现点击主界面右上角的”中”和”EN”按钮之间的切换，步骤如下：

首先，将两张图片命名为”language_en.png”和”language_zh.png”分别代表中文和英文环境下的图片，”language_ en.png”中”en”代表的是语言环境，不可以随意命名。

然后，在样式文件 default.xml 中通过 language_$locale$ 字符串表示要切换的图片。在中文环境下使用 language_zh.png，在英文环境下使用 language_en.png，代码如下：

```
<!-- CleanAir-Demo/res_800_480/assets/default/raw/styles/default.xml -->
<button>
<style name="language_btn" >
        <normal     icon="language_$locale$" />
        <pressed    icon="language_$locale$"/>
        <over       icon="language_$locale$" />
    </style>
</button>
```

就这样就完成了中英文环境下图片的切换。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

9.3 炫酷图表

9.3.1 功能详解

炫酷图表实现的功能点主要有下面几点：

- 仪表盘
- 饼图
- 曲线图
- 柱状图

9.3.2 应用实现

本案例使用目录结构、编译和运行步骤与第 2 章介绍的 hello world 应用类似，这里就不重复介绍了。

本案例使用 XML 设计界面，业务逻辑使用 C 语言实现。主界面中间的五个按钮，通过注册点击事件进入相应的界面。其中只有仪表盘界面使用了 AWTK 提供的 guage 控件实现，剩下的饼图、曲线图和柱状图是通过调用 AWTK 提供的接口自绘而成，这也体现了 AWTK 功能的强大，没有的控件可以自己实现。运行的效果详见图 10.5。



图 9.5 炫酷图表

本案例使用到的代码均可以在 Chart-Demo 目录找到。

1. 实现主界面逻辑

主界面实现的功能点如下：

- 中英文环境切换
- 界面切换，通过注册点击事件，然后进入相应的界面

下面，介绍如何实现这些功能点。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

(1) 中英文环境切换

本案例实现中英文环境切换与洁净新风系统的类似，这里是通过改变样式的方式实现图片切换（比较复杂，代码如下），而洁净新风系统是通过在样式文件 default.xml 中给图片加个”$locale$”后缀完成（比较简单，推荐使用这种方式）。由于在上面章节已经介绍过了，这里不重复了。

```
/* Chart-Demo/src/window_main.c */
/* 改变样式 */
static ret_t change_style(widget_t* win, const char* name, const char* style, bool_
↪t flag) {
  widget_t* w = widget_lookup(window_manager(), name, TRUE);
  if (w) {
    if (flag) {
      char style_en[20];
      tk_snprintf(style_en, sizeof(style_en), "%s_en", style);
      widget_use_style(w, style_en);
    } else {
      widget_use_style(w, style);
    }
  }

  return RET_OK;
}
```

(2) 界面切换

通过函数 widget_on 给相应的控件注册”EVT_CLICK”点击事件，进入仪表盘、饼图、曲线图和柱状图界面，代码如下：

```
/* Chart-Demo/src/window_main.c */
/* 子控件初始化(主要是设置click回调、初始显示信息) */
static ret_t init_widget(void* ctx, const void* iter) {
  widget_t* widget = WIDGET(iter);
  (void)ctx;

  if (widget->name != NULL) {
    const char* name = widget->name;
    if (tk_str_eq(name, "meter") || tk_str_eq(name, "meter_image")) {
      widget_t* win = widget_get_window(widget);
      widget_on(widget, EVT_CLICK, on_meter, win);
    } else if (tk_str_eq(name, "pie") || tk_str_eq(name, "pie_image")) {
      widget_t* win = widget_get_window(widget);
      widget_on(widget, EVT_CLICK, on_pie, win);
    } else if (tk_str_eq(name, "graph") || tk_str_eq(name, "graph_image")) {
      widget_t* win = widget_get_window(widget);
      widget_on(widget, EVT_CLICK, on_graph, win);
    } else if (tk_str_eq(name, "histogram") || tk_str_eq(name, "histogram_image"))
↪{
      widget_t* win = widget_get_window(widget);
      widget_on(widget, EVT_CLICK, on_histogram, win);
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    }
  }

  return RET_OK;
}
```

2. 实现仪表盘逻辑

仪表盘界面实现的功能点如下：

- 绘制仪表盘
- 启动仪表盘
- 停止仪表盘

下面，介绍如何实现这些功能点。

(1) 绘制仪表盘

通过 AWTK 自带的 guage 控件实现仪表盘的绘制，代码如下：

```
<!-- Chart-Demo/res_800_480/assets/default/raw/ui/window_meter.xml -->
<window anim_hint="htranslate" tr_text="meter_title">
    <view name="guage_function_view" x="0" y="0" w="87%" h="90%" layout="r:1 c:2 s:
25" style="dark">
        <guage  image="guage_bg_1" >
            <gauge_pointer name="left_pointer" x="c" y="m" w="12" h="160"
                        value="-128" image="gauge_pointer_1"
                        animation="value(from=-128, to=128, yoyo_times=0,
↪forever=TRUE, duration=3000,
                        delay=1000)"/>
        </guage>
        <guage image="guage_bg_2" >
            <gauge_pointer name="right_pointer" x="c" y="m" w="12" h="160"
                        value="-128" image="gauge_pointer_2"
            animation="value(from=-128, to=128, yoyo_times=0, forever=TRUE,
↪duration=3000,
                        delay=1000)"/>
        </guage>
    </view>
    ...
</window>
```

(2) 启动仪表盘（界面右边的第一个按钮）

完成调用 widget_start_animator 开始动画，实现启动仪表盘，代码如下：

```
/* Chart-Demo/src/window_meter.c */
/* 点击开始按钮 */
static ret_t on_start(void* ctx, event_t* e) {
  widget_t* win = (widget_t*)ctx;
  set_btn_style(win, e);
  widget_start_animator(NULL, NULL);
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  return RET_OK;
}
```

(3) 停止仪表盘（界面右边的第二个按钮）

调用 widget_stop_animator 停止动画，实现停止仪表盘，代码如下：

```
/* Chart-Demo/src/window_meter.c */
/* 点击停止按钮 */
static ret_t on_stop(void* ctx, event_t* e) {
  widget_t* win = (widget_t*)ctx;
  set_btn_style(win, e);
  widget_stop_animator(NULL, NULL);

  return RET_OK;
}
```

运行效果如下图所示：

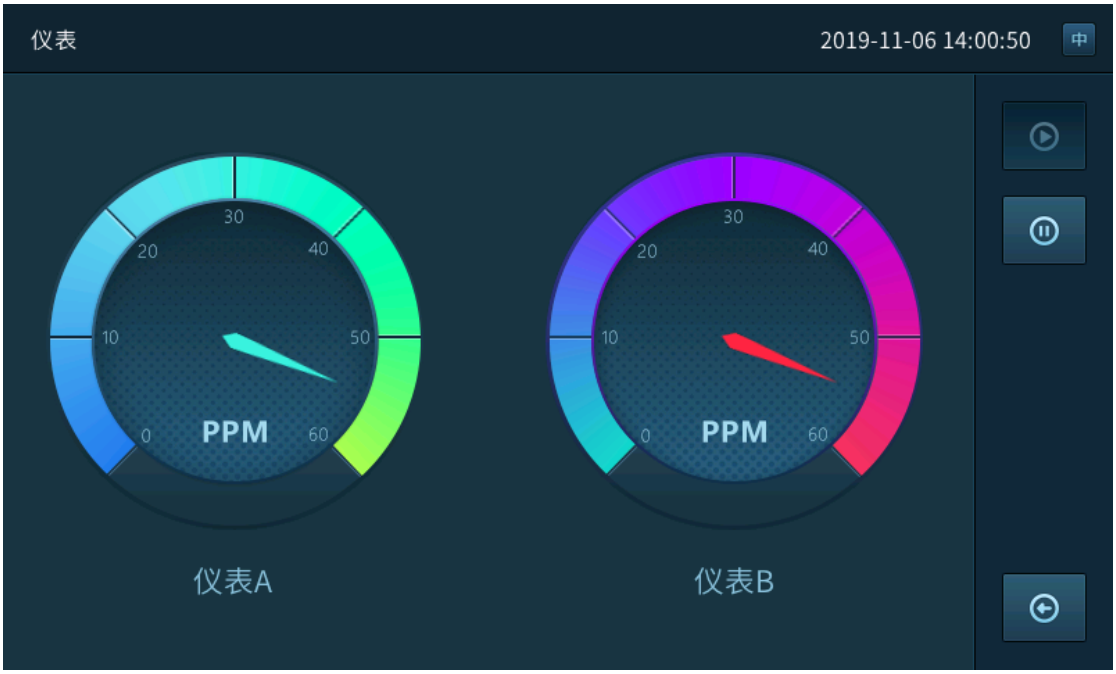


图 9.6　仪表盘

3. 实现饼图逻辑

饼图界面实现的功能点如下：

- 绘制饼图
- 生成新的饼图/环形图
- 饼图和环形图之间的切换

下面，介绍如何实现这些功能点。

(1) 绘制饼图

饼图是一个自定义控件，通过重新定义控件的虚函数表（主要是 create、on_paint_self、on_destroy、set_prop、get_prop 等函数），并使用 widget_factory_register 注册到 AWTK 中，即

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

可在 xml 中直接使用，代码如下：

```
/* Chart-Demo/src/custom_widgets/pie_slice/pie_slice.c */
static ret_t pie_slice_on_paint_self(widget_t* widget, canvas_t* c) {
  ...
  if (vg != NULL && (has_image || color.rgba.a)) {
    xy_t cx = widget->w / 2;
    xy_t cy = widget->h / 2;
    float_t end_angle = 0;
    float_t r = 0;
    bool_t ccw = pie_slice->counter_clock_wise;
    float_t start_angle = TK_D2R(pie_slice->start_angle);
    float_t angle = (M_PI * 2 * pie_slice->value) / pie_slice->max;

    if (ccw) {
      end_angle = start_angle - angle + M_PI * 2;
    } else {
      end_angle = start_angle + angle;
    }

    vgcanvas_save(vg);
    vgcanvas_translate(vg, c->ox, c->oy);
    if (end_angle > start_angle) {
      vgcanvas_set_fill_color(vg, color);
      vgcanvas_begin_path(vg);
      r = tk_min(cx, cy);
      r -= pie_slice->explode_distancefactor;

      if (pie_slice->is_semicircle) {
        if (ccw) {
          start_angle = M_PI * 2 - (start_angle + angle) / 2;
          end_angle = start_angle + angle / 2;
          cy = cy + r * 0.5;
          r += pie_slice->explode_distancefactor * 1.5;
          vgcanvas_arc(vg, cx, cy, r, start_angle, end_angle, !ccw);
          r -= pie_slice->inner_radius;
          vgcanvas_arc(vg, cx, cy, r, end_angle, start_angle, ccw);
        }
      } else {
        vgcanvas_arc(vg, cx, cy, r, start_angle, end_angle, ccw);
        if (r - pie_slice->inner_radius <= 0) {
          vgcanvas_line_to(vg, cx, cy);
        } else {
          r -= pie_slice->inner_radius;
          vgcanvas_arc(vg, cx, cy, r, end_angle, start_angle, !ccw);
        }
      }

      vgcanvas_close_path(vg);
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
      if (has_image) {
        vgcanvas_paint(vg, FALSE, &img);
      } else {
        vgcanvas_fill(vg);
      }
    }
    vgcanvas_restore(vg);
  }
  ...
}
```

(2) 饼图/环形图的值动画（界面右边的第一个按钮）

生成新的饼图/环形图，实现步骤如下：

首先，通过创建值动画将原来的饼图/环形图还原到原点，代码如下：

```
/* Chart-Demo/src/window_pie.c */
/* 创建扇形还原到原点动画 */
static ret_t create_animator_to_zero(widget_t* win) {
  set_btn_enable(win, FALSE);
  widget_t* pie_view = widget_lookup(win, "pie_view", TRUE);
  if (pie_view) {
    WIDGET_FOR_EACH_CHILD_BEGIN_R(pie_view, iter, i)
    value_t v;
    widget_get_prop(iter, PIE_SLICE_PROP_IS_EXPLODED, &v);
    save_pie_exploded[i] = value_bool(&v);

    pie_slice_t* pie_slice = PIE_SLICE(iter);
    if (pie_slice->is_exploded) {
      pie_slice_set_exploded(iter);
    }
    int32_t delay = 80;
    delay = delay * (nr - 1 - i);
    char param[100];
    tk_snprintf(param, sizeof(param),
                "value(to=0, duration=50, delay=%d, easing=sin_out)", delay);
    widget_create_animator(iter, param);
    WIDGET_FOR_EACH_CHILD_END();
  }

  return RET_OK;
}
```

然后，通过定时器判断上面的动画是否已经全部还原到原点，如果是重置饼图/环形图的值数据，代码如下：

```
/* Chart-Demo/src/window_pie.c */
/* 点击创建新饼图或者环形图定时器 */
static ret_t on_new_pie_timer(const timer_info_t* timer) {
  pie_value_t* pie_data = NULL;
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  widget_t* win = WIDGET(timer->ctx);

  uint32_t count = widget_animator_manager_count(widget_animator_manager());
  if (count == 0) {
    value_t v;
    value_t v1;
    ret_t result = widget_get_prop(win, "is_new", &v);
    bool_t flag = (result == RET_NOT_FOUND) ? FALSE : value_bool(&v);
    if (flag) {
      pie_data = old_pie_data;
      widget_set_prop(win, "is_new", value_set_bool(&v1, FALSE));
    } else {
      pie_data = new_pie_data;
      widget_set_prop(win, "is_new", value_set_bool(&v1, TRUE));
    }
    widget_t* pie_view = widget_lookup(win, "pie_view", TRUE);
    if (pie_view) {
      WIDGET_FOR_EACH_CHILD_BEGIN(pie_view, iter, i)
      pie_slice_t* pie_slice = PIE_SLICE(iter);
      int32_t delay = DELAY_TIME;
      int32_t duration = DURATION_TIME;
      delay = delay * i;
      const pie_value_t* new_pie = pie_data + nr - 1 - i;
      pie_slice_set_start_angle(iter, new_pie->start_angle);
      char param[100];
      tk_snprintf(param, sizeof(param),
                  "value(name=%s, to=%d, duration=%d, delay=%d, easing=sin_out)",
                  SAVE_EXPLODED,
                  new_pie->value, duration, delay);
      widget_create_animator(iter, param);
      WIDGET_FOR_EACH_CHILD_END();
    }
    timer_add(on_save_exploded_timer, win, 1000 / 60);

    return RET_REMOVE;
  }

  return RET_REPEAT;
}
```

(3) 饼图/环形图的切换动画（界面右边的第二个按钮）

饼图和环形图之间的切换，实现步骤如下：

首先，通过创建值动画将原来的饼图/环形图还原到原点。

然后，通过定时器判断上面的动画是否已经全部还原到原点，如果是就进行饼图和环形图之间的切换，并调用 pie_slice_set_semicircle 设置是否是环形图，代码如下：

```
/* Chart-Demo/src/window_pie.c */
/* 环形图定时器 */
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
static ret_t on_arch_timer(const timer_info_t* timer) {
  widget_t* win = WIDGET(timer->ctx);

  uint32_t new_inner_radius = win->h / 5;
  uint32_t inner_radius = 0;

  widget_t* pie_view = widget_lookup(win, "pie_view", TRUE);
  uint32_t count = widget_animator_manager_count(widget_animator_manager());
  if (count == 0) {
    value_t v;
    value_t v1;
    ret_t result = widget_get_prop(win, "is_arch", &v);
    bool_t flag = (result == RET_NOT_FOUND) ? FALSE : value_bool(&v);
    if (flag) {
      widget_set_prop(win, "is_arch", value_set_bool(&v1, FALSE));
      inner_radius = 550;
    } else {
      widget_set_prop(win, "is_arch", value_set_bool(&v1, TRUE));
      inner_radius = new_inner_radius;
    }
    widget_t* pie_view = widget_lookup(win, "pie_view", TRUE);
    if (pie_view) {
      WIDGET_FOR_EACH_CHILD_BEGIN(pie_view, iter, i)
      pie_slice_t* pie_slice = PIE_SLICE(iter);
      pie_slice_set_inner_radius(iter, inner_radius);

      if (flag) {
        pie_slice_set_semicircle(iter, FALSE);
        pie_slice_set_counter_clock_wise(iter, FALSE);
      } else {
        pie_slice_set_semicircle(iter, TRUE);
        pie_slice_set_counter_clock_wise(iter, TRUE);
      }

      int32_t delay = DELAY_TIME;
      int32_t duration = DURATION_TIME;
      delay = delay * i;
      const pie_value_t* new_pie = old_pie_data + nr - 1 - i;
      pie_slice_set_start_angle(iter, new_pie->start_angle);
      char param[100];
      tk_snprintf(param, sizeof(param),
                  "value(name=%s, to=%d, duration=%d, delay=%d, easing=sin_out)",␣
↪SAVE_EXPLODED,
                  new_pie->value, duration, delay);
      widget_create_animator(iter, param);
      WIDGET_FOR_EACH_CHILD_END();
    }
    timer_add(on_save_exploded_timer, win, 1000 / 60);
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    return RET_REMOVE;
  }

  return RET_REPEAT;
}
```

运行效果如下图所示：

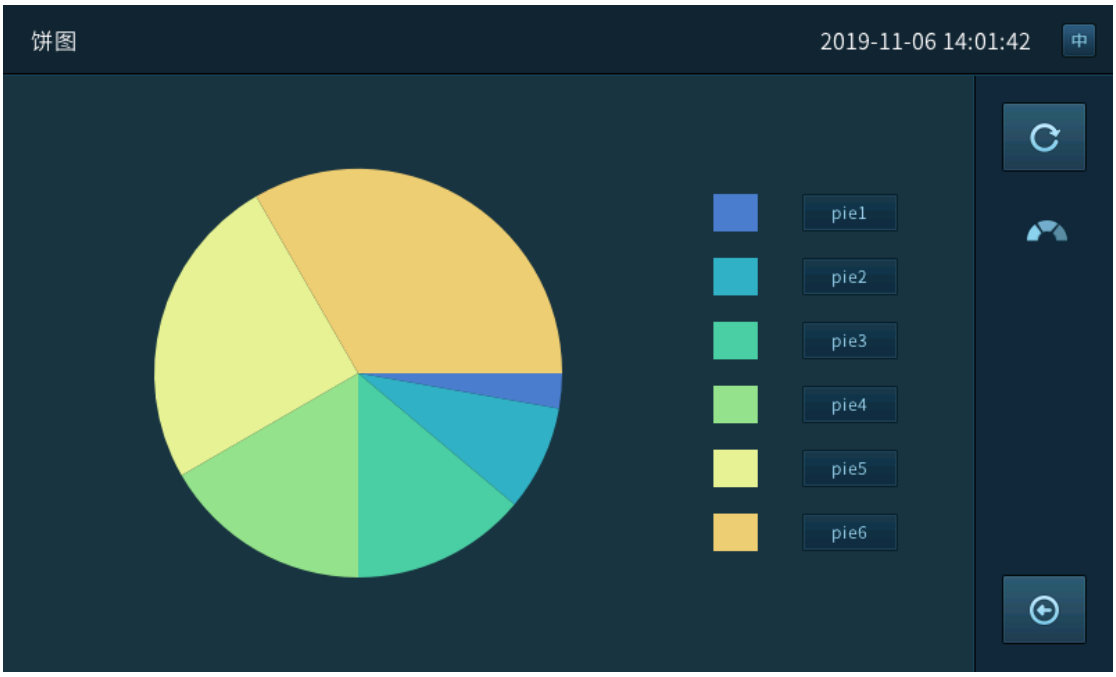


图 9.7　饼图

4. 实现曲线图逻辑

曲线图界面实现的功能点如下：

- 绘制曲线图
- 随机生成新的曲线图
- 设置是否显示曲线
- 设置是否显示闭合区域
- 设置是否显示圆点
- 设置是否显示平滑曲线

下面，介绍如何实现这些功能点。

(1) 绘制曲线图

曲线图同样是一个自定义控件，该控件中 X 轴、Y 轴、曲线等是一个个独立的元素，各自完成自身的绘制（比如曲线 line_series 仅负责将数据点按指定波形显示），但布局受父控件 chart_view 控制。代码如下：

```
/* Chart-Demo/src/custom_widgets/chart/line_series.c */
static ret_t line_series_on_paint_self(widget_t* widget, canvas_t* c) {
  line_series_t* series = LINE_SERIES(widget);
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  return_value_if_fail(series != NULL, RET_BAD_PARAMS);

  line_series_start_init_if_not_inited(widget);
  series_p_reset_fifo(widget);

  if (series->base.display_mode == SERIES_DISPLAY_COVER) {
    return series_p_on_paint_self_cover(widget, c);
  } else {
    return series_p_on_paint_self_push(widget, c);
  }
}
```

(2) 随机生成新的曲线图（界面右边的第一个按钮）

通过 widget_on 给该按钮注册 on_series_rset_rand_ufloat_data 响应事件，然后在函数 on_series_rset_rand_ufloat_data 中间接调用 rand 函数生成随机值，最后将生成的随机值通过 series_push 函数设置给控件，代码如下：

```
/* Chart-Demo/src/window_line_series.c */
static void init_normal_line_series_window(widget_t* widget) {
  widget_t* new_graph = widget_lookup(widget, "new_graph", TRUE);
  bool_t bring_to_top = strstr(widget->name, "_more_axis") != NULL ? TRUE : FALSE;

  if (new_graph != NULL) {
    widget_on(new_graph, EVT_CLICK, on_series_rset_rand_ufloat_data, widget);
  }
  ...
}
```

```
/* Chart-Demo/src/window_series_common.c */
ret_t on_series_rset_rand_ufloat_data(void* ctx, event_t* e) {
  widget_t* win = WIDGET(ctx);
  widget_t* chart_view = widget_lookup(win, "chartview", TRUE);
  if (chart_view) {
    on_series_rset_ufloat_data(chart_view, get_series_capacity_min(chart_view));
  }
  return RET_OK;
}

void on_series_rset_ufloat_data(widget_t* widget, uint32_t count) {
  on_series_rset_data(widget, count, sizeof(float_t), generate_ufloat_data);
}

static void generate_ufloat_data(void* buffer, uint32_t size) {
  uint32_t i;
  float_t* b = (float_t*)buffer;
  for (i = 0; i < size; i++) {
    b[i] = (float_t)(rand() % 120 + 10);
  }
}
```

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

(3) 设置是否显示曲线（界面右边的第二个按钮）

通过设置控件的 SERIES_PROP_LINE_SHOW 属性是否显示曲线，代码如下：

```
/* Chart-Demo/src/window_series_common.c */
ret_t on_series_line_show_changed(void* ctx, event_t* e) {
  return on_series_prop_changed(ctx, e, SERIES_PROP_LINE_SHOW, "line", "line",
↪"line_select");
}

static ret_t on_series_prop_changed(void* ctx, event_t* e, const char* prop, const␣
↪char* btn_name,
                                    const char* style, const char* style_select) {
  widget_t* win = WIDGET(ctx);
  widget_t* chart_view = widget_lookup(win, "chartview", TRUE);
  if (chart_view) {
    WIDGET_FOR_EACH_CHILD_BEGIN(chart_view, iter, i)
    if (widget_is_series(iter)) {
      value_t v;
      if (widget_get_prop(iter, prop, &v) == RET_OK) {
        value_set_bool(&v, !value_bool(&v));
        widget_set_prop(iter, prop, &v);

        widget_t* btn = widget_lookup(win, btn_name, TRUE);
        if (btn) {
          widget_use_style(btn, value_bool(&v) ? style_select : style);
        }
      }
    }
    WIDGET_FOR_EACH_CHILD_END()
  }

  return RET_OK;
}
```

(4) 设置是否显示闭合区域（界面右边的第三个按钮）

通过设置控件的 SERIES_PROP_LINE_AREA_SHOW 属性是否显示闭合区域，代码如下：

```
/* Chart-Demo/src/window_series_common.c */
ret_t on_series_area_show_changed(void* ctx, event_t* e) {
  return on_series_prop_changed(ctx, e, SERIES_PROP_LINE_AREA_SHOW, "area", "area",␣
↪"area_select");
}
```

(5) 设置是否显示圆点（界面右边的第四个按钮）

通过设置控件的 SERIES_PROP_SYMBOL_SHOW 属性是否显示圆点，代码如下：

```
/* Chart-Demo/src/window_series_common.c */
ret_t on_series_symbol_show_changed(void* ctx, event_t* e) {
  return on_series_prop_changed(ctx, e, SERIES_PROP_SYMBOL_SHOW, "symbol", "symbol
↪",
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
                                    "symbol_select");
}
```

(6) 设置是否显示平滑曲线（界面右边的第五个按钮）

通过设置控件的 SERIES_PROP_LINE_SMOOTH 属性是否显示平滑曲线，代码如下：

```
/* Chart-Demo/src/window_series_common.c */
ret_t on_series_smooth_changed(void* ctx, event_t* e) {
  return on_series_prop_changed(ctx, e, SERIES_PROP_LINE_SMOOTH, "smooth", "smooth
↪",
                                    "smooth_select");
}
```

运行效果如下图所示：

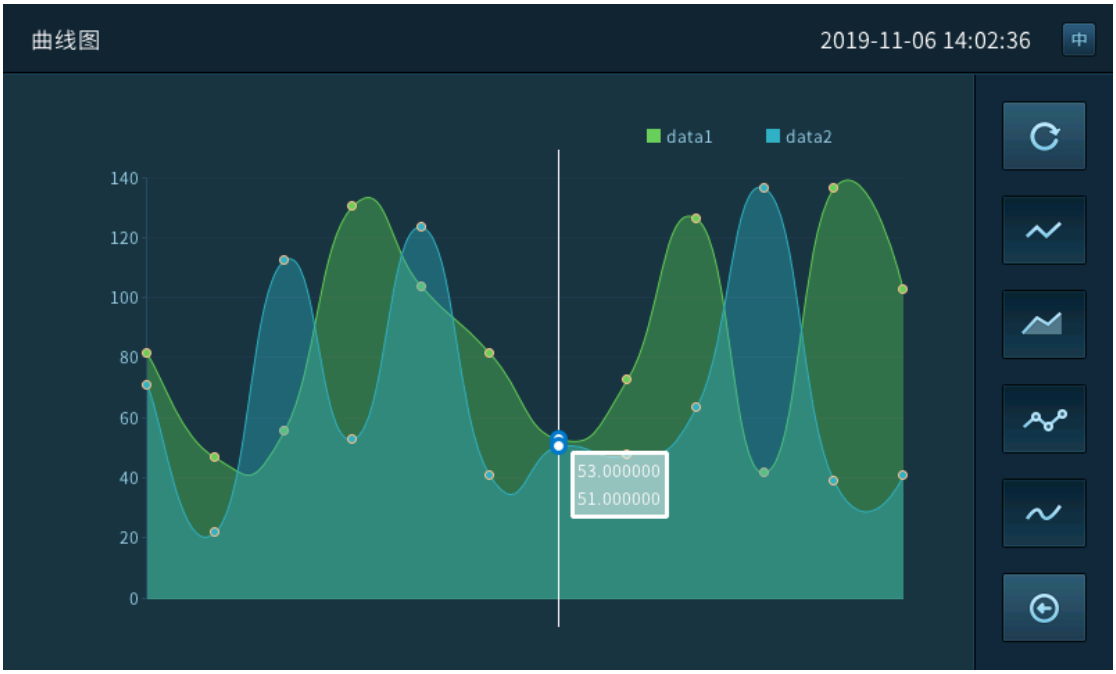


图 9.8　曲线图

5. 实现柱状图逻辑

柱状图界面实现的功能点如下：

- 绘制柱状图
- 随机生成新的柱状图

下面，介绍如何实现这些功能点。

(1) 绘制柱状图

柱状图与上面曲线图原理类似，不同的是数据显示改为使用 bar_series，bar_series 负责将数据点显示为一组组的柱条。代码如下：

```
/* Chart-Demo/src/custom_widgets/chart/bar_series.c */
static ret_t bar_series_on_paint(widget_t* widget, canvas_t* c, float_t ox, float_t␣
↪oy,
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
                                    fifo_t* fifo, uint32_t index, uint32_t size, rect_
↪t* clip_rect) {
  return bar_series_on_paint_internal(widget, c, ox, oy, fifo, index, size, clip_
↪rect, FALSE);
}
```

(2) 随机生成新的柱状图（界面右边的第一个按钮）

生成新的柱状图的逻辑与曲线图页面的差不多，这里就不累赘了，代码如下：

```
/* Chart-Demo/src/window_bar_series.c */
static void init_normal_bar_series_window(widget_t* widget) {
  widget_t* new_graph = widget_lookup(widget, "new_graph", TRUE);
  if (new_graph != NULL) {
    widget_on(new_graph, EVT_CLICK, on_series_rset_rand_ufloat_data, widget);
  }
}
```

运行效果如下图所示：

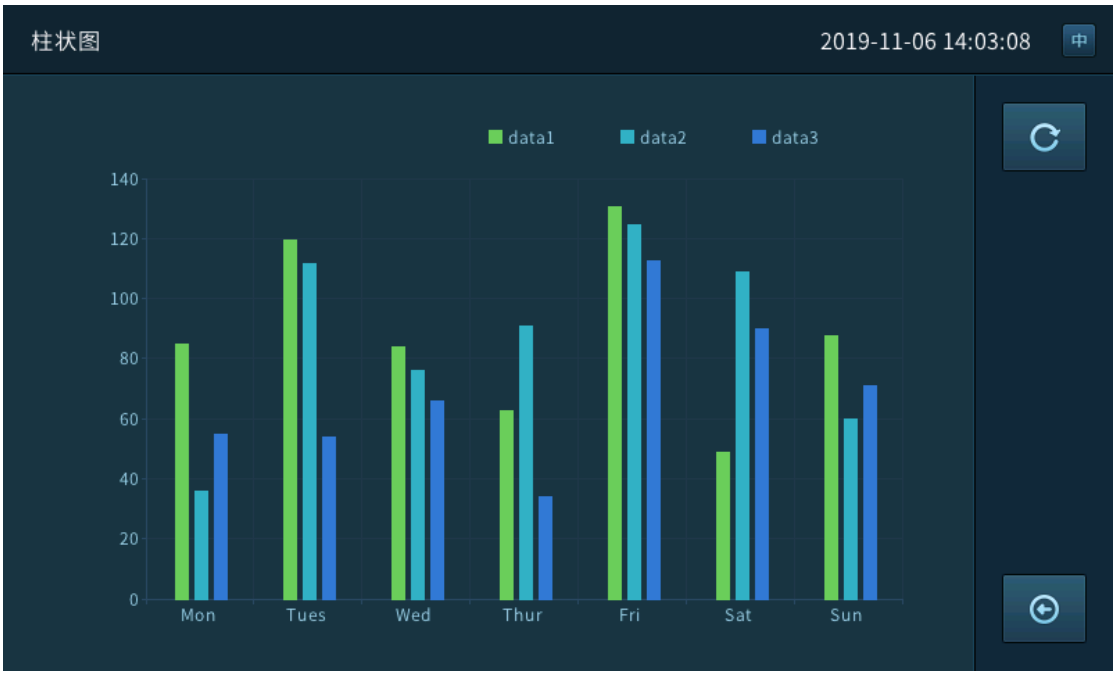


图 9.9　柱状图

9.4　音乐播放器

9.4.1　功能详解

音乐播放器实现的功能点主要有下面几点：

- 黑胶唱片动画效果
- 显示歌词
- 切换歌曲
- 播放列表

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

- 均衡器
- 旋钮音频参数调节

9.4.2 应用实现

本案例使用目录结构、编译和运行步骤与第 2 章介绍的 hello world 应用类似，这里就不重复介绍了。

本案例使用 XML 设计界面，业务逻辑使用 C 语言实现。主界面中左侧中部的黑胶唱片音乐播放场景主要通过 widget_animation 和 image_animation 动画配合 image、gauge_pointer、framer_view 控件实现。主界面右侧中部的歌词主要通过 list_view、scroll_view 和 list_item 的组合使用实现。主界面底部为播放功能选项，其中左侧三个按钮实现前一首、播放/暂停、下一首的功能；中间部分用两个 label 显示时间，一个 silder 显示进度，右侧两个按钮分别实现切换播放模式以及打开歌曲列表，功能主要通过注册点击事件实现。在主界面的右上方为三个按钮, 从上到下第一个按钮实现黑胶唱片场景动画的开启或者关闭，其余两个则按钮通过注册点击事件进入相应的界面。运行的效果详见下图。

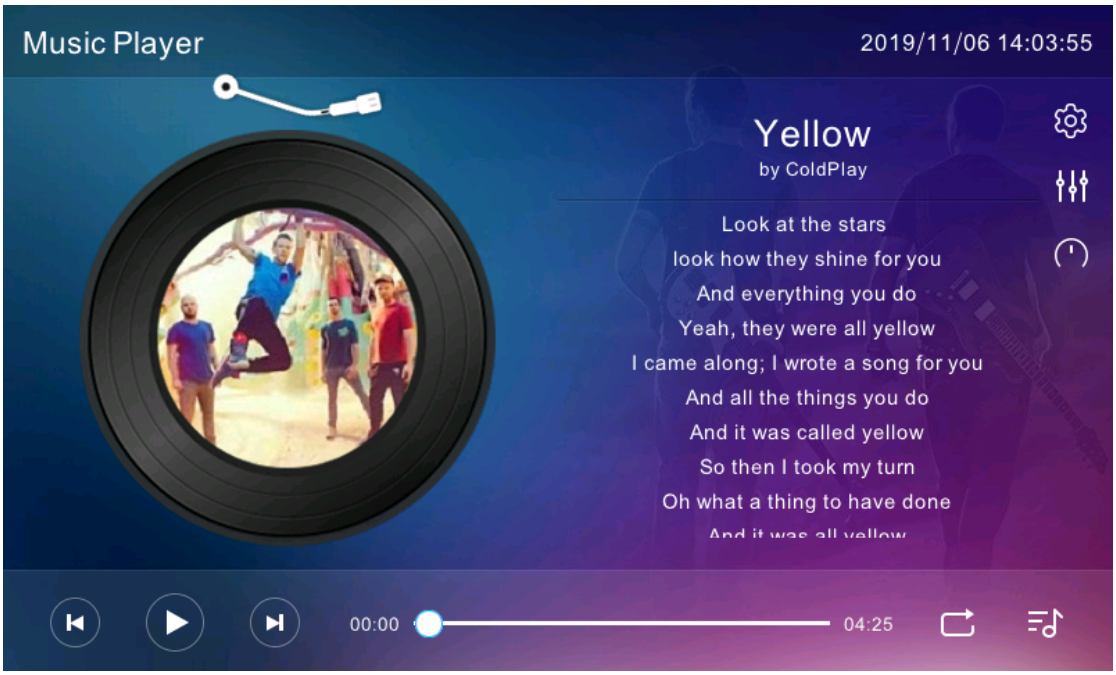


图 9.10 音乐播放器

本案例使用的代码均可以在 MusicPlay-Demo 目录找到。

1. 实现黑胶唱片动画效果

(1) 先创建两个 image 控件和一个 gauge_pointer 控件来分别实现唱片、歌曲封面和唱针的绘制。代码如下：

```
<!-- MusicPlayer-Demo/res_800_480/assets/default/raw/ui/main.xml -->
<frame_view  name="frame_menu"  x="-20" y="90" w="450" h="300" align_v="bottom" >
        <view x="0" y="0" w="300" h="300">
            <image name="song_y" w="188" h="188" image="yellow" draw_type="default"
↪/>
            <image name="Vinyl" image="Vinyl" w="300" h="300" draw_type="default" /
↪>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
        </view>
        <view x="0" y="0" w="300" h="300">
            <image name="song_q" w="188" h="188"  image="see_you_again" draw_type=
↪"default" />
            <image name="Vinyl" image="Vinyl" w="300" h="300" draw_type="default" /
↪>
        </view>
        <view x="0" y="0" w="250" h="250">
            <image name="song_n" w="188" h="188"  image="let_it_go" draw_type=
↪"default" />
            <image name="Vinyl" image="Vinyl" w="300" h="300" draw_type="default" /
↪>
        </view>
</frame_view>
<gauge_pointer name="vinyl_head" x="165" y="50" w="20" h="20" angle="-70"  image=
↪"gauge_pointer"/>
```

(2) 唱片、歌曲封面和唱针的旋转摆动主要通过 widget_create_animator 创建控件动画来实现，代码如下：

```
/* MusicPlayer-Demo/src/window_main.c */
/* 封面旋转动画 */
ret_t swtich_frame_rotation_animator(widget_t* win, bool_t start_anim) {
  return_value_if_fail(win != NULL, RET_BAD_PARAMS);
  widget_t* frame_menu = widget_lookup(win, "frame_menu", TRUE);
  frame_view_t* frame_view = FRAME_VIEW(frame_menu);
  widget_t* btn_anim_switch = widget_lookup(win, "btn_anim_switch", TRUE);

  widget_t* frame_child = frame_menu->children->elms[frame_view->value];
  if (start_anim && tk_str_eq(btn_anim_switch->style, "s_anim_switch_p")) {
    widget_create_animator(frame_child->children->elms[0],
                           "rotation(from=0, to=628,  duration=800000, repeat_
↪times=0)");
  } else {
    widget_stop_animator(frame_child->children->elms[0], "rotation");
    widget_destroy_animator(frame_child->children->elms[0], "rotation");
  }
  return RET_OK;
}

/* 唱针动画 */
ret_t swtich_frame_vinyl_head_animator(widget_t* win, bool_t start_anim) {
  return_value_if_fail(win != NULL, RET_BAD_PARAMS);
  widget_t* vinyl_head = widget_lookup(win, "vinyl_head", TRUE);
  if (start_anim) {
    widget_create_animator(vinyl_head, "value(from=-70, to=-35, duration=300,␣
↪delay=0)");
  } else {
    widget_create_animator(vinyl_head, "value(from=-35, to=-70, duration=300,␣
↪delay=0)");
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  }
}
```

2. 实现显示歌词

歌词的显示主要由控件 list_view、list_item 和 scroll_view 来实现，其中 list_item 的添加不在 XML 中进行，主要通过在代码中解析歌词文件，每解析一行便创建一个 list_time 作为 list_view 的子控件。代码如下：

```
<!-- MusicPlayer-Demo/res_800_480/assets/default/raw/ui/main.xml -->
<list_view name="lrc_view" x="45%"  y="30%" w="55%" h="50%" item_height="25" style=
↪"empty">
    <scroll_view name="lrc_scroll" x="0" y="0" w="100%" h="100%"
                 xslidable="FALSE" yslidable="TRUE"  style="empty">
    </scroll_view>
    <scroll_bar name="bar" x="right" y="0" w="0" h="0" value="0"/>
</list_view>
```

```
/* MusicPlayer-Demo/src/custom_function/music_manager.c */
/* 歌词解析 */
static ret_t parse_lrc_line(widget_t* win, const char* name) {
    ......
    while (right != NULL && left != NULL) {
      strncpy(buff, left + 1, right - left - 4);
      widget_set_text_utf8(slider_max_label, buff);
      p2 = strrchr(p, ']');
      if (p2 != NULL) {
        p2++;
        value_set_int32(&v_lrc_time, timetosec(buff));
        if (lrc_scroll->children == NULL || lrc_scroll->children->size < n + 1)
        list_item_create(lrc_scroll, 0, 0, 0, 0);  //添加list_item
        widget_use_style(lrc_scroll->children->elms[n], "empty");
        widget_invalidate(lrc_scroll->children->elms[n], NULL);
        setlocale(LC_CTYPE, "");
        widget_set_prop(lrc_scroll->children->elms[n], "lrc_time", &v_lrc_time);
        widget_set_text_utf8(lrc_scroll->children->elms[n], p2); //写入歌词
        widget_set_visible(lrc_scroll->children->elms[n], TRUE, FALSE);
        slider_set_max(slider, value_int32(&v_lrc_time));
        n++;
      }
    ......
  }
  ......
  return RET_OK;
}
```

3. 实现切换歌曲

(1) 实现切换歌曲封面

此功能点主要使用自定义容器控件 frame_view 实现，关于自定义控件的定义、注册请

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

参考 10.3.2 章节．此处便不再赘述。此容器主要实现的是类似于幻灯片的动画效果，首先，创建一个 frame_view 容器控件；其次，在 frame_view 中添加待播放子控件；最后，通过函数 frame_view_set_then 来切换上一个或者下一个子控件，代码如下：

```
/* MusicPlayer-Demo/src/custom_widgets/photo_frame/frame_view.c */
ret_t  frame_view_set_then(widget_t* widget, bool_t operate) {
  frame_view_t* frame_view = FRAME_VIEW(widget);
  return_value_if_fail(frame_view != NULL, RET_BAD_PARAMS);

  int32_t xoffset_end = 0;
  if (operate)
    xoffset_end = widget->h * -1;
  else
    xoffset_end = widget->h;
  frame_view_scroll_to(WIDGET(frame_view), xoffset_end);
  return RET_OK;
}
```

(2) 实现切换歌曲时间和歌词

在歌曲封面滚动动画完成后，可通过获取 frame_view 当前显示的子控件（也就是歌曲封面）的序号（frame_view->value）来选择对应的歌词进行解析，最后将歌曲封面、歌词和歌曲时间信息同步到界面上，代码如下：

```
/* MusicPlayer-Demo/src/window_main.c */
static ret_t on_frame_menu_vchange(void* ctx, event_t* e) {
  widget_t* win = WIDGET(ctx);
  widget_t* frame_menu = WIDGET(e->target);
  frame_view_t* frame_view = FRAME_VIEW(frame_menu);
  init_player(win);
  load_song(win, frame_view->value, TRUE);//通过frame_view->value加载对应歌曲信息
  return RET_OK;
}
```

4. 实现播放列表

(1) 实现播放列表

通过 list_view、list_scroll、list_item 控件实现播放列表的绘制，代码如下：

```
<!-- MusicPlayer-Demo/res_800_480/assets/default/raw/ui/play_list.xml-->
<list_view name="song_list_view" x="0"  y="16%" w="100%" h="83%" item_height="35"␣
↪style="empty">
 <scroll_view name="song_list_scroll" x="0" y="0" w="100%" h="100%"
              xslidable="FALSE" yslidable="TRUE" style="empty">
    <list_item name="0_song"  style="empty_list" layout="r1 c0">
        <image draw_type="icon" w="10%" image="icon_list" visible="FALSE"/>
        <label w="90%" text="Yellow" style="left_list"/>
    </list_item>
    <list_item name="1_song"  style="empty_list" layout="r1 c0">
        <image draw_type="icon" w="10%" image="icon_list" visible="FALSE"/>
        <label w="90%" text="see you again" style="left_list"/>
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
    </list_item>
    <list_item name="2_song"  style="empty_list" layout="r1 c0" image="icon_list">
        <image draw_type="icon" w="10%" image="icon_list" visible="FALSE"/>
        <label w="90%" text="let it go" style="left_list"/>
    </list_item>
 </scroll_view>
</list_view>
```

(2) 通过播放列表切换歌曲

首先为 list_item 控件注册事件，在所注册的事件函数中进一步响应 frame_view 所注册事件函数 on_frame_menu_vchange 切换所选择的歌曲进行播放，代码如下：

```
/* MusicPlayer-Demo/src/window_play_list.c */
static ret_t on_song_item_down(void* ctx, event_t* e) {
  if (m_win == NULL) return RET_FAIL;
  ......
  if (frame_view->value != iter_index) {
    if (vh->angle == -70) {
      widget_create_animator(vinyl_head, "value(from=-70, to=-35, duration=300,
↪delay=0)");
      widget_start_animator(vinyl_head, "value");
    }
    init_player(m_win);
    swtich_frame_rotation_animator(m_win, FALSE);
    frame_view_set_value(frame_menu, iter_index);//响应EVT_VALUE_CHANGED事件
  }
  return RET_OK;
}
```

```
/* MusicPlayer-Demo/src/window_main.c */
static ret_t on_frame_menu_vchange(void* ctx, event_t* e) {
  widget_t* win = WIDGET(ctx);
  widget_t* frame_menu = WIDGET(e->target);
  frame_view_t* frame_view = FRAME_VIEW(frame_menu);
  init_player(win);
  load_song(win, frame_view->value, TRUE);//切换歌曲
  return RET_OK;
}
```

运行效果如下图所示：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

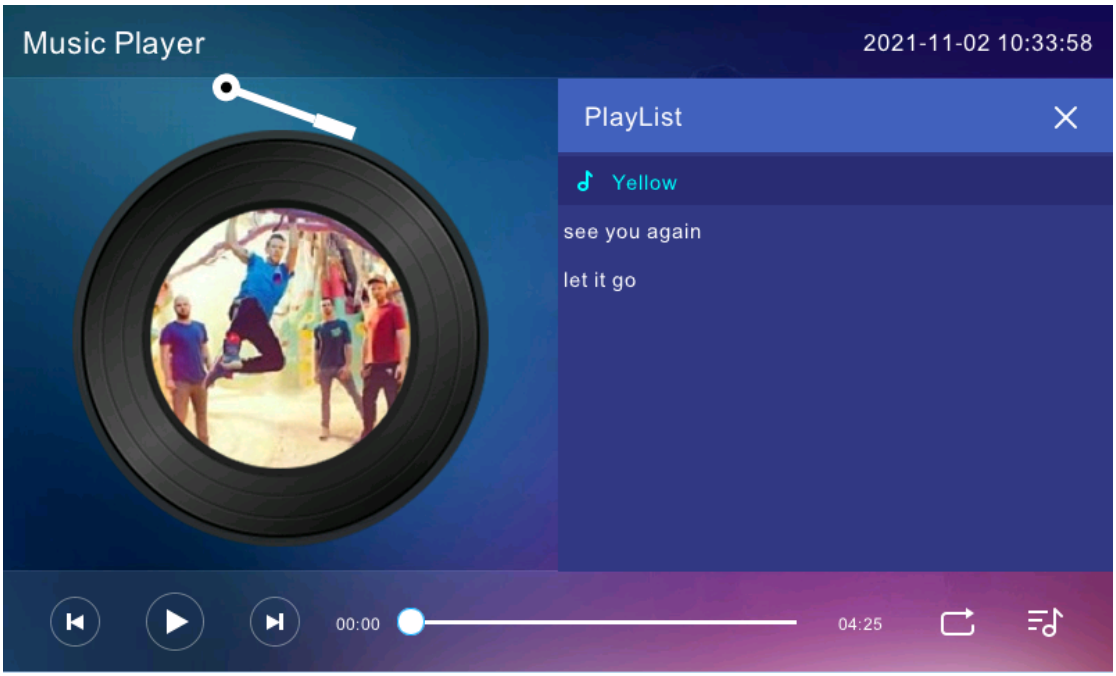


图 9.11　播放列表

5. 实现均衡器逻辑

(1) 绘制滑动条和曲线图

通过 slider 控件实现滑动条的绘制，曲线图是一个自定义控件，布局受到 char_view 影响，代码如下：

```
<!-- MusicPlayer-Demo/res_800_480/assets/default/raw/ui/equalizer.xml -->
<dialog anim_hint="fade"  self_layout="default(x=400,y=53,w=400,h=355)" style="bg_
↪setting">
    <chart_view name="chartview" x="0" y="75%" w="100%" h="25%" style="default">
        <x_axis name="x" w="100%" h="100%" axis_type="value"  min="0" max="8"
                    tick="{show:false}" split_line="{show:false}"  label="
↪{show:false}"
                            data="[0,0,0,0,0,0,0,0,0]"/>
        <y_axis name="y" w="100%" h="100%" axis_type="value"  min="-15" max="120"
                    tick="{show:false}" split_line="{show:false}" label="
↪{show:false}"
                            data="[0,10,20,30,40,50,60,70,80]"/>
        <line_series name="s1"  w="100%" h="100%" capacity="10" line="{style:s1,␣
↪smooth:true}"
                    area="{show:false}" symbol="{show:false}"/>
        <tooltip />
    </chart_view>
</dialog>
```

(2) 改变频率并生成对应频率波形

首先通过 widget_on 注册 slider 的 EVT_VALUE_CHANGING 事件处理函数，然后在该函数（set_series_data）中完成 slider 数值和波形图的绘制，代码如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
/* awtk_projects/MusicPlayer-Demo/src/dialog_equalizer.c */
static ret_t on_slider_changing(void* ctx, event_t* e) {
  widget_t* win = WIDGET(ctx);
  widget_t* child = WIDGET(e->target);
  char name_buf[BUFF_LEN] = {0};
  strcpy(name_buf, child->name);
  strtok(name_buf, "_");
  int32_t i = atoi(strtok(NULL, "_"));
  val_slider[i] = widget_get_value(child);

  widget_t* chart_view = widget_lookup(win, "chartview", TRUE);
  if (chart_view) {
    set_series_data(val_slider, chart_view, SLIDER_COUNT);
  }
  return RET_OK;
}
```

运行的效果如下图所示：



图 9.12　均衡器

1. 实现旋钮音频参数调节

(1) 绘制表盘指针和进度圆环控件

创建 gauge_pointer 和 progress_circle 控件实现旋钮和旋钮刻度线的绘制，代码如下：

```
<!-- MusicPlayer-Demo/res_800_480/assets/default/raw/ui/advanced.xml-->
<view name="gauge_pointer_view" x="0" y="10%" w="100%" h="100%">
    <view name="guage_low" x="20%" y="10%" w="120" h="120">
        <label x="left" y="middle:35" w="10" h="10" text="-"/>
        <label x="right" y="middle:35" w="10" h="10" text="+"/>
        <label x="center" y="bottom" w="10" h="10" tr_text="Low"/>
```

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
        <progress_circle name="circle_low"  w="120" h="120" max="360" style="blue"␣
↪start_angle="150" value="120" line_width="2"/>
        <gauge_pointer  x="17" y="17" name="low_gauge_pointer" w="86" h="86" ␣
↪image="btn_round"/>
    </view>
   ...
    <view name="guage_deep" x="60%" y="50%" w="120" h="120">
        <label x="left" y="middle:35" w="10" h="10" text="-"/>
        <label x="right" y="middle:35" w="10" h="10" text="+"/>
        <label x="center" y="bottom" w="10" h="10" tr_text="SURR-Depth"/>
        <progress_circle name="circle_deep" w="120" h="120" max="360" style="blue"␣
↪start_angle="150" value="120" line_width="2"/
        <gauge_pointer x="17" y="17" name="deep_gauge_pointer" w="86" h="86" ␣
↪image="btn_round"/>
    </view>
</view>
```

(2) 实现旋钮转动

通过注册 gauge_pointer 的 EVT_POINTER_DOWN 和 EVT_POINTER_MOVE 事件函数获取实时坐标点来计算所转动的角度，然后重新绘制 gauge_pointer 和 progress_circle，代码如下：

```
/* MusicPlayer-Demo/src/custom_function/knob_function.c */
ret_t knob_angle_change(widget_t* circle, widget_t* knob_pointer, point_t p) {
  ..
  if (tk_str_eq(value_str(&val_pointer_name), knob_pointer->name) &&
      value_str(&val_pointer_name) != NULL) {
    int32_t angle_temp = value_int32(&val_angle) + (mv - value_int32(&val_start_
↪P));
    if ((widget_get_prop(knob_pointer, "s_pmove", &val) == RET_OK && value_bool(&
↪val) == TRUE)) {
      if (mv > 150 || mv < -150) {
        value_set_bool(&val, FALSE);
        value_set_int32(&val_angle, angle_temp);
        widget_set_prop(knob_pointer, "s_pmove", &val);
        widget_set_prop(knob_pointer, "val_angle", &val_angle);
        return RET_OK;
      }
      if (angle_temp > 120) angle_temp = 120;
      if (angle_temp < -120) angle_temp = -120;
      gauge_pointer_set_angle(knob_pointer, angle_temp);//重新绘制gauge_pointer角度
      progress_circle_set_value(circle, angle_temp + 120);// 重新绘制progress_
↪circle角度
    } else {
      if (mv <= 150 && mv >= -150) {
        value_set_bool(&val, TRUE);
        value_set_int32(&val_start_P, mv);
        widget_set_prop(knob_pointer, "s_pmove", &val);
        widget_set_prop(knob_pointer, "val_start_P", &val_start_P);
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
      }
    }
  }
  return RET_OK
}
```

运行效果如下图所示：

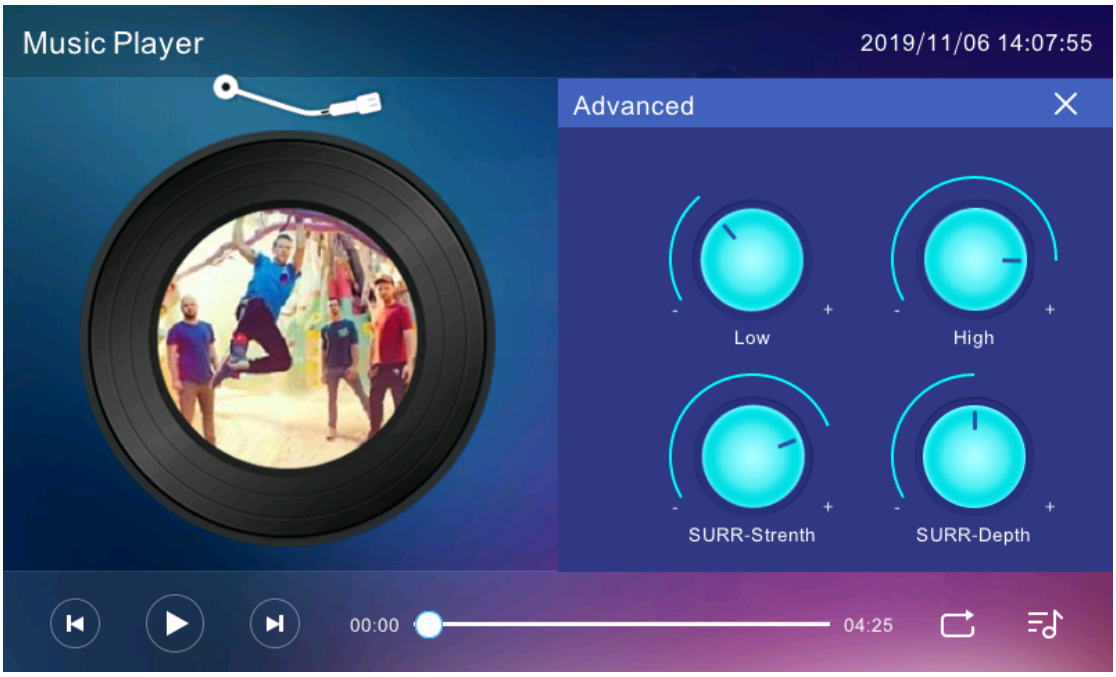


图 9.13　高级设置

9.5　智能手表

9.5.1　功能详解

智能手表实现的功能点主要有下面几点：

- 表盘：右滑切换表盘、长按预览及选择表盘、下滑打开消息列表、上滑打开快捷操作界面、左滑打开应用列表
- 日历应用：右滑返回、上滑切换到下个月、下滑切换到上个月、天气应用
- 音乐应用：右滑返回、播放与暂停、播放模式、切歌、改变音量
- 健康应用：右滑返回、设定目标
- 心率检测：右滑返回、检测心率

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

9.5.2 应用实现

本应用包括四个表盘：数字和模拟表盘各有两个。每个表盘都具有不同功能，具体如下：

- 模拟表盘一：具有动态显示电量、查看天气及记录和动态显示健康数据的功能，详见下图：



图 9.14 模拟表盘一

- 数字表盘一：具有显示时间、动态更新刻度（刻度值表示秒）的功能，详见下图：



图 9.15 数字表盘一

- 数字表盘二：具有显示天气、日历和健康数据、播放音乐以及动态显示心率图的功能，

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

详见下图：



图 9.16　数字表盘二

- 模拟表盘二：具有显示天气和日历以及播放音乐的功能，其中音乐圆环进度与音乐应用同步，下面几个消息提示图标也是动态变化，详见下图：



图 9.17　模拟表盘二

1. 实现手势识别

通过 gesture 手势控件来实现的，通过比对手势按下与松开时的坐标，从而判断其方向及距离，代码如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
/* SmartWatch-Demo/src/custom_widgets/ gesture.c */
static ret_t gesture_on_event(widget_t* widget, event_t* e) {
  uint16_t type = e->type;
  gesture_t* gesture = GESTURE(widget);
  ...
  case EVT_POINTER_UP: {
    pointer_event_t* pointer_event = (pointer_event_t*)e;
    (void)pointer_event;
    if (gesture->child_graded) {
      return RET_OK;
    }
    if (gesture->pressed) {
      pointer_event_t evt = *(pointer_event_t*)e;
      if (evt.x != gesture->press_point.x || evt.y != gesture->press_point.y) {
        float_t distance_x = gesture->press_point.x - evt.x;
        float_t distance_y = gesture->press_point.y - evt.y;
        distance_x = distance_x > 0 ? distance_x : -distance_x;
        distance_y = distance_y > 0 ? distance_y : -distance_y;
        if (distance_x > distance_y) {
          if (distance_x < 40 && distance_x < widget->w * 0.2f) {
            gesture_pointer_up_cleanup(widget);
            break;
          }
          if (evt.x > gesture->press_point.x) {
            log_debug("slide right\n");
            evt.e = event_init(EVT_SLIDE_RIGHT, widget);
            widget_dispatch(widget, (event_t*)&evt);
          } else {
            log_debug("slide left\n");
            evt.e = event_init(EVT_SLIDE_LEFT, widget);
            widget_dispatch(widget, (event_t*)&evt);
          }
        } else {
          if (distance_y < 40 && distance_y < widget->h * 0.2f) {
            gesture_pointer_up_cleanup(widget);
            break;
          }
          if (evt.y > gesture->press_point.y) {
            log_debug("slide down\n");
            evt.e = event_init(EVT_SLIDE_DOWN, widget);
            widget_dispatch(widget, (event_t*)&evt);
          } else {
            log_debug("slide up\n");
            evt.e = event_init(EVT_SLIDE_UP, widget);
            widget_dispatch(widget, (event_t*)&evt);
          }
        }
      }
    ...
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
}
```

2. 实现表盘长按预览

表盘预览窗口主要用了 slide_menu 控件布局，表盘长按预览的效果详见下图，代码如下：



图 9.18　表盘预览及选择

```
<!--SmartWatch-Demo/res_800_480/assets/raw/ui/select_dial.xml-->
<window anim_hint="slide_right" style="select_dial"  full_screen="false" x="c" y="m
↪" w="390" h="390"
        theme="select_dial_styles">
  <view x="0" y="0" w="100%" h="100%" name="select:gesture">
    <slide_menu name="select:slideview" x="c" y="m" w="240%" h="70%" style="dot"␣
↪min_scale="1.0">
      <view x="0" y="0" w="100%" h="100%">
        <button name="watch_04" style="watch_04" x="c" y="m" w="258" h="258"/>
      </view>
      <view x="0" y="0" w="100%" h="100%">
        <button name="watch_01" style="watch_01" x="c" y="m" w="258" h="258"/>
      </view>
      <view x="0" y="0" w="100%" h="100%">
        <button name="watch_02" style="watch_02" x="c" y="m" w="258" h="258"/>
      </view>
      <view x="0" y="0" w="100%" h="100%">
        <button name="watch_06" style="watch_06" x="c" y="m" w="258" h="258"/>
      </view>
    </slide_menu>
    <view name="select:digit_btns" x="c" y="m" w="80%" h="80%" visible="false"␣
↪sensitive="false"
          style="digit_btns" >
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
      <button name="select:top" x="126" y="45" w="60" h="60" text="Top" />
      <button name="select:bottom" x="126" y="165" w="60" h="60" text="Bottom" />
      <button name="select:left" x="66" y="105" w="60" h="60" text="Left" />
      <button name="select:right" x="186" y="105" w="60" h="60" text="Right" />
    </view>
    <view name="select:time_btns" x="c" y="m" w="80%" h="80%" visible="false"␣
↪sensitive="false"
          style="time_btn" >
      <button name="select:1" x="31" y="30" w="60" h="60" text="1" />
      <button name="select:2" x="31" y="210" w="60" h="60" />
      <button name="select:3" x="126" y="210" w="60" h="60" />
      <button name="select:4" x="221" y="210" w="60" h="60" />
    </view>
    <button name="select:done" style="music" x="c" y="bottom" w="100%" h="30" text=
↪"完成"
            visible="false" sensitive="false" />
  </view>
</window>
```

3. 实现音乐应用

切歌动画主要通过两个动画实现，先把当前歌曲切走，再把新歌曲切回来，详见下图：



图 9.19　音乐应用界面

切歌实现的步骤如下：

步骤一：首先记住音乐的播放状态，切歌过程中是不播放的，通过动画把歌名标签移动到视野外，代码如下：

```
/* SmartWatch-Demo/src/application/ music.c */
static ret_t on_music_next(void* ctx, event_t* e) {
  (void)e;
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
  widget_t* win = WIDGET(ctx);

  music_state_t* music_state = &(global_data.music_state);
  music_state->play_progress = 0; /* 进度变为0 */

  widget_t* widget = widget_lookup(win, "music:progress", TRUE);
  if (widget) {
    health_circle_set_value_b(widget, music_state->play_progress);
    widget_invalidate(widget, NULL);
  }
  /* 切歌 */
  widget = widget_lookup(win, "music:mode", TRUE);
  if (widget != NULL) {
    music_state->song_index = next_index(music_state);
  }
  widget_t* label = widget_lookup(win, "music:song_name", TRUE);
  if (label != NULL) {
    const char* name = "music_next";
    widget_create_animator_with(label, "x(name=%s, from=%d, to=%d, duration=%d)",
      name, 0, label->w, 250);
    widget_animator_t* animator = widget_find_animator(label, name);
    if (animator != NULL) {
      uint8_t play_state = music_state->play_state;
      if (music_state->play_state == 1) {
        music_state->play_state = 0; // 先暂停
      }
      widget_animator_on(animator, EVT_ANIM_END, on_music_next_play_anim_end,
                         tk_pointer_from_int(play_state));
    }
  }
  return RET_OK;
}
```

步骤二：然后再用动画将已经更换歌名的标签从视野外移动到中间，代码如下：

```
/* SmartWatch-Demo/src/application/music.c */
static ret_t on_music_next_play_anim_end(void* ctx, event_t* e) {
  (void)e;
  uint8_t play_state = tk_pointer_to_int(ctx);
  music_state_t* music_state = &(global_data.music_state);
  timer_info_t timer;
  timer.ctx = music_win;
  on_update_timer(&timer);
  widget_t* label = widget_lookup(music_win, "music:song_name", TRUE);
  if (label != NULL) {
    widget_create_animator_with(label, "x(from=%d, to=%d, duration=%d)",
      -label->w, 0, 250);
  }
  music_state->play_state = play_state;
  return RET_OK;
```


©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
}
```

4. 实现天气应用

（1）显示天气的效果详见下图



图 9.20　天气应用界面

（2）小时天气界面主要用到了 slide_view 布局，其中小时天气用了 hour_weather 自定义控件，其中点击中间环会更新温度值，效果详见下图，代码如下：



图 9.21　小时天气界面

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
/* SmartWatch-Demo/src/custom_widgets/hour_weather.c */
static ret_t hour_weather_on_event(widget_t* widget, event_t* e) {
  uint16_t type = e->type;
  hour_weather_t* hour_weather = HOUR_WEATHER(widget);

  switch (type)
  {
    case EVT_POINTER_DOWN: {
      hour_weather->pressed = TRUE;
      break;
    }

    case EVT_POINTER_UP: {
      pointer_event_t* pointer_event = (pointer_event_t*)e;
      point_t point = { pointer_event->x, pointer_event->y };
      widget_to_local(widget, &point);
      log_debug("(%d, %d) clicked!\n", (int32_t)point.x, (int32_t)point.y);

      xy_t x_c = point.x - widget->w / 2;
      xy_t y_c = point.y - widget->h / 2;
      hour_weather->selected = hour_health_get_click_select(widget, x_c, y_c);

      on_change_select(widget);
      hour_weather->pressed = FALSE;

      event_t event = event_init(EVT_CHANGE_SELECT, widget);
      widget_dispatch(widget, &event);

      break;
    }

    default:
      break;
  }

  return RET_OK;
}
```

5. 实现健康应用

健康应用主要是 slide_view 布局，包括了四个界面，下面主要看看其中的两个界面。

（1）主界面中的分时控件使用了自定义的 bar_graph 控件，详见下图：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.



图 9.22　主界面

（2）活动界面使用了自定义的 health_circle 圆环控件（基于 progress_circle 改造），加了拖动可以动态改变数值的功能，详见下图：



图 9.23　活动界面

6. 实现日历应用

日历应用布局主要通过 view 控件实现，详见下图：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.



图 9.24　日历应用界面

其中，切换月份通过自定义动画实现，步骤如下：

步骤一：先通过动画把当前月份界面移走，代码如下：

```
/* SmartWatch-Demo/src/application/calendar.c */
static ret_t on_calendar_down_prev(void* ctx, event_t* e) {
  (void)e;
  widget_t* window = widget_get_window(WIDGET(ctx));
  widget_t* widget = widget_lookup(window, s_calendar_labels, TRUE);
  if (widget != NULL) {
    widget_t* calendar_date = widget_lookup(window, s_calendar_date, TRUE);
    wh_t h = widget->h;
    (void)h;
    if (calendar_date != NULL) {
      const char* name = "down";
      widget_create_animator_with(widget, "y(name=%s, from=%d, to=%d, duration=%d)
↪",
        name, widget->y - 10, window->h, 500);
      widget_animator_t* animator = widget_find_animator(widget, name);
      if (animator != NULL) {
        widget_animator_on(animator, EVT_ANIM_END, on_calendar_down, window);
      }
    }
  }
  return RET_OK;
}
```

步骤二：切走当前月份后再切换到下月界面，更换完成也是通过动画载入下月界面，代码如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
/* SmartWatch-Demo/src/application/calendar.c */
static ret_t on_calendar_down(void* ctx, event_t* e) {
  (void)e;
  if (date_current.month == 12) {
    date_current.month = 1;
    date_current.year++;
  } else {
    date_current.month += 1;
  }
  calendar_cal(ctx, &date_current);

  widget_t* window = widget_get_window(WIDGET(ctx));
  widget_t* widget = widget_lookup(window, s_calendar_labels, TRUE);
  if (widget != NULL) {
    widget_t* calendar_date = widget_lookup(window, s_calendar_date, TRUE);
    wh_t h = widget->h;
    if (calendar_date != NULL) {
      widget_create_animator_with(widget, "y(from=%d, to=%d, duration=%d)",
        -h, calendar_date->h, 500);
    }
  }

  return RET_OK;
}
```

7. 实现心率应用

（1）心率图主要通过自定义控件 bar_graph 实现，bar_graph 通过 data 字段实现数据持久化，图表数据放在全局域中，每次加载窗口就动态载入，详见下图：



图 9.25　心率图

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

（2）心率检测窗口中的心跳动画用了 image_animation 控件，检测完成或者右滑返回心率窗口，效果详见下图，代码如下：



图 9.26　心率检测界面

```
<!--SmartWatch-Demo/res_800_480/assets/raw/ui/heart_rate_check.xml-->
<window anim_hint="slide_left(start_alpha=0, end_alpha=200)" style="heart_rate"
↪full_screen="false"
        x="c" y="m" w="390" h="390" theme="heart_rate_styles">
  <gesture name="heart_rate_check_gesture" x="c" y="m" w="390" h="390" >
    <label name="show" style="check_top" x="c" y="60" w="139" h="34" text="正在检测
↪"/>
    <image_animation x="c" y="m" w="238" h="238" image="heart_" start_index="0"
                          end_index="2" format="%s%05d" auto_play="true" interval=
↪"100" delay="0"/>
    <label name="bottom" style="check_bottom" x="c" y="308" w="194" h="26" text=
↪"请保持手腕平衡"/>
  </gesture>
</window>
```

8. 实现通知列表

通知列表主要通过 list_view 控件实现，居中效果则通过不可见的空项实现。效果详见下图，代码如下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.



图 9.27　通知消息界面

```
<!--SmartWatch-Demo/res_800_480/assets/raw/ui/message.xml-->
<window anim_hint="slide_down" style="message" full_screen="false" x="c" y="m" w=
↪"390" h="390"
        theme="message_styles">
  <gesture  x="0" y="0" h="100%" w="100%" name="message_window" >
    <list_view x="c" y="-55" w="100%" h="120%" item_height="100">
      <scroll_view name="view" x="c" y="m" w="100%" h="100%">
        <list_item style="a" children_layout="default(rows=1,cols=1)" >
          <view x="c" y="m" w="100%" h="95%" style="dragger" visible="false">
          </view>
        </list_item>
        <list_item style="a" children_layout="default(rows=1,cols=1)">
          <view x="c" y="m" w="100%" h="95%" style="dragger">
            <column x="50" y="m" w="100%" h="100%">
              <image x="0" y="m" w="88" h="88" image="app_cloudy_normal_88"/>
              <row x="90" y="m" w="200" h="100%">
                <digit_clock x="0" y="0" w="100%" h="50%" format="hh:mm" style=
↪"time"/>
                <label x="0" y="50%" w="100%" h="50%" text="天气晴朗" style="text"/
↪>
              </row>
            </column>
          </gesture>
</window>
```

滑动删除操作则通过自定义动画实现，代码详见 SmartWatch-Demo/src/slide_appliction/message.c 这里就不列举了。

9. 实现应用菜单

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

应用菜单主要用了自定义控件 slide_menu_v 布局，与 slide_menu 效果基本一样，只不过滑动方向改为上下，效果详见下图，代码详见 SmartWatch-Demo/src/custom_widgets/slide_menu_v.c 这里就不列举了。



图 9.28　应用选择界面

10. 实现快捷操作

快捷操作界面主要用了 slide_view 布局，效果详见下图，代码如下：



图 9.29　快捷操作界面

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
<!--SmartWatch-Demo/res_800_480/assets/raw/ui/quick.xml-->
<window anim_hint="slide_up" full_screen="false" style="pay" x="c" y="m" w="390" h=
↪"390"
        theme="quick_styles">
  <gesture name="gesture" x="0" y="0" w="100%" h="100%">
    <slide_view x="m" y="c" w="100%" h="100%">
      <view x="c" y="m" w="100%" h="100%">
        <label x="c" y="62" w="213" h="34" text="支付宝付款码" style="text"/>
        <button x="c" y="125" w="306" h="156" name="alipay" style="alipay"/>
      </view>
      <view x="c" y="m" w="100%" h="100%">
        <label x="c" y="62" w="70" h="34" text="请靠近读卡器" style="text" />
        <button name="gongshang" style="gongshang" x="c" y="125" w="274" h="173" />
      </view>
    </slide_view>
    <slide_indicator_arc x="0" y="b" w="100%" h="20" spacing="5"/>
  </gesture>
</window>
```

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

10. 把应用部署到嵌入式开发环境

本章导读：

AWTK 已经在多个嵌入式平台做好了移植，并提供了相应的模板工程，如：嵌入式 *Linux*、*AWorksOS*、*RT1052* 裸系统以及 *STM32* 裸系统等。*AWTK* 适用于嵌入式领域，本文将简述如何将 *AWTK* 应用部署到嵌入式开发环境中。

10.1 把应用部署到嵌入式 Linux

AWTK 应用程序在 Windows 调试好之后，部署到嵌入式 Linux 平台非常方便，只需要简单的几个步骤就可以了，下面将介绍如何将自己的应用移植到嵌入式 Linux 平台上。

（1）下载 AWTK 整合包

在 AWTK 官网下载最新的 AWStudio[26]后，可以在 AWStudio/AWTK/SDK 中可以找到 awtk 目录与 awtk-linux-fb 目录。将这两个目录拷贝到 Linux 系统目录中（如：/home/user）：

```
/home/user/
      |-- awtk/
      |-- awtk-linux-fb/
      |-- user_apps/
```

（2）编辑 awtk_config.py 设置工具链的路径

```
TSLIB_LIB_DIR='/opt/28x/tslib/lib'        #tslib目录
TSLIB_INC_DIR='/opt/28x/tslib/include'    #tslib目录
TOOLS_PREFIX='/opt/28x/gcc-4.4.4-glibc-2.11.1-multilib-1.0/arm-fsl-linux-gnueabi/
↪bin/arm-linux-' #交叉编译链工具目录（用户视自己情况填写）
```

注：如果不需要 tslib，不定义 TSLIB_LIB_DIR 和 TSLIB_INC_DIR 即可。如：

```
#TOOLS_PREFIX=''
#TSLIB_LIB_DIR=''
```

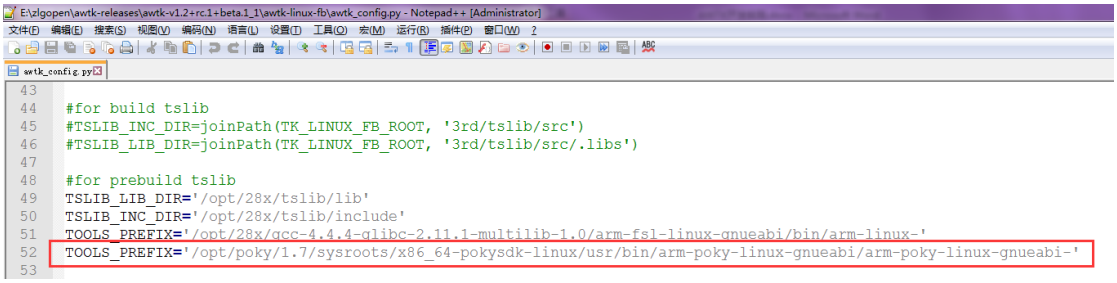


图 10.1 设置 TOOLS_PREFIX

在 OS_FLAGS 中加入需要的编译参数，例如 CPU 支持硬浮点，则开启”-mfloat-abi=hard“，如下图所示：

[26] http://awtk.zlg.cn

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

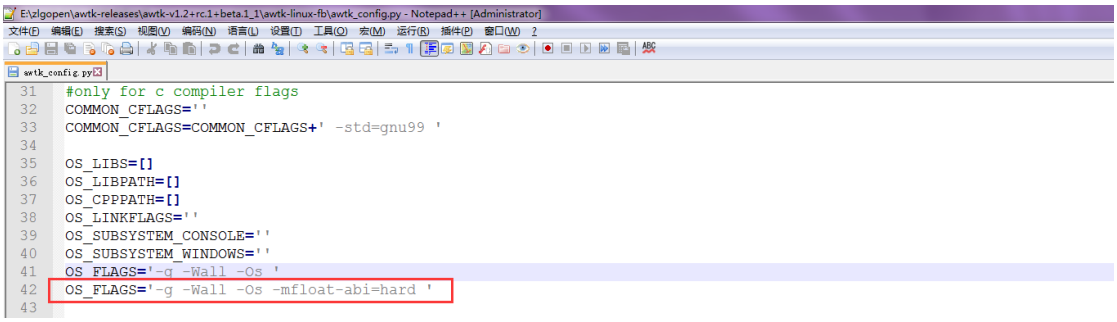


图 10.2　支持硬浮点

（3）修改输入设备文件名

修改/home/user/awtk-linux-fb/awtk-port/main_loop_linux.c 文件中的设备文件名：

```
/* framebuffer */
#ifndef FB_DEVICE_FILENAME
#define FB_DEVICE_FILENAME "/dev/fb0"
#endif /*FB_DEVICE_FILENAME*/

/* tslib */
#ifndef TS_DEVICE_FILENAME
#define TS_DEVICE_FILENAME "/dev/input/event0"
#endif /*TS_DEVICE_FILENAME*/

/* 键盘 */
#ifndef KB_DEVICE_FILENAME
#define KB_DEVICE_FILENAME "/dev/input/event1"
#endif /*KB_DEVICE_FILENAME*/

/* 鼠标 */
#ifndef MICE_DEVICE_FILENAME
#define MICE_DEVICE_FILENAME "/dev/input/mouse0"
#endif /*MICE_DEVICE_FILENAME*/
```

注: 在有些平台下，如果设置 #define MICE_DEVICE_FILENAME “/dev/input/mice”，会出现触摸不灵的问题。通过 hexdump /dev/input/mice 命令发现，按下触摸屏或操作鼠标都会打印信息，即/dev/input/mice 会同时接收触摸和鼠标事件。可通过”hexdump /dev/input/xx”命令选择正确的鼠标设备文件名。

（4）编译 awtk

在进行应用程序的编译前，需要先编译 awtk，进入 awtk-linux-fb 目录后执行下面的命令对 awtk 进行编译

```
cd /home/user/awtk-linux-fb
scons
```

（5）编译应用程序

将要部署的应用（如：HelloDesigner-Demo，示例程序可以在 AWStudio 中下载）复制到 user_apps 目录下。进入 awtk-linux-fb 目录后执行下面的命令，目标文件将输出到 awtk-linux-fb/bin 目录下：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

```
cd /home/user/awtk-linux-fb
scons APP=../user_apps/HelloDesigner-Demo
```

（6）打包应用程序

编译完成后，还需要把应用程序和资源文件打包在一起，才能下载到板子上运行，执行下面的命令，生成压缩包 release.tar.gz：

```
cd /home/user/awtk-linux-fb
./release.sh ../user_apps/HelloDesigner-Demo/res demo
```

（7）运行应用程序

把 release.tar.gz 上传到开发板并解压，然后通过 ssh 或串口调助手让板子运行：

```
./release/bin/demo
```

注: 如果想具体了解更多嵌入式 Linux 移植方面的信息，请看 awtk-linux-fb/README.md。

10.2 把应用部署到 AWorksOS

AWTK 应用程序在 Windows 调试好之后，部署到 AWorksOS 平台非常方便，**只需屏蔽工程中自带的示例代码文件（通常名字是 xxx_demo），然后往工程中添加用户自己的应用代码就可以了**，下面以 RT1052 平台为例介绍如何将自己的应用移植到 AWorksOS 平台上。

从广州致远电子有限公司[27]官网下载最新的M1052 核心板[28]（RT1052）光盘资料，里面包含自带移植好的 AWTK Eclipse 工程和 AWTK Keil 工程。

注: 在使用 Designer 制作 UI 界面时，默认使用全字库，可能会导致嵌入式开发环境编译时空间不足。用户应根据应用的实际需求，在 Designer 中对字库进行裁剪，重新打包后再部署到嵌入式开发环境中。

10.2.1 使用 Eclipse 构建应用

找到 AWorksOS SDK 开发包下的 aw_m1052_disk-2.0.0 或 aw_easyarmrt1052_disk-2.0.1 目录，按下面步骤使用 Eclipse 把自己的应用添加到 AWTK 工程中，具体操作如下：

1. 打开工程

（1）打开 Eclipse（如双击”eclipse_neon_2016q3_x86/startup_eclipse_neon.bat），然后选择工程存放目录，如：“E:/mytools/eclipse/awtk_demo”，详见下图：

[27] https://www.zlg.cn
[28] https://www.zlg.cn/ipc/ipc/product/id/217.html

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

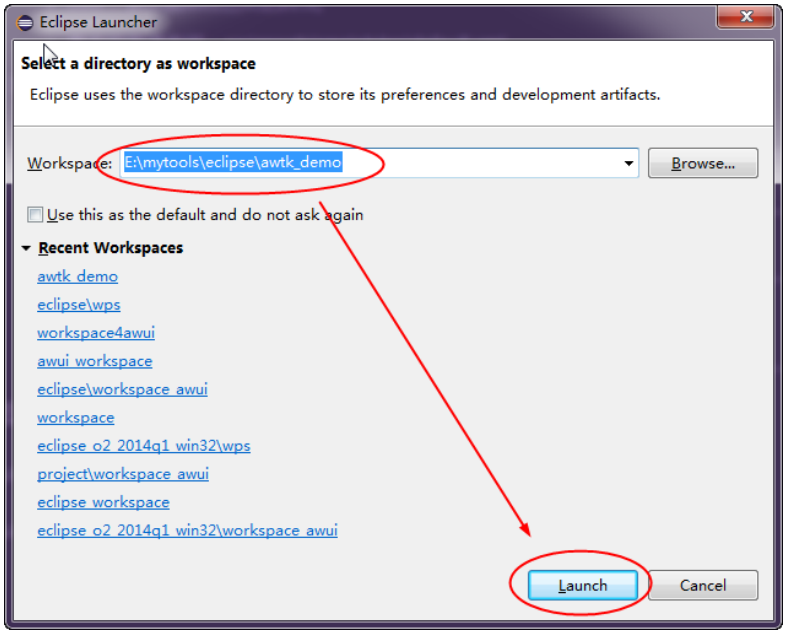


图 10.3　建立 Workspace

（2）选择菜单 File → Import 后，详见下图：

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

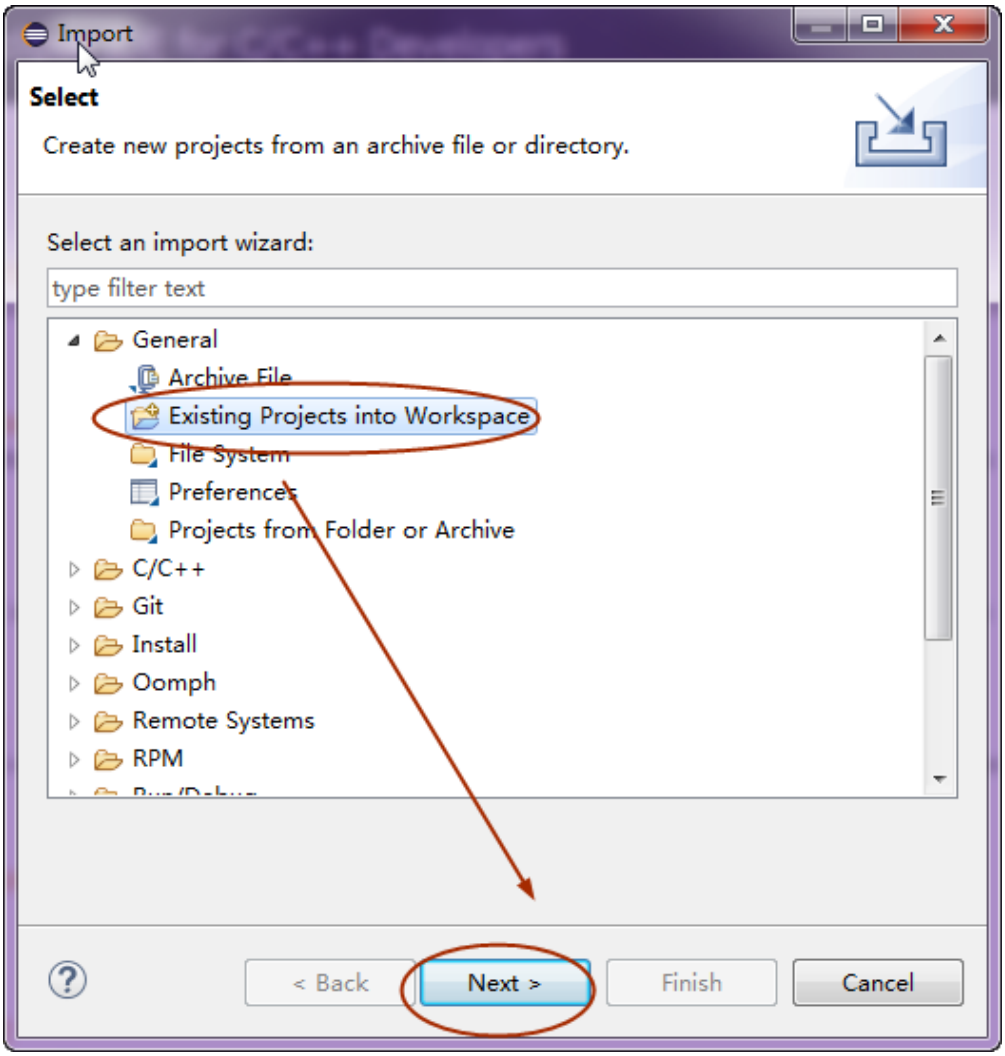


图 10.4　导入工程

（3）选择”aw_m1052_disk-2.0.0/examples/application/app_awtk_demo/prj/m1052_awtk”工程所在目录，详见下图：

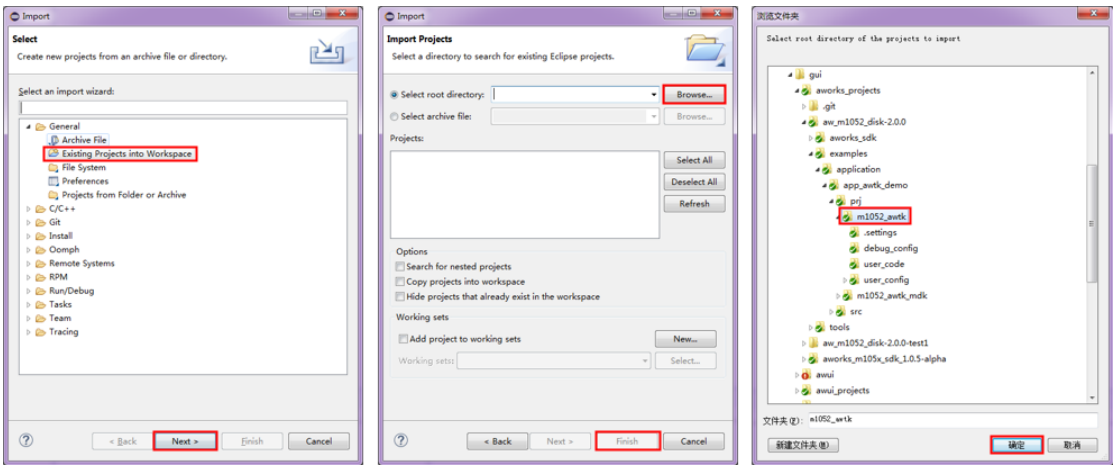


图 10.5　选择工程所在目录

（4）如果最后显示下图，则表示开发环境已经搭建好了。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

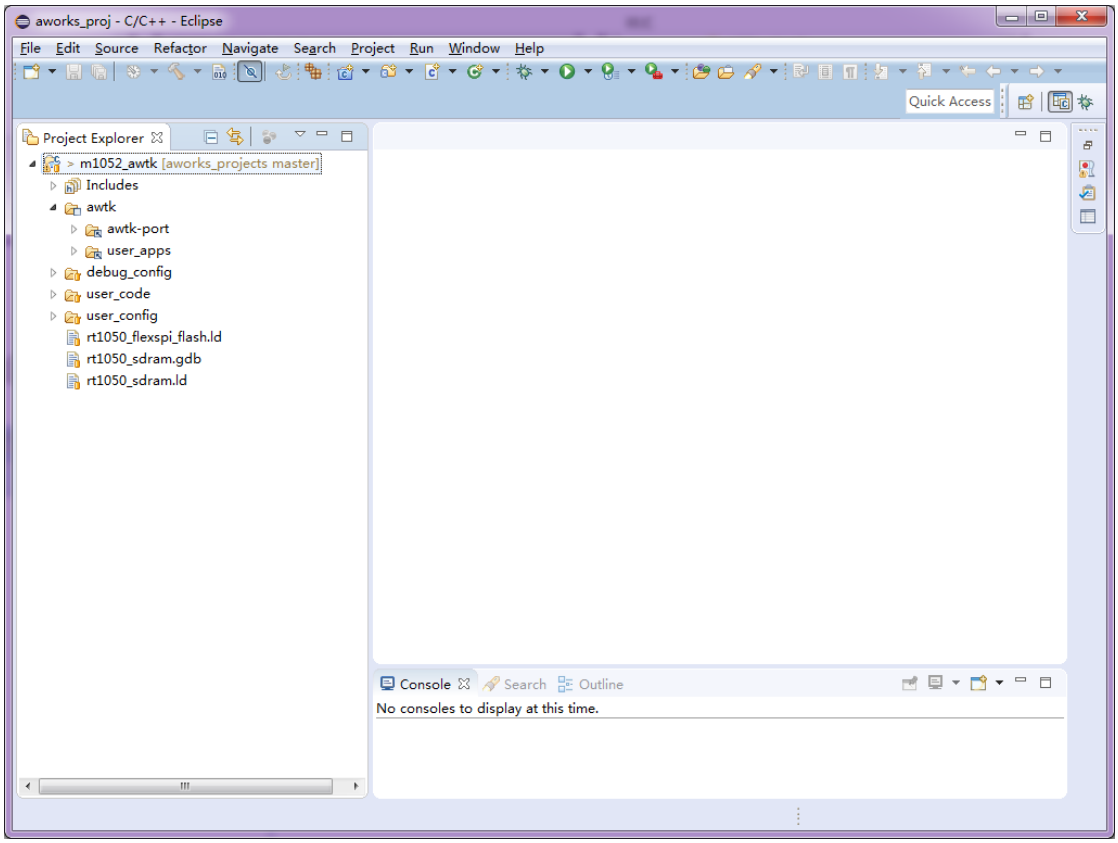


图 10.6　导入成功

2. 把应用加入工程

（1）屏蔽 demos 示例

由于 m1052_awtk 工程已经自带 demos 应用示例，但本工程同时只能运行一个 AWTK 应用。如果想运行自己的应用程序，需要将 demos 屏蔽掉，按下图所示操作：

- 选择 Eclipse 工程里的 demos 文件夹，弹出右键菜单
- 选择 Resource ConfigurationsExclude from Build…
- 点击”Select All”选中所有配置

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

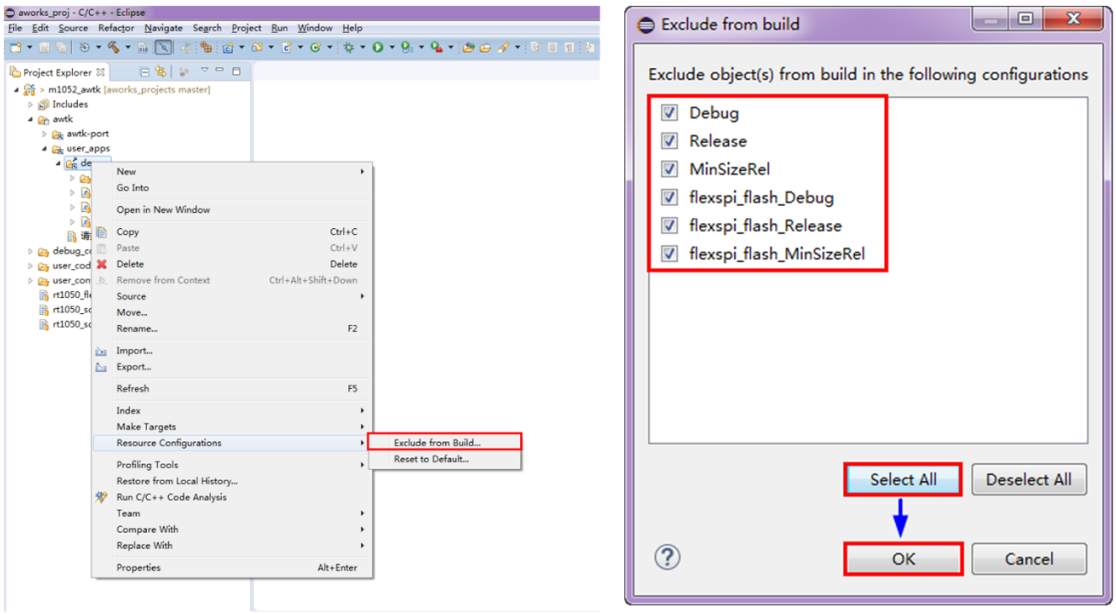


图 10.7　屏蔽 demos 示例

（2）把应用加入工程

例如，需要编译 HelloDesigner-Demo，将 HelloDesigner-Demo 文件夹复制到：aw_m1052_disk-2.0.0-test/ examples/ application/ app_awtk_demo/ src/ user_apps 目录下，然后刷新工程（右键 m1052_awtk → Refresh），最终如下图所示。

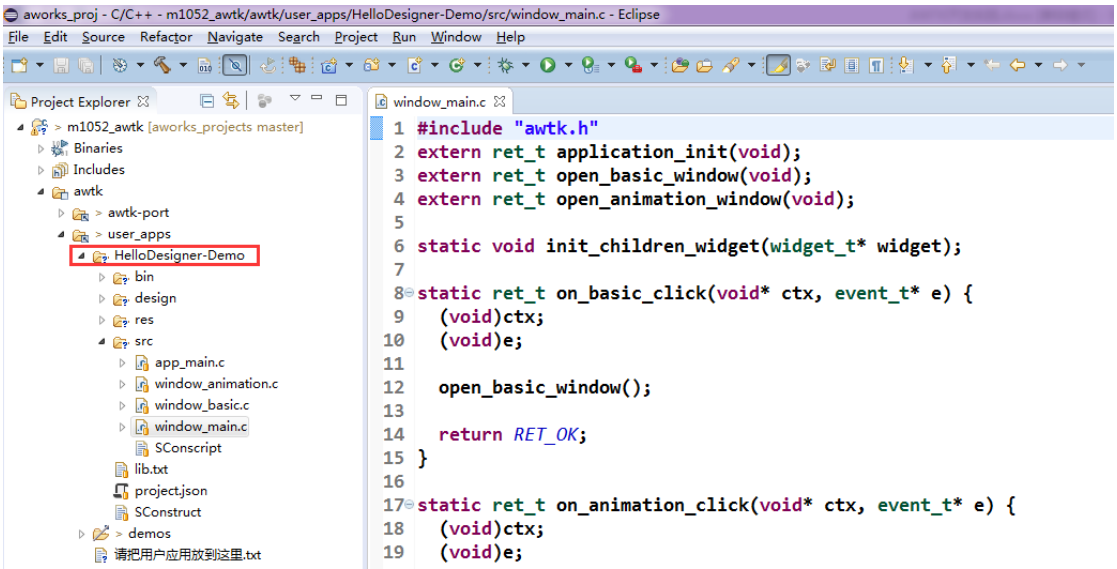


图 10.8　把应用加入工程

（3）编译工程

右键”m1052_awtk”后，按下图所示操作，编译工程。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

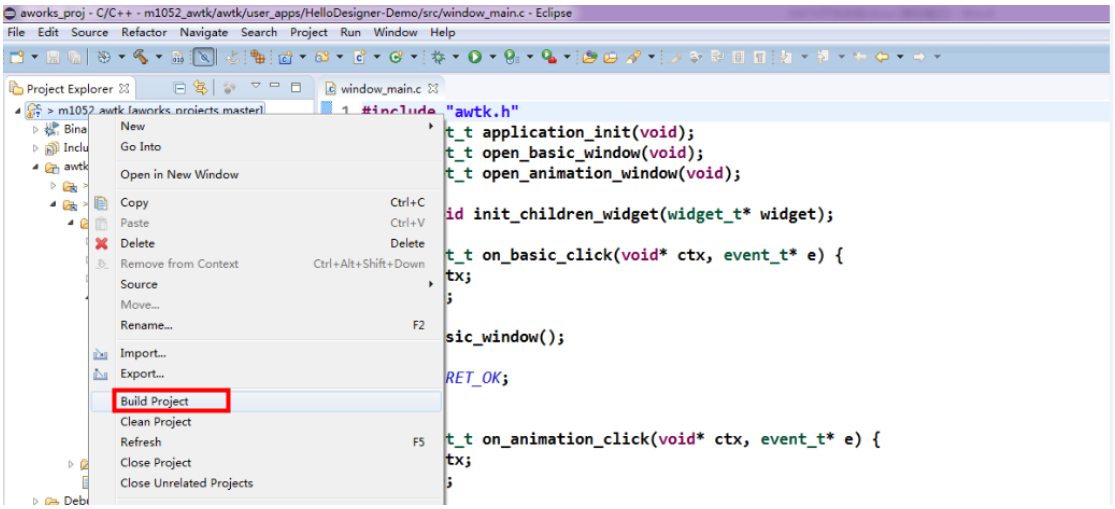


图 10.9　编译工程

（4）下载并调试工程

板子连接好仿真器并上电后，按下图所示操作，调试工程。

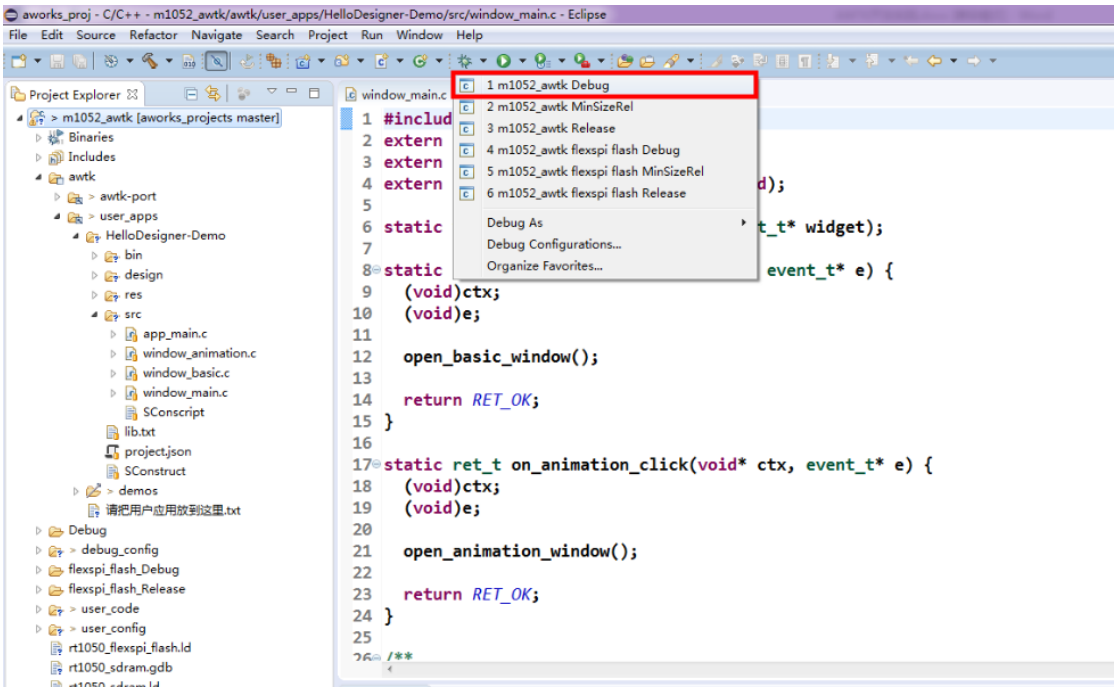


图 10.10　调试工程

10.2.2　使用 Keil 构建应用

找到 AWorksOS SDK 开发包下的 aw_m1052_disk-2.0.0 或 aw_easyarmrt1052_disk-2.0.1 目录，按下面步骤使用 Keil 把自己的应用添加到 AWTK 工程中，具体操作如下：

（1）用 Keil 打开 easy_arm_rt1052.uvprojx 工程，目录如下：

```
examples/application/app_awtk_demo/prj/easy_arm_rt1052_awtk_mdk
```

（2）屏蔽 awtk-demo 示例

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

在 easy_arm_rt1052 工程中，默认提供了一个 awtk-demo 示例。如果需要将自己的应用添加到该工程，先屏蔽 awtk-demo 文件夹，可按下图所示操作。

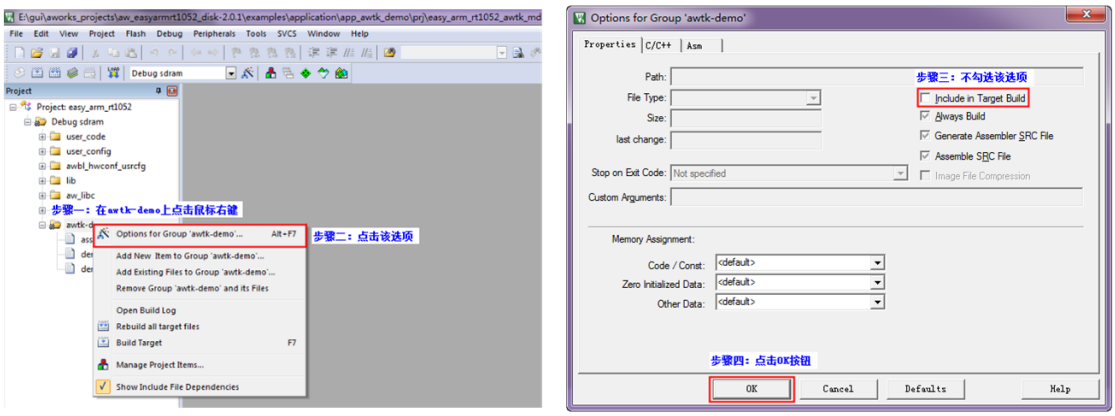


图 10.11　屏蔽 awtk-demo 示例

（3）把应用加入工程

例如，将 HelloDesigner-Demo 中的源代码（通常在 src 目录下），添加到这个 Keil 工程中，可按下图所示操作。

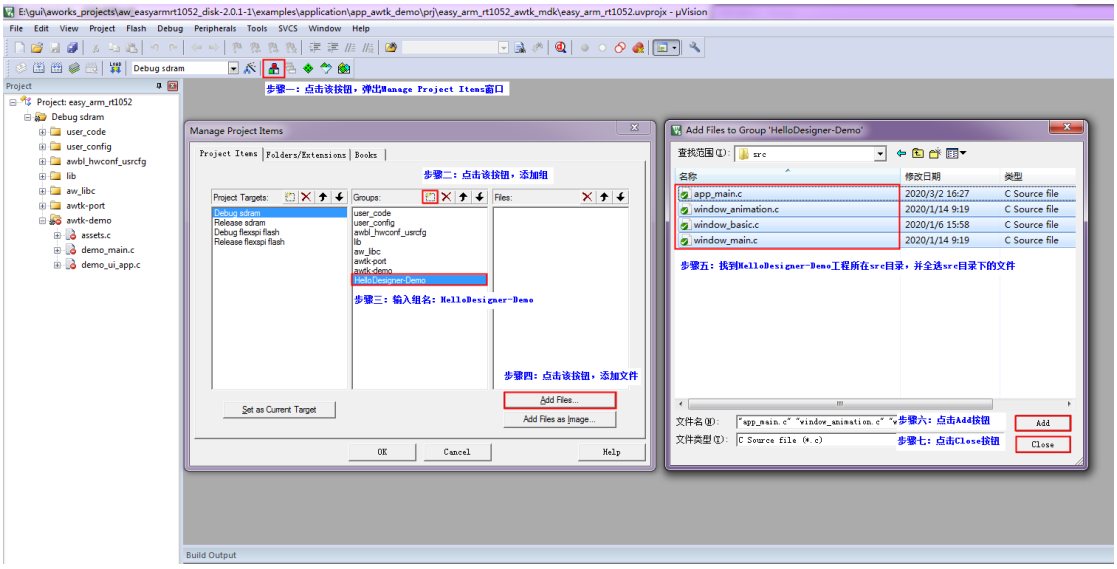


图 10.12　把应用加入工程

（4）编译工程

添加好应用后，就可以编译工程了，可按下图所示操作。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

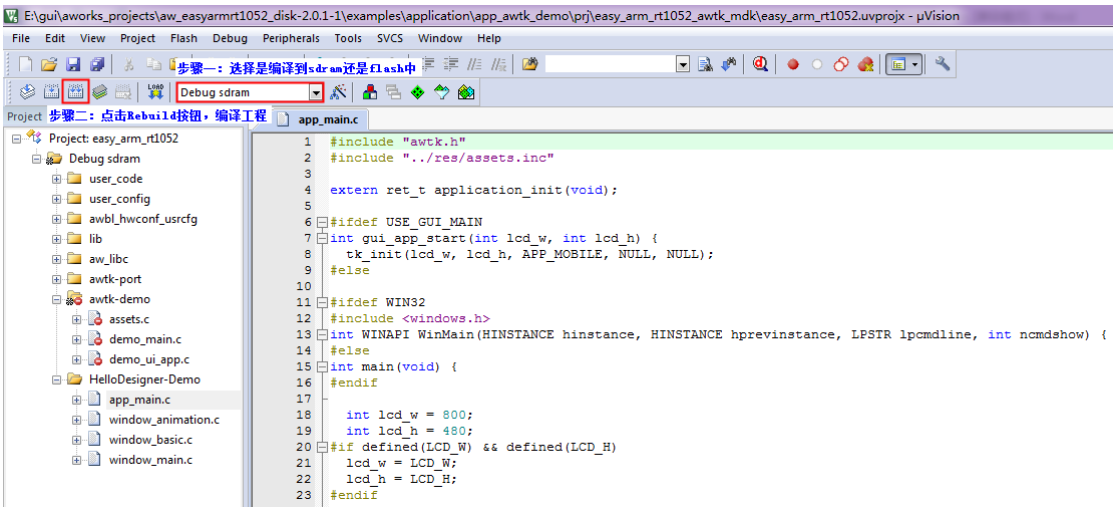


图 10.13　编译工程

10.2.3　如何升级 AWTK

板卡原厂提供的 Eclipse 或 Keil 工程一般内置了指定版本的 AWTK，为避免硬件不兼容的问题，我们不建议用户自行升级这个 AWTK，**你可以咨询板卡原厂提供最新的完整的 SDK 包**。若用户仍然想自行升级，需要自己把 AWTK 代码移植到目标平台上并编译，详细请参考《AWTK 移植及裁剪指南》。

10.3　把应用部署到其他平台

AWTK 已经被移植到多个平台，并提供了对应的 Keil 的工程，例如：

- 用于 RT1052 裸系统的 awtk-easyarm-rt1052-raw 工程
- 用于 STM32 裸系统的 awtk-stm32f429igtx-raw、awtk-stm32f767igtx-raw 等工程

以上这些工程都可以在 AWTK 的官网或者 GitHub 找到。如果想把自己的应用添加到这些 Keil 工程上编译运行，方法都大同小异，**即先屏蔽工程中自带的示例代码文件（通常名字是 xxx_demo），然后往工程中添加用户自己的应用代码及资源文件**。应用代码和资源文件通常是指 AWTK Designer 工程中 src 文件夹下的所有代码，其中二进制资源文件会以头文件的形式包含进来。

注：AWTK Designer 默认使用全字库，可能会导致嵌入式开发环境编译时报告空间不足的错误。用户应根据应用的实际需求，对字库进行裁剪，重新打包后再部署到嵌入式开发环境中。

©2025 Guangzhou ZHIYUAN Electronics Co., Ltd.

诚信共赢，持续学习，客户为先，专业专注，只做第一

广州致远电子股份有限公司

更多详情请访问 www.zlg.cn

欢迎拨打全国服务热线 400-888-4005